

计算机系统导论期中考试 2012 年 11 月 7 日

姓名:

学号:

成绩:

Problem A -- Multiple Choice (18 pts, 1pt or 2pts each)

1. Let `int x = -31/8` and `int y = -31 >> 3`. What are the values of `x` and `y`? (m-f10-a 1-5) C
 - a) `x = -3, y = -3`
 - b) `x = -4, y = -4`
 - c) `x = -3, y = -4`
 - d) `x = -4, y = -3`
2. For which values can `X` not be equal to `Z` in the code below (circle all that apply) (f-f09 1-B) A&D

```
int X = CONSTANT;
float Y = X;
int Z = Y;
```

 - a) For large positive values of `CONSTANT` (e.g., >1,000,000,000)
 - b) For large negative values of `CONSTANT` (e.g., >-100)
 - c) For small positive values of `CONSTANT` (e.g., <100)
 - d) For small negative values of `CONSTANT` (e.g., <-1,000,000,000)
 - e) None of the above (i.e., `X==Z` in all of these cases)
3. Consider the following code, what is the output of the `printf`? (m-f10-a 1-1) B

```
int x = 0x15213F10 >> 4;
char y = (char) x;
unsigned char z = (unsigned char) x;
printf("%d, %u", y, z);
```

 - a) -241, 15
 - b) -15, 241
 - c) -241, 241
 - d) -15, 15
4. After executing the following code, which of the variables are equal to 0? (m-f11 1-3) CD or CDE
 - a) `unsigned int a = 0xffffffff;`
 - b) `unsigned int b = 1;`
 - c) `unsigned int c = a + b;`
 - d) `unsigned long d = a + b;`
 - e) `unsigned long e = (unsigned long)a + b`
5. How does x86 assembly store the return value when a function is finished? (f-f10 1-4) B
 - a) The `ret` instruction stores it in a special `retval` register.
 - b) By convention, it is always in `%eax`.
 - c) It is stored on the stack just above the `(%ebp)` of the callee.
 - d) It is stored on the stack just above all the arguments to the function
6. Which of the following is FALSE concerning x86-64 architecture? (f-f10 1-9) D
 - a) A double is 64 bits long.
 - b) Registers are 64 bits long.

- c) Pointers are 64 bits long.
 - d) Pointers point to locations in memory that are multiples of 64 bits apart.
7. Denormalized floating point numbers are **(m-f10-a 1-18) A**
- a) Very close to zero (small magnitude)
 - b) Very far from zero (large magnitude)
 - c) Un-representable on a number line
 - d) Zero.
8. What is the minimum (most negative) value of a 32-bit two's complement integer? **(m-f11 1-1) C**
- a) -2^{32}
 - b) $-2^{32} + 1$
 - c) -2^{31}
 - d) $-2^{31} + 1$
9. What is the difference between the mov and lea instructions? **(m-f11 1-2) B**
- a) lea dereferences an address, while mov doesn't.
 - b) mov dereferences an address, while lea doesn't.
 - c) lea can be used to copy a register into another register, while mov cannot.
 - d) mov can be used to copy a register into another register, while lea cannot.
10. Which of the following 8 bit floating point numbers (1 sign, 3 exponent, 4 fraction) represent NaN? **(m-s11 1-2) B**
- a) 1 000 1111
 - b) 0 111 1111
 - c) 0 100 0000
 - d) 1 111 0000
11. `%rsp` is 0xdeadbeefdeadd0d0. What is the value in `%rsp` after the following instruction executes?
`pushq %rbx` **(m-s11 1-3) D**
- a) 0xdeadbeefdeadd0d4
 - b) 0xdeadbeefdeadd0d8
 - c) 0xdeadbeefdeadd0cc
 - d) 0xdeadbeefdeadd0c8
12. Consider an `int *a` and an `int n`. If the value of `%ecx` is `a` and the value of `%edx` is `n`, which of the following assembly snippets best corresponds to the C statement `return a[n]`? **(m-s11 1-1) C**
- a) `ret (%ecx,%edx,4)`
 - b) `leal (%ecx,%edx,4),%eax`
`ret`
 - c) `mov (%ecx,%edx,4),%eax`
`ret`
 - d) `mov (%ecx,%edx,1),%eax`
`ret`
13. What is the C equivalent of `mov 0x10 (%rax,%rcx,4), %rdx` **(2pts) (m-s11 1-8) C**
- a) `rdx = rax + rcx + 4 + 10`
 - b) `*(rax + rcx + 4 + 10) = rdx`
 - c) `rdx = *(rax + rcx*4 + 0x10)`
 - d) `rdx = *(rax + rcx + 4 + 0x10)`

14. In what section of an ELF binary are initialized variables located? **(2pts) (e2b-s11) B**
- a) .symtab
 - b) .data
 - c) .bss
 - d) .text
15. A system uses a two-way set-associative cache with 16 sets and 64-byte blocks. Which set does the byte with the address 0xdeadbeef map to? **(2pts) (e2-f10) B**
- a) Set 7
 - b) Set 11
 - c) Set 13
 - d) Set 14

Problem B -- Bits & Bytes, Int, and FP (20 pts)

1. For each of the following propositions, write in all comparisons that make it always true among the four possibilities:

`< > == !=`

If none are guaranteed to hold, please indicate that explicitly by marking it with an X. We have filled in the first two for you as examples. Assume the variables are declared with

`int x, y;`

and initialized to some unknown values. You may assume that int's are 32 bits wide, char's are 8 bits wide and that right shift is arithmetical on signed numbers and logical on unsigned numbers.

(m-s07 1) (4pts)

(1) !=

(2) <

!=

(3) >

!=

(4) !=

If $x > y$ then y	$\left\{ \begin{array}{c} < \\ != \end{array} \right\}$	x
If $x < 0$ then $x+1$	$\left\{ \begin{array}{c} \mathbf{X} \end{array} \right\}$	0
If $x > 0$ then $x+1$	$\left\{ \begin{array}{c} \end{array} \right\}$	0
If $(x \gg 31) < 0$ then x	$\left\{ \begin{array}{c} \end{array} \right\}$	0
If $((x \ll 31) \gg 31) < 0$ then $x \& 1$	$\left\{ \begin{array}{c} \end{array} \right\}$	0
If $x > y$ then (unsigned) x	$\left\{ \begin{array}{c} \end{array} \right\}$	y

2. Floating point encoding. Consider the following 5-bit floating point representation based on the IEEE floating point format. This format does not have a sign bit – it can only represent nonnegative numbers.

- There are $k = 3$ exponent bits. The exponent bias is 3.
- There are $n = 2$ fraction bits.

Recall that numeric values are encoded as a value of the form $V = M \times 2^E$, where E is the exponent after biasing, and M is the significant value. The fraction bits encode the significant value M using either a denormalized (exponent field 0) or a normalized representation (exponent field nonzero). The exponent E is given by $E = 1 - \text{Bias}$ for denormalized values and $E = e - \text{Bias}$ for normalized values, where e is the value of the exponent field exp interpreted as an unsigned number.

Below, you are given some decimal values, and your task is to encode them in floating point format. In addition, you should give the rounded value of the encoded floating point number. To get credit, you must give these as whole numbers (e.g., 17) or as fractions in reduced form (e.g., $3/4$). Any rounding of the significand is based on round-to-even, which rounds an unrepresentable value that lies halfway between two representable values to the nearest even representable value.

(8pts)

Value	FP bits	Rounded value
3	100 10	3 normalized, exact
9	110 00	8 normalized, round down, because 10 is odd
3/16	000 11	3/16 exact, denorm
15/2	110 00	8 normalized, round up, change exponent

Value	Floating Point Bits	Rounded value
9/32	001 00	1/4
3		
9		
3/16		
15/2		

3. Conversion from float to int is a notoriously expensive operation. Not all processors do this in hardware. A clever technique uses the normalization step in double-precision floats. Imagine if you could normalize a (non-negative) floating point number so that the low-order significant bit represents 2^0 . Then the significant would be an integer. **(8 pts) (f-f09 7)**

Assume a double where, after taking the exponent into account, the low-order bit represents 1 (2^0). What does the “implied leading 1” of the significant represent?

2^{52}

Now, assume a double x is known to be in the range $0 \leq x < 232$. To force the low-order bit of x to represent 1 (2^0), you should add by **2^{52}** ?

After this operation, you can store the double to memory and read the (circle one:)

- Upper 32 bits (the end with the sign bit == 0), or
- **Lower 32 bits (the other end)**

as an unsigned integer to get the integer value of the original double. This algorithm will round (circle one)

- toward zero,
- up,
- down,
- **to the nearest even.**

Hints:

double-precision significant has 52 bits

double-precision exponent has 11 bits

double-precision sign-bit is (of course) 1 bit

Problem C Machine Programming (25 pts)

1. The function below is hand-written assembly code for a sorting algorithm. Fill in the blanks on the next page by converting this assembly to C code **(m-s09-4) (13pts)**

```

.globl mystery_sort    # exports the symbol so other .c files
                        # can call the function

mystery_sort:
    jmp     loop1_check

loop1:
    xor     %rdx, %rdx
    mov     %rsi, %rcx
    jmp     loop2_check

loop2:
    mov     (%rdi, %rcx, 8), %rax
    cmp     %rax, (%rdi, %rdx, 8)
    jg      loop2_check
    mov     %rcx, %rdx

loop2_check:
    dec     %rcx
    test    %rcx, %rcx
    jnz     loop2

    dec     %rsi
    mov     (%rdi, %rsi, 8), %rax
    mov     (%rdi, %rdx, 8), %rcx
    mov     %rcx, (%rdi, %rsi, 8)
    mov     %rax, (%rdi, %rdx, 8)

loop1_check:
    test    %rsi, %rsi
    jnz     loop1

    ret
    
```

```

void mystery_sort (long* array, long len)
{
    long a, b, tmp;
    while(len>0)
    {
        a = 0;
        for(b=len-1; b>0; b--)
        {
            if(array[b]>array[a])
            {
                a=b;
            }
        }
        len--;
        tmp = array[len];
        array[len] = array[a];
        array[a] = tmp;
    }
}
    
```

```

void mystery_sort (long* array, long len)
{
    long a, b, tmp;

    while (____ > ____ )
    {
        a = ____;

        for (b = ____; b > ____; b--)
        {
            if (array[____] > array{____])
            {
                ____ = ____;
            }
        }

        len--;

        tmp = array[____];
        array[____] = array[____];
        array[____] = tmp;
    }
}

```

2. Below is some assembly code to a famous algorithm. Please briefly read the code then answer the questions on the following page. **(m-s09-6) 12pts**

```

0000000000400498 <mystery>:
400498: 41 b8 00 00 00 00  mov    $0x0,%r8d
40049e: eb 22              jmp    4004c2 <mystery+0x2a>
4004a0: 89 c8              mov    %ecx,%eax
4004a2: c1 e8 1f          shr    $0x1f,%eax
4004a5: 01 c8              add    %ecx,%eax
4004a7: d1 f8              sar    %eax          ; arith shift right 1 bit
4004a9: 42 8d 0c 00       lea    (%rax,%r8,1),%ecx
4004ad: 48 63 c1          movslq %ecx,%rax
4004b0: 8b 04 87          mov    (%rdi,%rax,4),%eax
4004b3: 39 d0              cmp    %edx,%eax
4004b5: 7d 05              jge    4004bc <mystery+0x24>
4004b7: 41 89 c8          mov    %ecx,%r8d
4004ba: eb 06              jmp    4004c2 <mystery+0x2a>
4004bc: 39 d0              cmp    %edx,%eax
4004be: 7e 10              jle    4004d0 <mystery+0x38>
4004c0: 89 ce              mov    %ecx,%esi
4004c2: 89 f1              mov    %esi,%ecx
4004c4: 44 29 c1          sub    %r8d,%ecx
4004c7: 85 c9              test   %ecx,%ecx
4004c9: 7f d5              jg     4004a0 <mystery+0x8>
4004cb: b9 ff ff ff ff    mov    $0xffffffff,%ecx
4004d0: 89 c8              mov    %ecx,%eax
4004d2: c3                retq

```

- a) Please write a single line of C code to represent the instruction `lea (%rax,%r8,1),%ecx` (Use C variables named `rax`, `r8`, and `ecx`, you can ignore types).
`ecx = rax + r8`
- b) Please write a single line of C code to represent the instruction `mov (%rdi,%rax,4),%eax` (Use C variables named `rdi` and `rax`, you can ignore types).
`eax = *(rdi + rax) // rdi is int *`
`eax = *(rdi + 4 * rax) // rdi is char *`
- c) Commonly found in assembly is the leave instruction; why is that instruction not in this code?
`no stack to return from, no push %ebp`
- d) You learned about two different architectures in class, IA32 and x86 64. What architecture is this code written for and what major downside would occur from using the other architecture
`x86_64`
`32-bit has less registers than 64-bit mode, so local variables would need to be stored on the stack. Stack accessing takes extra instructions and extra time.`

Problem D Stack (10 pts)

Stack discipline. Consider the following C code and assembly code for a recursive function . **(m-f11-5) 10pts**

```

int gcd(int a, int b)      0x08048394 <+0>:    push    %ebp
{                          0x08048395 <+1>:    mov     %esp, %ebp
    if(!b)                0x08048397 <+3>:    sub     $0x10, %esp
    {                      0x0804839a <+6>:    mov     0x8(%ebp), %eax
        return a;          0x0804839d <+9>:    mov     0xc(%ebp), %ecx
    }                     0x080483a0 <+12>:   test    %ecx, %ecx
                          0x080483a2 <+14>:   je      0x80483b7 <gcd+35>
    return gcd(b, a % b);  0x080483a4 <+16>:   mov     %eax, %edx
                          0x080483a6 <+18>:   sar     $0x1f, %edx
                          0x080483a9 <+21>:   idiv    %ecx
                          0x080483ab <+23>:   mov     %edx, 0x4(%esp)
                          0x080483af <+27>:   mov     %ecx, (%esp)
                          0x080483b2 <+30>:   call    0x8048394 <gcd>
                          0x080483b7 <+35>:   leave
                          0x080483b8 <+36>:   ret
    
```

Imagine that a program makes the procedure call `gcd(213, 18)`. Also imagine that prior to the invocation, the value of `%esp` is `0xffff1000`—that is, `0xffff1000` is the value of `%esp` immediately before the execution of the call instruction.

A. Note that the call `gcd(213, 18)` will result in the following function invocations: `gcd(213, 18)`, `gcd(18, 15)`, `gcd(15, 3)`, and `gcd(3, 0)`. Using the provided code and your knowledge of IA32 stack discipline, fill in the stack diagram with the values that would be present immediately before the execution of the `leave` instruction for `gcd(15, 3)`. Supply numerical values wherever possible, and cross out each blank for which there is insufficient information to complete with a numerical value.

Hints: The following set of C style statements describes an approximation of the operation of the instruction `idiv %ecx`, where `/` is the division operator and `%` is the modulo operator:

`%eax = %eax / %ecx`

`%edx = %eax % %ecx`

Also, recall that `leave` is equivalent to `movl %ebp, %esp; popl %ebp`

Part A:

+	-----+	
	unknown	0xffff1008
+	-----+	
	18	0xffff1004
+	-----+	
	213	0xffff1000
+	-----+	
	unknown	0xffff0ffc
+	-----+	
	unknown	0xffff0ff8
+	-----+	
	unknown	0xffff0ff4

unknown	0xffff0ff0
15	0xffff0fec
18	0xffff0fe8
0x080483b7	0xffff0fe4
0xffff0ff8	0xffff0fe0
unknown	0xffff0fdc
unknown	0xffff0fd8
3	0xffff0fd4
15	0xffff0fd0
0x080483b7	0xffff0fcc
0xffff0fe0	0xffff0fc8
unknown	0xffff0fc4
unknown	0xffff0fc0
0	0xffff0fbc
3	0xffff0fb8
unknown	0xffff0fb4
unknown	0xffff0fb0

B. What are the values of %esp and %ebp immediately before the execution of the ret instruction for gcd(15, 3)?

esp: 0xffff0fcc
ebp: 0xffff0fe0

Problem E -- The Hit or Miss Question (15 pts)

Given a 32-bit Linux system that has a 2-way associative cache of size 128 bytes with 32 bytes per block. Long longs are 8 bytes. For all parts, assume that table starts at address 0x0.

```
int i;
```

```
int j;
```

```
long long table[4][8];
```

```
for (j = 0; j < 8; j++) {
    for (i = 0; i < 4; i++) {
        table[i][j] = i + j;
    }
}
```

- A. This problem refers to code sample 1. In the table below write down in each space whether that element's access will be a hit or a miss. Indicate hits with a 'H' and misses with a 'M'.

	0	1	2	3	4	5	6	7
0	M	M	M	M	M	M	M	M
1	M	M	M	M	M	M	M	M
2	M	M	M	M	M	M	M	M
3	M	M	M	M	M	M	M	M

What is the miss rate of this code sample?

1

```

int i;

int j;

int table[4][8];

for (j = 0; j < 8; j++) {
    for (i = 0; i < 4; i++) {
        table[i][j] = i + j;
        table[j][i]?
    }
}

```

- B. This problem refers to code sample above. In the table below write down in each space whether that element's access will be a hit or a miss. Indicate hits with a 'H' and misses with a 'M'.

	0	1	2	3	4	5	6	7
0	M	H	H	H	H	H	H	H
1	M	H	H	H	H	H	H	H
2	M	H	H	H	H	H	H	H
3	M	H	H	H	H	H	H	H

What is the miss rate of this code sample?

1/8

- C. One code sample performs better than the other. Why is this?

The second sample performs better than the first because it makes better use of the cache. In the first, heavy conflict results in a large number of evictions. In the second, the entire array fits in the cache, and the only misses are cold misses.

Problem F – Linking (12 pts)

For each of the following code snippets, write down all symbols in the resulting object files from compilation. Write whether it is a weak global, strong global, or local variable, and what section of the final compiled ELF binary the variable will go into. Fill in the value if you have enough information to determine the value.

A.

main.c	foo.c
<pre>#include <stdio.h> int x; int y; int z = 0; int main() { printf("%x\n", x); printf("%x\n", y); x = 0xdeadbeef; printf("%x\n", x); printf("%x\n", y); return 0; }</pre>	<pre>short x = 5; short y = 2;</pre>

File	Symbol	Strength / scope	Value	ELF Section
Main.o	X	Weak global	-	.data
	Y	Weak global	-	.data
	Z	Strong global	0	.data
	Main	Strong global	-	.text
foo.o	X	Strong global	5	.data
	Y	Strong global	2	.data

第一题 选择题（每小题 2 分，共 34 分）

（每小题有一个或多个正确答案）

1、变量 x 的值为 $0x01234567$ ，地址 $\&x$ 为 $0x100$ ；则该变量的值在 x86 和 Sun 机器内存中的存储排列顺序正确的是（ ）

选项	机器类型	地址			
		0x100	0x101	0x102	0x103
A	x86	67	45	23	01
	Sun	01	23	45	67
B	x86	76	54	32	10
	Sun	01	23	45	67
C	x86	01	23	45	67
	Sun	67	45	23	01
D	x86	01	23	45	67
	Sun	01	23	45	67

答案：A

考察大端、小端；同时 sun 是大端、x86 是小端

2、假设下列 int 和 unsigned 数均为 32 位，

`int x = 0x80000000;`

`unsigned y = 0x00000001;`

`int z = 0x80000001;`

以下表达式正确的是（ ）

A. $(-x) < 0$

B. $(-1) > y$

C. $(z < 3) == (z * 8)$

D. $y * 24 == z < 5 - z < 3$

答案:ABCD；考虑到运算符的优先顺序，选 ABC 也算对

A. int 中 $0x80000000$ 的相反数还是自己

B. signed (-1) 和 unsigned y 比较，都按照 unsigned，所以强制类型转换后 (-1) 很大

C: unsigned, signed 左移三位 $= *8$

D: 应该是相等关系；signed 左移之后，和 unsigned $y * 24$ 相比都看成 unsigned

3、对 $x = 1\frac{1}{8}$ 和 $y = 1\frac{3}{8}$ 进行小数点后两位取整（rounding to nearest even），结果正确的是（ ）

A. $1\frac{1}{4}$, $1\frac{1}{4}$

B. 1, $1\frac{1}{4}$

C. $1\frac{1}{4}$, $1\frac{1}{2}$

D. 1, $1\frac{1}{2}$

答案：D

$x = 1.00\underline{100}$ half way and down --> 1.00

$y = 1.01\underline{100}$ half way and up-->1.10

4、在完成 Bomb Lab 的时候，通常先执行 gdb bomb 启动调试，然后执行 ____ explode_bomb 命令以防引爆炸弹，之后在进行其他必要的设置后，最后执行 ____ 命令以便开始执行程序。上述两个空格对应的命令是（ ）

A. st, ru B. br, go C. br, ru D. st, go

答案: c

说明: 根据之前的讨论, 出一道题目检查同学们是否自己做过 lab

5、已知函数 `int x(int n) { return n*____; }` 对应的汇编代码如下:

```
lea(%rdi,%rdi,4),%rdi
lea(%rdi,%rdi,1),%eax
retq
```

请问横线上的数字应该是 ()

A. 4 B. 5 C. 2 D. 10

答案: D

说明: 此题目考察对于乘法的转换, 难度较低, 适合出选择题。还可以把乘法换成除法, 就可以出大题或者简答题。

6、32 位 x86 计算机、Windows 操作系统下定义的一个 structure S 包含三个部分: `double a, int b, char c`, 请问 S 在内存空间中最多和最少分别能占据多少个字节 (32 位 Windows 系统按 1、4、8 的原则对齐 char、int、double)? 答: ()

A. 16, 13
B. 16, 16
C. 24, 13
D. 24, 16

答案: D 考虑对齐,windows double 按 8 字节对齐, 最长 c, a, b, 最短 a,b,c

7、x86 体系结构的内存寻址方式有多种格式, 请问下列哪些指令是正确的: ()

A. `movl $34, (%eax)`
B. `movl (%eax), %eax`
C. `movl $23, 10(%edx,%eax)`
D. `movl (%eax), 8(%ebx)`

答案: ABC, 寻址不支持内存到内存的访问

8、x86 体系结构中, 下面哪些选项是错误的? 答: ()

A. `leal` 指令只能够用来计算内存地址
B. x86_64 机器可以使用栈来给函数传递参数
C. 在一个函数内, 改变任一寄存器的值之前必须先将其原始数据保存在栈内
D. 判断两个寄存器中值大小关系, 只需要 SF (符号) 和 ZF (零) 两个 conditional code

答案: ACD

9、下面对 RISC 和 CISC 的描述中, 错误的是: ()

A. CISC 指令系统中的指令数目较多, 有些指令的执行周期很长; 而 RISC 指令系统中通常指令数目较少, 指令的执行周期都较短。
B. CISC 指令系统中的指令编码长度不固定; RISC 指令系统中的指令编码长度固定, 这样使得 RISC 机器可以获得了更短的代码长度。
C. CISC 指令系统支持多种寻址方式, RISC 指令系统支持的寻址方式较少。
D. CISC 机器中的寄存器数目较少, 函数参数必须通过栈来进行传递; RISC 机器中的

寄存器数目较多，可以通过寄存器来传递参数，避免了不必要的存储访问。

答案: BD

10、下面对流水线技术的描述，正确的是：()

- A. 流水线技术不仅能够提高执行指令的吞吐率，还能减少单条指令的执行时间。
- B. 不断加深流水线级数，总能获得性能上的提升。
- C. 流水级划分应尽量均衡，吞吐率会受到最慢的流水级影响。
- D. 指令间的数据相关可能会引发数据冒险，可以通过数据转发或暂停流水线来解决。

答案: CD

(11-13)、在教材所描述的流水线处理器 (the PIPE processor) 上分别运行如下四段 Y86 程序代码。请分析其中数据冒险的具体情况，并回答后续3个小题。

#Program 1:	#Program 2:
mrmovl 8(%ebx), %edx	mrmovl 8(%ebx), %edx
rmmovl %edx, 16(%ecx)	nop
	rmmovl %edx, 16(%ecx)
#Program 3:	#Program 4:
mrmovl 8(%ebx), %edx	mrmovl 8(%ebx), %edx
nop	nop
nop	nop
rmmovl %edx, 16(%ecx)	nop
	rmmovl %edx, 16(%ecx)

11、对于每段程序，请指出是否会因为数据冒险导致流水线停顿 (Stall)。

Program 1: (), Program 2: (), Program 3: (), Program 4:
();

A. Stall B. No-Stall

答案: A, B, B, B

12、对于每段程序，请指出流水线处理器内是否会产生数据转发 (Forwarding)。

Program 1: (), Program 2: (), Program 3: (), Program 4:
();

A. Forwarding B. No-Forwarding

答案: A, A, A, B

13、对于每段程序，请指出流水线处理器内使用哪个信号进行数据转发，如果不进行数据转发，则用none表示。

Program 1: (), Program 2: (), Program 3: (), Program 4:
();

A. m_valM B. W_valM C. none

答案: A, A, B, C

14、下面哪些选项是错误的？答：()

A. 同一个任务采用时间复杂度为 $O(\log N)$ 算法一定比采用复杂度为 $O(N)$ 算法的执行时间短

- B. 编译器进行程序优化时，总是可以使用算数结合律来减少计算量
- C. 增大循环展开（loop unrolling）的级数，有可能降低程序的执行性能（即增加执行时间）
- D. 分支预测时，“总是预测不跳转”（branch not taken）一定比“总是预测跳转”（branch taken）预测准确率高

答案：ABD

15、以下哪些程序优化编译器总是可以自动进行？（假设 `int i`, `int j`, `int A[N]`, `int B[N]`, `int m` 都是局部变量，`N` 是一个整数型常量，`int foo(int)` 是一个函数）答：（ ）

	优化前	优化后
A.	<pre>for (j = 0 ; j < N ; j++) m += i*N*j;</pre>	<pre>int temp = i*N; for (j= 0 ; j < N ; j++) m += temp * j;</pre>
B.	<pre>for (j = 0 ; j < N ; j++) B[i] *= A[j];</pre>	<pre>int temp = B[i]; for (j= 0 ; j < N ; j++) temp *= A[j]; B[i] = temp;</pre>
C.	<pre>for (j = 0 ; j < N ; j++) m = (m + A[j]) + B[j];</pre>	<pre>for (j = 0 ; j < N ; j++) m = m + (A[j] + B[j]);</pre>
D.	<pre>for (j = 0 ; j < foo(N) ; j++) m ++;</pre>	<pre>int temp = foo(N); for (j= 0 ; j < temp ; j++) m ++;</pre>

答案：AC

16、如果直接映射高速缓存大小是 4KB，并且块（block）大小为 32 字节，请问它每组（set）有多少行（line）？答：（ ）

- A. 128 B. 64 C. 32 D. 1

答案：D

17、关于局部性（locality）的描述，不正确的是：（ ）

- A. 数组通常具有很好的时间局部性
- B. 数组通常具有很好的空间局部性
- C. 循环通常具有很好的时间局部性
- D. 循环通常具有很好的空间局部性

答案：A

第二题（8 分）

1) 判断下表中每一行表达式对或错。如果错，请举出反例或简要说明原因（每行 1 分）

int x, y;

unsigned u, v;

	True or false	原因或举出反例
if $x < 0$, then $x * 2 < 0$		
$u \leq -1$		
if $x > y$, then $-x < -y$		
if $u > v$, then $-u > -v$		

答案

	True or false	原因或举出反例
if $x < 0$, then $x * 2 < 0$	F	$X = -2^w - 1$
$u \leq -1$	T	-1 作为无符号来比大于 u
if $x > y$, then $-x < -y$	F	$X = 0, y = -2^w - 1$
if $u > v$, then $-u > -v$	F	$U = 2, v = 1$

2) 请按 IEEE 浮点标准的单精度浮点数表示下表中的数值，首先写出形如 $(-1)^s \times M \times 2^E$ 的表达式，然后给出十六进制的表示。（每格 1 分）

注：单精度浮点数的字段划分如下：

符号位 (s): 1-bit; 阶码字段 (exp): 8-bit; 小数字段 (frac): 23-bit; 偏置值 (bias): 127。

Value	$(-1)^s \times M \times 2^E, 1 \leq M < 2$	Hex representation
$-1 \frac{1}{2}$		
2^{-149}		

答案

Value	$(-1)^s \times M \times 2^E$ $1 \leq M < 2$	Hex representation
$-1 \frac{1}{2}$	$(-1) \times 1.1 \times 2^0$	0xBFC00000
2^{-149}	1.0×2^{-149}	0x00000001

第三题 （11 分）

阅读下面的 C 代码：

```
/*
 * Copyright (C) 2013 Davidlohr Bueso <davidlohr.bueso@hp.com>
 *
 * Based on the shift-and-subtract algorithm for computing integer
 * square root from Guy L. Steele.
 */
/**
 * int_sqrt - rough approximation to sqrt
 * @x: integer of which to calculate the sqrt
 *
 * A very rough approximation to the sqrt() function.
 */
unsigned long int_sqrt(unsigned long x)
{
    unsigned long b, m, y = 0;

    if (x <= 1)
        return x;

    m = 1UL << (BITS_PER_LONG - 2);
    while (m != 0) {
        b = y + m;
        y >>= 1;

        if (x >= b) {
            x -= b;
            y += m;
        }
        m >>= 2;
    }

    return y;
}
```

1) 在 64 位的机器上 BITS_PER_LONG 的定义为 long 类型的二进制位数，它是多少位？

2) 填写下面反汇编中的缺失的内容：

```
<int_sqrt>:
4004c4:    push    %rbp
4004c5:    mov     %rsp,%rbp
4004c8:    mov     %rdi,-0x28(%rbp)

4004cc:    movq    __ (1) __,-0x8(%rbp)
4004d4:    cmpq    $0x1,-0x28(%rbp)

4004d9:    ja      __ (2) __ <int_sqrt+??>
4004db:    mov     -0x28(%rbp),%rax

4004df:    jmp     __ (3) __ <int_sqrt+??>
4004e1:    movl    $0x0,-0x10(%rbp)

4004e8:    movl    __ (4) __,-0xc(%rbp)

4004ef:    jmp     __ (5) __ <int_sqrt+??>
4004f1:    mov     -0x10(%rbp),%rax
4004f5:    mov     -0x8(%rbp),%rdx
```

```

4004f9:    lea    (6),%rax
4004fd:    mov     %rax,-0x18(%rbp)
400501:    shrq   -0x8(%rbp)
400505:    mov     -0x28(%rbp),%rax
400509:    cmp     -0x18(%rbp),%rax

40050d:    jb      (7)<int_sqrt+??>
40050f:    mov     -0x18(%rbp),%rax
400513:    sub     %rax,-0x28(%rbp)
400517:    mov     -0x10(%rbp),%rax
40051b:    add     %rax,-0x8(%rbp)
40051f:    shrq   (8), -0x10(%rbp)
400524:    cmpq    $0x0,-0x10(%rbp)
400529:    jne     (9)<int_sqrt+??>
40052b:    mov     -0x8(%rbp),(10)
40052f:    leaveq
400530:    retq

```

答案:

1、 答： 64

2、

<int_sqrt>:

```

4004c4:    push    %rbp
4004c5:    mov     %rsp,%rbp
4004c8:    mov     %rdi,-0x28(%rbp)
4004cc:    movq    $0x0,-0x8(%rbp)
4004d4:    cmpq    $0x1,-0x28(%rbp)
4004d9:    ja      4004e1 <int_sqrt+0x1d>
4004db:    mov     -0x28(%rbp),%rax
4004df:    jmp     40052f <int_sqrt+0x6b>
4004e1:    movl    $0x0,-0x10(%rbp)
4004e8:    movl    $0x40000000,-0xc(%rbp)
4004ef:    jmp     400524 <int_sqrt+0x60>
4004f1:    mov     -0x10(%rbp),%rax
4004f5:    mov     -0x8(%rbp),%rdx
4004f9:    lea     (%rdx,%rax,1),%rax
4004fd:    mov     %rax,-0x18(%rbp)
400501:    shrq   -0x8(%rbp)
400505:    mov     -0x28(%rbp),%rax
400509:    cmp     -0x18(%rbp),%rax
40050d:    jb      40051f <int_sqrt+0x5b>
40050f:    mov     -0x18(%rbp),%rax
400513:    sub     %rax,-0x28(%rbp)
400517:    mov     -0x10(%rbp),%rax
40051b:    add     %rax,-0x8(%rbp)
40051f:    shrq    $0x2,-0x10(%rbp)
400524:    cmpq    $0x0,-0x10(%rbp)
400529:    jne     4004f1 <int_sqrt+0x2d>
40052b:    mov     -0x8(%rbp),%rax
40052f:    leaveq
400530:    retq

```


第四题（10 分）

阅读下面的汇编代码：

<f>:

```
4004c4:    push    %rbp
4004c5:    mov     %rsp,%rbp
4004c8:    sub     $0x10,%rsp
4004cc:    mov     %edi,-0x4(%rbp)
4004cf:    cmpl    $0x1,-0x4(%rbp)
4004d3:    ja      4004dc <f+0x18>
4004d5:    mov     $0x1,%eax
4004da:    jmp     40052d <f+0x69>
4004dc:    mov     -0x4(%rbp),%eax
4004df:    and     $0x1,%eax
4004e2:    test    %eax,%eax
4004e4:    jne     4004f5 <f+0x31>
4004e6:    mov     0x200440(%rip),%eax    # 60092c <x.1604>
4004ec:    add     $0x1,%eax
4004ef:    mov     %eax,0x200437(%rip)    # 60092c <x.1604>
4004f5:    mov     -0x4(%rbp),%eax
4004f8:    and     $0x1,%eax
4004fb:    test    %al,%al
4004fd:    je      40050e <f+0x4a>
4004ff:    mov     0x20042b(%rip),%eax    # 600930 <y.1605>
400505:    add     $0x1,%eax
400508:    mov     %eax,0x200422(%rip)    # 600930 <y.1605>
40050e:    mov     -0x4(%rbp),%eax
400511:    sub     $0x1,%eax
400514:    mov     %eax,%edi
400516:    callq   4004c4 <f>
40051b:    mov     0x20040f(%rip),%edx    # 600930 <y.1605>
400521:    lea     (%rax,%rdx,1),%edx
400524:    mov     0x200402(%rip),%eax    # 60092c <x.1604>
40052a:    lea     (%rdx,%rax,1),%eax
40052d:    leaveq
40052e:    retq
```

1) 程序

```
main()
{
    unsigned int n;
    for (n=1; n< 4; n++) {
        printf("f(%d) = %x\n", n, f(n));
    }
}
```

的运行结果为：f(1)=1, f(2)=4e, f(3)=9f, 请填写 f 函数所需要的内容（每空 1 分）：

```
#define N    (1) _____
#define M    (2) _____
```

```
struct P1 {char c[N]; char *d[N]; char e[N]; } P1;
struct P2 {int i[M]; char j[M]; short k[M]; } P2;
```

```
unsigned int f(unsigned int n)
{
    (3) _____ unsigned int x = sizeof(P1);
    (4) _____ unsigned int y = sizeof(P2);
```

```

if ( (5) )
    return 1;

if ( (6) )
    x++;

if ( (7) )
    y++;

return (8);
}

```

2、程序

```

main()
{
    printf("%x, %x\n", f(2), f(2));
}

```

的运行结果为：（2分）

1、答案：

```

#define N    3
#define M    5

```

```

struct P1 {char c[N]; char *d[N]; char e[N]; } P1;
struct P2 {int i[M]; char j[M]; short k[M]; } P2;

```

```

unsigned int f(unsigned int n)
{
    static unsigned int x = sizeof(P1);
    static unsigned int y = sizeof(P2);

    if (n<=1)
        return 1;

    if ((n & 1) == 0)
        x++;

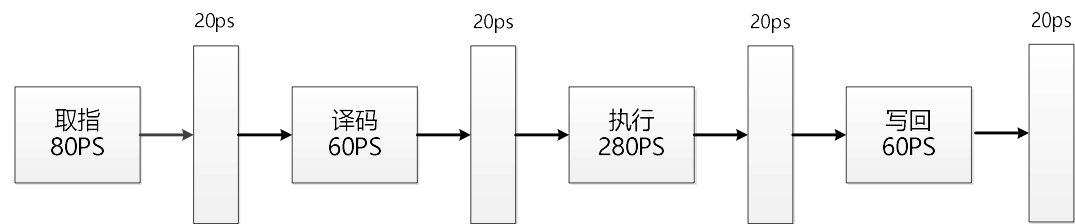
    if ((n & 1) == 1)
        y++;

    return f(n-1) + (y) + (x);
}

```

2、答案： 4f, 4e （回答 4e, 4f 给一半的分）

第五题（9 分）



在“取指-译码-执行-写回”的四级流水线中，各流水级的工作内容和延迟如上图所示，寄存器的延迟也已标出。数据和指令分别存放在不同的存储器中。Cycle N 写入寄存器文件的数据 Cycle N+1 才可读出。请问：

- 1) 若不考虑流水线填充和清空时间，请计算该处理器的吞吐率。（1 分）
- 2) 若将该处理器改造为单周期处理器（SEQ），请计算 SEQ 处理器的吞吐率。（1 分）
- 3) 在上述流水线中，执行阶段包含了访问数据存储的时间。对于如下的 Y86 程序段，指令间存在哪些数据相关（dependence），会引起哪些数据冒险（hazard）？（5 分）

Prog:
 irmovl\$128,%edx #instr1
 irmovl\$3,%ecx #instr2
 rmmovl %ecx,0(%edx) #instr3
 irmovl\$10,%ebx #instr4
 mrmovl 0(%edx),%eax #instr5
 addl %ebx,%eax #instr6

- 4) 以上的数据冒险，可以通过转发（forward）的方法解决。请结合上述程序代码和流水线结构图逐个说明解决方案。（2 分）

答案：

- 1) $1000/(280 + 20) = 1000/300 = 3.33\text{GIPS}$
- 2) $1000/(80 + 60 + 280 + 60 + 20) = 1000/500 = 2\text{ GIPS}$
- 3)

Prog:
 irmovl\$128,%edx ; 1、dx= 128
 irmovl\$3,%ecx ; 2、cx= 3
 rmmovl %ecx,0(%edx) ; 3、[dx] = cx
 irmovl\$10,%ebx ; 4、bx= 10
 mrmovl 0(%edx),%eax ; 5、ax= [dx]
 addl %ebx,%eax ; 6、bx = bx + ax

相关：1-3，2-3，1-5，4-6，5-6

冒险：1-3，2-3，4-6，5-6

- 4) 2-3，5-6：执行到译码的转发通路解决；1-3,4-6：写回到译码的转发通路解决。

第六题（9 分）

请分析Y86 ISA中定义的两条指令（cmovXX、call）和一条新加入Y86 ISA的IA32指令（decl：将操作数减1）。若在教材所描述的SEQ处理器上执行这些指令，请按下表填写每个阶段进行的操作。如果在某一阶段没有任何操作，请填写none指明。

注1、所用到的指令编码为：

cmovXX rA, rB	2	fn	rA	rB	
call Dest	8	0	Dest		
decl rA	C	0	rA	F	

注2、需说明的信号包括：icode, ifun, rA, rB, valA, valB, valC, valE, valP; the register file R[], data memory M[], Program counter PC, condition codes CC。

（每格0.5分）

Stage	cmovXX rA, rB	call Dest	decl rA
Fetch	icode:ifun $\leftarrow$ M ₁ [PC] rA:rB $\leftarrow$ M ₁ [PC+1] valP $\leftarrow$ PC+2	icode:ifun $\leftarrow$ M ₁ [PC] valC $\leftarrow$ M ₄ [PC+1] valP $\leftarrow$ PC+5	icode:ifun $\leftarrow$ M ₁ [PC] rA:rB $\leftarrow$ M ₁ [PC+1] valP $\leftarrow$ PC+2
Decode	valA $\leftarrow$ R[rA]	valB $\leftarrow$ R[%esp]	valA $\leftarrow$ R[rA]
Execute	valE $\leftarrow$ 0+valA Cnd $\leftarrow$ Cond(CC,ifun) （也可以写Set CC）	valE $\leftarrow$ valB+(-4)	valE $\leftarrow$ valA+(-1) Cnd $\leftarrow$ Cond(CC,ifun) （也可以写Set CC）
Memory	none	M ₄ [valE] $\leftarrow$ valP	none
Write back	if(Cnd) R[rB] $\leftarrow$ valE	R[%esp] $\leftarrow$ valE	R[rA] $\leftarrow$ valE
PC update	PC $\leftarrow$ valP	PC $\leftarrow$ valC	PC $\leftarrow$ valP

第七题（10 分）

已知如下的汇编程序实现了函数 `transform(char* src, char* tgt, char delta)`

`transform:`

`jmp L2`

`L1:`

`add %edx, %eax`

`add $1, %rdi`

`mov %al, (%rsi)`

`add $1, %rsi`

`L2:`

`movzbl (%rdi), %eax`

`test %al, %al`

`jne L1`

`movb $0, (%rsi)`

`retq`

参考信息：64 位指令集中传递前三个参数分别使用寄存器 `%rdi`, `%rsi` 和 `%rdx`

1) 写出 `transform` 函数对应的 C 语言版本（2 分）

2) 假设读写访存指令延迟为 20 个时钟周期，其他指令延迟为 2 个时钟周期，所有分支预测都成功。同时 CPU 包含足够多的部件来实现指令集并行，那么在最理想情况下 CPE 最低应该是多少（2 分）？为什么（2 分）？

3) 已知 `src` 对应字符串中每个字符 `c` 都满足 $0 < c \leq 80$ 且 $0 \leq \text{delta} \leq 5$ 。通过下面的改写，可以把 `transform` 程序 CPE 的理论下限降低一半，请填空。假设程序运行在小端法机器上。（每空 1 分）

```
void transform(char* src, char* tgt, char delta) {
    short x = _____;
    while(*src && _____) {
        *(short*)tgt = *(short*)src + x;
        src += 2;
        tgt += 2;
    }
    *(short*)tgt = _____ ? *(short*)src + delta : *(short*)tgt & _____;
}
```

答案：

(1) `{while(*src) *tgt++ = *src++ + delta; *tgt = 0;}`

(2) 42

每次循环的关键路径为 读内存、做加法、写内存，该路径需要 42 个时钟周期。

本题陷阱：同学可能会受书上的例子误导认为做加法可以和下一个时钟周期的读内存并行，使得 CPE 降到 40，但实际上因为 `src` 和 `tgt` 指向的位置可能重叠，我们不能把下一次迭代的读操作移动到这一次的写操作之前。

(3) `((short) delta) << 8 + delta`

`*(src+1)`

`0xFF00`

`*src`

第八题（9 分）

假设存在一个能够存储四个数据的 Cache，每一个 line 的长度（B）为 2 字节。假设内存空间的地址一共是 32 字节，既内存空间地址长度一共是 5 个比特：从 0(00000)到 31(11111)，一共有 8 个数据读取操作，每个操作的地址按顺序如下所示（单位是字节），数据替换采用 LRU（least recently used）策略。

数据访问地址： 1 -> 4 -> 17 -> 2 -> 8 -> 16 -> 9 -> 0

1) 如果 Cache 的结构是 directed mapped (S=4, E=1)，如下图所示，请在下图空白处填入，访问上述数据序列访问后 Cache 的状态。（注：TAG 使用二进制格式，V=1 代表数据有效，用[A-B]表示地址 A 到 B 之间对应的数据）（4 分）

V	TAG	DATA
1	00	M[0-1]
1	00	M[2-3]
1	00	M[4-5]
0		

2) 如果 cache 的结构如下图所示既(S = 2, E = 2)， 请填入访问后的状态 （2 分）

V	TAG	DATA	V	TAG	DATA
1	000	M[0-1]	1	010	M[8-9]
1	000	M[2-3]	0		

在这种情况下，数据访问一共产生了多少次 Miss 6 （1 分）

（顺序 M-> M-> M-> M-> H->H-> M）

3) 如果 cache 的结构变成 (S=1, E=4)，最终存储在 Cache 里面的数据有那些（注：只需要填写数据部分，顺序不限）？

_____, _____, _____, _____ （2 分）

M[0-1], M[8-9], M[16-17], M[2-3]

说明：1 -> 4 -> 17 -> 2 -> 8 -> 16 -> 9 -> 0 （后四个不重复数据）

北京大学信息科学技术学院考试试卷

考试科目： 计算机系统导论 姓名： _____ 学号： _____

考试时间： 2014 年 11 月 13 日 任课教师： _____

题号	一	二	三	四	五	六	七	八	总分
分数									
阅卷人									

北京大学考场纪律

1、考生进入考场后，按照监考老师安排隔位就座，将学生证放在桌面上。无学生证者不能参加考试；迟到超过 15 分钟不得入场。在考试开始 30 分钟后方可交卷出场。

2、除必要的文具和主考教师允许的工具书、参考书、计算器以外，其它所有物品（包括空白纸张、手机、或有存储、编程、查询功能的电子用品等）不得带入座位，已经带入考场的必须放在监考人员指定的位置。

3、考试使用的试题、答卷、草稿纸由监考人员统一发放，考试结束时收回，一律不准带出考场。若有试题印制问题请向监考教师提出，不得向其他考生询问。提前答完试卷，应举手示意请监考人员收卷后方可离开；交卷后不得在考场内逗留或在附近高声交谈。未交卷擅自离开考场，不得重新进入考场答卷。考试结束时间到，考生立即停止答卷，在座位上等待监考人员收卷清点后，方可离场。

4、考生要严格遵守考场规则，在规定时间内独立完成答卷。不准交头接耳，不准偷看、夹带、抄袭或者有意让他人抄袭答题内容，不准接传答案或者试卷等。凡有违纪作弊者，一经发现，当场取消其考试资格，并根据《北京大学本科考试工作与学术规范条例》及相关规定严肃处理。

5、考生须确认自己填写的个人信息真实、准确，并承担信息填写错误带来的一切责任与后果。

学校倡议所有考生以北京大学学生的荣誉与诚信答卷，共同维护北京大学的学术声誉。

以下为试题和答题纸，共 14 页。

得分

第一题 单项选择题（每小题 2 分，共 32 分）

1、假设下列 unsigned 和 int 数均为 32 位，

`unsigned x = 0x00000001;`

`int y = 0x80000000;`

`int z = 0x80000001;`

以下表达式正确的是

A. `(-1) < x`

B. `(- y) > -1`

C. `~ y + y == -1`

D. `(z<<4) > (z*16)`

答：（ ）

答案：C

A 错误 signed (-1) 和 unsigned x 比较，都按照 unsigned，所以强制类型转换后 (-1) 很大

B 错误 int 中 0x80000000 的相反数还是自己，是最小的负数

C 正确 相等关系；`~y + y == 0xFFFFFFFF` 是 -1 的补码表示

D 错误 应该是相等关系 unsigned, signed 左移 4 位 相当于 *16

2、下面说法正确的是：

A. 数 0 的反码表示是唯一的

B. 数 0 的补码表示不是唯一的

C. 1000, 1111, 1110, 1111, 1100, 0000, 0000, 0000 表示唯一的整数是 0x8FEFC000

D. 1000, 1111, 1110, 1111, 1100, 0000, 0000, 0000 如果是单精度浮点表示，则表示的是 $-(1.110111111)_2 \times 2^{31-127}$

答：（ ）

答案：D

考察反码、补码

A. 错误。若用反码表示，则可表示为 00000000 或 11111111。

B. 错误。数 0 的补码表示是唯一的：+0 的补码=+0 的反码=+0 的原码=00000000；0 的原码和反码有两种，补码只有一种。

- C. 错误。如果是无符号数表示，则表示的是 **0x8FEFC000**；如果是补码表示，则表示的是 **-0x70104000**
- D. 正确。

3、下面表达式中为“真”的是：

- A. (unsigned) -1 < -2
- B. 2147483647 > (int) 2147483648U
- C. (0x80005942 >> 4) == 0x09005942
- D. 2147483647 + 1 != 2147483648

答：()

答案：B

4、下列的指令组中，那一组指令只改变条件码，而不改变寄存器的值？

- A. CMP, SUB
- B. TEST, AND
- C. CMP, TEST
- D. LEAL, CMP

答：()

答案：C

SUB 和 AND 都同时会改变条件码和寄存器的值，LEAL 不改变改变条码。

5、下列指令中，寻址方式不正确的是

- A. MOVB %ah, 0x20(, %ecx, 8)
- B. LEAL (0xA, %eax), %ebx
- C. SUBB 0x1B, %bl
- D. INCL (%ebx, %eax)

答：()

答案：B

存储器数的基址应该存放在一个基址寄存器中。

6、有如下定义的结构，在 x86-64 下，下述结论中错误的是？

```
struct {  
    char c;  
    union {  
        char vc;
```

- ```

 double value;
 int vi;
 } u;
 int i;
} sa;

```
- A. `sizeof(sa) == 24`
  - B. `(&sa.i - &sa.u.vi) == 8`
  - C. `(&sa.u.vc - &sa.c) == 8`
  - D. 优化成员变量的顺序，可以做到“`sizeof(sa) == 16`”

答: (        )

**答案: B**

由于对齐的需求，在 **x86-64** 下，要保证 **sa.u.value** (**double** 类型变量) 的地址必须 8 字节对齐，而在 **ia32** 的 **Linux** 系统中，可 4 字节对齐。故 **sizeof(sa)** 在 **x86-64** 下要占用三个 8 字节的空间共 24 字节。**sa.u.vc** 与 **sa.c** 之间和 **sa.i** 与 **sa.u.vi** 之间的地址值之差均为 8，但即使不知道 **(&sa.i - &sa.u.vi)** 表示之间可以放置多少个整数 (值为 2)，也可通过优化成员变量顺序后有 **sizeof(sa) == 16**，用排除法得出正确答案。

7、关于如何避免缓冲区溢出带来的程序风险，下述错误的做法为？

- A. 编程时定义大的缓冲区数组
- B. 编程时避免使用 `gets`，而采用 `fgets`
- C. 程序运行时随机化栈的偏移地址
- D. 在硬件级别引入不可执行代码段的机制

答: (        )

**答案: A**

**B、C、D** 均为讲义中所提及的解决缓冲区溢出风险的方法。**A** 策略则无法从根本上解决缓冲区溢出问题，只要输入足够长数据就仍然可以实现缓冲区溢出攻击。

8、对简单的 `switch` 语句常采用跳转表的方式实现，在 **x86-64** 系统中，下述最有可能正确的 `switch` 分支跳转汇编指令为哪个？

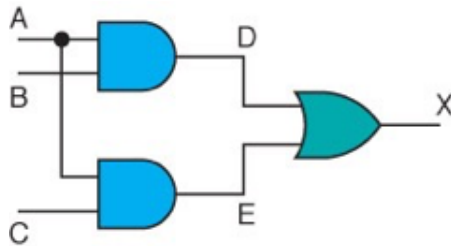
- A. `jmp .L3(, %eax, 4)`
- B. `jmp .L3(, %eax, 8)`
- C. `jmp *.L3(, %eax, 4)`
- D. `jmp *.L3(, %eax, 8)`

答：( )

答案：D

A 与 B 都是错误的基于跳转表跳转的指令正确格式。C、D 的指令格式正确，但在 x86-64 系统中，每个地址占 8 个字节，因此更有可能的答案是 D。

9、对应下述组合电路的正确 HCL 表达式为



A.  $\text{Bool } X = (A \mid\mid B) \ \&\& \ (A \mid\mid C)$

B.  $\text{Bool } X = A \mid\mid (B \ \&\& \ C)$

C.  $\text{Bool } X = A \ \&\& \ (B \mid\mid C)$

D.  $\text{Bool } X = A \mid\mid B \mid\mid C$

答：( )

答案：C

10、若处理器实现了三级流水线，每一级流水线实际需要的运行时间分别为 2ns、2ns 和 1ns，则此处理器不停顿地执行完毕 10 条指令需要的时间为：

A. 21ns

B. 22ns

C. 23ns

D. 24ns

答：( )

答案：D

$2+2+2*10 = 24$

11、关于 RISC 和 CISC 的描述，正确的是：

A. CISC 指令系统的指令编码可以很短，例如最短的指令可能只有一个字节，因此 CISC 的取指部件设计会比 RISC 更为简单。

- B. CISC 指令系统中的指令数目较多, 因此程序代码通常会比较长; 而 RISC 指令系统中通常指令数目较少, 因此程序代码通常会比较短。
- C. CISC 指令系统支持的寻址方式较多, RISC 指令系统支持的寻址方式较少, 因此用 CISC 在程序中实现访存的功能更容易。
- D. CISC 机器中的寄存器数目较少, 函数参数必须通过栈来进行传递; RISC 机器中的寄存器数目较多, 只需要通过寄存器来传递参数。

答: ( )

**答案: C**

**考查对 CISC 和 RISC 基本特点的描述, A 和 B 都是描述反了, D 则是太绝对, RISC 也有可能用栈来传递参数。**

12、关于流水线技术的描述, 正确的是:

- A. 指令间数据相关引发的数据冒险, 一定可以通过暂停流水线来解决。
- B. 流水线技术不仅能够提高执行指令的吞吐率, 还能减少单条指令的执行时间。
- C. 增加流水线的级数, 一定能获得性能上的提升。
- D. 流水级划分应尽量均衡, 不均衡的流水线会增加控制冒险。

答: ( )

**答案: A**

**说明: B 会增加单条指令的执行时间, C 可能会降低性能, D 和控制冒险没有关系**

13、下面关于程序性能的说法中, 哪种是正确的?

- A. 处理器内部只要有多个功能部件空闲, 就能实现指令并行, 从而提高程序性能。
- B. 同一个任务采用时间复杂度为  $O(\log N)$  算法一定比采用复杂度为  $O(N)$  算法的执行时间短
- C. 转移预测总是能带来好处, 不会产生额外代价, 对提高程序性能有帮助。
- D. 增大循环展开 (loop unrolling) 的级数, 有可能降低程序的性能 (即增加执行时间)

答: ( )

**答案: D**

14、仅考虑以下代码, 哪些程序优化总是被编译器自动进行? (假设 `int i, int j, int A[N], int B[N], int m, int *p` 都是局部变量, `N` 是一个整数型

常量, int foo(int) 是一个函数)

| 优化前                                                        | 优化后                                                                            |
|------------------------------------------------------------|--------------------------------------------------------------------------------|
| A.   for (j = 0 ; j < N ; j<br>++)<br>B[i] *= A[j];        | int temp = B[i];<br>for (j= 0 ; j < N ; j ++)<br>temp *= A[j];<br>B[i] = temp; |
| B.   for (j = 0 ; j < N ; j<br>++)<br>m += i*N*j;          | int temp = i*N;<br>for (j= 0 ; j < N ; j ++)<br>m += temp * j;                 |
| C.   i = foo(N);<br>j = foo(N);<br>if (*p != 0)<br>m = j ; | j = foo(N);<br>if (*p != 0)<br>m = j ;                                         |
| D.   for (j = 0 ; j < foo(N) ;<br>j ++)<br>m ++;           | int temp = foo(N);<br>for (j= 0 ; j < temp ;<br>j ++)<br>m ++;                 |

答: (        )

答案: B

15、以下关于存储结构的讨论, 那个是正确的

- A. 增加额外一级存储, 数据存取的延时一定不会下降
- B. 增加存储的容量, 数据存取的延时一定不会下降
- C. 增加额外一级存储, 数据存取的延时一定不会增加
- D. 以上选项都不正确

答: (        )

答案: D

16、关于 cache 的 miss rate, 下面那种说法是错误的。

- A. 保持 E 和 B 不变, 增大 S, miss rate 一定不会增加
- B. 保持总容量和 B 不变, 提高 E, miss rate 一定不会增加
- C. 保持总容量和 E 不变, 提高 B, miss rate 一定不会增加
- D. 如果不采用“LRU”, 使用“随机替换策略”, miss rate 可能会降低

答: (        )

答案: C

注：实际答案 B 的说法也是有错误的。例如：保持总容量和 B 不变，提高 E，就意味着以前的两组可能并成一组。假设 A 和 B 组并成了新的一组。现在假设有一个访问序列  $b_1, b_2, a_1, a_2, a_3, \dots, a_n, b_1, b_2$ 。其中  $b_i$  是会被放到 B 组的块， $a_i$  是会被放到 A 组的块。n 足够大使得可以把合并后的组中的所有块都换出去。那么按合并后的情况全部都不命中，而按合并前的情况最后两次访问  $b_1, b_2$  仍然能命中。

因此，选 C 或 B 都算正确。

|    |
|----|
| 得分 |
|    |

## 第二题 (10 分)

1) 假设下列 unsigned 和 int 数均为 5 位 (有符号整型用补码运算表示), 在下表中填入正确答案 (每空 1 分, 共 6 分)

```
int y = - 7;
unsigned z = y;
```

|       | Decimal Representation | Binary Representation |
|-------|------------------------|-----------------------|
| z     |                        |                       |
| y - z |                        |                       |
| TMin  |                        |                       |

答案

|       | Decimal Representation | Binary Representation |
|-------|------------------------|-----------------------|
| z     | 25                     | 1 1001                |
| y - z | 0                      | 0 0000                |
| TMin  | -16                    | 1 0000                |

2) 请按 IEEE 浮点标准的单精度浮点数表示下表中的数值, 首先写出形如  $(-1)^s \times M \times 2^E$  的表达式, 然后给出十六进制的表示。(每格 1 分, 共 4 分)

注: 单精度浮点数的字段划分如下:

符号位 (s): 1-bit; 阶码字段 (exp): 8-bit; 小数字段 (frac): 23-bit; 偏置值 (bias): 127。

| Value | $(-1)^s \times M \times 2^E$ , $1 \leq M < 2$ | Hex representation |
|-------|-----------------------------------------------|--------------------|
| 0.375 |                                               |                    |
| -12.5 |                                               |                    |

答案

| Value | $(-1)^s \times M \times 2^E$<br>$1 \leq M < 2$ | Hex representation |
|-------|------------------------------------------------|--------------------|
| 0.375 | $1.1 \times 2^{-2}$                            | 0x3EC00000         |
| -12.5 | $(-1) * 1.1001 * 2^3$                          | 0xC1480000         |

|    |
|----|
| 得分 |
|    |

### 第三题（10 分）

阅读以下代码。假设代码运行在 IA32 的计算机上，字长为 4，请给出各个变量在内存中的十六进制字节表示（地址从小到大）。

注意 tiny\_float 是一种 8 位的浮点数，1 个符号位，4 个指数位，3 个尾数位。

```
int main()
{
 int a = 0x15213;
 unsigned char b = ((char)-5);
 tiny_float c = 19;
 float d;
 if (b < 4)
 d = -0.96875;
 else
 d = 769;
```

| a  |    |    |    |
|----|----|----|----|
| 13 | 52 | 01 | 00 |

| b  |
|----|
| FB |

| c  |
|----|
| 5A |

| d  |    |    |    |
|----|----|----|----|
| 00 | 40 | 40 | 44 |

每空 1 分



|    |
|----|
| 得分 |
|    |

#### 第四题（10 分）

一个函数如下，其中部分代码被隐去，请通过gdb调试信息补全代码（4分）。

```
int f(int n, int m) {
 if (m > 0) {
 if (n > 1) {
 int r = f(n - 1, m);
 return (r - 1 + m) % n + 1;
 }
 else if (n == 1) {
 return 1;
 }
 }
 return 0;
}
```

{考点：x86-64函数调用、参数传递及栈的使用。函数调用通过rdi和rsi传递第一和第二个参数，栈中只记录函数的返回地址。由于是递归调用，需要用栈保存递归过程中参数变量的值。同时考察xor、test、lea、idiv、sete等指令的使用。难点1：两个判断“n > 1”和“n == 1”在汇编代码中只有一次比较，第二次判断相等是通过sete使得n==1时返回值为1，否则返回值为0实现的。难点2：表达式“(r - 1 + m) % n + 1”比较复杂，需要综合多条语句的信息才能分析出来，并且变量r对应于f(n-1,m)的返回值即寄存器%eax，因而不存在相应的赋值指令。}

如下是通过“gcc -g -O2”命令编译后，在gdb中通过“disas f”命令得到的反汇编代码，其中有两个汇编指令不全，请补全这两条汇编指令（2分）。

```
0x00000000004004e0 <f+0>: mov %rbx,-0x10(%rsp)
0x00000000004004e5 <f+5>: mov %rbp,-0x8(%rsp)
0x00000000004004ea <f+10>: xor %eax,%eax
0x00000000004004ec <f+12>: sub $0x10,%rsp
0x00000000004004f0 <f+16>: test %esi,%esi
0x00000000004004f2 <f+18>: mov %edi,%ebp
0x00000000004004f4 <f+20>: mov %esi,%ebx
0x00000000004004f6 <f+22>: jle 0x400513 <f+51>
0x00000000004004f8 <f+24>: cmp $0x1,%edi
```

```

0x000000000004004fb <f+27>: jle 0x400521 <f+65>
0x000000000004004fd <f+29>: lea -0x1(%rbp),%edi
0x00000000000400500 <f+32>: callq 0x4004e0 <f>
0x00000000000400505 <f+37>: lea -0x1(%rax,%rbx,1),%edx
0x00000000000400509 <f+41>: mov %edx,%eax
0x0000000000040050b <f+43>: sar $0x1f,%edx
0x0000000000040050e <f+46>: idiv %ebp
0x00000000000400510 <f+48>: lea 0x1(%rdx),%eax
0x00000000000400513 <f+51>: mov (%rsp),%rbx
0x00000000000400517 <f+55>: mov 0x8(%rsp),%rbp
0x0000000000040051c <f+60>: add $0x10,%rsp
0x00000000000400520 <f+64>: retq
0x00000000000400521 <f+65>: sete %al
0x00000000000400524 <f+68>: movzbl %al,%eax
0x00000000000400527 <f+71>: jmp 0x400513 <f+51>

```

{考察点：函数中使用到了%rbp和%rbx寄存器，两者都是callee保存的寄存器，使用前需要压栈，函数返回时需要弹栈恢复寄存器的值。通过前后汇编代码的对比，应该可以猜出两个空分别填写什么；但要注意，压栈和弹栈时，%rsp寄存器的值不同，因而对应的地址表示也不同。}

已知在调用函数  $f(4, 3)$  时，我们在函数  $f$  中指令 `retq` 处设置了断点，下面列出的是程序在第一次运行到断点处暂停时时，相关通用寄存器的值。请根据你对函数及其汇编代码的理解，填写当前栈中的内容。如果某些内存位置处内容不确定，请填写  $x$ 。（4分）

|                  |                            |
|------------------|----------------------------|
| <code>rax</code> | <code>0x1</code>           |
| <code>rbx</code> | <code>0x3</code>           |
| <code>rcx</code> | <code>0x3</code>           |
| <code>rdx</code> | <code>0x309c552970</code>  |
| <code>rsi</code> | <code>0x3</code>           |
| <code>rdi</code> | <code>0x1</code>           |
| <code>rbp</code> | <code>0x2</code>           |
| <code>rsp</code> | <code>0x7fffffff340</code> |
| <code>rip</code> | <code>0x400520</code>      |

{ 考察点：递归调用的返回地址共三处是明确的，并且相同，值可以从反汇编代码中确定（1分）；三次递归调用程序栈中，压入的 `%rbx(m)` 的值不变，压入的 `%rbp(n)` 的值为每次减小 1（1分）；注意  $x86-64$ ，栈中的数据都是 64 位的，但因为数值均比较小，所以这 9 个位置处的高 4 字节均为 0（1分）；其余位置的内容均是不确定的（1分）。 }

|                            |                         |
|----------------------------|-------------------------|
| <code>0x7fffffff38c</code> | $x$                     |
| <code>0x7fffffff388</code> | $x$                     |
| <code>0x7fffffff384</code> | $x$                     |
| <code>0x7fffffff380</code> | $x$                     |
| <code>0x7fffffff37c</code> | $x$                     |
| <code>0x7fffffff378</code> | $x$                     |
| <code>0x7fffffff374</code> | <code>0x0</code>        |
| <code>0x7fffffff370</code> | <code>0x00400505</code> |
| <code>0x7fffffff36c</code> | <code>0x0</code>        |
| <code>0x7fffffff368</code> | <code>0x4</code>        |
| <code>0x7fffffff364</code> | <code>0x0</code>        |
| <code>0x7fffffff360</code> | <code>0x3</code>        |
| <code>0x7fffffff35c</code> | <code>0x0</code>        |
| <code>0x7fffffff358</code> | <code>0x00400505</code> |
| <code>0x7fffffff354</code> | <code>0x0</code>        |
| <code>0x7fffffff350</code> | <code>0x3</code>        |
| <code>0x7fffffff34c</code> | <code>0x0</code>        |
| <code>0x7fffffff348</code> | <code>0x3</code>        |
| <code>0x7fffffff344</code> | <code>0x0</code>        |
| <code>0x7fffffff340</code> | <code>0x00400505</code> |
| <code>0x7fffffff33c</code> | <code>0x0</code>        |
| <code>0x7fffffff338</code> | <code>0x2</code>        |
| <code>0x7fffffff334</code> | <code>0x0</code>        |
| <code>0x7fffffff330</code> | <code>0x3</code>        |
| <code>0x7fffffff32c</code> | $x$                     |
| <code>0x7fffffff328</code> | $x$                     |
| <code>0x7fffffff324</code> | $x$                     |
| <code>0x7fffffff320</code> | $x$                     |

|    |
|----|
| 得分 |
|    |

### 第五题（8 分）

阅读下面的汇编代码，根据汇编代码填写 C 代码中缺失的部分，然后描述该程序的功能。

```

pushl %ebp
movl %esp,%ebp
movl $0x0, %ecx
cmpl $0x0, 8(%ebp)
jle .L1
.L2
movl $0x0, %edx
movl 8(%ebp), %eax
divl $0x0a
addl %edx, %ecx
movl %eax, 8(%ebp)
cmpl $0x0, 8(%ebp)
jg .L2
.L1
movl 0x0, %edx
movl %ecx, %eax
divl 0x3
cmpl 0x0, %edx
jne .L3
movl 0x1, %eax
jmp .L4
.L3
movl 0x0, %eax
.L4

```

```

int fun(unsigned x) {
 int bit_sum = 0;
 while ((int) x > 0) {
 bit_sum += x % 10;
 x = x / 10;
 }
 if (bit_sum % 3 == 0)
 return 1;
 else
 return 0;
}

```

红色下划线部分为答案，每空 1 分。

该程序用来判断一个不大于  $2^n-1$  的非负整数是否为 3 的倍数，如果大于  $2^n-1$ ，则直接返回 1（3 分）。

|    |
|----|
| 得分 |
|    |

第六题（10 分）

请分析Y86 ISA中新加入的一条指令：**caddXX**，条件加法。其功能可以参考**add**和**cmovXX**两条指令。

|               |          |           |           |           |
|---------------|----------|-----------|-----------|-----------|
| <b>caddXX</b> | <b>C</b> | <b>fn</b> | <b>rA</b> | <b>rB</b> |
|---------------|----------|-----------|-----------|-----------|

若在教材所描述的SEQ处理器上执行这条指令，请按下表填写每个阶段进行的操作。需说明的信号包括：**icode**, **ifun**, **rA**, **rB**, **valA**, **valB**, **valC**, **valE**, **valP**, **Cnd**; the register file **R[]**, data memory **M[]**, Program counter **PC**, condition codes **CC**。其中对存储器的引用必须标明字节数。如果在某一阶段没有任何操作，请填写**none**指明。

| Stage      | <b>caddXX rA, rB</b>                                                                                                                            |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Fetch      | <b>icode:ifun <math>\leftarrow M_1[PC]</math></b><br><b>rA:rB <math>\leftarrow M_1[PC+1]</math></b><br><b>valP <math>\leftarrow PC+2</math></b> |
| Decode     | <b>valA <math>\leftarrow R[rA]</math></b><br><b>valB <math>\leftarrow R[rB]</math></b>                                                          |
| Execute    | <b>valE <math>\leftarrow valA+valB</math></b><br><b>Cnd <math>\leftarrow Cond(CC,ifun)</math></b>                                               |
| Memory     | <b>none</b>                                                                                                                                     |
| Write back | <b>if(Cnd) <math>R[rB] \leftarrow valE</math></b>                                                                                               |
| PC update  | <b>PC <math>\leftarrow valP</math></b>                                                                                                          |

（每个操作 1 分）

注：**Execute** 阶段可以写上 **if(Cnd) Set CC**，不计分，但如果只写了 **Set CC** 是要扣分的。**Write back** 阶段的操作不可以写成  **$R[rB] \leftarrow Cnd? valE: valB$**

|    |
|----|
| 得分 |
|    |

### 第七题 (10 分)

如下是使用 C 语言描述的链表结构的声明，链表的结尾使用空指针来表示。同时使用函数 `int length (List *p)` 来计算链表的长度。为简化起见，假设该链表是非循环的。

```
typedef struct LIST {
 struct LIST *next;
 int data;
} List;
```

- 1) 函数 `count_pos1` 用来计算链表中 `data` 为正数的元素个数，并将结果存放在地址 `k`。以下的程序可能存在问题导致效率很低或程序出错，请指出并修改。

(4 分)

```
void count_pos1 (List *p, int *k) {
 int i;
 for (i = 0; i < length(p); i++) {
 if (p->data > 0)
 *k++;
 p = p->next;
 }
}
```

- 2) 为提高程序性能，可以考虑删除变量 `i` 以消除函数调用。请修改上述程序达到该目的。(2 分)

- 3) 上述程序内层循环的汇编片段如下所示。假设该链表不为空且大部分数据都为正数，转移预测全部正确，设计中有足够多的部件来实现指令并行。其中访存操作全部 `cache` 命中，时延为 3 `cycle`，其他指令时延为 1`cycle`。请计算以下程序的 `CPE` 下限，并给出文字说明。(4 分)

```
.L1:
 movl 4(%eax), %ecx
 testl %ecx, %ecx
 jle .L2
 incl %edx
.L2:
```

```

movl (%eax), %eax
testl %eax, %eax
jne .L1

```

答案：

1)

```

void count_pos1 (List *p, int *k) {
 int l, num=0, len;
 len = length(p)
 for (i = 0; i < len; i++) {
 if (p->data > 0)
 num++;
 p = p->next;
 }
 *k = num
}

```

( 4 分)

2) while(p) 或其他相同功能的语句 (2 分)

3) L1 和 L2 的代码没有数据依赖，完全可以并行。(2 分)

CPE 的下限为 3+1=4。(2 分)

|    |
|----|
| 得分 |
|    |

### 第八题（10 分）

假设存在一个能够存储四个 Block 的 Cache，每一个 Block 的长度为 2Byte。假设内存空间大小共是 16Byte，即内存空间地址长度一共是 4bit，可访问地址为(0~15)，数据访问地址序列如下所示，访问数据单位是 Byte，默认替换策略是 LRU。

2 3 10 9 6 8

1) 如果 Cache 的结构是下图所示（S=2， E=2），请在下图空白处填入访问上述六次数据访问后 Cache 的状态。注：用[0-1]表示地址 0 至 1 上对应的数据（4 分）

|      | V | TAG | Block | V | TAG | Block   |
|------|---|-----|-------|---|-----|---------|
| set0 | 1 | 10  | [8-9] | 0 |     |         |
| set1 | 1 | 01  | [6-7] | 1 | 10  | [10-11] |

2) 这六次数据访问一共产生了多少次 Miss \_\_\_\_\_（2 分）

答案：4

3) 如果 Cache 的替换策略改成 MRU（即，最近使用的数据被替换出去），请在下图空白处填入访问上述六次数据访问后 Cache 的状态（2 分）。

|      | V | TAG | Block | V | TAG | Block |
|------|---|-----|-------|---|-----|-------|
| set0 | 1 | 10  | [8-9] | 0 |     |       |
| set1 | 1 | 00  | [2-3] | 1 | 01  | [6-7] |

4) 这六次数据访问一共产生了多少次 Miss \_\_\_\_\_（2 分）

答案：4



# 北京大学信息科学技术学院考试试卷

考试科目：计算机系统导论 姓名：\_\_\_\_\_ 学号：\_\_\_\_\_

考试时间：2015 年 11 月 09 日 小班教师：\_\_\_\_\_

| 题号  | 一 | 二 | 三 | 四 | 五 | 总分 |
|-----|---|---|---|---|---|----|
| 分数  |   |   |   |   |   |    |
| 阅卷人 |   |   |   |   |   |    |

## 北京大学考场纪律

1、考生进入考场后，按照监考老师安排隔位就座，将学生证放在桌面上。无学生证者不能参加考试；迟到超过 15 分钟不得入场。在考试开始 30 分钟后方可交卷出场。

2、除必要的文具和主考教师允许的工具书、参考书、计算器以外，其它所有物品（包括空白纸张、手机、或有存储、编程、查询功能的电子用品等）不得带入座位，已经带入考场的必须放在监考人员指定的位置。

3、考试使用的试题、答卷、草稿纸由监考人员统一发放，考试结束时收回，一律不准带出考场。若有试题印制问题请向监考教师提出，不得向其他考生询问。提前答完试卷，应举手示意请监考人员收卷后方可离开；交卷后不得在考场内逗留或在附近高声交谈。未交卷擅自离开考场，不得重新进入考场答卷。考试结束时间到，考生立即停止答卷，在座位上等待监考人员收卷清点后，方可离场。

4、考生要严格遵守考场规则，在规定时间内独立完成答卷。不准交头接耳，不准偷看、夹带、抄袭或者有意让他人抄袭答题内容，不准接传答案或者试卷等。凡有违纪作弊者，一经发现，当场取消其考试资格，并根据《北京大学本科考试工作与学术规范条例》及相关规定严肃处理。

5、考生须确认自己填写的个人信息真实、准确，并承担信息填写错误带来的一切责任与后果。

学校倡议所有考生以北京大学学生的荣誉与诚信答卷，共同维护北京大学的学术声誉。

以下为试题和答题纸，共 页。

|    |
|----|
| 得分 |
|    |

第一题 单项选择题（每小题 1 分，共 20 分）

注：请将选择题答案填写在下表中

|    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|
| 题号 | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
| 答案 |    |    |    |    |    |    |    |    |    |    |
| 题号 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| 答案 |    |    |    |    |    |    |    |    |    |    |

1. 给定一个实数，会因为该实数表示成单精度浮点数而发生误差。不考虑 NaN 和 Inf 的情况，该绝对误差的最大值为

- A.  $2^{103}$
- B.  $2^{104}$
- C.  $2^{230}$
- D.  $2^{231}$

参考信息：单精度浮点数阶码 8 位，尾数 23 位

答案：a。计算过程：浮点数阶码 8 位，即最大指数为 127，尾数为 23 位，即最低位表示的数为  $2^{-23} * 2^{127} = 2^{104}$ 。那么误差在刚好落在最低位的一半的时候最大，即  $2^{103}$ 。其他选项为干扰项，如果求最大指数的时候忘记非规格化数或者对偶数舍入理解不深会算出 b，如果以为指数为非规格化数会算出 c 或 d。

2. 以下说法错误的是

- A. 负数加上负数结果可能为正数
- B. 正数加上正数结果可能为负数
- C. 用 & 和 ~ 可以表示所有的逻辑与或非操作
- D. 用 & 和 | 可以表示所有的逻辑与或非操作

答案：d。无法用 & 和 | 表示逻辑非

3. 在 32 位平台上，按 C90 标准以下语句中，结果为假的是

- A. `return INT_MIN < INT_MAX;`
- B. `return -2147483648 < 2147483647;`
- C. `int a = -2147483648; return a < 2147483647;`
- D. `return -2147483647-1 < 2147483647;`

参考信息：

C90 的转换顺序: int -> long -> unsigned -> unsigned long  
 $2^{31} = 2147483648$

答案: b。因为 2147483648 超出了有符号数的表示范围, C90 的 C 语言会将其识别为 unsigned。注意在 32 位机器上 long 和 int 都是 32 位。

4. 关于浮点数, 以下说法正确的是

- A. 给定任意浮点数 a, b 和 x, 如果  $a > b$  成立 (求值为 1), 则一定  $a+x > b+x$  成立
- B. 不考虑结果为 NaN、Inf 或运算过程发生溢出的情况, 高精度浮点数一定得到比低精度浮点数更精确或相同的结果
- C. 不考虑输入为 NaN、Inf 的情况, 高精度浮点数一定得到比低精度浮点数更精确或相同的结果
- D. 给定任意浮点数 a, b 和 x, 如果  $a > b$  不成立 (求值为 0), 则一定  $a+x > b+x$  不成立。

答案: d。如果不出现 NaN 和 Inf, 浮点数是满足单调性的, 这时 a 和 d 都成立。如果出现 NaN 和 Inf, 那么任何时候比较运算的操作数有 NaN 和 Inf 就为 0, 就只有 d 成立。关于 b 和 c, 由于运算的偶然性, 高精度不一定比低精度精确。

5. 已知下面的数据结构, 假设在 Linux/IA32 下要求对齐, 这个结构的总的大小是多少个字节? 如果重新排列其中的字段, 最少可以达到多少个字节?

```
struct {
 char a;
 double *b;
 double c;
 short d;
 long long e;
 short f;
};
```

- A. 32, 28
- B. 36, 32
- C. 28, 26
- D. 26, 26

选择 A, Linux/IA32 要求 double 和 long long 在 4 字节边界上对齐, 其中 a 占 1 字节, 后面填充 1 字节, b 占 4 字节, 填充 2 个字节, c 占 8 字节, d 占 2 字节, 填充 2 字节, e 占 8 字节, f 占 2 字节, 再填充 2 字节, 共 32 字节; 重新排列时, 可以按字节数从大到小排列, 最后, 最少需要  $8+8+4+2+2+1$  再加 3 字节满足 4 字节对齐要求, 共 28 个字节。

6. 下列寻址模式中, 正确的是:

- A. (%eax, , 4)
- B. (%eax, %esp, 3)
- C. 123
- D. \$1(%ebx, %ebp, 1)

选择 C。A 中不能省略 Ei, B 中比例因子错误, D 中偏移地址表示错误。

7. 假设存储器按“大端法”存储数据对象, 已知如下的 C 语言数据结构: union { char c[2]; int i; }; 当 c 的值为 0x01, 0x23 时, i 的值为:

- A. 0x0123
- B. 0x2301
- C. 0x01230000
- D. 不确定

选择 D。因为 i 的两个高地址字节的值未知。

8. 假设某条 C 语言 switch 语句编译后产生了如下的汇编代码及跳转表:

|                     |           |
|---------------------|-----------|
| movl 8(%ebp), %eax  | .L7:      |
| subl \$48, %eax     | .long .L3 |
| cmpl \$8, %eax      | .long .L2 |
| ja .L2              | .long .L2 |
| jmp *.L7(, %eax, 4) | .long .L5 |
|                     | .long .L4 |
|                     | .long .L5 |
|                     | .long .L6 |
|                     | .long .L2 |
|                     | .long .L3 |

在源程序中, 下面的哪些(个)标号出现过:

- A. '2', '7'
- B. 1
- C. '3'
- D. 5

选择 C。标号的取值范围为'0' - , '1'、'2'、'7'、'9'、...等没有出现。

9. x86 体系结构中, 下面哪个说法是正确的?

- A. leal 指令只能用来计算内存地址
- B. x86\_64 机器可以使用栈来给函数传递参数

- C. 在一个函数内, 改变任一寄存器的值之前必须先将其原始数据保存在栈内
- D. 判断两个寄存器中值大小关系, 只需要 SF 和 ZF 两个条件码

答案: B

A.leal 指令做普通算术运算; C.caller saved 寄存器才需要; D.还需要 OF

10. 下列的指令组中, 哪一组指令只改变条件码, 而不改变寄存器的值?

- A. CMP, SUB
- B. TEST, AND
- C. CMP, TEST
- D. LEAL, CMP

答案: C

SUB 和 AND 都同时会改变条件码和寄存器的值, LEAL 不改变改变条码。

11. 下面有关指令系统设计的描述正确的是:

- A. 采用 CISC 指令比 RISC 指令代码更长。
- B. 采用 CISC 指令比 RISC 指令运行时间更短
- C. 采用 CISC 指令比 RISC 指令译码电路更加复杂
- D. 采用 CISC 指令比 RISC 指令的流水线吞吐更高

答案: C, CISC 的比 RISC 代码复杂 (如不定长), 因此译码也更复杂, 优点是代码更短。运行时间和吞吐都谁更高要依据实际应用。

12. 一个功能模块包含组合逻辑和寄存器, 组合逻辑单元的总延迟是 100ps, 单个寄存器的延时是 20ps, 该功能模块执行一次并保存执行结果, 理论上能达到的最短延时和最大吞吐分别是多少?

- A. 20ns, 50GIPS
- B. 120ns, 50GIPS
- C. 120ns, 10GIPS
- D. 20ps, 10GIPS

答案: B, 主要考察延时和吞吐的区别, 延时最小是 logic + 1 个 register, 吞吐最高时是把 logic 划分成无穷多份, 每一级需要 20ps,  $1000/20ps = 50GIPS$

13. 关于流水线技术的描述, 错误的是:

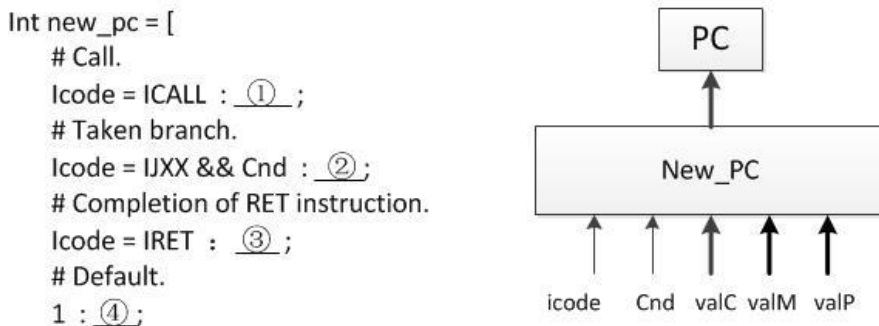
- A. 流水线技术能够提高执行指令的吞吐率, 但也同时增加单条指令的执行时间。
- B. 减少流水线的级数, 能够减少数据冒险发生的几率。
- C. 指令间数据相关引发的数据冒险, 都可以通过 data forwarding 来解决。

D. 现代处理器支持一个时钟内取指、执行多条指令，会增加控制冒险的开销。

答：C，load-use 冒险 不能通过 data forwarding 来解决

14. 在 Y86 的 SEQ 实现中，PC (Program Counter，程序计数器) 更新的逻辑

结构如下图所示，请根据 HCL 描述为①②③④选择正确的数据来源。



其中：Icode 为指令类型，Cnd 为条件是否成立，valC 表示指令中的常数值，valM 表示来自返回栈的数据，valP 表示 PC 自增。

- A. valC, valM, valP, valP
- B. valC, valC, valP, valP
- C. valC, valC, valM, valP
- D. valM, valC, valC, valP

答：C

15. 下面关于存储器的说法，错误的是\_\_\_\_\_

- A. SDRAM 的速度比 FPM DRAM 快
- B. SDRAM 的 RAS 和 CAS 请求共享相同的地址引脚
- C. 磁盘的寻道时间和旋转延迟大致在一个数量级
- D. 固态硬盘的随机读写性能基本相当

答案：D

说明：本题考查学生对内存、硬盘基本特性的掌握。

16. 某磁盘的旋转速率为 7200 RPM，每条磁道平均有 400 扇区，则一个扇区的平均传送时间为\_\_\_\_\_

- A. 0.02 ms
- B. 0.01 ms
- C. 0.03 ms
- D. 0.04 ms

答案：A

说明： $60 / 7200 \text{ RPM} \times 1 / 400 \text{ sectors/track} \times 1000 \text{ ms/sec}$   
 $\approx 0.02 \text{ ms}$

17. 某高速缓存 ( $E=2, B=4, S=16$ )，地址宽度 14，当引用地址  $0x9D28$  处的 1 个字节时，tag 位应为：

- A. 01110100
- B. 001110000
- C. 1110000
- D. 10011100

答案：A

说明： $0x9D28=10\underline{01110100}101000$

18. 关于高速缓存的说法正确的是 \_\_\_\_\_

- A. 直写 (write through) 比写回 (write back) 在电路实现上更复杂
- B. 固定的高速缓存大小，较大的块可提高时间局部性好的程序的命中率
- C. 随着高速缓存组相联度的不断增大，失效率不断下降
- D. 以上说法全不正确

答案：D

说明：本题考查学生对高速缓存基本性质和实现机制的理解。

19. 关于局部性 (locality) 的描述，不正确的是：

- A. 循环通常具有很好的时间局部性
- B. 循环通常具有很好的空间局部性
- C. 数组通常具有很好的时间局部性
- D. 数组通常具有很好的空间局部性

答案：C

数组本身只能体现出空间局部性

20. 以下哪些程序优化编译器总是可以自动进行？（假设  $\text{int } i, \text{int } j, \text{int } A[N], \text{int } B[N], \text{float } m$  都是局部变量， $N$  是一个整数型常量， $\text{int } \text{foo}(\text{int})$  是一个函数）

|    | 优化前                                                          | 优化后                                                                                |
|----|--------------------------------------------------------------|------------------------------------------------------------------------------------|
| A. | <pre>for (j = 0 ; j &lt; N ; j ++)<br/>    m += i*N*j;</pre> | <pre>int temp = i*N;<br/>for (j= 0 ; j &lt; N ; j ++)<br/>    m += temp * j;</pre> |

|    |                                                                     |                                                                                         |
|----|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| B. | <pre>for (j = 0 ; j &lt; N ; j ++)     B[i] *= A[j];</pre>          | <pre>int temp = B[i]; for (j= 0 ; j &lt; N ; j ++)     temp *= A[j]; B[i] = temp;</pre> |
| C. | <pre>for (j = 0 ; j &lt; N ; j ++)     m = (m + A[j]) + B[j];</pre> | <pre>for (j = 0 ; j &lt; N ; j ++)     m = m + (A[j] + B[j]);</pre>                     |
| D. | <pre>for (j = 0 ; j &lt; foo(N) ; j ++)     m++;</pre>              | <pre>int temp = foo(N); for (j= 0 ; j &lt; temp ; j ++)     m++;</pre>                  |

答案: A

说明: 考察 procedure, memory aliasing, 和 floating 的精度问题



|    |
|----|
| 得分 |
|    |

## 第二题 (20 分)

1. 对于下面的每一个表达式, 请选择以下选项中的一个或多个 (即“不定项”), 使得该表达式恒成立, 如果没有满足条件的选项则选 E。

A. <      B. >      C. ==      D. !=      E. none

题目中出现的变量定义如下 (浮点数保证不是 NaN 或者 Inf):

```
int x, y;
unsigned ux = x;
double d;
```

- 1) 如果  $x > 0$ , 则  $x + 1$  \_\_\_\_ 0
- 2) 如果  $x > y$ , 则  $ux$  \_\_\_\_  $y$
- 3) 如果  $((x \ll 31) \gg 31) < 0$ , 则  $x \& 1$  \_\_\_\_ 0
- 4) 如果  $((\text{unsigned char})x \gg 1) < 64$ , 则  $(\text{char})x$  \_\_\_\_ 0
- 5) 如果  $d < 0$ , 则  $d * 2$  \_\_\_\_ 0
- 6) 如果  $d < 0$ , 则  $d * d$  \_\_\_\_ 0
- 7)  $x \wedge y \wedge (\sim x) - y$  \_\_\_\_  $y \wedge x \wedge (\sim y) - x$
- 8)  $(( (!!ux)) \ll 31) \gg 31$  \_\_\_\_  $(( (!!x)) \ll 31) \gg 31$

(前 6 题每题一分, 漏选错选不得分; 7、8 两题每题 2 分, 漏选得 1 分; 共 10 分)

2. 考虑一种 12-bit 长的浮点数, 此浮点数遵循 IEEE 浮点数格式, 请回答下列问题。

注: 浮点数的字段划分如下:

符号位 (s): 1-bit; 阶码字段 (exp): 4-bit; 小数字段 (frac): 7-bit。

- 1) 请写出在下列区间中包含多少个用上面规则精确表示的浮点数(每空 2 分, 共 4 分)

A:  $[1, 2)$  \_\_\_\_\_

B:  $[2, 3)$  \_\_\_\_\_

- 2) 请写出下面浮点数的二进制表示 (每空 1 分, 共 6 分)

| 数字               | 二进制表示 |
|------------------|-------|
| 最小的正规格化数         |       |
| 最大的非规格化数         |       |
| $17\frac{1}{16}$ |       |

|                   |  |
|-------------------|--|
| $-\frac{1}{8192}$ |  |
| $20\frac{3}{8}$   |  |
| $-\infty$         |  |

答案:

1. 1) D      2) D      3) B, D      4) E      5) A, D

6) E    当两个数绝对值都很小时，相乘以后由于精度不够会变成 0

7) C      8) C

2. 1) A. 128      B. 64

2)

| 数字                | 二进制表示          |
|-------------------|----------------|
| 最小的正规格化数          | 0 0001 0000000 |
| 最大的非规格化数          | 0 0000 1111111 |
| $17\frac{1}{16}$  | 0 1011 0001000 |
| $-\frac{1}{8192}$ | 1 0000 0000001 |
| $10\frac{3}{8}$   | 0 1010 0100110 |
| $-\infty$         | 1 1111 0000000 |

|    |
|----|
| 得分 |
|    |

### 第三题（20 分）

1. 考虑下面的union的声明，回答后面的问题。

```
union ELE {
 struct {
 int x;
 int *p;
 }e1;
 struct {
 union ELE * next;
 int y;
 }e2;
};
```

(注：32位机器)

这个union总共大小为多少\_\_\_\_\_字节 (2分)

2. 假设编译器为process的主体产生了如下代码，请补充完整下面的过程：(6分)

（只有一个不需要任何强制类型转换且不违反任何类型限制的答案）

```
movl 8(%ebp),%eax
movl (%eax),%ecx
movl 4(%ecx),%edx
movl (%edx),%edx
subl 4(%eax),%edx
movl %edx, (%ecx)
```

```
void process(union ELE * up)
```

```
{
 up->_____ = _____ - _____;
}
```

3. 请查看下文完成如下功能的汇编代码，定位错误语句并进行更正： (6 分)

给出  $n$  (在 `%ebp+8` 位置,  $n \geq 1$ ),  $up$  (在 `%ebp+12` 位置, `ELE*` 类型), 假设以  $*up$  为头元素 (设  $*up$  为第 0 个), 由声明中的 `next` 连接形成了一个链表, 请将第  $n$  个元素 (假设链表足够长) 的  $x$  的值放入 `%eax` 中

X86 代码

```
xorl %ecx,%ecx
movl 8(%edx),%ebp
movl 12(%ebp),%eax
```

LOOP:

```
movl (%eax),%eax
add $1,%ecx
test %ecx,%edx
jne LOOP
```

```
movl (%eax),%eax
```

Y86 代码

```
mrmovl 8(%edx),%ebp
mrmovl 12(%ebp),%eax
irmovl $1,%ecx
```

LOOP:

```
mrmovl (%eax),%eax
test %ecx,%edx
jne LOOP
```

```
mrmovl (%eax),%eax
```

4. 阅读下列代码，回答后面的问题

```
typedef struct {
 short x[A][B];
 int y;
```

```

}str1;

typedef struct {
 char array[B];
 int t;
 short s[B];
 int u;
}str2;

void setVal(str1 *p,str2 *q) {
 int v1=q->t;
 int v2=q->u;
 p->y=v1+v2;
}

```

(short 以 2 字节计算)

GCC 为 setVal 的主体产生下面的代码：

```

movl 12(%ebp),%eax
movl 28(%eax),%edx
addl 8(%eax),%edx
movl 8(%ebp),%eax
movl %edx,44(%eax)

```

请直接写出A和B的值各是多少？(6分)

答案

1. 8
2.  $up \rightarrow next \rightarrow x = * (up \rightarrow next \rightarrow p) - (up \rightarrow y)$
3. (3个错，每个错2分)

x86代码

```

xorl %ecx,%ecx
movl 8(%edx),%ebp ||应为: movl 8(%ebp),%edx
movl 12(%ebp),%eax

```

```

LOOP:
movl (%eax),%eax
add $1,%ecx
test %ecx,%edx
jne LOOP

```

```

movl (%eax),%eax

```

Y86代码

```

mrmovl 8(%edx),%ebp ||应为: mrmovl 8(%ebp),%edx
mrmovl 12(%ebp),%eax
irmovl $1,%ecx

```

```

LOOP:
mrmovl (%eax),%eax
test %ecx,%edx ||应为: subl %ecx,%edx
jne LOOP

```

```

mrmovl (%eax),%eax

```

4.

A=3 B=7

|    |
|----|
| 得分 |
|    |

#### 第四题（20 分）

请分析Y86 ISA中新加入的一条指令：NewJE，其格式如下。

|       |   |   |    |    |      |
|-------|---|---|----|----|------|
| NewJE | C | 0 | rA | rB | Dest |
|-------|---|---|----|----|------|

其功能为：如果 $R[rA] = R[rB]$ ，则跳转到Dest继续执行，否则顺序执行。

1. 若在教材所描述的SEQ处理器上执行这条指令，请按下表补全每个阶段的操作。需说明的信号可能会包括：icode, ifun, rA, rB, valA, valB, valC, valE, valP, Cnd; the register file R[], data memory M[], Program counter PC, condition codes CC。其中对存储器的引用必须标明字节数。如果在某一阶段没有任何操作，请填写none指明。

| Stage      | NewJE rA, rB, Dest                                                                                                       |
|------------|--------------------------------------------------------------------------------------------------------------------------|
| Fetch      | $icode:ifun \leftarrow M_1[PC]$<br>$rA:rB \leftarrow M_1[PC+1]$<br>$valC \leftarrow M_4[PC+2]$<br>$valP \leftarrow PC+6$ |
| Decode     | $valA \leftarrow R[rA]$<br>$valB \leftarrow R[rB]$                                                                       |
| Execute    | $valE \leftarrow valA - valB$<br>（注：也可以是 $valB - valA$ ）                                                                 |
| Memory     | none                                                                                                                     |
| Write back | none                                                                                                                     |
| PC update  | $PC \leftarrow valE==0 ? valC : valP$                                                                                    |

（红字处每行 1 分，共 5 分）

2. 若在教材所描述的PIPE处理器上执行NewJE指令，如果跳转条件不满足，一共

会错误执行\_\_2\_\_条指令。

为了减小错误预测的代价，现将教材所描述的PIPE处理器做如下改进：在Decode阶段增加一个比较器，用于判断  $(R[rA] = R[rB])$  条件，比较器的输出信号为d\_equal。如果相等，则d\_equal = 1，反之 d\_equal = 0。

此时，如果执行NewJE指令时跳转条件不满足，一共会错误执行\_\_1\_\_条指令。  
(每空 1 分，共 2 分)

3. 在教材所描述的PIPE处理器上执行JXX指令时，发生转移预测错误的判断条件和各级流水线寄存器的控制信号如下所示：

|                     |                         |  |  |  |  |
|---------------------|-------------------------|--|--|--|--|
| Condition           | Trigger                 |  |  |  |  |
| Mispredicted Branch | E_icode = IJXX & !e_Cnd |  |  |  |  |

|                     |        |        |        |        |        |
|---------------------|--------|--------|--------|--------|--------|
| Condition           | F      | D      | E      | M      | W      |
| Mispredicted Branch | normal | bubble | bubble | normal | normal |

在第（2）小题所述的改进后的处理器上执行 NewJE 指令，发生转移预测错误的判断条件和各级流水线寄存器的控制信号应如何设置？

|                     |                             |  |  |  |  |
|---------------------|-----------------------------|--|--|--|--|
| Condition           | Trigger                     |  |  |  |  |
| Mispredicted Branch | D_icode = INewJE & !d_equal |  |  |  |  |

|                     |        |        |        |        |        |
|---------------------|--------|--------|--------|--------|--------|
| Condition           | F      | D      | E      | M      | W      |
| Mispredicted Branch | normal | bubble | normal | normal | normal |

(每空 1 分，共 5 分)

4. 在第（2）小题所述的改进后的处理器上执行如下代码，

```
0x000: mrmovl 0(%eax), %edx
```



```

0x006: NewJE %edx, %eax, t
0x00c: irmovl $1, %eax # Fall through
0x012: nop
0x013: nop
0x014: nop
0x015: halt
0x016: t:irmovl $3, %edx # Target (Should not execute)
0x01c: irmovl $4, %ecx # Should not execute
0x022: irmovl $5, %edx # Should not execute

```

会发生 load-use 和 misprediction 组合的 hazard 情况，如下图所示



请问此时，各级流水线寄存器的控制信号应如何设置？

| Condition     | F     | D     | E      | M      | W      |
|---------------|-------|-------|--------|--------|--------|
| Combination C | stall | stall | bubble | normal | normal |

(每空 1 分，共 3 分)

5. 在教材 PIPE 处理器设计中，data memory 实际是高速缓存(cache)。假设在执行上述(4)中代码时，0x000 指令中的 0(%eax) 地址中的数据不在 data memory 中，则 data memory 会将输出信号 m\_datamiss 置为 1，直到数据从内存中取回到 data memory，再将 m\_datamiss 置为 0。(m\_datamiss 的默认值为 0)

这种情况的判断条件如下，请问各级流水线寄存器的控制信号应如何设置？

| Condition | Trigger                                     |
|-----------|---------------------------------------------|
| Data Miss | M_icode in { IMRMOVL, IPOPL } && m_datamiss |

| Condition | F     | D     | E     | M     | W      |
|-----------|-------|-------|-------|-------|--------|
| data miss | stall | stall | stall | stall | bubble |

(每空 1 分，共 5 分)

|    |
|----|
| 得分 |
|    |

### 第五题（20 分）

（每个填空和选择 1 分，表格每相邻两格 1 分）

#### 1. 仔细阅读下面的程序，根据条件回答下列各题（10 分）

- 地址宽度为 7，数组的起始地址为 0x1000000
- Block Size = 4 Byte, Set = 4，两路组相连.
- 替换算法位 LRU（最近最少使用）

```
#define LENGTH 8
void clear4x4 (char array[LENGTH][LENGTH]) {
 int row, col ;
 for (col = 0 ; col < 4; col++) {
 for (row = 0; row < 4 ; row++) {
 array[row] [col] = 0;
 }
 }
}
```

#### 1) 以上程序执行会引起多少次失效？

4

#### 2) 如果 LENGTH 改为 16，会引起多少次失效？

16

#### 3) 如果 LENGTH 变为 17，与 2) 相比，下面描述正确的是：A，会引起 14 次失效。

- A) 16×16 比 17×17 产生更多的失效次数
- B) 16×16 和 17×17 产生的失效次数相同
- C) 16×16 比 17×17 产生更少的失效次数

#### 4) 请画出 3) 运行后 cache 中 set0 和 set1 的最终状态。

| V | Tag | Data                        | V | Tag | Data                        |
|---|-----|-----------------------------|---|-----|-----------------------------|
| 1 | 101 | Array[1][0]~<br>Array[1][3] | 1 | 100 | Array[0][0]~<br>Array[0][3] |
| 1 | 110 | Array[2][0]~<br>Array[2][3] | 1 | 111 | Array[3][0]~<br>Array[3][3] |

2. 改变假设条件，回答下列各题（10 分）

- 地址宽度为 8，数组的起始地址为 0x10000000
- Cache 容量为 16 Byte，Block Size = 4 Byte，全相联 Cache
- 替换算法位 LRU（最近最少使用）

- 1) Tag 的位数为 6
- 2) 如果执行上述程序，当 LENGTH=8 时，会引起多少次失效？  
4
- 3) 如果 LENGTH 改为 16，会引起多少次失效？  
4
- 4) 如果 LENGTH 变为 17，与 3) 相比，下面描述正确的是： C
  - A) 16×16 比 17×17 产生更多的失效次数
  - B) 16×16 和 17×17 产生的失效次数相同
  - C) 16×16 比 17×17 产生更少的失效次数

5) 请画出 4) 执行后 cache 的最终状态

| V | Tag    | Data                     |
|---|--------|--------------------------|
| 1 | 100000 | Array[0][0]~ Array[0][3] |
| V | Tag    | Data                     |
| 1 | 100101 | Array[1][0]~ Array[1][3] |
| V | Tag    | Data                     |
| 1 | 101001 | Array[2][0]~ Array[2][3] |
| V | Tag    | Data                     |
| 1 | 101101 | Array[3][0]~ Array[3][3] |

# 北京大学信息科学技术学院考试试卷

考试科目：计算机系统导论 姓名：\_\_\_\_\_ 学号：\_\_\_\_\_

考试时间：2016 年 11 月 07 日 小班教师：\_\_\_\_\_

|     |   |   |   |   |   |    |
|-----|---|---|---|---|---|----|
| 题号  | 一 | 二 | 三 | 四 | 五 | 总分 |
| 分数  |   |   |   |   |   |    |
| 阅卷人 |   |   |   |   |   |    |

## 北京大学考场纪律

1、考生进入考场后，按照监考老师安排隔位就座，将学生证放在桌面上。无学生证者不能参加考试；迟到超过 15 分钟不得入场。在考试开始 30 分钟后方可交卷出场。

2、除必要的文具和主考教师允许的工具书、参考书、计算器以外，其它所有物品（包括空白纸张、手机、或有存储、编程、查询功能的电子用品等）不得带入座位，已经带入考场的必须放在监考人员指定的位置。

3、考试使用的试题、答卷、草稿纸由监考人员统一发放，考试结束时收回，一律不准带出考场。若有试题印制问题请向监考教师提出，不得向其他考生询问。提前答完试卷，应举手示意请监考人员收卷后方可离开；交卷后不得在考场内逗留或在附近高声交谈。未交卷擅自离开考场，不得重新进入考场答卷。考试结束时间到，考生立即停止答卷，在座位上等待监考人员收卷清点后，方可离场。

4、考生要严格遵守考场规则，在规定时间内独立完成答卷。不准交头接耳，不准偷看、夹带、抄袭或者有意让他人抄袭答题内容，不准接传答案或者试卷等。凡有违纪作弊者，一经发现，当场取消其考试资格，并根据《北京大学本科考试工作与学术规范条例》及相关规定严肃处理。

5、考生须确认自己填写的个人信息真实、准确，并承担信息填写错误带来的一切责任与后果。

学校倡议所有考生以北京大学学生的荣誉与诚信答卷，共同维护北京大学的学术声誉。

以下为试题和答题纸，共      页。

装订线内

不要答题

|    |
|----|
| 得分 |
|    |

第一题 单项选择题和填空题（每小题 1 分，共 20 分）

注：选择题的回答填写在下表中，填空题的回答填写在题目提供的空格中。

|    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|
| 题号 | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
| 回答 |    |    |    |    |    |    |    |    |    |    |
| 题号 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| 回答 |    |    |    |    | /  | /  | /  | /  | /  | /  |

1. 在下列指令中，其执行会影响条件码中的 CF 位的是：

- A. jmp NEXT      B. jc NEXT      C. inc %bx      D. shl \$1,%ax

答案：D

2. 下列关于比较指令 CMP 说法中，正确的是：

- A. 专用于有符号数比较      B. 专用于无符号数比较  
C. 专用于串比较      D. 不区分比较的对象是有符号数还是无符号数

答案：D

3. 在如下代码段的跳转指令中，目的地址是：

```
400020: 74 F0 je _____
400022: 5d pop %rbp
```

- A. 400010      B. 400012      C. 400110      D. 400112

答案：B（有符号跳转，向后跳转-0x10）

4. 对于如下的 C 语言中的条件转移指令，它所对应的汇编代码中至少包含几条条件转移指令：if (a > 0 && a != 1 || a < 0 && a != -1) b=a;

- A. 2 条      B. 3 条      C. 4 条      D. 5 条

答案：B（这个条件相当于!(a==0 || a==1 || a==-1)）

5. 将 AX 清零，下列指令错误的是（ ）

- A. sub %ax, %ax      B. xor %ax, %ax  
C. test %ax, %ax      D. and \$0, %ax

答案：C（test 相当于 and）

6. 在如下 switch 语句对应的跳转表中, 哪些标号没有出现在分支中 ( )

```
addq $1, %rdi
cmpq $8, %rdi
ja .L2
jmp *.L4(, %rdi, 8)
.L4: .quad .L9 .quad .L5 .quad .L6 .quad .L7
 .quad .L2
 .quad.L7 .quad .L8 .quad .L2 .quad .L5
```

- A. 3, 6                      B. -1, 4                      C. 0, 7                      D. 2, 4

答案: A

7. 已知短整型数组 s 的起始地址和下标 i 分别存放在寄存器 %rdx 和 %rcx, 将 &s[i] 存放在寄存器 %rax 中所对应的汇编代码是 ( )

- A. leaq (%rdx, %rcx, 1), %rax                      B. movw (%rdx, %rcx, 2), %rax  
C. leaq (%rdx, %rcx, 2), %rax                      D. movw (%rdx, %rcx, 1), %rax

答案: C

8. 下面对指令系统的描述中, 错误的是: ( )

- A. CISC 指令系统中的指令数目较多, 有些指令的执行周期很长; 而 RISC 指令系统中通常指令数目较少, 指令的执行周期都较短。  
B. CISC 指令系统中的指令编码长度不固定; RISC 指令系统中的指令编码长度固定, 这样使得 CISC 机器可以获得了更短的代码长度。  
C. CISC 指令系统支持多种寻址方式, RISC 指令系统支持的寻址方式较少。  
D. CISC 机器中的寄存器数目较少, 函数参数必须通过栈来进行传递; RISC 机器中的寄存器数目较多, 只需要通过寄存器来传递参数, 避免了不必要的存储访问。

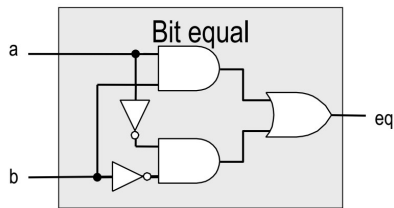
答案: D

9. 下面对流水线技术的描述, 正确的是: ( )

- A. 流水线技术不仅能够提高执行指令的吞吐率, 还能减少单条指令的执行时间。  
B. 不断加深流水线级数, 总能获得性能上的提升。  
C. 流水级划分应尽量均衡, 吞吐率会受到最慢的流水级影响。  
D. 指令间的数据相关可能会引发流水线停顿, 但总是可以通过调度指令来解决。

答案: C

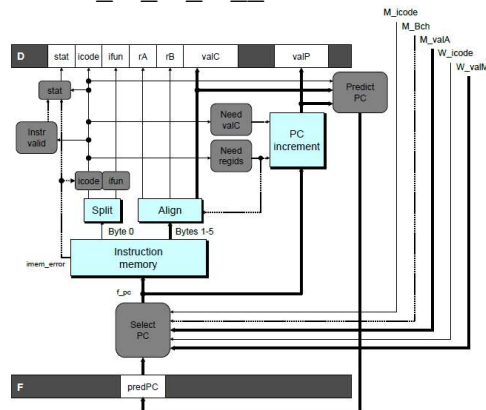
10. 对应下述组合电路的正确 HCL 表达式为:



- A. Bool eq = ( a or b ) and ( !a or !b )
- B. Bool eq = ( a and b ) or ( !a and !b )
- C. Bool eq = ( a or !b ) and ( !a or b )
- D. Bool eq = ( a and !b ) or ( !a and b )

答案: B

11. 流水线数据通路中的转移预测策略为总是预测跳转。如果转移预测错误,需要恢复流水线,并从正确的目标地址开始取值。其中,用来判断转移预测是否正确的信号是\_①\_和\_②\_,用来获得正确的目标地址的信号是\_③\_。



- A. ① M\_icode ② M\_Bch ③ M\_valA
- B. ① W\_icode ② M\_Bch ③ M\_valA
- C. ① W\_icode ② M\_Bch ③ W\_valM
- D. ① M\_icode ② M\_Bch ③ W\_valM

答案: A

12. 若处理器实现了三级流水线,每一级流水线实际需要的运行时间分别为 1ns、2ns 和 3ns,则此处理器不停顿地执行完毕 10 条指令需要的时间为:

- A. 21 ns B. 12 ns C. 24 ns D. 36 ns



答案 D  $3+3+10*3=36ns$

13. 以下关于存储结构的讨论, 那个是正确的

- A. 增加额外一级存储, 数据存取的延时一定不会下降
- B. 增加存储的容量, 数据存取的延时一定不会下降
- C. 增加额外一级存储, 数据存取的延时一定不会增加
- D. 以上选项都不正确

答: ( )

答案: D

14. 关于局部性 (locality) 的描述, 不正确的是:

- A. 循环通常具有很好的时间局部性
- B. 循环通常具有很好的空间局部性
- C. 数组通常具有很好的时间局部性
- D. 数组通常具有很好的空间局部性

答案: C

数组本身只能体现出空间局部性

15. 假定编译器规定 int 和 short 型长度分别为 32 位和 16 位, 执行下列语句:

```
unsigned short x = 65530;
```

```
unsigned int y = x;
```

得到 y 的机器数为\_\_\_\_\_。(用 16 进制表示, 勿省略前导的 0)

答案: 0000FFFA

解析: x 的机器数是 FFFA

y 的机器数是 0000FFFA, 注意无符号数使用零扩展

16. 一个 C 语言程序在一台 32 位机器上运行。程序中定义了三个变量 x、y 和 z, 其中 x 和 z 为 int 型, y 为 short 型。当 x=127, y=-9 时, 执行赋值语句 z=x+y 后, z 的值是\_\_\_\_\_ (用 16 进制表示, 勿省略前导的 0)

答案: 00000076

解析: x 的机器数是 0000007F

y 的机器数是 FFF7, 和 int 型运算时符号扩展为 FFFFFFF7

z 的机器数是 0000007F+FFFFFFF7=00000076 (最高位进位自然丢弃)

也可直接由  $127-9=118$  得到结果

17. 若按 IEEE 浮点标准的单精度浮点数 (符号位 1 位, 阶码字段 exp 占据 8 位,

小数字段 frac 占据 23 位) 表示-8.25, 结果是\_\_\_\_\_ (用 16 进制表示)

答案: C1040000

解析:  $-8.25 = (-1) * 2^3 * (1 + \frac{1}{32})$

符号位: 1

阶码字段: 1000 0010 (127+3=130)

小数字段: 000 0100 0000 0000 0000 0000 (1/32)

结果: 1100 0001 0000 0100 0000 0000 0000 0000

18. 若我们采用基于 IEEE 浮点格式的浮点数表示方法, 阶码字段 exp 占据 k 位, 小数字段 frac 占据 n 位, 则最大的非规格的正数是\_\_\_\_\_ (结果用含有 n, k 的表达式表示)

答案:  $(1 - 2^{-n}) * 2^{-2^{k-1} + 2}$

解析: 阶码字段 exp: 00...00

小数字段 frac: 11...11

偏置值 bias:  $2^{k-1} - 1$

19. 如果直接映射高速缓存大小是 4KB, 并且块 (block) 大小为 32 字节, 那么它每组 (set) 有\_\_\_\_\_行 (line)。

答案: 1。这是直接映射高速缓存的基本特征, 和容量无关

20. 如果直接映射高速缓存大小是 4KB, 并且块 (block) 大小为 32 字节, 那么它每路 (way) 有\_\_\_\_\_行 (line)。

答案: 128, 容量=路容量\*路数=cache 行大小\*Set 数\*路数->set 数=4KB/32B=2<sup>7</sup>=128

|    |
|----|
| 得分 |
|    |

## 第二题 (20 分)

(边凯归, 周明辉)

1. 在 64 位机器上, 判断下列等式是否恒成立

```
/* random_int() 函数返回一个随机的 int 类型值 */
int x = random_int();
int y = random_int();
int z = random_int();
unsigned ux = (unsigned)x;
long lx = (long)x; /* long 为 64 位 */
long ly = (long)y;
double dx = (double)x;
double dy = (double)y;
double dz = (double)z;
```

| Expression                                                    | Always True? |   |
|---------------------------------------------------------------|--------------|---|
| $(x \geq 0) \    \ (3 * x < 0)$                               | Y            | N |
| $(x \geq 0) \    \ (x < ux)$                                  | Y            | N |
| $((x \gg 1) \ll 1) \leq x$                                    | Y            | N |
| $((x - y) \ll 3) + (x \gg 1) - y == 8 * x - 9 * y + x / 2$    | Y            | N |
| $(x - y > 0) == ((y + \sim x + 1) \gg 31 == 1)$               | Y            | N |
| $dx + dy == (double) (y + x)$                                 | Y            | N |
| $dx + dy + dz == dz + dy + dx$                                | Y            | N |
| $(int)((lx + ly) \gg 1) == ((x \& y) + ((x \wedge y) \gg 1))$ | Y            | N |

(前 6 题每题 1 分; 7、8 两题每题 2 分; 共 10 分)

答案

N 考虑  $x = (1 \ll 31) \gg 1$

N 考虑  $x = -1$

Y

N 考虑  $x = -1 \ y = 0$

N 考虑  $x - y = T_{min}$

N 考虑  $x+y$  溢出

Y 恒成立, 每一个 `int` 型都可以由一个 `double` 型精确表示

Y 恒成立

2. 假设 C 语言中新定义了一种数据类型 T, 该类型为 12-bit 长的浮点数, 此浮点数遵循 IEEE 浮点数格式, 其字段划分如下:

符号位 (s): 1-bit; 阶码字段 (exp): 6-bit; 小数字段 (frac): 5-bit。

1) 若将该格式下能表示的所有正规格数从小到大依次排列, 则

相邻两数之间差值的最小值为\_\_\_\_\_

相邻两数之间差值的最大值为\_\_\_\_\_

2) 现定义了如下变量:

T a = -15.875;

T b = (1<<28) + (1<<24) + (1<<22);

T c = a\*b;

请写出各变量的二进制表示:

| 变量 | 二进制表示 |
|----|-------|
| a  |       |
| b  |       |
| c  |       |

(每空 2 分; 共 10 分)

答案:

1)

$2^{-35}$

$2^{26}$

2)

1 100011 00000 (发生进位)

0 111011 00010 (发生舍入)

1 111111 00000 (溢出,  $+\infty$ )

|    |
|----|
| 得分 |
|    |

### 第三题（20 分）

（出题人：熊英飞，审核人：陈钟、焦文品）

(1) 观察下面C语言函数和它相应的x86-64汇编代码

```
int foo(int x, int i)
{
 switch(i)
 {
 case 1:
 x -= 10;
 case 2:
 x *= 8;
 break;
 case 3:
 x += 5;
 case 5:
 x /= 2;
 break;
 case 0:
 x &= 1;
 default:
 x += i;
 }
 return x;
}

00000000004004a8 <foo>:
4004a8: mov %edi,%edx
4004aa: cmp $0x5,%esi
4004ad: ja 4004d4 <foo+0x2c>
4004af: mov %esi,%eax
4004b1: jmpq *0x400690(,%rax,8)
4004b8: sub $0xa,%edx
4004bb: shl $0x3,%edx
4004be: jmp 4004d6 <foo+0x2e>
4004c0: add $0x5,%edx
```

```

4004c3: mov %edx,%eax
4004c5: shr $0x1f,%eax
4004c8: lea (%rdx,%rax,1),%eax
4004cb: mov %eax,%edx
4004cd: sar %edx
4004cf: jmp 4004d6 <foo+0x2e>
4004d1: and $0x1,%edx
4004d4: add %esi,%edx
4004d6: mov %edx,%eax
4004d8: retq

```

调用gdb命令x/kg \$rsp 将会检查从rsp中的地址开始的k个8字节字，请填写下面gdb命令的输出（每空一分）。

>(gdb) x/6g 0x400690

```

0x400690: 0x_____ 0x_____
0x4006a0: 0x_____ 0x_____
0x4006b0: 0x_____ 0x_____

```

(2) 右边的汇编代码是由左边程序中的 m 函数编译而成。回答如下问题。

|                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                                                         |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> typedef struct _list {     struct _list* next;     int value; } list;  int m(list* p) {     int r = 100;     while ( __①__ ) {         r = __②__;         p = __③__;     }     if (p != 0) r = __④__;     return r; } </pre> | <pre> m:     testq %rdi, %rdi     je .L6     movq (%rdi), %rdx     movl \$100, %eax     testq %rdx, %rdx     jne .L4     jmp .L3 .L14:     movq (%rdi), %rdx     testq %rdx, %rdx     je .L3 .L4:     movl 8(%rdx), %r8d     ⑤ 8(%rdi), %r8d     ⑤ %r8d, %eax     movq (%rdx), %rdi     testq %rdi, %rdi     jne .L14     ret .L3:     ⑤ 8(%rdi), %eax     ret .L6:     movl \$100, %eax     ret </pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

已知访存延迟 1 个指令周期。不访存的时候 addl 延迟 4 个指令周期，imull 延

迟 8 个指令周期，其他指令延迟 1 个指令周期。指令访存时延迟的执行周期为不访存的时候延迟的指令周期和访存延迟的周期之和。函数 m 中的循环在处理链表 p 的时候 CPE 为 4 个指令周期。⑤处的指令为 movl, addl, imull 中的一个。请填写①-⑤处的代码（⑤空 2 分，其他 3 分）。

- ① \_\_\_\_\_
- ② \_\_\_\_\_
- ③ \_\_\_\_\_
- ④ \_\_\_\_\_
- ⑤ \_\_\_\_\_

答案：

(1) 每空 1 分

|           |                    |                    |
|-----------|--------------------|--------------------|
| 0x400690: | 0x00000000004004d1 | 0x00000000004004b8 |
| 0x4006a0: | 0x00000000004004bb | 0x00000000004004c0 |
| 0x4006b0: | 0x00000000004004d4 | 0x00000000004004c3 |

(2) ⑤空 2 分，其他 3 分

① `p != 0 && p->next != 0`

② `r * (p->value * p->next->value)` 注意：答案中运算可以交换，但不可结合

③ `p->next->next`

④ `r * p->value`

⑤ `imull` （因为每次处理两个元素，所以关键路径的时间周期为 8。关键路径要么为两次访存和两次 `mov`，要么为⑤处的运算。前者不可能达到 8，所以只能是 `imull`。）

给分说明：如果②④的答案和⑤一致，但是⑤答错了就②④各给 2 分。

|    |
|----|
| 得分 |
|    |

#### 第四题（20 分）

（出题人：汪小林，易江芳）

请分析32位的Y86 ISA中新加入的一组条件返回指令：cretXX，其格式如下。

cretXX    9    fun

类似cmovXX，该组指令只有当条件码 (Cnd) 满足时，才执行函数返回；如果条件不满足，则顺序执行。

1. 若在教材所描述的SEQ处理器上执行这条指令，请按下表补全每个阶段的操作。需说明的信号可能会包括：icode, ifun, rA, rB, valA, valB, valC, valE, valP, Cnd; the register file R[], data memory M[], Program counter PC, condition codes CC。其中对存储器的引用必须标明字节数。如果在某一阶段没有任何操作，请填写none指明。

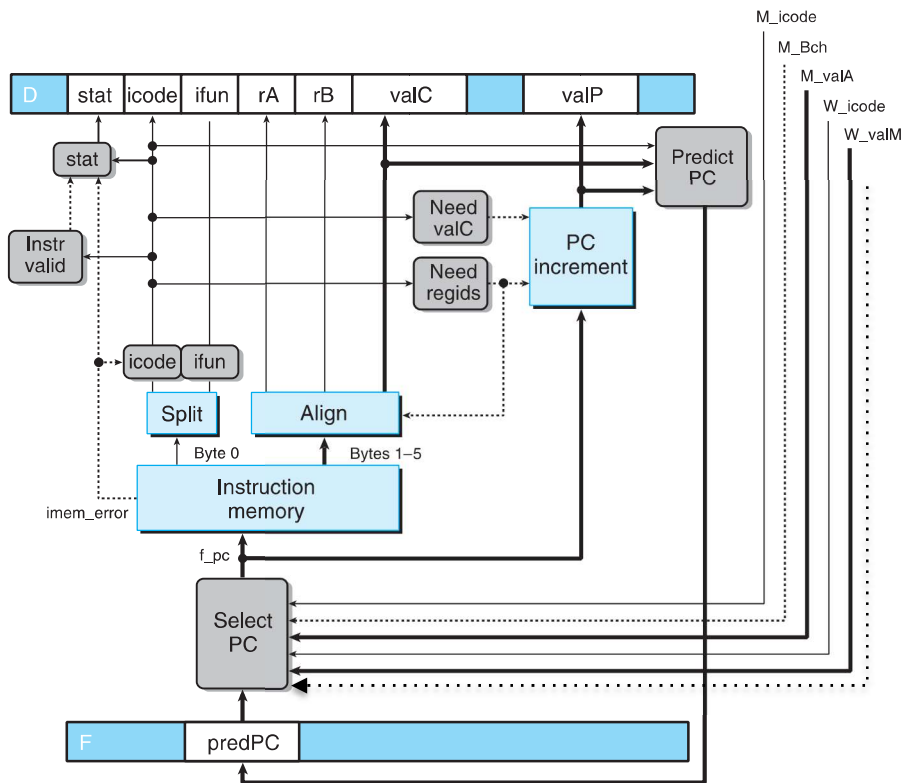
| Stage      | cretXX Offset                                                                                     |
|------------|---------------------------------------------------------------------------------------------------|
| Fetch      | <u>icode:ifun <math>\leftarrow M_1[PC]</math></u><br><u>valP <math>\leftarrow PC+1</math></u>     |
| Decode     | <u>valB <math>\leftarrow R[\%esp]</math></u><br><u>valA <math>\leftarrow R[\%esp]</math></u>      |
| Execute    | <u>valE <math>\leftarrow valB + 4</math></u><br><u>Cnd <math>\leftarrow Cond(CC, ifun)</math></u> |
| Memory     | <u>valM <math>\leftarrow M_4[valA]</math></u>                                                     |
| Write back | <u>if (Cnd) R[%esp] <math>\leftarrow valE</math></u>                                              |
| PC update  | <u>PC <math>\leftarrow Cnd ? valM : valP</math></u>                                               |

（每个空 1 分，共 9 分）

2. 为了执行cretXX指令，我们需要改进教材所描述的PIPE处理器，在W（Write



Back) 阶段引入流水线寄存器 W\_Cnd (填W\_Bch也算对) , 并将其连接到PC选择器 (Select PC) 以便有条件地更新PC。假设改进后的处理器总是预测函数返回条件不满足, 则如果返回条件满足时, 一共会错误取指 3 条指令。(每空1分, 共2分)



3. 在2中改进的PIPE处理器上执行cretXX指令时, 发生预测错误时的判断条件和各级流水线寄存器的控制信号应如何设置?

| Condition         | Trigger                                                                            |
|-------------------|------------------------------------------------------------------------------------|
| Mispredicted cret | $(E\_icode = ICRETXX \ \&\& \ e\_Cnd) \    \ (M\_icode = ICRETXX \ \&\& \ M\_Cnd)$ |

(算两个空, 每空 1 分, 共 2 分)

| Condition         | F      | D      | E      | M      | W      |
|-------------------|--------|--------|--------|--------|--------|
| Mispredicted cret | Normal | bubble | bubble | normal | normal |

(每空 1 分, 共 3 分)

4. PIPE 处理器上处理器上执行如下代码片段,

```
0x000: xorl %eax, %eax
```

```
0x002: popl %esp
```

```
0x004: cretne
```

是否会发生 load-use 和 misprediction cret 组合的 hazard 情况? (1分)

答: 不会

如果此时“popl %esp”在流水线的 Execute 阶段, 请问此时, 各级流水线寄存器的控制信号应如何设置?

| Condition   | F     | D     | E      | M      | W      |
|-------------|-------|-------|--------|--------|--------|
| Combination | stall | stall | bubble | normal | normal |

(每空 1 分, 共 3 分)

|    |
|----|
| 得分 |
|    |

第五题（20 分）

（孙广宇，陆俊林）

现有一个能够存储4个Block的Cache，每一个Cache Block的长度为2 Byte（既  $B = 2$ ）。内存空间的大小是32Byte，既内存空间地址范围如下：

$$0_{10} \quad (00000_2) \quad \text{--} \quad 31_{10} \quad (11111_2)$$

现有一程序，访问内存地址序列如下所示，单位是Byte。

$$2_{10} \quad 23_{10} \quad 10_{10} \quad 9_{10} \quad 9_{10} \quad 11_{10} \quad 3_{10}$$

1. Cache的结构如下图所示（ $S=2$ ， $E=2$ ），初始状态为空，替换策略LRU。请在下图空白处填入上述数据访问后Cache的状态。

（TAG使用二进制格式；Data Block使用10进制格式，例：M[6-7]表示地址 $6_{10}$ - $7_{10}$ 对应的数据）

|      |   |     |            |
|------|---|-----|------------|
|      | V | TAG | Data Block |
| set0 | 1 | 010 | M[8-9]     |
|      | 0 |     |            |

|      |   |     |          |
|------|---|-----|----------|
|      | V | TAG | Block    |
| set1 | 1 | 010 | M[10-11] |
|      | 1 | 000 | M[2-3]   |

上述数据访问一共产生了多少次 Hit：\_\_2\_\_ (2pts)

2. 如果Cache的替换策略改成MRU（即Most Recently Used，最近使用的数据被替换出去），请在下图空白处填入访问上述数据访问后Cache的状态。

|   |     |            |
|---|-----|------------|
| V | TAG | Data Block |
|---|-----|------------|

|      |   |     |        |
|------|---|-----|--------|
| set0 | 1 | 010 | M[8-9] |
|      | 0 |     |        |

|      |   |     |          |
|------|---|-----|----------|
|      | V | TAG | Block    |
| set1 | 1 | 000 | M[2-3]   |
|      | 1 | 010 | M[10-11] |

上述数据访问一共产生了多少次 Hit: \_\_3\_\_ (2pts)

3. 现增加一条新规则：地址区间包含 5 的倍数的 block 将不会被缓存，仍旧使用 MRU 替换策略，这 7 次数据访问一共产生了多少次 Hit \_\_2\_\_ (2pts)

4. 在第 3 小题的基础上，现又增加一条数据预取规则：每当地址为 10 的数据被访问时，地址为 8 的数据将会被放入缓存，这 7 次数据访问一共产生了多少次 Hit \_\_3\_\_ (2pts)

# 北京大学信息科学技术学院考试试卷

考试科目：计算机系统导论 姓名：\_\_\_\_\_ 学号：\_\_\_\_\_

考试时间：2017 年 11 月 6 日 小班号：\_\_\_\_\_ 小班教师：\_\_\_\_\_

|     |   |   |   |   |   |    |
|-----|---|---|---|---|---|----|
| 题号  | 一 | 二 | 三 | 四 | 五 | 总分 |
| 分数  |   |   |   |   |   |    |
| 阅卷人 |   |   |   |   |   |    |

## 北京大学考场纪律

1、考生进入考场后，按照监考老师安排隔位就座，将学生证放在桌面上。无学生证者不能参加考试；迟到超过 15 分钟不得入场。在考试开始 30 分钟后方可交卷出场。

2、除必要的文具和主考教师允许的工具书、参考书、计算器以外，其它所有物品（包括空白纸张、手机、或有存储、编程、查询功能的电子用品等）不得带入座位，已经带入考场的必须放在监考人员指定的位置。

3、考试使用的试题、答卷、草稿纸由监考人员统一发放，考试结束时收回，一律不准带出考场。若有试题印制问题请向监考教师提出，不得向其他考生询问。提前答完试卷，应举手示意请监考人员收卷后方可离开；交卷后不得在考场内逗留或在附近高声交谈。未交卷擅自离开考场，不得重新进入考场答卷。考试结束时间到，考生立即停止答卷，在座位上等待监考人员收卷清点后，方可离场。

4、考生要严格遵守考场规则，在规定时间内独立完成答卷。不准交头接耳，不准偷看、夹带、抄袭或者有意让他人抄袭答题内容，不准接传答案或者试卷等。凡有违纪作弊者，一经发现，当场取消其考试资格，并根据《北京大学本科考试工作与学术规范条例》及相关规定严肃处理。

5、考生须确认自己填写的个人信息真实、准确，并承担信息填写错误带来的一切责任与后果。

学校倡议所有考生以北京大学学生的荣誉与诚信答卷，共同维护北京大学的学术声誉。

以下为试题和答题纸，共 \_\_\_\_\_ 页。

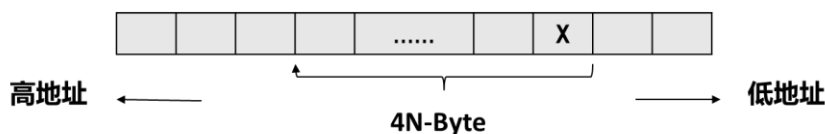
|    |
|----|
| 得分 |
|    |

第一题 单项选择题（每小题 2 分，共 30 分）

注：选择题的回答请填写在下表中。

|    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|
| 题号 | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
| 回答 |    |    |    |    |    |    |    |    |    |    |
| 题号 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| 回答 |    |    |    |    |    | /  | /  | /  | /  | /  |

1. 假定一个特殊设计的计算机，将 int 型数据的长度从 4-Byte 扩展为 4N-Byte，采用大端法（Big Endian）。现将该 int 型所能表示的最小负数写入内存中，如下图所示。其中每个小矩形代表一个 Byte，请问 x 位置这个 Byte 中的值是多少？



- A.  $00000000_2$     B.  $01111111_2$     C.  $10000000_2$     D.  $11111111_2$

答案：C

解析：最小的负数为  $10000000...$ ，大端法将最高位放到最低地址

2. 以下说法正确的是：

- A. 负数加上负数结果都为负数  
 B. 正数加上正数结果都为正数  
 C. 用 & 和 ~ 可以表示所有的逻辑与或非操作  
 D. 用 & 和 | 可以表示所有的逻辑与或非操作

答案：C

3. 若我们采用基于 IEEE 浮点格式的浮点数表示方法，阶码字段 (exp) 占据 k 位，小数字段 (frac) 占据 n 位，则最小的规格化 (normalized) 的正数是 \_\_\_\_\_（结果用含有 n, k 的表达式表示）

- A.  $(1 - 2^{-n}) * 2^{-2^{k-1}+2}$                       B.  $1 * 2^{-2^{k-1}+2}$

C.  $2^{-n} * 2^{-2^{k-1}+2}$

D.  $(1 - 2^{-n}) * 2^{-2^k+1}$

答案: B

解析: 阶码字段 `exp: 00...01`; 小数字段 `frac: 00...00`; 偏置值 `bias:  $2^{k-1}-1$`

4. 在下列的 x86-64 汇编代码中, 错误的是:

A. `movq %rax, (%rsp)`

B. `movl $0xFF, (%ebx)`

C. `movsbl (%rdi), %eax`

D. `leaq (%rdx, 1), %rdx`

答案: B/D 都对

解析: B: 地址寄存器必须是 64 位寄存器; D: 括号里少了一个逗号

5. 在下列关于条件传送的说法中, 正确的是:

A. 条件传送可以用来传送字节、字、双字、和 4 字的数据

B. C 语言中的“?:”条件表达式都可以编译成条件传送

C. 使用条件传送总可以提高代码的执行效率

D. 条件传送指令不需要用后缀 (例如 `b, w, l, q`) 来表明操作数的长度

答案: D

解析: A. 条件传送不支持单字节传送; B. 如果“?:”涉及到的两个表达式中有一个出错或者有副作用, 用条件传送会导致非法行为; C. 如果被旁路的分支的计算量很大, 计算就白做了; D. 从目标寄存器的名字可以推断出条件传送指令的操作数长度

6. 有如下代码段:

```
int func(int x, int y);
int (*p) (int a, int b);
p = func;
p(0,0);
```

对应的下列 x86-64 过程调用正确的是:

A. `call *%rax`

B. `call (%rax)`

C. `call *(%rax)`

D. `call func`

答案: A/D 都对

解析: 用 `00` 编译可以得到 A, 用 `0g` 编译可以得到 D。

7. 有 A 的定义: `int A[3][2] = {{1,2}, {3,3}, {2,1}};`

那么 `A[2]` 的值为:

A. &A+16

B. A+16

C. \*A+4

D. \*A+2

答案: C

解析: 参见书 p177 页表格, 机器在计算指针与常数的运算时, 会将常数乘以指针指向的元素大小。&A 常数扩大的倍数为  $\text{sizeof}(A[3][2]) = 3*2*4$ ; A 常数扩大的倍数为  $\text{sizeof}(A[0])=2*4$ ; \*A 常数扩大的倍数为  $\text{sizeof}(\text{int}) = 4$ 。正确的答案应为 A+2 或 \*A+4, 故应选择 C。

程序验证如下:

```
int main(){
 int A[3][2] = {{1,2},{3,3},{2,1}};
 cout << "&A+16: " << &A+16 << endl;
 cout << "A+16: " << A+16 << endl;
 cout << "*A+4: " << *A+4 << endl;
 cout << "*A+2: " << *A+2 << endl;
 cout << A[2] << endl;
}
```

```
&A+16: 0xb5fee0
A+16: 0xb5fde0
*A+4: 0xb5fd70
*A+2: 0xb5fd68
0xb5fd70
```

8. 在 x86-64 架构下, 有如下变量:

```
union {char c[8], int i;} x;
```

在  $x.i=0x41424344$  时,  $x.c[2]$  的值为多少 (提示:  $'A'=0x41$ ):

A. 'A'

B. 'B'

C. 'C'

D. 'D'

答案: B

9. 下面关于 RISC 和 CISC 的描述中, 正确的是:

A. CISC 和早期 RISC 在寻址方式上相似, 通常只有基址和偏移量寻址

B. CISC 指令集可以对内存和寄存器操作数进行算术和逻辑运算, 而 RISC 只能寄存器操作数进行算术和逻辑运算

C. CISC 和早期的 RISC 指令集都有条件码, 用于条件分支检测

D. CISC 机器中的寄存器数目较少, 函数参数必须通过栈来进行传递; RISC 机器中的寄存器数目较多, 只需要通过寄存器来传递参数, 避免了不必要的存储访问

答案: B



10. 在 Y86 的 SEQ 实现中, 对仅考虑 IRMMOVQ, ICALL, IPOPQ, IRET 指令, 对 mem\_addr 的 HCL 描述正确的是:

```
word mem_addr = [
 icode in { (1), (2) } : valE;
 icode in { (3), (4) } : valA;
];
```

- A. (1) IRMMOVQ    (2) IPOPQ    (3) IRET    (4) ICALL
- B. (1) IRMMOVQ    (2) IRET    (3) IPOPQ    (4) ICALL
- C. (1) ICALL    (2) IPOPQ    (3) IRMMOVQ    (4) IRET
- D. (1) IRMMOVQ    (2) ICALL    (3) IPOPQ    (4) IRET

**答案: D**

11. 关于流水线技术的描述, 错误的是:

- A. 流水线技术能够提高执行指令的吞吐率, 但也同时增加单条指令的执行时间
- B. 增加流水线级数, 不一定能获得总体性能的提升
- C. 指令间数据相关引发的数据冒险, 不一定可以通过暂停流水线来解决。
- D. 流水级划分应尽量均衡, 吞吐率会受到最慢的流水级影响, 均衡的流水线能提高吞吐量。

**答案: C**

12. 假设已有声明 int i, int j, const int n, int r, int a[n], int b[n], int mul(int, int) 以下程序优化编译器一般不会进行的是:

|    | 优化前                                                 | 优化后                                                                     |
|----|-----------------------------------------------------|-------------------------------------------------------------------------|
| A. | for (j = 0; j < n; ++j)<br>a[n * 8] += b[j];        | int tmp = (n << 3);<br>for (j = 0; j < n; ++j)<br>a[tmp] += b[j];       |
| B. | for (j = 0; j < n; ++j)<br>r = (r * a[j]) * b[j];   | for (j = 0; j < n; ++j)<br>r = r * (a[j] * b[j]);                       |
| C. | for (j = 0; j < n; ++j)<br>a[mul(n, i) + j] = b[j]; | int ni = mul(n, i);<br>for (j = 0; j < n; ++j)<br>a[ni + j] = b[j];     |
| D. | for (j = 1; j < n; ++j)<br>a[0] += a[j];            | int tmp = 0;<br>for (j = 1; j < n; ++j)<br>tmp += a[j];<br>a[0] += tmp; |

**答案: C**

13. 以下计算机部件中，通常不用于存储器层次结构（Memory Hierarchy）的是：

- A. 高速缓存      B. 内存      C. 硬盘      D. 优盘（U 盘）

答案：D

14. 关于局部性（locality）的描述，正确的是：

- A. 数据的时间局部性或数据空间局部性，在任何有意义的程序中都能体现  
B. 指令的时间局部性或数据空间局部性，在任何有意义的程序中都能体现  
C. 数据的时间局部性，在任何循环操作中都能体现  
D. 数据的空间局部性，在任何数组操作中都能体现

答案：B

注：B 选项有笔误（“数据空间局部性”应为“指令空间局部性”），所以所有答案都不对。按任何选项都对计分。

15. 在高速缓存存储器中，关于全相联和直接映射结构，以下论述正确的是：

- A. 如果配备同样容量、技术的高速缓存，配备全相联高速缓存的计算机总是比配备直接映射高速缓存的计算机性能低  
B. 如果配备同样容量、技术的高速缓存，配备全相联高速缓存的计算机总是比配备直接映射高速缓存的计算机性能高  
C. 如果配备同样容量、技术的高速缓存，当数据在缓存中时，配备全相联高速缓存的计算机总是比配备直接映射高速缓存的计算机读数据慢  
D. 如果配备同样容量、技术的高速缓存，当数据在缓存中时，配备全相联高速缓存的计算机总是比配备直接映射高速缓存的计算机读数据快

答案：C

|    |
|----|
| 得分 |
|    |

第二题（15 分）

考虑有一种基于 IEEE 浮点格式的 9 位浮点表示格式 A。格式 A 有 1 个符号位，k 个阶码位，n 个小数位。现在已知  $\frac{-9}{16}$  的位模式可以表示为“101100010”，请回答

以下问题：（注：阶码偏移量为  $2^{k-1}-1$ ）

1. 求 k 和 n 的值。（1 分）

答案：k = 4，n = 4

2. 基于格式 A，请填写下表。值的表示可以写成整数（如 16），或者写成分数（如  $17/64$ ）。（注：每格 2 分）

| 描述       | 二进制位表示      | 值       |
|----------|-------------|---------|
| 最大的非规格化数 | 0 0000 1111 | 15/1024 |
| 最小的正规格化数 | 0 0001 0000 | 1/64    |
| 最大的规格化数  | 0 1110 1111 | 248     |

3. 假设格式 A 变为 1 个符号位，k+1 个阶码位，n-1 个小数位，那么能表示的实数数量会怎样变化，数值的精度会怎样变化？（回答增加、降低或不变即可）（2 分）

答案：小数位变少后，NaN 的数量减少了，所以实数数量增加（1 分），数值精度降低（1 分）。

|    |
|----|
| 得分 |
|    |

### 第三题（15 分）

分析下面C语言程序和相应的x86-64汇编程序。其中缺失部分代码（被遮挡），请在对应的横线上填写缺失的内容。（注：每格1分）

```
#include <stdio.h>
#include "string.h"

void myprint(char *str)
{
 char buffer[16];
 ██████████(buffer, str);
 printf("%s \n", buffer);
}

void alert(void)
{
 printf("██████████\n");
}

int main(int argc, char *argv[])
{
 myprint("1234567123456712345671234567\xaa\x84\x04\x08");
 return 0;
}

.section .rodata
.LC0:
.string "██████████"
.text
.globl myprint
.type myprint, @function
myprint:
.LFB0:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
subq $48, ██████████
movq %rdi, -40(%rbp)
movq %fs:40, ██████████
movq %rax, -8(%rbp)
xorl %eax, %eax
movq ██████████, %rdx
leaq -32(%rbp), %rax
movq %rdx, ██████████
██████████
call strcpy
██████████ -32(%rbp), %rax
movq %rax, %rsi
movl $.LC0, %edi
movl $0, %eax
call printf
```

① strcpy

② Where am I?

③ %s \n

④ %rsp

⑤ %rax

⑥ -40(%rbp)

⑦ %rsi

⑧ movq %rax, %rdi

⑨ leaq

```

 nop
 [redacted], %rax
 xorq [redacted]
 je [redacted]
 call __stack_chk_fail
.L2:
 leave
 .cfi_def_cfa 7, 8
 ret
 .cfi_endproc
.LFE0:
 .size myprint, .-myprint
 .section .rodata
.LC1:
 .string "Where am I?"
 .text
 .globl alert
 .type alert, @function
alert:
.LFB1:
 .cfi_startproc
 pushq %rbp
 .cfi_def_cfa_offset 16
 .cfi_offset 6, -16
 movq %rsp, %rbp
 .cfi_def_cfa_register 6
 movl $.LC1, %edi
 call puts
 nop
 popq %rbp
 .cfi_def_cfa 7, 8
 [redacted]
 .cfi_endproc
.LFE1:
 .size alert, .-alert
 .section .rodata
 .align 8
.LC2:
 .string "1234567123456712345671234567\252\[redacted]\004\b"
 .text
 .globl main
 .type main, @function
main:
.LFB2:
 .cfi_startproc
 pushq %rbp
 .cfi_def_cfa_offset 16
 .cfi_offset 6, -16
 movq %rsp, %rbp
 .cfi_def_cfa_register 6
 subq $16, %rsp
 movl %edi, -4(%rbp)
 movq %rsi, -16(%rbp)
 movl $.LC2, %edi
 [redacted]
 movl $0, %eax
 leave
 .cfi_def_cfa 7, 8
 ret
 .cfi_endproc
.LFE2:
 .size main, .-main

```

⑩ movq -8(%rbp)

⑪ %fs:40, %rax

⑫ .L2

注：下面这种回答也对：

⑩ movq %fs:40

⑪ -8(%rbp), %rax

⑬ ret

⑭ 204

⑮ call myprint

|    |
|----|
| 得分 |
|    |

#### 第四题 (20 分)

分析64位的Y86 ISA中新加入的条件内存传送指令：crrmmovqXX和cmrrmovqXX。crrmmovqXX和cmrrmovqXX指令在条件码满足所需要的约束时，分别执行和rmmovq以及mrrmovq同样的语义。其格式如下：

|            |   |    |    |    |         |
|------------|---|----|----|----|---------|
| rmmovq     | 4 | 0  | rA | rB | D (8字节) |
| crrmmovqXX | 4 | fn | rA | rB | D (8字节) |
| mrrmovq    | 5 | 0  | rA | rB | D (8字节) |
| cmrrmovqXX | 5 | fn | rA | rB | D (8字节) |

1. 请按下表补全每个阶段的操作。需说明的信号可能会包括: icode, ifun, rA, rB, valA, valB, valC, valE, valP, Cnd; 寄存器堆R[], 存储器M[], 程序计数器PC, 条件码CC。其中对存储器的引用必须标明字节数。 (10分)

| 阶段   | rmmovq rA, D(rB)                                                                                                                                                                    | cmrrmovqXX D(rB), rA                                                                                                          |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| 取指   | $\begin{aligned} \text{icode:ifun} &\leftarrow M_1[PC] \\ \text{rA:rB} &\leftarrow M_1[PC+1] \\ \text{valC} &\leftarrow M_8[PC+2] \\ \text{valP} &\leftarrow PC + 10 \end{aligned}$ |                                                                                                                               |
| 译码   | $\begin{aligned} \text{valA} &\leftarrow R[\text{rA}] \\ \text{valB} &\leftarrow R[\text{rB}] \end{aligned}$                                                                        |                                                                                                                               |
| 执行   | $\text{valE} \leftarrow \text{valB} + \text{valC}$                                                                                                                                  | $\text{valE} \leftarrow \text{valB} + \text{valC}$<br>$\underline{\text{Cnd} \leftarrow \text{Cond}(\text{CC}, \text{ifun})}$ |
| 访存   | $M_8[\text{valE}] \leftarrow \text{valA}$                                                                                                                                           | $\text{valM} \leftarrow M_8[\text{valE}]$<br>或<br>$\text{if}(\text{Cnd}) \text{ valM} \leftarrow M_8[\text{valE}]$            |
| 写回   | none                                                                                                                                                                                | $\text{if}(\text{Cnd}) R[\text{rA}] \leftarrow \text{valM}$                                                                   |
| 更新PC | $PC \leftarrow \text{valP}$                                                                                                                                                         |                                                                                                                               |

2. 为了执行上述新增指令，我们需要改进教材所描述的PIPE处理器，在回写（W: Write Back）阶段引入寄存器以保持流水线信号 W\_Cnd（请参考教材对信号的命名规则书写），以便有条件地更新寄存器内容。在如此改进的PIPE处理器上，请写出如下信号的HCL代码。（6分）

| 信号       | HCL代码                                                                                            |
|----------|--------------------------------------------------------------------------------------------------|
| F_stall  | (E_icode in {IMRMVQ, IPO PQ}   <br>( <u>①</u> )) &&<br>E_dstM in <u>②</u>   <br>IRET in <u>③</u> |
| E_bubble | ( <u>④</u> )   <br>(E_icode in {IMRMVQ, IPO PQ}   <br>( <u>①</u> )) &&<br>E_dstM in <u>②</u>     |
| M_bubble | m_stat in <u>⑤</u>   <br>W_stat in <u>⑤</u>                                                      |

附HCL描述中的常数值编码表如下：

|          |                |          |                 |
|----------|----------------|----------|-----------------|
| IHALT    | halt指令的代码      | INOP     | nop指令的代码        |
| IRRMVQ   | rrmovq指令的代码    | IIRMOVQ  | irmovq指令的代码     |
| IRMMOVQ  | rmmovq指令的代码    | IMRMVQ   | mrmmovq指令的代码    |
| ICRMMOVQ | crmmovqXX指令的代码 | ICMRMOVQ | cmrmmovqXX指令的代码 |
| IOPL     | 整数运算指令的代码      | IJXX     | 跳转指令的代码         |
| ICALL    | call指令的代码      | IRET     | ret指令的代码        |
| IPUSHQ   | pushq指令的代码     | IPOPQ    | popq指令的代码       |
| FNONE    | 默认功能码          | RNONE    | 表示没有寄存器文件访问     |
| ALUADD   | 表示加法运算         | RRSP     | 表示%rsp寄存器ID     |
| SAOK     | 正常地址操作状态码      | SADR     | 地址异常状态码         |
| SINS     | 非法指令异常状态码      | SHLT     | halt状态码         |

① E\_icode == ICMRMOVQ && e\_Cnd

② {d\_srcA, d\_srcB}

③ {D\_icode, E\_icode, M\_icode}

④ E\_icode == IJXX && !e\_Cnd

⑤ {SADR, SINS, SHLT}

3. 对于下面的Y86汇编代码, 请使用上述条件内存传送指令将其修改为不带跳转的汇编代码序列。假设下面的代码片段在教材所描述的PIPE处理器上运行, 不考虑该片段前后代码的影响以及高速缓存 (cache) 失效的情况, 假设%rsi初值为0, 处理器设计使用总是选择 (always taken) 的预测策略。原始代码片段预计运行\_\_\_11\_\_\_周期, 改进代码片段预计执行\_\_\_9 (或8)\_\_\_周期。(4分)

注: 由于第2小题的HCL描述, 要求带Cond, 因为这是一个更优化的设计, 而且和教材上对IJXX的考虑一致。基于此, 第3小题可以有两种指令序列, cmrmovqne在前的序列执行需要8个周期, cmrmovqe在前的指令序列需要9个周期, 因此最后一空填8或9都对。

| 原始代码                                                                                                        | 改进代码                                                                                                                                                                                          |
|-------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> andq %rsi %rsi jne L1 mrmovq 8(%rdx), %rax jmp L2 L1: mrmovq 8(%rdx), %rbx L2: addq %rax, %rbx </pre> | <pre> (9个周期的版本) andq %rsi %rsi cmrmovqe 8(%rdx), %rax cmrmovqne 8(%rdx), %rbx addq %rax, %rbx  (8个周期的版本) andq %rsi %rsi cmrmovqne 8(%rdx), %rbx cmrmovqe 8(%rdx), %rax addq %rax, %rbx </pre> |



|    |
|----|
| 得分 |
|    |

第五题（20 分）

现有一个能够存储 4 个 Block 的 Cache，每一个 Cache Block 的大小为 2 Byte（即  $B = 2$ ）。内存空间的大小是 32 Byte，即内存空间地址范围如下：

$$0_{10} \quad (00000_2) \quad \text{--} \quad 31_{10} \quad (11111_2)$$

现有一程序，访问内存地址序列如下所示，单位是 Byte。

$0_{10} \quad 3_{10} \quad 4_{10} \quad 7_{10} \quad 16_{10} \quad 19_{10} \quad 21_{10} \quad 22_{10} \quad 8_{10} \quad 10_{10} \quad 13_{10} \quad 14_{10} \quad 24_{10} \quad 26_{10} \quad 29_{10} \quad 30_{10}$

1. Cache 的结构如下图所示（ $S=2$ ， $E=2$ ），初始状态为空，替换策略 LRU。请在下图空白处填入上述数据访问后 Cache 的状态。（12 分）

（TAG 使用二进制格式；Data Block 使用十进制格式，例：M[6-7] 表示地址  $6_{10}-7_{10}$  对应的数据）

|      | V | TAG | Data Block | V | TAG | Data Block |
|------|---|-----|------------|---|-----|------------|
| set0 | 1 | 110 | M[24-25]   | 1 | 111 | M[28-29]   |
| set1 | 1 | 110 | M[26-27]   | 1 | 111 | M[30-31]   |

上述数据访问一共产生了多少次 Hit：\_\_0\_\_（2 分）

2. 在第 1 小题的基础上，现增加一条数据预取规则：每当 cache 访问出现 miss 时，被访问地址及其后续的一个 cache block 都会被放入缓存，即当 M[0-1] 访问发生 miss，则把 M[0-1] 和 M[2-3] 都放入缓存中。那么，这 16 次数据访问一共产生了多少次 Hit：\_\_8\_\_（2 分）

3. 在第 1 小题的基础上，如果每一个 Cache Block 的大小扩大为 4 Byte（即  $B = 4$ ，cache 大小变为原来的 2 倍），这 16 次数据访问一共产生了多少次 Hit：\_\_8\_\_（2 分）

4. 在第 3 小题的基础上，考虑增加 2 中的数据预取规则，这 16 次数据访问一共产生了多少次 Hit：\_\_12\_\_（2 分）

# 北京大学信息科学技术学院考试试卷

考试科目：计算机系统导论 姓名：\_\_\_\_\_ 学号：\_\_\_\_\_

考试时间：2018年11月14日 小班号：\_\_\_\_ 小班教师：\_\_\_\_\_

| 题号  | 一 | 二 | 三 | 四 | 五 | 六 | 总分 |
|-----|---|---|---|---|---|---|----|
| 分数  |   |   |   |   |   |   |    |
| 阅卷人 |   |   |   |   |   |   |    |

## 北京大学考场纪律

1、考生进入考场后，按照监考老师安排隔位就座，将学生证放在桌面上。无学生证者不能参加考试；迟到超过 15 分钟不得入场。在考试开始 30 分钟后方可交卷出场。

2、除必要的文具和主考教师允许的工具书、参考书、计算器以外，其它所有物品（包括空白纸张、手机、或有存储、编程、查询功能的电子用品等）不得带入座位，已经带入考场的必须放在监考人员指定的位置。

3、考试使用的试题、答卷、草稿纸由监考人员统一发放，考试结束时收回，一律不准带出考场。若有试题印制问题请向监考教师提出，不得向其他考生询问。提前答完试卷，应举手示意请监考人员收卷后方可离开；交卷后不得在考场内逗留或在附近高声交谈。未交卷擅自离开考场，不得重新进入考场答卷。考试结束时间到，考生立即停止答卷，在座位上等待监考人员收卷清点后，方可离场。

4、考生要严格遵守考场规则，在规定时间内独立完成答卷。不准交头接耳，不准偷看、夹带、抄袭或者有意让他人抄袭答题内容，不准接传答案或者试卷等。凡有违纪作弊者，一经发现，当场取消其考试资格，并根据《北京大学本科考试工作与学术规范条例》及相关规定严肃处理。

5、考生须确认自己填写的个人信息真实、准确，并承担信息填写错误带来的一切责任与后果。

学校倡议所有考生以北京大学学生的荣誉与诚信答卷，共同维护北京大学的学术声誉。

以下为试题和答题纸，共      页。

|    |
|----|
| 得分 |
|    |

第一题 单项选择题（每小题 2 分，共 30 分）

注：选择题的回答请填写在下表中。

|    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|
| 题号 | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
| 回答 |    |    |    |    |    |    |    |    |    |    |
| 题号 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| 回答 |    |    |    |    |    | /  | /  | /  | /  | /  |

1. 下列哪种类型转换既可能导致溢出、又可能导致舍入？

A) int 转 float B) float 转 int C) int 转 double D) float 转 double

答案：b，本题考查浮点数类型转换，参见中文版 86 页。

2. 在采用小端法存储机器上运行下面的代码，输出的结果将会是？

（int, unsigned 为 32 位长, short 为 16 位长, 0~9 的 ASCII 码分别是 0x30~0x39）

```
char *s = "2018";
int *p1 = (int *)s;
short s1 = (*p1)>>12;
unsigned u1 = (unsigned) s1;
printf("0x%x\n", u1);
```

A) 0x00002303 B) 0x00032303 C) 0xffff8313 D) 0x00008313

答案：C

本题考查大小端存储，以及整数类型转化。字符串在存储时不区分大小端，前面的字符存储在低地址，而在被转化成整型的时候低地址被视为低位，因此 \*p1 为 0x38313032，s1 为 0x8313。在由 short 转化为 unsigned 的时候，我们要先改变大小，之后完成有符号到无符号的转换，因此 u1 为 0xffff8313。

3. 考虑如下函数

```
void XOR(int x, int y) {
 y = x ^ y;
 x = x ^ y;
 y = x ^ y;
 printf(x, y);
}
```

```
}
```

XOR(a, b)的输出结果为?

- A) a, b      B) b, a      C) b, 0      D) b, a^b

答案: C

本题考查 xor 的特殊用法,  $x \text{ xor } x = 0$ ,  $0 \text{ xor } x = x$ ;

答案修订: 应为 B, 该代码将 x, y 值进行交换。另外, 题面应为 `printf(“%d, %d”, x, y);`

4. 在 x86-64 下, 以下哪个选项的说法是错误的?

- A) `movl` 指令以寄存器作为目的时, 会将该寄存器的高位 4 字节设置为 0  
B) `cvtq` 指令的作用是将 `%eax` 符号扩展到 `%rax`  
C) `movabsq` 指令只能以寄存器作为目的  
D) `movswq` 指令的作用是将零扩展的字传送到四字节的

答案: D

`movswq` 应该是符号扩展

5. 以下代码的输出结果是

```
union {
 double d;
 struct {
 int i;
 char c[4];
 } s;
} u;
u.d = 1;
printf(“%d\n”, u.s.c[2]);
```

- A) 0      B) -16      C) 240      D) 191

答案: B

`c[2]` 保存的数据是  $(11110000)_2$ , `char` 保存的数据是有符号数, 所以应该是 -16

答案修订: ABC 均正确。

题目未说明大小端,若为大端法,选择 A。若为小端法,u. s. c[2]为(11110000)<sub>2</sub>,由于 char 在不同的编译器上会被实现成 signed char 或 unsigned char,因此扩展为 int 时-16 和 240 均有可能,B, C 均正确。

6. 下列关于 C 语言中的结构体(struct)以及联合(union)的说法中,正确的是:
- A) 对于任意 struct,将其成员按照其实际占用内存大小从小到大的顺序进行排列不一定会使之内存占用最小
  - B) 对于任意 struct,将其成员按照其实际占用内存大小从小到大的顺序进行排列一定不会使之内存占用最大
  - C) 对于任意 union,将其成员按照其实际占用内存大小从小到大的顺序进行排列不一定会使之内存占用最小
  - D) 对于任意 union,将其成员按照其实际占用内存大小从小到大的顺序进行排列一定不会使之内存占用最大

答案: A。联合以及只含一个基本数据类型成员的结构体的内存占用与其成员排列方式无关,即任意排列方式都可使得内存占用最小(最大),故 b、c、d 错误。对于 struct,应当将成员按照其存储单元所占内存大小从小到大(或从大到小)的顺序进行排列才能使内存占用最小,故 a 正确。

7. 下列关于程序控制结构的机器代码实现的说法中,正确的是:
- A) 使用条件跳转(conditional jump)语句实现的程序片段比使用条件赋值(conditional move)语句实现的同一程序片段的运行效率高
  - B) 使用条件跳转语句实现的程序片段与使用条件赋值语句实现的同一程序片段虽然效率可能不同,但在 C 语言的层面上看总是有着相同的行为
  - C) 一些 switch 语句不会被 gcc 用跳转表的方式实现
  - D) 以上说法都不正确

答案: C。条件跳转本身开销大于条件赋值,但条件赋值会将两个分支中的运算都完成,故分支中的运算较为复杂时,使用条件赋值语句实现的程序效率较低,故 a 错误。分支中的运算带有副作用时,条件跳转语句和条件赋值语句实现的程序行为不同,故 b 错误。switch 语句中的 case 若比较稀疏,则不会被用跳转表的方式实现,故 c 正确。

8. 下列关于条件码的描述中,不正确的是( )
- A) 所有算术指令都会改变条件码
  - B) 所有比较指令都会改变条件码
  - C) 所有与数据传送有关的指令都会改变条件码

D) 条件码一般不会直接读取，但可以直接修改

答案：C，leaq 是 movq 指令的变形，它只传送地址，不改变条件码。

答案修订：ABCD 均给分。

AB 没有指明是 x86 指令系统；x86 指令系统中有 SIMD 类指令不改变条件码；

C 数据传送一般不改变条件码；

D 正确，SAHF、STC、CLC、CMC 等指令均可以直接写条件码。

9. 请比较 RISC 和 CISC 的特点，回答下述问题：

假设编译技术处于发展初期，程序员更愿意使用汇编语言编程来解决问题，那么程序员会更倾向于选用 \_\_\_\_\_ ISA。

假设你设计的处理器速度非常快，但存储系统设计使得取指令的速度非常慢（也许只是处理单元的十分之一）。这时你会更倾向于选用 \_\_\_\_\_ ISA。

A) RISC、RISC B) CISC、CISC C) RISC、CISC D) CISC、RISC

答案：B

//知识点 1：CISC 有更多的指令，有些更接近高级语言

//知识点 2：CISC 的指令功能更复杂，指令执行需要更多的周期，一定程度上可以平衡处理速度与指令访存的速度差异。不过，通常处理器设计中，主要通过多层次的存储体系结构来弥补两者之间的速度差异。

10. Y86 指令 **popl ra** 的 SEQ 实现如下图所示，其中①和②分别为：

|            |                                                                                                      |
|------------|------------------------------------------------------------------------------------------------------|
| Fetch      | $icode:ifun \leftarrow M_1[PC]$<br>$ra:rb \leftarrow M_1[PC+1]$<br>$valP \leftarrow \textcircled{1}$ |
| Decode     | $valA \leftarrow R[\%esp]$<br>$valB \leftarrow R[\%esp]$                                             |
| Execute    | $valE \leftarrow \textcircled{2}$                                                                    |
| Memory     | $valM \leftarrow M_4[valA]$                                                                          |
| Write Back | $R[\%esp] \leftarrow valE$<br>$R[ra] \leftarrow valM$                                                |
| PC Update  | $PC \leftarrow valP$                                                                                 |

A)  $PC + 4$   $valA + 4$

B)  $PC + 4$   $valA + (-4)$

C)  $PC + 2$   $valB + 4$

D)  $PC + 2$   $valB + (-4)$

答案：C

//知识点：popl 弹栈指令，栈结构

1) popl 是双字节指令，下一条指令位置 PC+2

2) Y86 是 32 位体系结构，popl 弹出 4 字节，弹栈栈指针增加，valA + 4

11. 假设已有声明 `int i, int sum, int *p, int *q, int *r, const int n = 100, float a[n], float b[n], float c[n], int foo(int), void bar()`，以下哪项程序优化编译器总是可以进行？

|   |                                                                                          |                                                                                                             |
|---|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| A | <pre>for(i = 0; i &lt; n; ++i) {<br/>    a[i] += b[i];<br/>    a[i] += c[i];<br/>}</pre> | <pre>float tmp;<br/>for(i = 0; i &lt; n; ++i) {<br/>    tmp = b[i] + c[i];<br/>    a[i] += tmp;<br/>}</pre> |
| B | <pre>*p += *q;<br/>*p += *r;</pre>                                                       | <pre>int tmp;<br/>tmp = *q + *r;<br/>*p += tmp;</pre>                                                       |
| C | <pre>for(i = 0; i &lt; n; ++i)<br/>    sum += i * 4;</pre>                               | <pre>int N = n * 4;<br/>for(i = 0; i &lt; N; i += 4)<br/>    sum += i;</pre>                                |
| D | <pre>for(i = 0; i &lt; foo(n); ++i)<br/>    bar();</pre>                                 | <pre>int tmp = foo(n);<br/>for(i = 0; i &lt; tmp; ++i)<br/>    bar();</pre>                                 |

答案：C

A 浮点数不满足结合律

B p, q, r 可能指向同一个地址

D foo 函数可能有副作用

12. 设一种缓存共包含 4 个缓存块，每个缓存块 32 字节，采用 LRU 替换算法。设缓存初始状态为空，对其进行下列 char 类型内存地址序列访存，0x5a7, 0x5b7, 0x6a6, 0x5b8, 0x7a5, 0x5b9。对于直接映射缓存和 2 路组相联缓存，分别能产生几次命中？

A) 1, 3

B) 1, 4

C) 2, 3

D) 2, 4

答案：A

考核地址中 TAG 位域/组索引位域/缓存块内偏移位域在直接映射和组相联的定义，LRU 替换算法的概念和过程。

13. 设一种全相联缓存共包含 4 个缓存块,如果循环地顺序访问 5 个不同的缓存块,下列哪种替换算法会产生最多的命中?

- A) 最近最少使用替换策略 LRU
- B) 先入先出替换策略 FIFO
- C) 随机替换策略 Random
- D) 后入先出替换策略 LIFO

答案: A

考核替换算法的概念和过程

答案修订: CD 均正确。

A LRU 命中率 0%;

B FIFO 命中率 0%;

C Random 每次新来一个块,随机换出的块会导致之后第 1, 2, 3, 4 次访存有一次 miss (等概率), 每次 miss 平均 2.5 次访存, 每次访存 miss 率 40%, 因此命中率 60%;

D LIFO 命中率 60%, 最后两个块交替发生 miss。

14. 以下关于存储的描述中, 正确的是 ( )

- A) 由于基于 SRAM 的内存性能与 CPU 的性能有很大差距, 因此现代计算机使用更快的基于 DRAM 的高速缓存, 试图弥补 CPU 和内存间性能的差距。
- B) SSD 相对于旋转磁盘而言具有更好的读性能, 但是 SSD 写的速度通常比读的速度慢得多, 而且 SSD 比旋转磁盘单位容量的价格更贵, 此外 SSD 底层基于 EEPROM 的闪存会磨损。
- C) 一个有 2 个盘片、10000 个柱面、每条磁道平均有 400 个扇区, 每个扇区有 512 个字节的双面磁盘的容量为 8GB。
- D) 访问一个磁盘扇区的平均时间主要取决于寻道时间和旋转延迟, 因此一个旋转速率为 6000RPM、平均寻道时间为 9ms 的磁盘的平均访问时间大约为 19ms。

答案: B

A 选项中 SRAM 和 DRAM 位置反了。

C 选项中, 硬盘容量  $1GB=10^9$  Byte, 因此容量应该为 8.192GB。

D 选项中, 平均旋转延迟为  $0.5 * (60s/6000RPM) = 5ms$ , 平均访问时间为 14ms。

15. 某计算机地址空间 12 位, L1 cache 大小为 256 字节, 组数 S=4, 路数 E=2, 现在在地址 0x0 处开始有一个 N 行 M 列的 int 类型的数组 int A[N][M], 有如下 C 代码:

```
int ans=0;
for(int j=0;j<M;++j)
 for(int i=0;i<N;++i)
 ans+=A[i][j];
```

不考虑并行、编译优化等等会影响访问 A 数组元素顺序的因素, 则如下 (N, M) 对中会导致全部 Cache Miss 的有 ( )



$(N, M) = (64, 4), (32, 8), (16, 16), (2, 128)$

- A) 4 个
- B) 3 个
- C) 2 个
- D) 1 个

答案: C

每个缓存块可以存放  $256/S/E/4=8$  个 int 类型的数据

1、 $(N, M)=(64, 4)$  时, 访问数组第一行元素会将第一、二行元素都放入缓存, 因此在访问数组第二行元素  $A[1][0]$  时不会 Cache Miss

2、 $(N, M)=(32, 8)$  时, 可以验证全部 Cache Miss

3、 $(N, M)=(16, 16)$  时, 可以验证全部 Cache Miss

4、 $(N, M)=(2, 128)$  时, 访问  $A[0][1]$  时不会 Cache Miss

|    |
|----|
| 得分 |
|    |

## 第二题（10 分）

在本题中，int 和 unsigned 均为 32 位。int 用补码表示，算术右移。float 为 32 位 IEEE754 标准，double 为 64 位 IEEE754 标准。

- （2 points）生成任意 int 类型的 x，然后将它转换为 unsigned 类型：

```
int x = random();
unsigned ux = (unsigned) x;
```

对于以下每个 C 语言表达式，你需要判断它的值是否恒为 1。如果是，圈出 Y；否则圈出 N。

| 表达式                            | 恒为 1?    |
|--------------------------------|----------|
| $(x \geq 0) \mid\mid (x < ux)$ | Y      N |
| $((x \gg 1) \ll 1) \leq x$     | Y      N |

- （2 point）请补全函数 nlz，使之可以用来计算 unsigned 类型数据的二进制表示中**前导零**的个数（提示：填写数字）：

```
// TODO: Complete the `nlz` function to
// count the number of leading zeros
int nlz(unsigned x) {
 double w = 0.5;
 double y = (double)x + w;
 long long z;
 memcpy(&z, &y, sizeof(y));
 return ____ - (z >> ____);
}
```

- （2 point）上式中 w 的取值并不唯一。请判断以下哪个/哪些数字也符合 w 的要求：  
(a) 0.2 (b) 0.3 (c) 0.9 (d) 1.2
- （4 points）下面所示的 strlen\_x 是字符串求长度的函数在**大端序**电脑中的一种实现，nlz 函数的意义如前所述。

```

int func_x(unsigned x) {
 unsigned u = 0x7F7F7F7F;
 unsigned y = (x & u) + u;
 y = ~(y | x | u);
 return nlz(y) >> 3;
}

size_t strlen_x(const char *s) {
 size_t t, n = 0;
 const unsigned *si = (const unsigned *)s;
 while ((t = func_x(*si++)) == 4) { n += t; }
 return n + t;
}

```

不改变 strlen\_x 的实现，请修改 func\_x 以使得 strlen\_x 适用于小端序的电脑（提示：仅使用常数、算术/逻辑运算符和已定义变量）：

```

int func_x(unsigned x) {
 unsigned u = 0x7F7F7F7F;
 unsigned y = (x & u) + u;
 y = ~(y | x | u);
 return (___ - nlz((___) & (y-1))) >> 3;
}

```

答案：

1.

| 表达式                            | 恒为 1?                                     |
|--------------------------------|-------------------------------------------|
| $(x \geq 0) \mid\mid (x < ux)$ | N: e.g. $x = -1$ .                        |
| $((x >> 1) << 1) <= x$         | Y: $x >> 1$ rounds toward minus infinity. |

2.  $32 - 1 - ((z \gg 52) - 1023)$ ，即应该填 **1054**。

3. w 的取值范围为  $[0.5, 1)$ ，因此只有 **c** 符合要求。

4. 原始的 func\_x 函数的意义在于在 unsigned 类型数据的四个字节中，从 most significant byte 到 least significant byte 寻找第一个等于 '\0' 的字节的位置 (0~3)，若找不到则返回 4，因此大端序机器上正好可以用来做 strlen\_x 的累加。而在一台小端序机器上，const unsigned 类型的指针会将顺序排列的四个 char 颠倒顺序来解读，因此 func\_x 的意义应改变为从 least significant byte 开始寻找第一个等于 '\0' 的字节的位置。仔细考察 y 的含义，可以构造出以下式子：

```
// TODO: Fill in the blank line
int func_x(unsigned x) {
 unsigned u = 0x7F7F7F7F;
 unsigned y = (x & u) + u;
 y = ~(y | x | u);
 return (32 - nlz(~y & (y-1))) >> 3;
}
```

|    |
|----|
| 得分 |
|    |

第三题（15 分，每空 1 分）

数据结构定义如下：（测试用机：64 位 Linux 机器）

```
typedef struct s1 {
 char cc[N];
 int ii[N];
 int *ip;
} S1;
```

S1 t1[N];

1、计算该数据结构的空间：

当 N=3 时，sizeof(S1) = (1)，sizeof(t1) = (2)

当 N=4 时，sizeof(S1) = (3)，sizeof(t1) = (4)

当 N=5 时，sizeof(S1) = (5)，sizeof(t1) = (6)

**答案：**

**(1) 24 (2) 72**

**(3) 32 (4) 128**

**(5) 40 (6) 200**

2、当 N=4 时，该数据结构初始化代码如下：

```
void init(int n)
{
 int i;
 for (i=0; i<n; i++) {
 t1[i].ip = &(t1[i].ii[i]);
 }
}
```

根据上述代码，填写下面汇编中缺失的内容：

```
init:
 movl $0, %ecx
```

```

 jmp .L2
.L3:
 movslq %ecx, %rax
 leaq ((1) ,%rax,8), %rsi
 leaq 0(,%rsi,4), %rdx
 addq $t1+4, %rdx
 salq (2) , %rax
 movq (3) , ((4))
 addl $1, %ecx
.L2:
 cmpl (5) , %ecx
 jl .L3
 rep ret

```

**答案:**

(1) %rax (2) \$5 (3) %rdx (4) t1+24(%rax) (5) %edi

答案修订: (4) 全给分, 卷面多了一对括号。

3、当 N=3 时, 函数 fun 的汇编代码如下:

fun:

```

 movslq %esi, %rax
 movslq %edi, %rdi
 leaq (%rdi,%rdi), %rdx
 leaq (%rdx,%rdi), %r8
 leaq (%r8,%r8), %rcx
 addq %rcx, %rax
 movl %esi, t1+4(,%rax,4)
 addq %rdx, %rdi
 leaq 0(,%rdi,8), %rax
 movq t1+16(%rax), %rax
 movl %esi, (%rax)
 ret

```

根据上述代码, 填写函数 fun 的 C 语言代码:

```
void fun(int x, int y)
```

```
{
```

```
 (1) = (2);
```

```
 (3) = (4);
```

}

**答案:**

(1) `t1[x].ii[y]`

(2) `y`

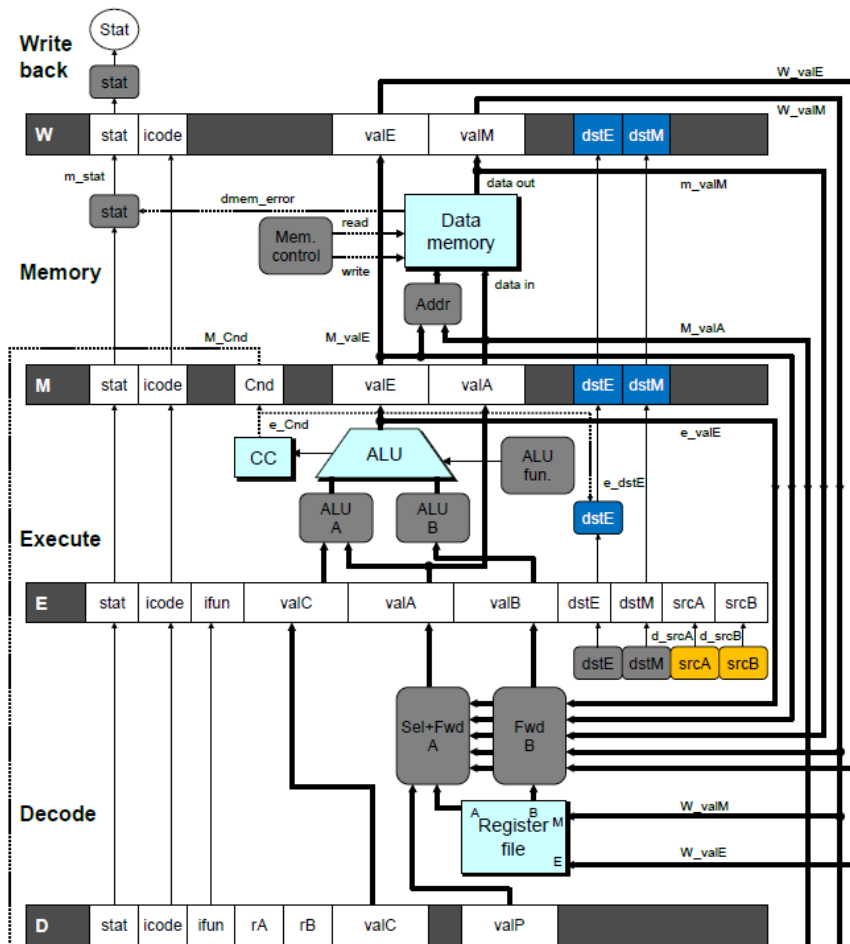
(3) `*(t1[x].ip)`

(4) `y`

|    |
|----|
| 得分 |
|    |

#### 第四题（15 分）

这是一款 Y86-32 流水线处理器的结构图（局部），请以此为基础，依次回答下列问题。



1、该处理器设计采用了前递（forwarding）技术，一定程度上解决了数据相关的问题，在上图中体现在 Sel+FwdA 和 FwdB 部件上。前者输出的信号会存到流水线寄存器 E 的 valA 域（即 E\_valA 信号），请补全该信号的 HCL 语言描述。

int E\_valA = [

D\_icode in { ICALL, IJXX } : \_\_\_\_\_ ; # ① 答案: D\_valP

d\_srcA == e\_dstE : \_\_\_\_\_ ; # ② 答案: e\_valE

d\_srcA == M\_dstM : \_\_\_\_\_ ; # ③ 答案: m\_valM



```

 d_srcA == M_dstE : M_valE ;
 d_srcA == W_dstM : W_valM ;
 ...
];

```

2、如果在该处理器上运行下面的程序，每条指令在不同时钟周期所处的流水线阶段如下表所示。在这种情况下，哪条指令的执行结果会有错误？写出该指令的地址：

0x01e。（1分）

|                             |   |   |   |   |   |   |   |   |   |    |    |    |
|-----------------------------|---|---|---|---|---|---|---|---|---|----|----|----|
| demo1.ys                    | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
| 0x000: irmovl \$128, %edx   | F | D | E | M | W |   |   |   |   |    |    |    |
| 0x006: irmovl \$3, %ecx     |   | F | D | E | M | W |   |   |   |    |    |    |
| 0x00c: rmmovl %ecx, 0(%edx) |   |   | F | D | E | M | W |   |   |    |    |    |
| 0x012: irmovl \$10, %ebx    |   |   |   | F | D | E | M | W |   |    |    |    |
| 0x018: mrmovl 0(%edx), %eax |   |   |   |   | F | D | E | M | W |    |    |    |
| 0x01e: addl %ebx, %eax      |   |   |   |   |   | F | D | E | M | W  |    |    |
| 0x020: halt                 |   |   |   |   |   |   | F | D | E | M  | W  |    |

3、如需检测出这个情况，需要增加逻辑电路，用 HCL 语言表达如下：

E\_icode in {IMRMOVL, IPOPL} && \_\_\_\_\_ in { \_\_\_\_\_ }

答案：E\_icode in {IMRMOVL, IPOPL} && E\_dstM in { d\_srcA, d\_srcB }，2分，全对才得分

4、当新增的电路检测出这个情况后，应对各流水线寄存器进行不同的设置，以便在尽可能少影响性能的前提下解决该问题。请填写下表，可选的设置包括 normal/bubble/stall 三种。

| F | D | E | M | W |
|---|---|---|---|---|
|   |   |   |   |   |

答案：stall, stall, bubble, normal, normal。3分，全对才得分

5、如果遇到下面程序代码所展示的情况，该处理器运行时仍然存在问题。因此，还需要新增检测电路。当新增的电路检测出这个情况后，应对各流水线寄存器进行不同的设置，以便在尽可能少影响性能的前提下解决该问题。请填写下表，可选的设置包括 normal/bubble/stall 三种。

```

demo2.ys
...

```

```

0x018: rmmovl %ecx, 0(%edx)
0x01e: irmovl $10, %ebx
0x024: popl %esp
0x026: ret

```

| F | D | E | M | W |
|---|---|---|---|---|
|   |   |   |   |   |

答案: stall, stall, bubble, normal, normal。3 分，全对才得分

|    |
|----|
| 得分 |
|    |

### 第五题（15 分）

Cache 为处理器提供了一个高性能的存储器层次框架。下面是一个 8 位存储器地址引用的列表（地址单位为字节，地址为 10 进制表示）：

3, 180, 43, 2, 191, 88, 190, 14, 181, 44

1. (7 分) 考虑如下 cache (S=2, E=2)，每个 cache block 大小为 2 个字节。假设 cache 初始状态为空，替换策略为 LRU。请填补下表：

(Tag 使用二进制格式；Data 使用十进制格式，例：M[6-7] 表示地址 6 和 7 对应的数据)

|       | V | Tag | Data | V | TAG | Data |
|-------|---|-----|------|---|-----|------|
| SET 0 | 1 |     |      | 1 |     |      |
| SET 1 | 1 |     |      | 1 |     |      |

共命中\_\_\_\_\_次 (1 分)，分别访问地址\_\_\_\_\_ (地址用 10 进制表示，2 分)

解答：

答案：

|       | V | Tag                  | Data | V | TAG                  | Data |
|-------|---|----------------------|------|---|----------------------|------|
| SET 0 | 1 | 101101<br>M[180-181] |      | 1 | 001011<br>M[44-45]   |      |
| SET 1 | 1 | 000011<br>M[14-15]   |      | 1 | 101111<br>M[190-191] |      |

共命中 3 次

(表格上每空格 1 分，tag 和 data 都正确才得分；)

命中次数回答正确得 1 分；

分别为访问地址 2、190、181 (三个地址完全正确得 2 分，答对两个得 1 分)

2. (4 分) 现在有另外两种直接映射的 cache 设计方案 C1 和 C2，每种方案的 cache 总大小都为 8 个字节，C1 块大小为 2 个字节，C2 块大小为 4 个字节。假设从内存加载一次数据到 cache 的时间为 25 个周期，访问一次 C1 的时间为 3 个周期，访问一次 C2 的时间为 5 个周期。针对第一问的地址访问序列，哪一种 cache 的设计更好？(请分别给出两种 cache 访问第一问地址序列的总时间以及 miss rate)

答案：

| Address | Binary address | C1 hit/miss | C2 hit/Miss |
|---------|----------------|-------------|-------------|
| 3       | 000000 11      | M           | M           |
| 180     | 101101 00      | M           | M           |
| 43      | 001010 11      | M           | M           |

|     |           |   |   |
|-----|-----------|---|---|
| 2   | 000000 10 | M | M |
| 191 | 101111 11 | M | M |
| 88  | 010110 00 | M | M |
| 190 | 101111 10 | H | H |
| 14  | 000011 10 | M | M |
| 181 | 101101 01 | H | M |
| 44  | 001011 00 | M | M |

C1 更好。(1 分)

C1: miss rate =  $8/10 = 80\%$ , (0.5 分) total cycles =  $8 * 25 + 10 * 3 = 230$  (1 分)

C2: miss rate =  $9/10 = 90\%$ , (0.5 分) total cycles =  $9 * 25 + 10 * 5 = 275$  (1 分)

3. (2 分)现在考虑另外一个计算机系统。在该系统中，存储器地址为 32 位，并采用如下的 cache:

| Cache datasize | Cache block size | Cache mode |
|----------------|------------------|------------|
| 32 KiB         | 8 Bytes          | 直接映射       |

此 cache 至少要占用\_\_\_\_\_Bytes. (datasize + (valid bit size + tag size) \* blocks)

答案:

cache block 为 8 bytes, 所以  $b=3$ ;

cache block 一共  $32 * 1024 / 8 = 4096$  个, 又因为是直接映射, 所以  $s=12$ ; 于是 tag 位一共  $t = 32 - s - 1 = 17$ 。所以总大小为:

$$\begin{aligned} \text{totalsize} &= \text{datasize} + (\text{valid bit size} + \text{tag size}) * \text{blocks} \\ &= 32 * 1024 + (1 + 17) * 4096 / 8 = 41984 \text{ (bytes)} \end{aligned}$$

|    |
|----|
| 得分 |
|    |

### 第六题（15 分）

在 PIPE 处理器上运行如下 Y86 代码

```
.L1 mrmov (%eax) %ebx
 addl %ebx %ecx
 addl %ecx %eax
 xorl %ecx %edx
 jne .L1
 irmov $1 %eax
 irmov $1 %eax
```

- 1) 假设上述代码中的循环部分，一共执行了  $N$  遍后跳出循环，未采用数据前递（data forwarding），分支每次都预测正确，访存每次都命中缓存，命中缓存时访存需要 1 个周期。请问共需执行多少个周期？在下面空格处，各填入一个数字。

$N * \underline{11} + \underline{6}$

答案： $4+N * (5+3+3)+2$ ，填充流水线 +4，一次 load-use 冒险 +3，一次 RAW 冒险+3，循环后 2 指令 +2

答案修订： $N*12+5$

$4+(N-1)*(5+3+3+1)+(5+3+3)+2$ ，填充流水线+4，一次 load-use 冒险+3(%ebx)，一次 RAW 冒险+3 (%ecx)，一次 RAW 冒险+1 (%eax)，循环后 2 指令+2，其中最后一次循环%eax 不会冒险。

- 2) 假设上述代码一共循环执行了  $N$  次后跳出循环，采用数据前递（data forwarding），分支每次都预测跳转（taken），访存每次都命中缓存，命中缓存时访存需要 1 个周期。请问共需执行多少个周期？

在下面空格处，各填入一个数字。

$N * \underline{6} + \underline{8}$

答案： $4+N * (5+1)+4$ ，填充流水线 +4，一次 load-use 冒险 +1，最后一次预测错误+2，循环后 2 指令 +2

- 3) 假设上述代码一共循环执行了  $N$  次后跳出循环( $N$  为偶数)，采用数据前递(data forwarding)，分支每次都预测不跳转(not taken)，数据访存时缓存命中率为 50%，指令访存全部命中缓存，命中缓存时访存需要 1 个周期，未命中缓存时访

存需要 3 个周期。请问共需执行多少个周期？

在下面空格处，各填入一个数字。

$N * \underline{9} + \underline{4}$

答案：4+0.5\*N \* (5+1+2)+0.5\*N\*(5+3+2) +2-2，填充流水线 +4，hit 一次 load-use 冒险 +1,miss 一次 load-use 冒险 +3,预测错误+2,循环后 2 指令 +2，最后一次预测正确-2

(每小题 5 分。N 相乘的那个数 3 分，常数 2 分)

# 北京大学信息科学技术学院考试试卷

考试科目：计算机系统导论 姓名：\_\_\_\_\_ 学号：\_\_\_\_\_

考试时间：2019 年 11 月 4 日 小班号：\_\_\_\_\_ 小班教师：\_\_\_\_\_

| 题号  | 一 | 二 | 三 | 四 | 五 | 六 | 总分 |
|-----|---|---|---|---|---|---|----|
| 分数  |   |   |   |   |   |   |    |
| 阅卷人 |   |   |   |   |   |   |    |

## 北京大学考场纪律

1、考生进入考场后，按照监考老师安排隔位就座，将学生证放在桌面上。无学生证者不能参加考试；迟到超过 15 分钟不得入场。在考试开始 30 分钟后方可交卷出场。

2、除必要的文具和主考教师允许的工具书、参考书、计算器以外，其它所有物品（包括空白纸张、手机、或有存储、编程、查询功能的电子用品等）不得带入座位，已经带入考场的必须放在监考人员指定的位置。

3、考试使用的试题、答卷、草稿纸由监考人员统一发放，考试结束时收回，一律不准带出考场。若有试题印制问题请向监考教师提出，不得向其他考生询问。提前答完试卷，应举手示意请监考人员收卷后方可离开；交卷后不得在考场内逗留或在附近高声交谈。未交卷擅自离开考场，不得重新进入考场答卷。考试结束时间到，考生立即停止答卷，在座位上等待监考人员收卷清点后，方可离场。

4、考生要严格遵守考场规则，在规定时间内独立完成答卷。不准交头接耳，不准偷看、夹带、抄袭或者有意让他人抄袭答题内容，不准接传答案或者试卷等。凡有违纪作弊者，一经发现，当场取消其考试资格，并根据《北京大学本科考试工作与学术规范条例》及相关规定严肃处理。

5、考生须确认自己填写的个人信息真实、准确，并承担信息填写错误带来的一切责任与后果。

学校倡议所有考生以北京大学学生的荣誉与诚信答卷，共同维护北京大学的学术声誉。

以下为试题和答题纸，共 \_\_\_\_\_ 页。

|    |
|----|
| 得分 |
|    |

第一题 单项选择题（每小题 2 分，共 30 分）

注：选择题的回答请填写在下表中。

|    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|
| 题号 | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
| 回答 |    |    |    |    |    |    |    |    |    |    |
| 题号 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| 回答 |    |    |    |    |    | /  | /  | /  | /  | /  |

1. 一个 IPv4 地址就是一个 32 位无符号整数。例如，10.2.155.253 对应的地址是 0x0a029bfd。协议规定，无论主机字节顺序如何，IP 地址在内存中总是以大端法来存放的。下面代码要实现的功能是检验 IP 是否符合 192.168.56.xx（xx 表示任意 0~255 的数）的模式，如果满足，则执行 if 语句内部指令。那么，在小端法的机器上，应该分别补充的数字是：

```

unsigned ip, mask;
// set ip
mask = _____;
if(ip & mask == _____)
{
 // do something
}

```

- A. 0x00ffffff      0x0038a8c0  
 B. 0x00ffffff      0x00838a0c  
 C. 0xffffffff00    0xc0a83800  
 D. 0xffffffff00    0xc8a8300

答案：A

解答：192.168.56.xx 对应的十六进制表示为 0xc0a838xx（大端），在小端法机器上对应的 32 位无符号整数就是 0xxx38a8c0。为了将某个数最高的 8 位掩去，可以把它与 0x00ffffff 进行按位与运算，这样会把高 8 位全部置为 0，而低 24 位不改变。再将其与 0x0038a8c0 进行比较，即可得出结果。

2. 已知 0x2019 和 0x12 是两个有符号 32 位整数。下列运算结果中，作为 32 位补码表示的整数最大的是：
- A. 0x2019 << 0x12  
 B. 0x2019 & 0x12



- C.  $0x2019 \mid 0x12$
- D.  $0x2019 \wedge 0x12$

答案: C

解答:  $0x2019$  左移 18 位后刚好最高位为 1, 对应补码表示的是负数; 另外三个选项的结果都是正数。运算数相同时, 运算结果中“1”可能出现的位置也相同。按位或运算的结果在每个这样的位置都为“1”, 而按位与和按位异或的结果中“1”的数量较少, 因此按位或运算的结果最大。也可将三个选项的结果计算出来进行比较。

3. 运行下面的代码, 输出结果是 (其中 float 类型表示 IEEE-754 规定的浮点数, 包括 1 位符号、8 位阶码和 23 位尾数):

```
for(float f = 1;; f = f + 1)
{
 if(f + 1 - f != (float)1)
 {
 printf("%.0f\n", f);
 break;
 }
}
```

- A.  $8388608(=2^{23})$
- B.  $16777216(=2^{24})$
- C.  $2147483647(=2^{31}-1)$
- D. 程序为死循环, 没有输出

答案: B

解答: 这种 float 类型的 23 位尾数意味着其与 23 位及以内的整数可以建立一一对应的联系, 自然  $f + 1 - f == 1$  成立; 当  $f$  超过这一范围时,  $f + 1$  的运算结果会发生舍入, 因而无法恢复到之前的结果。随着  $f$  自增, 最小的超过精度范围的整数是  $2^{24}$ , 故可以得出选项。

4. 以下关于 x86-64 指令的描述, 说法正确的有几项?

- a) 有符号除法指令 `idivq S` 将 `%rdx` (高 64 位) 和 `%rax` (低 64 位) 中的 128 位数作为被除数, 将操作数 `S` 的值作为除数, 做有符号除法运算; 指令将商存在 `%rdx` 寄存器中, 将余数存在 `%rax` 寄存器中。
- b) 我们可以使用指令 `jmp %rax` 进行间接跳转, 跳转的目标地址由寄存器 `%rax` 的值给出。
- c) 算术右移指令 `shr` 的移位量既可以是一个立即数, 也可以存放在单字节寄存器 `%cl` 中。
- d) `leaq` 指令不会改变任何条件码。

- A. 1
- B. 2
- C. 3
- D. 4

答案：A

本题考察 x86-64 中的一些基本指令，答案为 A。a 项错误，原因是 `idivq` 将余数存在 `%rdx` 中，将商存在 `%rax` 里。b 项错误，间接跳转的正确书写格式应为 `jmp *%rax`。C 项错误，算术右移指令应为 `sar`。

5. 已知函数 `func` 的参数超过 6 个。当 x86-64 机器执行完指令 `call func` 之后，`%rsp` 的值为 `S`。那么 `func` 的第  $k(k > 6)$  个参数的存储地址是？
- A.  $S + 8 * (k - 6)$
  - B.  $S + 8 * (k - 7)$
  - C.  $S - 8 * (k - 6)$
  - D.  $S - 8 * (k - 7)$

答案：A

本题考察 x86-64 运行时栈帧结构，答案为 A。当执行完 `call` 指令后，`S` 处存储的是函数的返回地址；再往上依次是第 7、第 8 个函数参数...故第  $k$  个函数参数存储在  $S + 8 * (k - 6)$  的地址处。

6. X86-64 指令提供了一组条件码寄存器；其中 `ZF` 为零标志，`ZF=1` 表示最近的操作得出的结构为 0；`SF` 为符号标志，`SF=1` 表示最近的操作得出的结果为负数；`OF` 为溢出标志，`OF=1` 表示最近的操作导致一个补码溢出（正溢出或负溢出）。当我们在一条 `cmpq` 指令后使用条件跳转指令 `jg` 时，那么发生跳转等价于以下哪一个表达式的结果为 1？
- A.  $\sim(SF \wedge OF) \ \& \ \sim ZF$
  - B.  $\sim(SF \wedge OF)$
  - C.  $SF \wedge OF$
  - D.  $(SF \wedge OF) \mid ZF$

答案：A。本题考察 x86-64 条件码，答案为 A。`cmpq a, b` 相当于通过  $b - a$  的值来设置条件码。 $SF \wedge OF$  为 1 表示  $b < a$ （减法结果要么负溢出要么为负数），于是  $\sim(SF \wedge OF)$  表示  $b \geq a$ ，再与上  $b \neq a$  的条件 ( $\sim ZF$ )，就可以得到最终结果 ( $b > a$ )。

7. 考虑以下 C 语言变量声明：

```
int *(*f[3])();
```

那么在一台 x86-64 机器上，`sizeof(f)` 和 `sizeof(*f)` 的值是多少？

- A. 8 24
- B. 24 8
- C. 8 8
- D. 8 不确定

答案：B 本题考察指针和数组的存储方式，答案为 B。f 是一个数组，每个元素都是一个函数指针，指向返回值为 `int *` 的函数。对于 f 而言，`sizeof(f)` 返回整个指针数组的大小，为  $3 * 8 = 24$ ；对于 \*f 而言，它是 `f[0]` 元素，是一个指针变量，大小为 8。

8. 大多数过程的栈帧是\_\_\_\_\_的，其长度在\_\_\_\_\_时确定。（注：此处的编译指从高级语言转化为汇编语言的过程）

- A. 定长，编译
- B. 定长，汇编
- C. 可变长，汇编
- D. 可变长，运行

答案：A。

说明：大多数过程的栈帧是定长的，在过程开始时通过减小栈指针的方式分配，减小的大小由编译器在编译时计算。大家常常在汇编代码中过程的开头看到

“`subq $24, %rsp`”，就是编译器计算出了栈帧大小并写在了汇编代码里。（书 P165）

9. `pushq %rbp` 的行为等价于以下（ ）中的两条指令。

- A. `subq $8, %rsp`    `movq %rbp, (%rdx)`
- B. `subq $8, %rsp`    `movq %rbp, (%rsp)`
- C. `subq $8, %rsp`    `movq %rax, (%rsp)`
- D. `subq $8, %rax`    `movq %rbp, (%rdx)`

答案：B。

说明：x86-64 系统中，`pushq %rbp` 指令将栈指针减 8，并向其中存入 `%rbp` 寄存器的值（书 P127）

10. 下面有三组对于指令集的描述，它们分别符合 ①\_\_\_\_，②\_\_\_\_，③\_\_\_\_ 的特点。

① 某指令集中，只有两条指令能够访问内存。

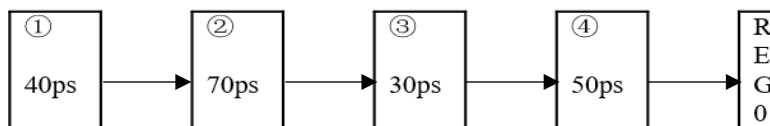
② 某指令集中，指令的长度都是 4 字节。

③ 某指令集中，可以只利用一条指令完成字符串的复制，也可以只利用一条指令查找字符串中第一次出现字母 K 的位置。

- A. CISC, CISC, CISC
- B. RISC, RISC, CISC
- C. RISC, CISC, RISC
- D. CISC, RISC, RISC

【答】B。①的访存模式单一，更加符合 RISC 的特点；②的指令长度固定，更加符合 RISC 的特点；③的指令功能丰富而复杂，更加符合 CISC 的特点。

11. 如下图所示，①~④为四个组合逻辑单元，对应的延迟已在图上标出，REG0 为一寄存器，延迟为 20ps。通过插入额外的 2 个流水线寄存器 REG1、REG2（延迟均为 20ps），可以对其进行流水化改造。改造后的流水线的吞吐率最大为 \_\_\_\_\_ GIPS。



- A. 7.69      B. 8.33      C. 10.00      D. 11.11

【答】C。额外的 2 个寄存器应当插入①②之间、②③之间，最慢的一级延迟为  $30+50+20=100\text{ps}$ ，吞吐率为  $1000/100=10\text{GIPS}$

12. 针对程序优化，请挑出下面唯一正确的陈述：

- A. 用 `add/sub` 和 `shift` 替代 `multiply/divide` 永远能提高程序的运行速度。
- B. 最有效的提高程序运行效率的方法是提高 `compiler` 的优化级别。
- C. 跨 `procedure` 优化的障碍之一是因为使用了全局变量。
- D. 程序中，`*a += *b; *a += *b;` 永远可以用 `*a += 2*(*b);` 代替。

答案：C

解析：A 错因为如果 `cpu` 支持硬件乘除，则用 `add/sub` 来模拟乘除通常并不划算。

B 错因为优化算法更能提高运行效率。

C 对

D 错因为 `a` 和 `b` 可能指向同一数据。

13. 对于 `loop-unrolling` 这种优化技巧，请指出下面哪一个陈述是错误的（一个）：

- A. `Loop-unrolling` 的原理是将尽量多的循环操作去掉相关性并重组，从而提高循环操作的并行性。
- B. `Loop-unrolling` 是一种将循环操作拆散的技术。
- C. `Loop-unrolling` 可以利用目标处理器的并行处理能力。
- D. 支持 `Loop-unrolling` 是有代价的，没有限制地增加并行支路数反而会降低运算速度。

答案：B

解析：B 是错误的因为简单地拆散 `loop` 并不能提高运行速度，只有拆散后做并行

合并才能提高运行速度。

14. 一个不含数组和指针访问的循环通常不能体现以下哪种局部性？

- A. 数据的时间局部性
- B. 数据的空间局部性
- C. 指令的时间局部性
- D. 指令的空间局部性

答案：B，见书 418 页和 419 页

15. 以下关于存储器的说法中，正确的是

- A. SRAM 单元比 DRAM 单元的能耗更大
- B. DRAM 通常用做高速缓存
- C. SDRAM 同时具备 SRAM 和 DRAM 的特点
- D. 固态硬盘的读写速度基本相当

答案：A，见书 401 页、402 页、404 页、414 页。

|    |
|----|
| 得分 |
|    |

## 第二题 整数、浮点数（10 分）

1. （6 points）假设 8-bit 整数，请填写以下表格（每空 1 分）

| 类型               | 最大的整数+1 | 39+(-127) | 39+(-127)是否溢出? |
|------------------|---------|-----------|----------------|
| Unsigned         | 二进制:    | 十进制:      |                |
| Two's Complement | 二进制:    | 十进制:      |                |

2. （4 point）假设 IEEE 754 浮点数标准，请回答以下问题（每空 1 分）

| 浮点数        | Decimal values |
|------------|----------------|
| 0x80000000 |                |
| 0x41180000 |                |

| 表达式                                                | 是否正确 |
|----------------------------------------------------|------|
| $(2 + 2^{50}) - 2^{50} \neq 2 + (2^{50} - 2^{50})$ |      |
| $2^{25} + 1 + 1 + 1 + 1 == 2^{25} + 4$             |      |

答案:

1.

| 类型               | 最大的整数+1        | 39+(-127) | 39+(-127)是否溢出? |
|------------------|----------------|-----------|----------------|
| Unsigned         | 二进制: 0000 0000 | 十进制: 168  | 是              |
| Two's Complement | 二进制: 1000 0000 | 十进制: -88  | 否              |

2.

| 浮点数        | Decimal values |
|------------|----------------|
| 0x80000000 | -0             |
| 0x41180000 | +9.5           |
| 表达式        | 是否正确           |

|                                                    |   |
|----------------------------------------------------|---|
| $(2 + 2^{50}) - 2^{50} \neq 2 + (2^{50} - 2^{50})$ | 是 |
| $2^{25} + 1 + 1 + 1 + 1 = 2^{25} + 4$              | 否 |

### IEEE 754 Floating Point Standard

The value of a real number can be represented in scientific binary notation as:

$$\text{Value} = (-1)^{\text{sign}} \times \text{Mantissa}_2 \times 2^{\text{Exponent}} = (-1)^S \times 1.M_2 \times 2^{E-\text{bias}}$$

The binary representation for floating point values uses three fields:

- **S:** encodes the *sign* of the number (0 for positive, 1 for negative)
- **E:** encodes the *exponent* in **biased notation** with a bias of  $2^{w-1}-1$
- **M:** encodes the *mantissa* (or *significand*, or *fraction*) – stores the fractional portion, but does not include the implicit leading 1.

|        | S     | E       | M       |
|--------|-------|---------|---------|
| float  | 1 bit | 8 bits  | 23 bits |
| double | 1 bit | 11 bits | 52 bits |

Associative: Only 23 bits of mantissa (M), so  $2 + 2^{50} = 2^{50}$  (2 gets rounded off). So LHS = 0, RHS = 2.

Cumulative: 1 is 25 powers of 2 away from  $2^{25}$ , so  $2^{25} + 1 = 2^{25}$ , but 4 is 23 powers of 2 away from  $2^{25}$ , so it doesn't get rounded off.

|    |
|----|
| 得分 |
|    |

### 第三题 机器级编程（15 分，每空 1 分）

下面的 C 程序包含 `main()`，`caller()`，`callee()` 三个函数。本题给出了该程序的部分 C 代码和 X86-64 汇编与机器代码。请分析给出的代码，补全空白处的内容，并回答问题。

注：汇编与机器码中的数字用 16 进制数填写

X86-64 汇编与机器代码：

答案填写处：

00000000004006cd <caller>:

|                              |       |                      |           |
|------------------------------|-------|----------------------|-----------|
| 4006cd: 55                   | push  | %rbp                 |           |
| 4006ce: 48 89 e5             | mov   | %rsp, %rbp           |           |
| 4006d1: 48 83 ec 50          | sub   | \$0x50, %rsp         |           |
| 4006d5: 48 89 7d b8          | mov   | %rdi, -0x48(%rbp)    |           |
| 4006d9: 64 48 8b 04 25 28 00 | mov   | %fs:0x28, %rax       |           |
| 4006e0: 00 00                |       |                      |           |
| 4006e2: 48 89 45 f8          | mov   | %rax, -0x8(%rbp)     |           |
| 4006e6: 31 c0                | xor   | %eax, %eax           |           |
| 4006e8: c6 45 d0 00          | movb  | \$0x0, -0x30(%rbp)   |           |
| 4006ec: c6 45 e0 00          | movb  | \$0x0, (1)           | (1) _____ |
| 4006f0: 48 8b 45 b8          | mov   | (2), %rax            | (2) _____ |
| 4006f4: 48 89 c7             | mov   | %rax, %rdi           |           |
| 4006f7:                      | callq | 400510 <strlen@plt>  |           |
| 4006fc: 89 45 cc             | mov   | (3), -0x34(%rbp)     | (3) _____ |
| 4006ff: 83 7d cc 0e          | cmpl  | \$0xe, -0x34(%rbp)   |           |
| 400703: 7f (4)               | jg    | 400752 <caller+0x85> | (4) _____ |
| 400705: 83 7d cc 09          | cmpl  | \$0x9, -0x34(%rbp)   |           |
| 400709:                      | jg    | 400720 <caller+0x53> |           |
| 40070b: 48 8b 55 b8          | mov   | -0x48(%rbp), %rdx    |           |
| 40070f: 48 8d 45 d0          | lea   | (5), %rax            | (5) _____ |
| 400713: 48 89 d6             | mov   | %rdx, %rsi           |           |
| 400716: 48 89 c7             | mov   | %rax, %rdi           |           |
| 400719:                      | callq | 400500 <strcpy@plt>  |           |
| 40071e:                      | jmp   | 40073b <caller+0x6e> |           |
| 400720: 48 8b 45 b8          | mov   | -0x48(%rbp), %rax    |           |
| 400724: 48 8d 50 0a          | lea   | 0xa(%rax), %rdx      |           |
| 400728: 48 8d 45 d0          | lea   | -0x30(%rbp), %rax    |           |
| 40072c: 48 83 c0 10          | add   | (6), %rax            | (6) _____ |



|                              |        |                               |      |           |
|------------------------------|--------|-------------------------------|------|-----------|
| 400730: 48 89 d6             | mov    | %rdx,                         | %rsi |           |
| 400733: 48 89 c7             | mov    | %rax,                         | %rdi |           |
| 400736:                      | callq  | 400500 <strcpy@plt>           |      |           |
| 40073b: ff 75 e8             | pushq  | -0x18(%rbp)                   |      |           |
| 40073e: ff 75 e0             | pushq  | -0x20(%rbp)                   |      |           |
| 400741: ff 75 d8             | pushq  | -0x28(%rbp)                   |      |           |
| 400744: ff 75 d0             | pushq  | -0x30(%rbp)                   |      |           |
| 400747: e8 (7)               | callq  | 400666 <callee>               | (7)  | _____     |
| 40074c: 48 83 c4 20          | add    | \$0x20,                       | %rsp |           |
| 400750:                      | jmp    | 400753 <caller+0x86>          |      |           |
| 400752: 90                   | nop    |                               |      |           |
| 400753: 48 8b 45 f8          | mov    | (8),                          | %rax | (8) _____ |
| 400757: 64 48 33 04 25 28 00 | xor    | %fs:0x28, %rax                |      |           |
| 40075e: 00 00                |        |                               |      |           |
| 400760:                      | je     | 400767 <caller+0x9a>          |      |           |
| 400762:                      | callq  | 400520 <__stack_chk_fail@plt> |      |           |
| 400767: c9                   | leaveq |                               |      |           |
| 400768: c3                   | retq   |                               |      |           |

C 代码:

答案填写处:

```
#include <stdio.h>
#include "string.h"
#define N (9) (9) _____
#define M (10) (10) _____
```

```
typedef union {char str_u[N]; long l;} union_e;
typedef struct {char str_s[M]; union_e u; long c;} struct_e;
```

```
void callee(struct_e s){
 char buf[M+N];
 strcpy(buf, s.str_s);
 strcat(buf, s.u.str_u);
 printf("%s\n",buf);
}
```

```

void caller(char *str){
 struct_e s;
 s.str_s[0]='\0';
 s.u.str_u[0]='\0';
 int len = strlen(str);
 if(len>= M+N)
 (11); (11)_____
 else if(len<N){
 strcpy(s.str_s, (12)); (12)_____
 }
 else{
 strcpy(s.u.str_u, (13)); (13)_____
 }
 callee(s);
}

int main(int argc, char *argv[]){
 caller("0123456789abcd");
 return 0;
}

```

caller 函数中，变量 s 所占的内存空间为: (14)\_\_\_\_\_

该程序运行后，printf 函数是否有输出？输出结果为: (15)\_\_\_\_\_

### 答案:

机器级编程（15 分，每空 1 分）

下面的 C 程序包含 main(), caller(), callee()三个函数。本题给出了该程序的部分 C 代码和 X86-64 汇编与机器代码。请分析给出的代码，补全空白处的内容，并回答问题。

注：汇编与机器码中的数字用 16 进制数填写

### X86-64 汇编与机器代码:

答案填写处:

00000000004006cd <caller>:

|                              |      |                   |
|------------------------------|------|-------------------|
| 4006cd: 55                   | push | %rbp              |
| 4006ce: 48 89 e5             | mov  | %rsp, %rbp        |
| 4006d1: 48 83 ec 50          | sub  | \$0x50, %rsp      |
| 4006d5: 48 89 7d b8          | mov  | %rdi, -0x48(%rbp) |
| 4006d9: 64 48 8b 04 25 28 00 | mov  | %fs:0x28, %rax    |

|                              |       |                      |                        |
|------------------------------|-------|----------------------|------------------------|
| 4006e0: 00 00                |       |                      |                        |
| 4006e2: 48 89 45 f8          | mov   | %rax, -0x8(%rbp)     |                        |
| 4006e6: 31 c0                | xor   | %eax, %eax           |                        |
| 4006e8: c6 45 d0 00          | movb  | \$0x0, -0x30(%rbp)   |                        |
| 4006ec: c6 45 e0 00          | movb  | \$0x0, (1)           | (1) <u>-0x28(%rbp)</u> |
| 4006f0: 48 8b 45 b8          | mov   | (2), %rax            | (2) <u>-0x48(%rbp)</u> |
| 4006f4: 48 89 c7             | mov   | %rax, %rdi           |                        |
| 4006f7:                      | callq | 400510 <strlen@plt>  |                        |
| 4006fc: 89 45 cc             | mov   | (3), -0x34(%rbp)     | (3) <u>%eax</u>        |
| 4006ff: 83 7d cc 0e          | cmpl  | \$0xe, -0x34(%rbp)   |                        |
| 400703: 7f (4)               | jg    | 400752 <caller+0x85> | (4) <u>4d</u>          |
| 400705: 83 7d cc 09          | cmpl  | \$0x9, -0x34(%rbp)   |                        |
| 400709:                      | jg    | 400720 <caller+0x53> |                        |
| 40070b: 48 8b 55 b8          | mov   | -0x48(%rbp), %rdx    |                        |
| 40070f: 48 8d 45 d0          | lea   | (5), %rax            | (5) <u>-0x30(%rbp)</u> |
| 400713: 48 89 d6             | mov   | %rdx, %rsi           |                        |
| 400716: 48 89 c7             | mov   | %rax, %rdi           |                        |
| 400719:                      | callq | 400500 <strcpy@plt>  |                        |
| 40071e:                      | jmp   | 40073b <caller+0x6e> |                        |
| 400720: 48 8b 45 b8          | mov   | -0x48(%rbp), %rax    |                        |
| 400724: 48 8d 50 0a          | lea   | 0xa(%rax), %rdx      |                        |
| 400728: 48 8d 45 d0          | lea   | -0x30(%rbp), %rax    |                        |
| 40072c: 48 83 c0 10          | add   | (6), %rax            | (6) <u>0x8</u>         |
| 400730: 48 89 d6             | mov   | %rdx, %rsi           |                        |
| 400733: 48 89 c7             | mov   | %rax, %rdi           |                        |
| 400736:                      | callq | 400500 <strcpy@plt>  |                        |
| 40073b: ff 75 e8             | pushq | -0x18(%rbp)          |                        |
| 40073e: ff 75 e0             | pushq | -0x20(%rbp)          |                        |
| 400741: ff 75 d8             | pushq | -0x28(%rbp)          |                        |
| 400744: ff 75 d0             | pushq | -0x30(%rbp)          |                        |
| 400747: e8 (7)               | callq | 400666 <callee>      | (7) <u>1a ff ff ff</u> |
| 40074c: 48 83 c4 20          | add   | \$0x20, %rsp         |                        |
| 400750:                      | jmp   | 400753 <caller+0x86> |                        |
| 400752: 90                   | nop   |                      |                        |
| 400753: 48 8b 45 f8          | mov   | (8), %rax            | (8) <u>-0x8(%rbp)</u>  |
| 400757: 64 48 33 04 25 28 00 | xor   | %fs:0x28, %rax       |                        |
| 40075e: 00 00                |       |                      |                        |

|            |        |                               |
|------------|--------|-------------------------------|
| 400760:    | je     | 400767 <caller+0x9a>          |
| 400762:    | callq  | 400520 <__stack_chk_fail@plt> |
| 400767: c9 | leaveq |                               |
| 400768: c3 | retq   |                               |

C 代码:

答案填写处:

```
#include <stdio.h>
```

```
#include "string.h"
```

```
#define N (9)
```

(9) 10

```
#define M (10)
```

(10) 5

```
typedef union {char str_u[N]; long l;} union_e;
```

```
typedef struct {char str_s[M]; union_e u; long c;} struct_e;
```

```
void callee(struct_e s){
```

```
 char buf[M+N];
```

```
 strcpy(buf, s.str_s);
```

```
 strcat(buf, s.u.str_u);
```

```
 printf("%s\n",buf);
```

```
}
```

```
void caller(char *str){
```

```
 struct_e s;
```

```
 s.str_s[0]='\0';
```

```
 s.u.str_u[0]='\0';
```

```
 int len = strlen(str);
```

```
 if(len>= M+N)
```

```
 (11);
```

(11) return

```
 else if(len<N){
```

```
 strcpy(s.str_s, (12));
```

(12) str

```
 }
```

```
 else{
```

```
 strcpy(s.u.str_u, (13));
```

(13) str+M

```
 }
```

```
 callee(s);
```

```
}
```

```
int main(int argc, char *argv[]){
 caller("0123456789abcd");
 return 0;
}
```

caller 函数中，变量 s 所占的内存空间为:

(14) 32 字节

该程序运行后，printf 函数是否有输出？输出结果为:

(15) abcd

|    |
|----|
| 得分 |
|    |

#### 第四题（15 分）

请分析 Y86-64 ISA 中新加入的一族算术指令： `irOpq V, rA, rB`，其格式如下：

|   |    |    |    |          |
|---|----|----|----|----------|
| C | Fn | rA | rB | V (8 字节) |
|---|----|----|----|----------|

与 `Opq` 类似，这族指令由四个指令组成，分别是 `iraddq`，`irsubq`，`irandq` 和 `irxorq`。其功能为：计算  $R[rA] \text{ OP } V$  并将结果存入  $R[rB]$  中，这里 `OP` 根据 `Fn` 的取值分别取 `+`，`-`，`&` 和 `^`，且此过程会设置条件码寄存器。

1. 若在教材所描述的 SEQ 处理器上执行这条指令，请按下表补全每个阶段的操作。需说明的信号可能会包括：`icode`，`ifun`，`rA`，`rB`，`valA`，`valB`，`valC`，`valE`，`valP`，`Cnd`；the register file `R[]`，data memory `M[]`，Program counter `PC`，condition codes `CC`。其中对存储器的引用必须标明字节数。如果在某一阶段没有任何操作，请填写 `none` 指明。（6 分）

| Stage      | <code>irOpq V, rA, rB</code>                                                                                                                                              |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Fetch      | <code>icode : ifun ← M<sub>1</sub>[PC]</code><br><code>rA : rB ← M<sub>1</sub>[PC + 1]</code><br><code>valC ← M<sub>8</sub>[PC + 2]</code><br><code>valP ← PC + 10</code> |
| Decode     | <code>valA ← R[rA]</code><br><code>valB ← R[rB]</code>                                                                                                                    |
| Execute    | <code>valE ← valA OP valC</code><br><code>Set CC</code>                                                                                                                   |
| Memory     | <code>none</code>                                                                                                                                                         |
| Write Back | <code>R[rB] ← valE</code>                                                                                                                                                 |
| Update PC  | <code>PC ← valP</code>                                                                                                                                                    |

2. 考虑如下一段 Y86-64 代码片段：

```

Loop: mrmovq (%rdi), %r10 # line 1
 rmmovq %r10, (%rsi) # line 2
 andq %r10, %r10 # line 3
 jle Npos # line 4
 irmovq $1, %r10 # line 5

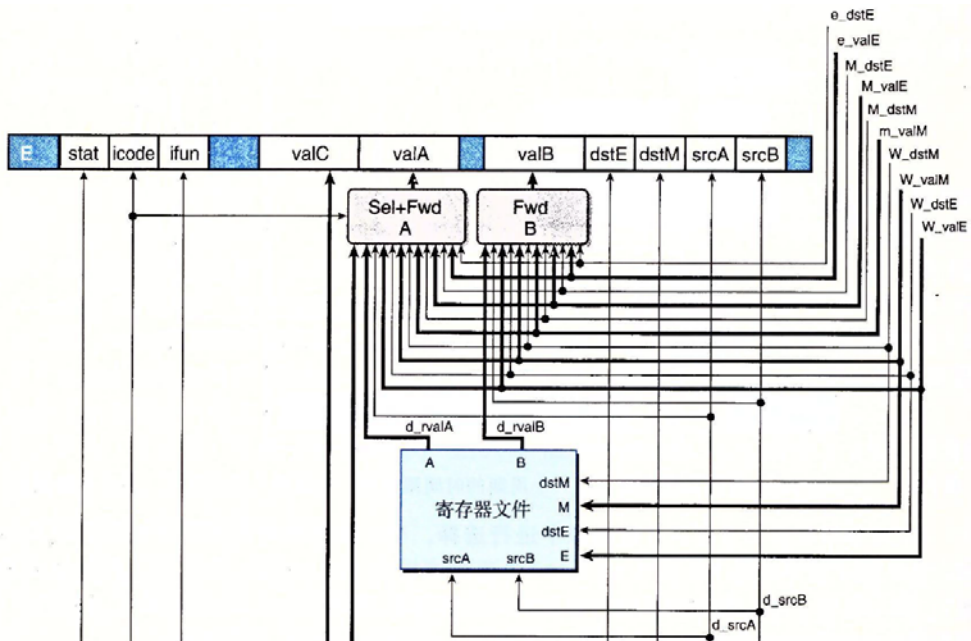
```

```

 addq %r10, %rax # line 6
Npos: irmovq $1, %r10 # line 7
 subq %r10, %rdx # line 8
 irmovq $8, %r10 # line 9
 addq %r10, %rdi # line 10
 addq %r10, %rsi # line 11
 andq %rdx, %rdx # line 12
 jg Loop # line 13
 ret # line 14

```

(1) 这段代码中存在一些指令间的数据相关，其中行 5 与行 6 的数据相关可以采用数据前递 (Forwarding) 技术解决，在下图中体现在 Sel+FwdA 和 FwdB 部件上。前者输出的信号会存到流水线寄存器 E 的 valA 域 (即 E\_valA 信号)，请选出该信号正确的 HCL 语言描述: B (2 分)



```

long d_valA = [
 D_icode in { ICALL, IJXX }: D_valP;
 _____;
 _____;
 _____;
 _____;
 _____;
 1: d_rvalA;

```

- ① d\_srcA == e\_dstE : e\_valE
- ② d\_srcA == M\_dstE : M\_valE
- ③ d\_srcA == M\_dstM : m\_valM
- ④ d\_srcA == W\_dstE : W\_valE
- ⑤ d\_srcA == W\_dstM : W\_valM

A ①②③④⑤      B ①③②⑤④      C ④⑤②③①      D ⑤④③②①

(2) 同样是数据相关，上述代码中行 1 与行 2 的情况不能用以上方法解决。为了检测这种情况，需要增加逻辑电路，用 HCL 语言表达如下：(3 分)

E\_icode in {IMRMOVQ, IPOPQ} && E\_dstM in {d\_srcA, d\_srcB}

(3) 假设该代码片段在教材所描述的 PIPE 处理器上运行，不考虑该片段代码前后代码的影响以及高速缓存 (cache) 失效的情况，假设 %rdx 初值为 10，%rdi 指向的内存中数组的元素均为正数，处理器设计使用总是选择 (always taken) 的预测策略。该代码片段预计运行 167 周期，若使用新增加的 irOpq 指令来优化这段代码，可以节省 30 周期的运行时间。(4 分)



|    |
|----|
| 得分 |
|    |

第五题（15 分）

1) 不同的 Cache 组织结构对访存性能有很大的影响。以直接映射 Cache 为例，L1 Cache 的容量为 128 字节，block size 为 16 个字节。为简化起见，使用 8 位地址访问该 Cache，地址如下图所示：

|     |       |   |   |              |   |   |   |
|-----|-------|---|---|--------------|---|---|---|
| 7   | 6     | 5 | 4 | 3            | 2 | 1 | 0 |
| tag | index |   |   | Block offset |   |   |   |

对于如下访问序列，请填写 Cache 中 Tag Ram 的变化，其中 B0 ~ B7 对应 8 个 Cache Block，表项中填写对应的 Tag。（6 分）

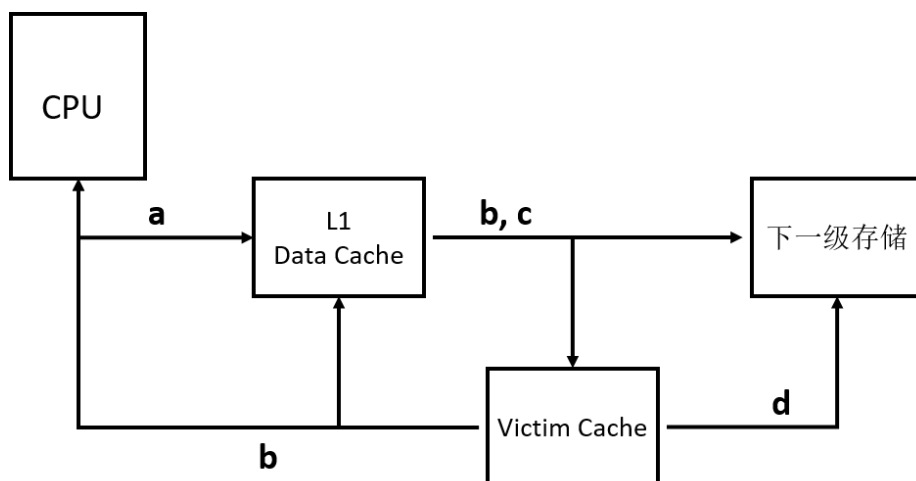
| 访问地址序列 | L1 Cache (Tag Ram) |    |    |    |    |    |    |    | 是否命中<br>(Y / N) |
|--------|--------------------|----|----|----|----|----|----|----|-----------------|
|        | B0                 | B1 | B2 | B3 | B4 | B5 | B6 | B7 |                 |
| A0     |                    |    | 1  |    |    |    |    |    | N               |
| 20     |                    |    | 0  |    |    |    |    |    | N               |
| A2     |                    |    | 1  |    |    |    |    |    | N               |
| 0      | 0                  |    |    |    |    |    |    |    | N               |
| 80     | 1                  |    |    |    |    |    |    |    | N               |
| 4      | 0                  |    |    |    |    |    |    |    | N               |

按行得分，每行 1 分。

填写变化，若空白处填写错误，该行不得分。

2) 由 1) 可见，直接映射结构的 Cache 失效率高。为改善性能，N. Jouppi 提出 Victim Cache 结构，即在 L1 Cache 之外接入一个很小的全相联 Cache 来为替换出 L1 Cache 的数据做备份，具体结构如下图，工作原理如下所述。

- 如果 L1 Cache 命中，则数据由 L1 Cache 传给 CPU；
- 如果 L1 Cache 失效，则访问 Victim Cache。如果在 Victim Cache 中命中，则数据由 Victim Cache 传给 CPU，并写回 L1 Cache，否则访问下一级存储；
- 如果 L1 Cache 发生替换，替换出去的数据放入全相联结构的 Victim Cache 中；
- Victim Cache 采用 FIFO（先进先出）替换算法，从 Victim Cache 中替换出去的数据，如果发生修改，直接写回下一级存储。



在题 1) 的基础上，为其添加一个 32 字节的全相联 Victim Cache，仍然使用题 1) 中的访问序列，请填写 Victim Cache 中的 Tag Ram 变化，并计算命中率。（9 分）

| 访问地址序列 | Victim Cache (Tag Ram) |      |                 |
|--------|------------------------|------|-----------------|
|        | B0                     | B1   | 是否命中<br>(Y / N) |
| A0     |                        |      | N               |
| 20     | 1010                   |      | N               |
| A2     |                        | 0010 | Y               |
| 0      |                        |      | N               |
| 80     | 0000                   |      | N               |
| 4      |                        | 1000 | Y               |

按行得分，每行 1 分。若空白处填写错误，该行不得分。

B0 和 B1 互换，也算正确。

若首行填写内容，本表格不得分。tag 应为二进制，否则本表格不得分。填写的 tag 位数错误，本表格不得分。

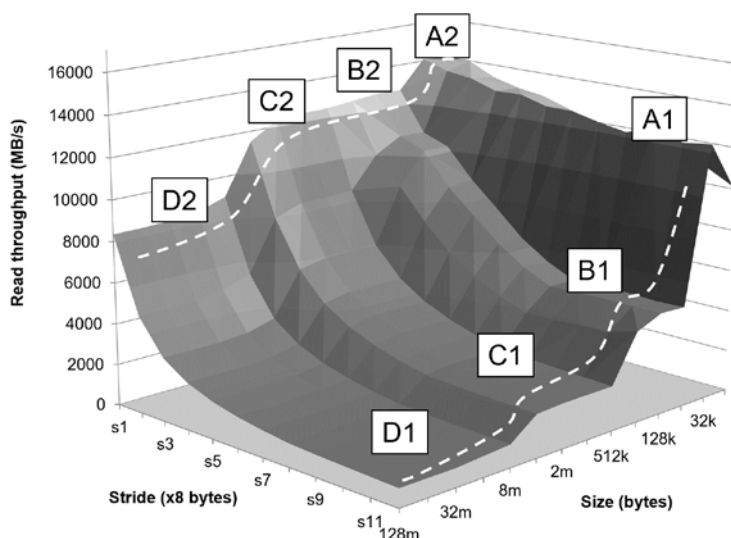
命中率为  $2/6 = 1/3$  或 33.3%（3 分）

|    |
|----|
| 得分 |
|    |

### 第六题（15 分）

下面这段代码可以用于评测计算机系统的存储系统性能。在某款现代的个人计算机上，采用该评测程序的不同的配置参数（数据集大小和访问步长），得到了如下图所示的性能表现。

```
long data[MAXElems]; /* Global array to traverse */
/* test - Iterate over first "elems" elements of
 * array "data" with stride of "stride", using
 * using 4x4 loop unrolling.
 */
int test(int elems, int stride) {
 long i, sx2=stride*2, sx3=stride*3, sx4=stride*4;
 long acc0 = 0, acc1 = 0, acc2 = 0, acc3 = 0;
 long length = elems, limit = length - sx4;
 /* Combine 4 elements at a time */
 for (i = 0; i < limit; i += sx4) {
 acc0 = acc0 + data[i];
 acc1 = acc1 + data[i+stride];
 acc2 = acc2 + data[i+sx2];
 acc3 = acc3 + data[i+sx3];
 }
 /* Finish any remaining elements */
 for (; i < length; i++) {
 acc0 = acc0 + data[i];
 }
 return ((acc0 + acc1) + (acc2 + acc3));
}
```



(1) 图中 A1-D1 这几个点有一些较为平缓的“平台”，这主要体现了访存行为的

什么特性? 时间局部性 (2分)

(2) 如果图中 D1 体现了内存的性能, 那 A1、B1、C1 分别体现存储层次结构中的什么部件的性能: A1: L1 Cache、B1: L2 Cache、C1: L3 Cache。

(每空 1 分)

其中, B1 所体现的部件, 容量可能是多大? b (2分)

a. 32KB   b. 256KB   c. 1MB   d. 8MB

C1 所体现的部件, 容量可能是多大? d (2分)

a. 32KB   b. 256KB   c. 1MB   d. 8MB

(3) 图中 B2 和 B1、C2 和 C1 之间的“斜坡”, 这主要是由于访存行为的什么特性导致的? 空间局部性 (2分)

(4) 图中 B2 和 C2 的性能指标差别不大, 不像 B1 和 C1 之间的差距那么大, 主要是什么原因导致的? 高速缓存的预取功能 (2分)

(5) 实际评测时, 需要把 test() 函数先运行一遍, 然后再运行一次获取评测数据, 这是为了避免什么现象的影响? 高速缓存的 cold miss (2分)

装  
订  
线  
内

不  
要  
答  
题

# 北京大学信息科学技术学院考试试卷

考试科目： 计算机系统导论 姓名： \_\_\_\_\_ 学号： \_\_\_\_\_

考试时间： 2014 年 1 月 7 日 任课教师： \_\_\_\_\_

| 题号  | 一 | 二 | 三 | 四 | 五 | 六 | 七 | 八 | 总分 |
|-----|---|---|---|---|---|---|---|---|----|
| 分数  |   |   |   |   |   |   |   |   |    |
| 阅卷人 |   |   |   |   |   |   |   |   |    |

## 北京大学考场纪律

1、考生进入考场后，按照监考老师安排隔位就座，将学生证放在桌面上。无学生证者不能参加考试；迟到超过 15 分钟不得入场。在考试开始 30 分钟后方可交卷出场。

2、除必要的文具和主考教师允许的工具书、参考书、计算器以外，其它所有物品（包括空白纸张、手机、或有存储、编程、查询功能的电子用品等）不得带入座位，已经带入考场的必须放在监考人员指定的位置。

3、考试使用的试题、答卷、草稿纸由监考人员统一发放，考试结束时收回，一律不准带出考场。若有试题印制问题请向监考教师提出，不得向其他考生询问。提前答完试卷，应举手示意请监考人员收卷后方可离开；交卷后不得在考场内逗留或在附近高声交谈。未交卷擅自离开考场，不得重新进入考场答卷。考试结束时间到，考生立即停止答卷，在座位上等待监考人员收卷清点后，方可离场。

4、考生要严格遵守考场规则，在规定时间内独立完成答卷。不准交头接耳，不准偷看、夹带、抄袭或者有意让他人抄袭答题内容，不准接传答案或者试卷等。凡有违纪作弊者，一经发现，当场取消其考试资格，并根据《北京大学本科考试工作与学术规范条例》及相关规定严肃处理。

5、考生须确认自己填写的个人信息真实、准确，并承担信息填写错误带来的一切责任与后果。

学校倡议所有考生以北京大学学生的荣誉与诚信答卷，共同维护北京大学的学术声誉。

以下为试题和答题纸，共    页。

|    |
|----|
| 得分 |
|    |

第一题 单项选择题（每小题 1.5 分，共 30 分）

1、对于 IEEE 浮点数，如果减少 1 位指数位，将其用于小数部分，将会有怎样的效果？答：（ ）

- A. 能表示更多数量的实数值，但实数值取值范围比原来小了。
- B. 能表示的实数数量没有变化，但数值的精度更高了。
- C. 能表示的最大实数变小，最小的实数变大，但数值的精度更高。
- D. 以上说法都不正确。

答案：C

因为 NaN 更多，所以能表示的实数值更少。

指数小了，所以最大实数变小，最小实数变大。

小数部分增加了，所以精度更高。

2、按照教材描述的原则，对于 x86\_64 程序，在 callq 指令执行后，函数的第一个参数一般存放在哪里？答：（ ）

- A. 8(%rsp)      B. 4(%rsp)      C. %rax      D. %rdi

答案：D

考察学生对 x86\_64 与 IA-32 在函数参数传递上的不同。

3、已知变量 x 的值已经存放在寄存器 eax 中，现在想把  $5x+7$  的值计算出来并存放到寄存器 ebx 中，如果不允许用乘法和除法指令，则至少需要多少条 IA-32 指令完成该任务？答：（ ）

- A. 1 条      B. 3 条      C. 2 条      D. 4 条

答案：A

考察学生对 leal 指令的用法是否熟悉。

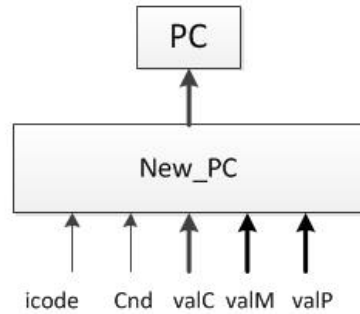
使用一条指令 “leal 7(%eax, %eax, 4), %ebx” 即可。

4、在 Y86 的 SEQ 实现中，PC (Program Counter, 程序计数器) 更新的逻辑结构如下图所示，请根据 HCL 描述为①②③④选择正确的数据来源。

```

Int new_pc = [
 # Call.
 Icode = ICALL : ①;
 # Taken branch.
 Icode = IJXX && Cnd : ②;
 # Completion of RET instruction.
 Icode = IRET : ③;
 # Default.
 1 : ④;

```



其中: Icode 为指令类型, Cnd 为条件是否成立, valC 表示指令中的常数值, valM 表示来自返回栈的数据, valP 表示 PC 自增。

- ①   A  : A) valC B) valM C) valP  
 ②   A  : A) valC B) valM C) valP  
 ③   B  : A) valC B) valM C) valP  
 ④   C  : A) valC B) valM C) valP

(5~6) 如果直接映射高速缓存 (Cache) 的大小是 4KB, 并且块大小 (block) 大小为 32 字节。

5、请问它每路 (way) 有多少行 (line)? 答: (      )

- A. 128                  B. 64                  C. 32                  D. 1

**答案: 选 A, 容量=路容量\*路数=cache 行大小\*Set 数\*路数->set 数=4KB/32B=2<sup>7</sup>=128**

6、如果数据访问的地址序列为 0->4->16->132->232->4096->160 (以字节为单位), 请问一共发生多少次替换? 答: (      )

- A. 0                  B. 1                  C. 2                  D. 3

**答案: 选 B, 连续访问 set 数相同的数据会发生替换, set 号有地址 5~11 位决定, 等于地址 A mod (128\*32), 只有 0 与 4096 位于同一 cache 行**

7、下列程序运行的结果是什么? 答: (      )

```

/* main.c */
int i=0;
int main()
{
 foo();
 return 0;
}

```

```

/* foo.c */
int i=1;
void foo()
{
 printf("%d", i);
}

```

- A. 编译错误
- B. 链接错误
- C. 段错误
- D. 有时打印输出 1, 有时打印输出 0;

**答案: B** 考点: 链接的基本概念, 符号解析的原则;

**注: 由于印刷版中引号的问题 (现已修正), 本题选 A 也算对**

8、在链接时, 对于什么样的符号一定不需要进行重定位? 答: ( )

- A. 不同 C 语言源文件中定义的函数
- B. 同一 C 语言源文件中定义的全局变量
- C. 同一函数中定义时不带 static 的变量
- D. 同一函数中定义时带有 static 的变量

**答案: C**

**说明: 考察需要进行重定位的条件。A 在链接前不在同一目标文件中, BD 都位于 .data 段中, 这些都需要重定位。C 位于栈中或者寄存器中, 不需要重定位。**

9、关于信号的描述, 以下不正确的是哪一个? 答: ( )

- A. 在任何时刻, 一种类型至多只会会有一个待处理信号
- B. 信号既可以发送给一个进程, 也可以发送给一个进程组
- C. SIGTERM 和 SIGKILL 信号既不能被捕获, 也不能被忽略
- D. 当进程在前台运行时, 键入 Ctrl-C, 内核就会发送一个 SIGINT 信号给这个前台进程

**答案: C**

**说明: 考察信号的基本了解, C 应该为“SIGKILL 和 SIGSTOP 信号既不能被捕获, 也不能被忽略”**

10、下面关于非局部跳转的描述, 正确的是 ( )

- A. setjmp 可以和 siglongjmp 使用同一个 jmp\_buf 变量
- B. setjmp 必须放在 main() 函数中调用
- C. 虽然 longjmp 通常不会出错, 但仍然需要对其返回值进行出错判断



D. 在同一个函数中既可以出现 `setjmp`，也可以出现 `longjmp`

答案：D

说明：考察非局部跳转的基本了解；D 选项：可以采用不同的 `jmp_buf`

11、假设有一台 64 位的计算机的物理页块大小是 8KB，采用三级页表进行虚拟地址寻址，它的虚拟地址的 VPO (Virtual Page Offset, 虚拟页偏移) 有 13 位，问它的虚拟地址的 VPN (Virtual Page Number, 虚拟页号码) 有多少位？

答：( )

A. 20                      B. 27                      C. 30                      D. 33

答案：选 C。

本题考查对页表组成的理解。

页块大小为 8KB，即  $2^{13}$  byte。在 64 位机器上，一个页表条目为 8byte。故共有页表条目  $2^{10}$  项，故每一级页表可以表示 10 位地址。因此三级页表存储，共需要  $10 \times 3 = 30$  位。

12、进程 P1 通过 `fork()` 函数产生一个子进程 P2。假设执行 `fork()` 函数之前，进程 P1 占用了 53 个（用户态的）物理页，则 `fork` 函数之后，进程 P1 和进程 P2 共占用\_\_\_\_\_个（用户态的）物理页；假设执行 `fork()` 函数之前进程 P1 中有一个可读写的物理页，则执行 `fork()` 函数之后，进程 P1 对该物理页的页表项权限为\_\_\_\_\_。上述两个空格对应内容应该是 ( )

A. 53，读写    B. 53，只读    C. 106，读写    D. 106，只读

答案：选 B。

`fork` 使用之后，只生成新的 `mm_struct` 等结构，不会将原来的数据也拷贝一份，而是采用了 `copy-on-write` 的策略，因而占用的物理内存不会是原来的两倍，同时这两个进程的页表项权限都只有读。

13、下列哪个例子是外部碎片？答：( )

- A. 分配块时为了字节对齐而多分配的空间
- B. 空闲块中互相指向的指针所占据的空间
- C. 多次释放后形成的不连续空闲块
- D. 用户分配后却从未释放的堆空间

答案：C

14、用带有 header 和 footer 的隐式空闲链表实现分配器时，如果一个应用请求一个 3 字节的块，下列说法哪一项是错误的？答：（ ）

- A. 搜索空闲链表时，存储利用率为：best fit > next fit > first fit
- B. 搜索空闲链表时，吞吐率为：next fit > first fit > best fit
- C. 在 x86 机器上，malloc(3) 实际分配的空闲块大小可能为 8 字节
- D. 在 x64 机器上，malloc(3) 返回的地址可能为 2147549777

**答案：D，malloc 在 64 位机器上返回的地址应该按 16 字节对齐。**

15、考虑如下代码，假设 result.txt 的初始内容是“123”。

```
int main(int argc, char** argv)
{
 int fd1 = open("result.txt", O_RDWR);
 char str[] = "abc";
 char c;
 write(fd1, str, 1);
 read(fd1, &c, 1);
 write(fd1, &c, 1);
 return 0;
}
```

在这段代码执行完毕之后，result.txt 的内容是什么？（假设所有的系统调用都会成功）答：（ ）

- A. a22      B. a21      C. a13      D. abb

**答案：A。考察文件操作过程中指针的移动情况。**

16、已知如下代码段

```
write(fd1, str1, strlen(str1));
write(fd2, str2, strlen(str2));
```

可以在原本为空的文件 ICS.txt 中写下字符串 I love ICS!

对于下面这些对于变量 fd1, fd2, str1, str2 的定义：

(1)

```
int fd1 = open("ICS.txt", O_RDWR);
int fd2 = open("ICS.txt", O_RDWR);
char *str1 = "I love ";
char *str2 = "ICS!";
```

(2)

```
int fd1 = open("ICS.txt", O_RDWR);
int fd2 = dup(fd1);
```

```

char *str1 = "I love ";
char *str2 = "ICS!";
(3)
int fd1 = open("ICS.txt", O_RDWR);
int fd2 = open("ICS.txt", O_RDWR);
char *str1 = "I love ";
char *str2 = "I love ICS!";
(4)
int fd1 = open("ICS.txt", O_RDWR);
int fd2 = dup(fd1);
char *str1 = "I love ";
char *str2 = "I love ICS!";

```

下面哪一个组合是正确的：( )

- A. (1) (4)      B. (2) (3)      C. (1) (2) (3) (4)      D. 都不正确

**答案：B**

**说明：考察两种不同文件描述符指向同一文件v的不同，一种是指向了同一的 open file table，一种是指向了同一的 v-node table。**

17、下列关于计算机网络概念的说法中，哪一项是正确的？答：( )

- A. 全球最大的计算机网络是互联网 Internet，所以计算机网络协议是 Internet Protocol 即 IP 协议。
- B. 计算机之间的网络通信是一个机器上的一个 process（如 client process）与另一个机器上的 process（如 server process）之间的通信。
- C. 网络应用程序有默认的端口号，大部分应用的端口号可以修改，而少部分知名应用如 Web 服务程序的端口号 80 是无法修改的。
- D. 一个域名只能对应一个 IP 地址；而一个 IP 地址可以对应多个域名。

**答案：B**

**解释：考察第一部分**

**A. IP 只是计算机网络众多协议中的一种；IP 是具有代表性的一种协议；但还有其他很多协议。**

**C. 端口号是可以修改的，默认端口号只是为了方便**

**D. 一个域名可以对应多个 IP；一个 IP 也可以对应多个域名。**

18、在 client-server 模型中，一个连接（connection）可以由 IP 地址，

端口号的组合来表示。假设一个访问网页服务器的应用，客户端 IP 地址为 128.2.194.242，目标服务器端 IP 地址为 208.216.181.15，用户设置的代理服务器 IP 地址为 155.232.108.39。目标服务器同时提供网页服务（默认端口 80），和邮件服务（默认端口 25）。当客户端向目标服务器发送访问网页的请求时，下面 connection socket pairs 正确的一组是？答：（ ）

|   | 客户端请求                                       | 代理请求                                         |
|---|---------------------------------------------|----------------------------------------------|
| A | (128.2.194.242:25,<br>155.232.108.39:80)    | (128.2.194.242:51213,<br>208.216.181.15:80)  |
| B | (128.2.194.242:51213,<br>155.232.108.39:80) | (128.2.194.242:12306,<br>208.216.181.15:80)  |
| C | (128.2.194.242:25,<br>208.216.181.15:80)    | (155.232.108.39:51213,<br>208.216.181.15:80) |
| D | (128.2.194.242:51213,<br>208.216.181.15:80) | (155.232.108.39:12306,<br>208.216.181.15:80) |

答案：D

解释：考察所有三部分内容

（客户端请求）这一段，客户端发送的请求，源地址是客户自己地址 128.2.194.242，端口号是临时端口号；目标地址仍是服务器地址 208.216.181.15:80 默认端口 80

（代理请求）这一段，代理发送的请求，源地址是代理自己地址 155.232.108.39，端口号是临时端口号；目标地址仍是服务器地址 208.216.181.15:80 默认端口 80。

所以，选项 D 正确。

（选项 A/B 中，目标地址不是代理的地址，而是目标服务器的地址。）

（选项 C 中，端口号 25 属于电子邮件应用默认端口号，不作为客户端的临时端口号。）

19、在 Pthread 线程包使用中，下列代码输出正确的是：（ ）

```
void *th_f(void * arg)
{
 printf("Hello World") ;
 pthread_exit(0) ;
}

int main(void)
{
```

```

pthread_t tid;
int st;
st = pthread_create(&tid, NULL, th_f, NULL);
if(st<0) {
 printf("Oops, I can not create thread\n");
 exit(-1);
}
sleep(1);
exit(0);
}

```

- A. Oops, I can not create thread
- B. Hello World
- Oops, I can not create thread
- C. Hello World
- D. 不输出任何信息

答案: C

20、有四个信号量，初值分别为: a=1, b=1, c=1, d=1

| 线程 1:   | 线程 2:   | 线程 3:   |
|---------|---------|---------|
| ① P(a); | ⑨ P(c); | ⑮ P(d); |
| ② P(b); | ⑩ P(b); | ⑯ P(a); |
| ③ P(d); | ⑪ V(c); | ⑰ V(d); |
| ④ P(c); | ⑫ V(b); | ⑱ P(b); |
| ⑤ V(d); | ⑬ P(a); | ⑲ V(a); |
| ⑥ V(a); | ⑭ V(a); | ⑳ V(b); |
| ⑦ V(b); |         |         |
| ⑧ V(c); |         |         |

上面的程序执行时，下列哪些执行轨迹不会产生死锁？

- A. ①②③⑨⑩⑮④
- B. ⑨①②③⑩
- C. ①②⑮⑨③
- D. ⑨①⑮②③

答案: 实际上四个选项都有问题，无正确答案，因此批卷时本题直接算作得分

|    |
|----|
| 得分 |
|    |

第二题 (8 分)

考虑一种新的遵从 IEEE 规范的浮点的格式，包含 3 位指数位，4 位小数位和 1 位符号位。请填写下面的表格。

| 描述     | 十进制数 (或分数) | 二进制表示      |
|--------|------------|------------|
| Bias   | 3          | -----      |
| 最小的正数  | 1/64       | 0 000 0001 |
| 最小的有限数 | -15.5      | 1 110 1111 |
| 最小规格正数 | 1/4        | 0 001 0000 |
| ---    | -11/32     | 1 001 0110 |
| ---    | 7/2        | 0 100 1100 |
| ---    | 63/128     | 0 010 0000 |
| ---    | 13.25      | 0 110 1010 |

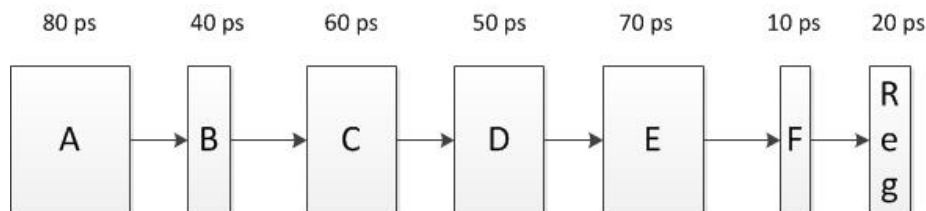
每行 1 分

考察学生对浮点数的掌握程度，包括舍入。

|    |
|----|
| 得分 |
|    |

### 第三题 （12 分）

如图所示，每个模块表示一个单独的组合逻辑单元，每个单元的延迟已在图中标出。通过在两个单元间添加寄存器的方式，可以对该数据通路进行流水化改造。假设每个寄存器的延迟为 20ps。



1) 如果改造为一个二级流水线（只插入一个寄存器），为获得最大的吞吐率，该寄存器应在哪里插入？请计算该流水线的吞吐率，并说明计算过程。

**插入在 CD 间（1 分）**

**$1000 / (80 + 40 + 60 + 20) = 1000 / 200 = 5 \text{ GIPS}$ （过程 1 分，结果 1 分）**

2) 如果改造为一个三级流水线（插入两个寄存器），为获得最大的吞吐率，寄存器应在哪里插入？请计算该流水线的吞吐率，并说明计算过程。

**插入在 BC 间和 DE 间（1 分）**

**$1000 / (80 + 40 + 20) = 1000 / 140 = 7.143 \text{ GIPS}$ （过程 1 分，结果 1 分）**

3) 如果改造为一个四级流水线（插入三个寄存器），为获得最大的吞吐率，寄存器应在哪里插入？请计算该流水线的吞吐率，并说明计算过程。

**插入 AB 间、CD 间、DE 间（1 分）**

**$1000 / (100 + 20) = 1000 / 120 = 8.33 \text{ GIPS}$ （过程 1 分，结果 1 分）**

4) 不改变单元划分，为获得最大性能，该设计至少需要划分成几级？请计算对应的吞吐率，并说明计算过程。

**至少划分成 5 级（1 分）**

**$1000 / (80 + 20) = 10 \text{ GIPS}$ （过程 1 分，结果 1 分）**

|    |
|----|
| 得分 |
|    |

#### 第四题 (10 分)

考虑如下两个程序 (fact1.c和fact2.c) :

```

/* fact1.c */
#define MAXNUM 12
int table[MAXNUM];
int fact(int n);
int main(int argc, char **argv) {
 int n;
 table[0] = 0;
 table[1] = 1;
 if (argc == 1) {
 printf("Error: missing argument\n");
 exit (0);
 }
 argv++;
 if (sscanf(*argv, "%d", &n) != 1 || n < 0 || n >= MAXNUM)
 {
 printf ("Error: %s not an int or out of range\n",
*argv);
 exit (0);
 }
 printf("fact(%d) = %d\n", n, fact(n));
}

/* fact2.c */
int* table;
int fact(int n) {
 static int num = 2;
 if (n >= num) {
 int i = num;
 while (i <= n) {
 table[i] = table[i-1] * i;
 i++;

```



```

 }
 num = i;
}
return table[n];
}

```

(1) 对于每个程序中的相应符号，给出它的属性（局部变量、强全局变量或弱全局变量），以及它在链接后位于 ELF 文件中的什么位置？（提示：如果某表项中的内容无法确定，请画 X）（6 分）

fact1.c

| 变量    | 类型                 | ELF Section      |
|-------|--------------------|------------------|
| table | <b>weak global</b> | <b>.bss</b>      |
| fact  | <b>weak global</b> | <b>X (或.bss)</b> |
| num   | <b>X</b>           | <b>X</b>         |

fact2.c

| 变量    | 类型                   | ELF Section  |
|-------|----------------------|--------------|
| table | <b>weak global</b>   | <b>.bss</b>  |
| fact  | <b>strong global</b> | <b>.text</b> |
| num   | <b>local</b>         | <b>.data</b> |

(2) 对上述两个文件进行链接之后，会对每个符号进行解析。请给出链接后下列符号被定义的模块（fact1 or fact2）。（2 分）

|       | 定义模块         |
|-------|--------------|
| table | <b>不确定</b>   |
| fact  | <b>fact2</b> |
| num   | <b>fact2</b> |

(3) 使用 gcc（命令：gcc -o fact fact1.c fact2.c）来编译之后得到的可执行文件是否能够正确执行？为什么？（2 分）

**不一定能正确执行。因为 table 在两个文件中都是 weak global，因此链接器会任选一个定义来解析 table。而因为在 fact2 模块中只给 table 预留了一个单字（4 字节）空间，因此如果选择了 fact2 来解析 table 的话，会出现 segmentation fault。【该问题已经过编译检查确认。】**

**考察知识点：linking 的基本概念、符号属性、链接的过程、符号的解析等。**

|    |
|----|
| 得分 |
|    |

### 第五题 (10 分)

#### Part I

请阅读以下程序，然后回答问题（假设程序中的函数调用都可以正确执行）：

```
int main() {
 printf("A\n");
 if (fork() == 0) {
 printf("B\n");
 }
 else {
 printf("C\n");
 A
 }
 printf("D\n");
 exit(0);
}
```

(1) 如果程序中的 A 位置的代码为空，列出所有可能的输出结果：(1 分)

**4 个：分别是 ABDCD ABCDD ACBDD ACDBD（错一个扣半分，多了也扣半分，最多扣 1 分）**

(2) 如果程序中的 A 位置的代码为：

```
waitpid(-1, NULL, 0);
```

列出所有可能的输出结果：(2 分)

**3 个：分别是 ABDCD ABCDD ACBDD（每个半分，多了扣一分，最多扣 2 分）**

(3) 如果程序中的 A 位置的代码为：

```
printf("E\n");
```

列出所有可能的输出结果：(2 分)

**7 个：分别是 ABDCE ABCEDD ACEBDD ACEDBD ACBEDD ACBEDD ABCDED（错一个扣半分，多了扣一分）**

## Part II

请阅读以下程序，然后回答问题（假设程序中的函数调用都可以正确执行，且每条语句都是原子动作）：

```
pid_t pid;
int even = 0;
int counter1 = 0;
int counter2 = 1;
void handler1(int sig) {
 if (even % 2 == 0) {
 printf("%d\n",
 counter1);
 counter1 = ____ A ____;
 } else {
 printf("%d\n",
 counter2);
 counter2 = ____ B ____;
 }
 even = ____ C ____;
}
void handler2(int sig) {
 if (____ D ____) {
 counter1 = even * even;
 } else {
 counter2 = even * even;
 }
}
int main() {
 signal(SIGUSR1, handler1);
 signal(SIGUSR2, handler2);
 if ((pid = fork()) == 0) {
 while (1) {}
 }
 while (even < 20) {
 kill(pid, ____ E ____);
 sleep(1);
 kill(pid, ____ F ____);
 sleep(1);
 even += 2;
 }
 kill(pid, SIGKILL);
 exit(0);
}
```

1). 完成程序，使得程序在输出前 20 个斐波那契 (Fibonacci) 数列，即  $F_0=0$ ,  $F_1=1$ , ...,  $F_n=F_{n-1}+F_{n-2}$ 。（如果存在对本次程序执行结果没有影响的语句，请在相应位置填写“无关”）（3 分）

A: counter1 + counter2

B: counter1 + counter2

C: even + 1

D: 无关

E: SIGUSR1

F: SIGUSR1

2). 完成程序，其中 A, B 处保持不变，使得程序可以分别输出前几个奇数或偶数的平方和。（如果存在对本次程序执行结果没有影响的语句，请在相应位置填写“无关”）（2 分）

其中：

若要输出奇数的平方和：even 的初始值为 3；

若要输出偶数的平方和，even 的初始值为 2。

C: even + 2

D: even % 2 == 1

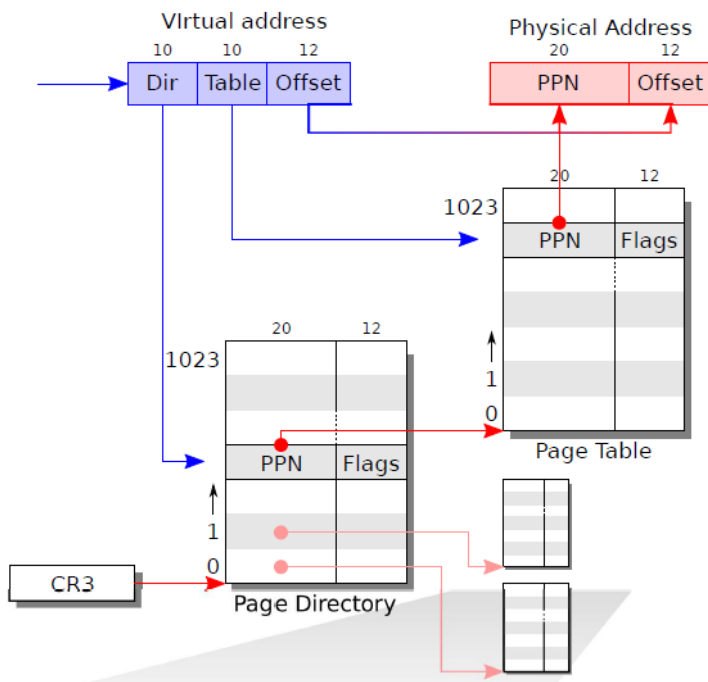
E: SIGUSR2

F: SIGUSR1

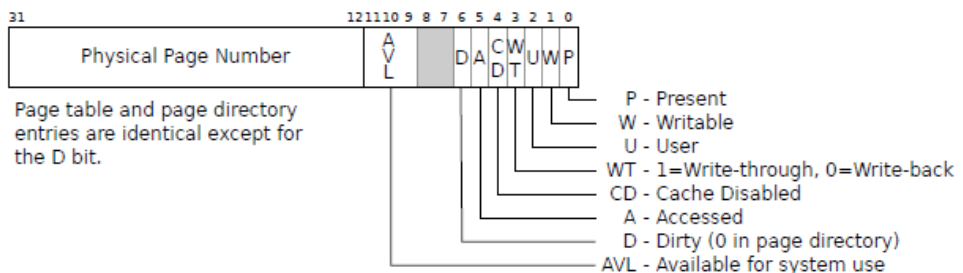
|    |
|----|
| 得分 |
|    |

### 第六题 (10 分)

Intel 的 IA32 体系结构采用二级页表，称第一级页表为页目录 (Page Directory)，第二级页表为页表 (Page Table)。其虚拟地址到物理地址的翻译方式如下图。先根据 CR3 找到页目录地址，然后依据偏移 Dir 找到一个页目录项，页目录项的高 20 位 (PPN) 为二级页表地址；在二级页表中根据偏移 Table 找到页表项，页表项中的高 20 位 (PPN) 即为物理地址的高 20 位，将这 20 位与虚拟地址的低 12 位拼在一起形成完整的物理地址。



页目录和页表均有 1024 项，每一项为 4 字节，含义如下：



页目录和页表由操作系统维护，通常情况下只能在内核态下访问，为了给用户提供一访问页表项和页目录项内容的接口，假设操作系统中已经执行过如下代码段：

```
#define UVPT 0xef400000
#define PDX(la) (((unsigned int) (la)) >> 22) & 0x3FF
.....
kern_pgdir[PDX(UVPT)] = PADDR(kern_pgdir) | PTE_U | PTE_P;
```

其中 kern\_pgdir 是操作系统维护的页目录数组，共 1024 项，每一项的类型为 unsigned int。PADDR(kern\_pgdir) 用于获得 kern\_pgdir 的物理地址，页目录在物理内存中正好占一页，所以 kern\_pgdir 的物理地址是 4KB 对齐的。PTE\_U 和 PTE\_P 代表了这个页目录项的权限，即用户态可访问（只读）。可以看到，这条语句将页目录的第 PDX(UVPT) 项指向了页目录自身。

利用这一点，对于给定的虚拟地址 va，可以获得 va 对应的页目录项和页表项内容，分别对应于函数 get\_pde 和 get\_pte，请完成这两个函数（每空 2 分）：

```
#define UVPT 0xef400000
// get_pde(va): 获取虚拟地址 va 对应的一级页表(页目录)中的页目录项内容
unsigned int get_pde(unsigned int va) {
 unsigned int pdx = (va >> [1]) & [2];
 unsigned int addr = UVPT + ([3]) + pdx * 4;
 return *((unsigned int *) (addr));
}

// get_pte(va): 获取虚拟地址 va 对应的二级页表中的页表项内容
unsigned int get_pte(unsigned int va) {
 unsigned int PGNUM = va >> [4];
 unsigned int addr = [5] + PGNUM;
 return *((unsigned int *) (addr));
}
```

● 答案：

[1]: 22                      [2] 0x3ff (或 1023)    [3] UVPT >> 10  
[4]: 10                      [5] UVPT

```

#define UVPT 0xef400000

unsigned int get_pde(unsigned int va) {
 unsigned int pdx = (va >> 22) & 0x3ff;
 unsigned int addr = UVPT + (UVPT >> 10) + pdx * 4;
 return *((unsigned int *) (addr));
}

unsigned int get_pte(unsigned int va) {
 unsigned int PGNUM = va >> 12;
 unsigned int addr = UVPT + PGNUM * 4;
 return *((unsigned int *) (addr));
}

```

要获取页目录项，需要让  $\text{addr} = \text{UVPT}[31:22] \mid \text{UVPT}[31:22] \mid \text{PDX} \mid 00$ ，即最高的 10 位为 UVPT 的高 10 位，中间的 10 位也是 UVPT 的高 10 位，接下来 10 位是 PDX (va 的高 10 位)，最后两位为 0。

这样，根据翻译机制，第一步取出 addr 的高 10 位 UVPT[31:22]，由于页目录中 UVPT[31:22] 指向的是页目录本身，所以这一步找到的二级页表仍然为页目录本身。第二步取出中间 10 位 UVPT[31:22]，重复刚才的过程，这一步找到的物理页仍然为页目录本身。最后加上偏移量  $\text{PDX} \mid 00$  就可以找到页目录中的第 PDX 项。

要获取页表项，让  $\text{addr} = \text{UVPT}[31:22] \mid \text{PDX} \mid \text{PTX} \mid 00$ ，其中 PTX 是 va 的中间 10 位。这样，第一步翻译后得到的二级页表仍然指向页目录本身，第二步翻译后得到的物理页指向的是 va 对应的二级页表。最后用偏移量  $\text{PTX} \mid 00$  即可获得二级页表中的第 PTX 项。

|    |
|----|
| 得分 |
|    |

第七题 (10 分)

(1) 一个服务器拥有两个独立的固定 IP 地址，那么它在 web 应用端口 80 上最多可以监听多少个独立的 socket 连接？ (2 分)

答案:  $2 \times 2^{48}$

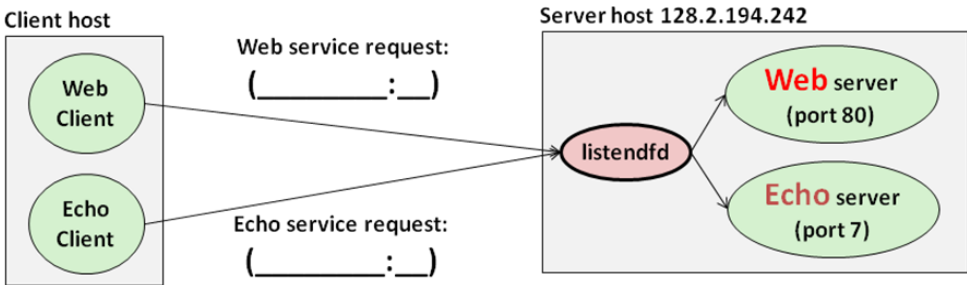
| 服务器端       | 客户端                               | 结果                   |
|------------|-----------------------------------|----------------------|
| 2 个独立固定 IP | 任意 32 位 IP<br>任意 16 位 port number | $2 \times 2^{32+16}$ |

(2) 该服务器在所有有 web 应用端口上最多可以监听多少个独立的 socket 连接？ (2 分)

答案:  $2 \times 2^{64}$

| 服务器端                              | 客户端                               | 结果                      |
|-----------------------------------|-----------------------------------|-------------------------|
| 2 个独立固定 IP<br>任意 16 位 port number | 任意 32 位 IP<br>任意 16 位 port number | $2 \times 2^{16+32+16}$ |

(3) 在下图中连线上填入正确的目标服务器的 socket 标识符 (2 分)



答案:

128.2.194.242:80

128.2.194.242:7

4) 在 Echo server 范例中, server 端通过 accept 函数接受了一个 client 的连接请求, 从而将网络描述符与该网络连接、socket 绑定, 然后进行网络数据传输。在下面的空格处填写正确的网络描述符, 每个空填写 **listenfd** 或 **connfd** (共 4 分, 每空 1 分)。

```
int main(int argc, char **argv) {
 int listenfd, connfd, port, clientlen;
 struct sockaddr_in clientaddr;
 struct hostent *hp;
 char *haddrp;
 unsigned short client_port;

 while (1) {
 clientlen = sizeof(clientaddr);
 _____ = Accept(_____, (SA *)&clientaddr,
 &clientlen);
 hp = Gethostbyaddr((const char*)
 &clientaddr.sin_addr.s_addr,
 sizeof(clientaddr.sin_addr.s_addr), AF_INET);
 haddrp = inet_ntoa(clientaddr.sin_addr);
 client_port = ntohs(clientaddr.sin_port);
 printf("server connected to %s (%s), port %u\n",
 hp->h_name, haddrp, client_port);
 echo(_____);
 Close(_____);
 }
}
```

答案: **connfd**, **listenfd**, **connfd**, **connfd**

**accept** 之后把参数里的 **listenfd** 转交给 **connfd**

**server** 无法支持多个 **connections**, 因为只有一个文件描述符 (**listenfd**)  
考察学生对 网络文件描述符、**listenfd**、**connfd** 的理解。



|    |
|----|
| 得分 |
|    |

#### 第八题（10 分）

某个城市为了解决市中心交通拥堵的问题，决定出台一项交通管制措施，对进入市中心区的机动车辆实行单双日限制行驶的办法。具体要求是，逢单日，只允许车辆牌号码为单数的机动车进入市中心区；同样，逢双日，只允许车辆牌号码为双数的机动车进入市中心区。有一个进入市中心区的交通路口，进入该路口的道路有一条，离开该路口的道路有两条，其中一条是通往市中心区的道路，而另一条是绕过市中心区的环路，在进入路口处设置了自动识别车辆牌号的识别设备与放行栅栏控制设备。在单日，遇到单号车辆进入路口车辆号码识别区，号码识别设备打开通往市中心区道路的放行栅栏；而遇到双号车辆，则打开绕过市中心区环路的放行栅栏。反之亦然。显然，只有在该路口车辆号码识别区中无车时，才允许一辆车进入车辆号码识别区。同时为了防止有车辆混过路口，两个放行栅栏平时处于关闭状态，只有在车辆号码识别区中的车辆已被识别出单双号之后，放行栅栏才会在识别设备的控制下，打开对应的放行栅栏，在车辆通过之后，该放行栅栏自行关闭。

```
vehicle_n int; /* 车辆号码 */
```

检查车辆牌号线程 T1:

```
while (1) {
 车辆到达识别区路口;
 ①
 车辆进入号码识别区;
 if (vehicle_n == 奇数)
 { ② }
 else
 { ③ }
};
```

市区放行栅栏线程 T2:

```
while (1) {
 ④
 允许车辆进入市中心区;
 ⑤
};
```

环路放行栅栏线程 T3:

```
while (1) {
 ⑥
 允许车辆绕行环路;
 ⑦
};
```

- 1) 请设计若干信号量, 给出每一个信号量的作用和初值。(3 分)

参考答案:

Check: 指示可否在车辆号码识别区中进入一辆汽车, 由于只能进入一辆, 其初值为 1。

Odd: 指示汽车号码是否为奇数, 其初值为 0, 表示不是奇数。

Even: 指示汽车号码是否为偶数, 其初值为 0, 表示不是偶数。

- 2) 请将信号量上对应的 PV 操作填写在代码中适当位置。(7 分)

参考答案:

① P(Check);

② V(Odd);

③ V(Even);

④ P(Odd);

⑤ V(Check);

⑥ P(Even);

⑦ V(Check);

装  
订  
线  
内

不  
要  
答  
题

# 北京大学信息科学技术学院考试试卷

考试科目： 计算机系统导论 姓名： \_\_\_\_\_ 学号： \_\_\_\_\_

考试时间： 2015 年 1 月 13 日 任课教师： \_\_\_\_\_

| 题号  | 一 | 二 | 三 | 四 | 五 | 六 | 七 | 八 | 总分 |
|-----|---|---|---|---|---|---|---|---|----|
| 分数  |   |   |   |   |   |   |   |   |    |
| 阅卷人 |   |   |   |   |   |   |   |   |    |

## 北京大学考场纪律

1、考生进入考场后，按照监考老师安排隔位就座，将学生证放在桌面上。无学生证者不能参加考试；迟到超过 15 分钟不得入场。在考试开始 30 分钟后方可交卷出场。

2、除必要的文具和主考教师允许的工具书、参考书、计算器以外，其它所有物品（包括空白纸张、手机、或有存储、编程、查询功能的电子用品等）不得带入座位，已经带入考场的必须放在监考人员指定的位置。

3、考试使用的试题、答卷、草稿纸由监考人员统一发放，考试结束时收回，一律不准带出考场。若有试题印制问题请向监考教师提出，不得向其他考生询问。提前答完试卷，应举手示意请监考人员收卷后方可离开；交卷后不得在考场内逗留或在附近高声交谈。未交卷擅自离开考场，不得重新进入考场答卷。考试结束时间到，考生立即停止答卷，在座位上等待监考人员收卷清点后，方可离场。

4、考生要严格遵守考场规则，在规定时间内独立完成答卷。不准交头接耳，不准偷看、夹带、抄袭或者有意让他人抄袭答题内容，不准接传答案或者试卷等。凡有违纪作弊者，一经发现，当场取消其考试资格，并根据《北京大学本科考试工作与学术规范条例》及相关规定严肃处理。

5、考生须确认自己填写的个人信息真实、准确，并承担信息填写错误带来的一切责任与后果。

学校倡议所有考生以北京大学学生的荣誉与诚信答卷，共同维护北京大学的学术声誉。

以下为试题和答题纸，共    页。

|    |
|----|
| 得分 |
|    |

第一题 单项选择题（每小题 1 分，共 20 分）

1. 下面关于 IEEE 浮点数标准说法正确的是（ ）

- A. 在位数一定的情况下，不论怎么分配 exponent bits 和 fraction bits，所能表示的数的个数是不变的
- B. 如果甲类浮点数有 10 位，乙类浮点数有 11 位，那么甲所能表示的最大数一定比乙小
- C. 如果甲类浮点数有 10 位，乙类浮点数有 11 位，那么甲所能表示的最小正数一定比乙小
- D. "0111000"可能是 7 位浮点数的 NAN 表示

答案：D

2. 假设有下面 x 和 y 的程序定义

```
int x = a >> 2;
```

```
int y = (x + a) / 4;
```

那么有多少个位于闭区间  $[-8, 8]$  的整数 a 能使得 x 和 y 相等？（ ）

- A. 12
- B. 13
- C. 14
- D. 15

答案：B

3. 左边的 C 函数中，在 x86\_64 服务器上采用 GCC 编译产生的汇编语言如右边所示。那么 (1) 和 (2) 的内容分别是：（ ）

```

 <arith>:
int arith(int x, int y) { lea (%rsi,%rdi,1),%eax
 return (x < y) ? (1) : (2); mov %esi,%edx
} sub %edi,%edx
 cmp %esi,%edi

```

```
cmovge %edx,%eax
```

```
retq
```

(提示: 第一个参数放在 rdi 寄存器中, 第二个参数放在 rsi 寄存器中)

A.  $x-y, x+y$    B.  $x+y, x-y$    C.  $x+y, y-x$    D.  $y-x, x+y$

**答案: C**

**说明: 考查 lea 和 cmov 的指令理解**

4. 假定 `struct P {int i; char c; int j; char d;};` 在 x86\_64 服务器的 Linux 操作系统上, 下面哪个结构体的大小与其它三个不同: 答: (        )

- A. `struct P1 {struct P a[3]};`
- B. `struct P2 {int i[3]; char c[3]; int j[3]; char d[3]};`
- C. `struct P3 {struct P *a[3]; char *c[3]};`
- D. `struct P4 {struct P *a[3]; int *f[3]};`

**答案: B**

**说明: 考查数据对齐, P 的 sizeof 为 16, A/C/D 都为 48, B 为 32**

5. 根据编译器安全优化的策略, 如下手工程序代码的优化, 哪个达不到优化效果? (        )

- A. 循环展开, 以减少循环的迭代次数
- B. 将函数调用移到循环内, 以提高程序的模块性
- C. 消除不必要的存储器引用, 减少访存开销
- D. 分离多个累计变量, 以提高并行性

**答案: B**

**(第五章) 考察安全优化策略**

6. 通常情况下, 下面的哪些表述是正确的? (        )

- A. 在一次读操作中, 返回的内容由高速缓存中的信息块决定
- B. 高速缓存利用了时间局部性
- C. 大部分情况下, 缓存需要用户程序采取显式的管理行为
- D. 一级高速缓存更看重命中率, 二级高速缓存更看重命中时间

**答案: B**

**(第六章) 考察高速缓存设计原理的理解**

7. 在代码中, 变量 `sum` 具有的特性是: ( )

```
int sumvec(int v[N])
{
 int i, sum = 0;

 for (i = 0; i < N; i++)
 sum += v[i];
 return sum;
}
```

- A. 良好的时间局部性
- B. 良好的空间局部性
- C. 同时具有良好的时间局部性和空间局部性
- D. 都不具有

**答案: A**

**(第六章) 考察对局部性的认识**

8. 下列关于静态库链接的描述中, 错误的是 ( )

- A. 链接时, 链接器只拷贝静态库中被程序引用的目标模块
- B. 使用库的一般准则是将它们放在命令行的结尾
- C. 如果库不是相互独立的, 那么它们必须排序
- D. 每个库在命令行只须出现一次即可

**【答案】D**

**【说明】如果相互调用的库, 在命令行必须重复出现。**

9. 在 `foo.c` 文件中的函数外, 如果添加如下一条语句:

```
static int count = 0xdeadbeef;
```

那么它在编译为 `foo.o` 后, 会影响到 ELF 可重定位目标文件中的除 `.text` 以外的哪些字段? ( )

- A. `.rodata`
- B. `.data, .symtab,`
- C. `.data, .symtab, .rel.data`
- D. `.rodata, .symtab, .rel.data`

**【答案】B**

**【说明】**这是一个本地静态全局变量，它在 `.data` 中占有位置，它不需要重定位，因为它的初始值是确定的，但是它在符号表中占有一个位置。

考察 `ELF` 文件格式。

10. 在系统调用成功的情况下，下列代码会输出几个 `hello`? ( )

```
void doit()
{
 if (fork() == 0) {
 printf("hello\n");
 fork();
 }
 return ;
}

int main()
{
 doit();
 printf("hello\n");
 exit(0) ;
}
```

A. 3      B. 4      C. 5      D. 6

**【答案】B**

**【说明】**考查学生对 `fork` 执行机制的理解和掌握。

11. 下列说法中哪一个是错误的? ( )

- A. 中断一定是异步发生的
- B. 异常处理程序一定运行在内核模式下
- C. 故障处理一定返回到当前指令
- D. 陷阱一定是同步发生的

**【答案】c**

**【说明】**

12. 下列这段代码的输出不可能是 ( )

```
void handler()
{
 printf("h");
}

int main()
{
 signal(SIGCHLD, handler) ;

 if (fork() == 0) {
 printf("a") ;
 } else {
 printf("b") ;
 }
 printf("c") ;
 exit(0) ;
}
```

- A. abcc      B. abch      C. bcach      D. bchac

**【答案】 D**

**【说明】**SIGCHLD 信号只有在 fork 的子进程结束时产生，因此 h 只会出现在 ac 之后。

13. 对于虚拟存储系统，一次访存过程中，下列命中组合不可能发生的是 ( )

- A. TLB 未命中，Cache 未命中，Page 未命中  
B. TLB 未命中，Cache 命中，Page 命中  
C. TLB 命中，Cache 未命中，Page 命中  
D. TLB 命中，Cache 命中，Page 未命中

**【答案】 D**

**【说明】**考察 TLB，Cache，页式虚拟存储器基本性质。

14. 有程序段如下：



```
int foo() {
 char str1[20], *str2;
 str2 = (char*)malloc(20*sizeof(char));
 free(str2);
}
```

下列说法中正确的是( )

- A. str1 和 str2 指向的内存都是分配在栈空间内的
- B. str1 和 str2 指向的内存都是分配在堆空间内的
- C. str1 指向的内存是分配在栈空间内的, str2 指向的内存是分配在堆空间内的
- D. str1 指向的内存是分配在堆空间内的, str2 指向的内存是分配在栈空间内的

**【答案】 C**

**【说明】 考察 malloc 函数是显式地分配和释放堆存储器**

15. ICS.txt 中包含 3000 个字符, 考虑如下代码段:

```
int main(int argc, char** argv) {
 int fd = open("ICS.txt", O_CREAT | O_RDWR, S_IRUSR |
S_IWUSR);
 write(fd, "ICS", 3);

 char buf[128];
 int i;
 for (i = 0; i < 10; i++) {
 int fd1 = open("ICS.txt", O_RDWR);
 int fd2 = dup(fd1);

 int cnt = read(fd1, buf, 128);
 write(fd2, buf, cnt);
 }
 return 0;
}
```

上述代码执行完后, ICS.txt 中包含多少个字符(假设所有系统调用都成功)?  
( )

- A. 3
- B. 256
- C. 3000
- D. 3072

**【答案】 C**

**【说明】 主要考查 open 函数的用法。open 不像 fopen, 不设置 O\_TRUNC 并不会清空文件。所以只会反复把文件中字符 1-128 写到字符 129-256, 字符数不变。几个干扰项分别考查 dup 的作用以及 buf 大小对于程序功能的影响。**

16. 下列系统 I/O 的说法中, 正确的是( )

- A. C 语言中的标准 I/O 函数在不同操作系统中的实现代码一样
- B. 对于同一个文件描述符，混用 RIO 包中的 `rio_readnb` 和 `rio_readn` 两个函数不会造成问题
- C. C 语言中的标准 I/O 函数是异步线程安全的
- D. 使用 I/O 缓冲区可以减少系统调用的次数，从而加快 I/O 的速度

**【答案】D**

**【说明】A** 中在不同操作系统需要用到不同系统调用。**B** 中两个函数一个从 **buffer** 读一个直接读，混用会造成错误。**C** 不是线程安全的。**D** 正确。

17. 如果两个局域网高层分别采用 TCP/IP 协议和 SPX/IPX 协议，那么可以选择的互连设备应是 ( )
- A. 网桥
  - B. 集线器
  - C. 路由器
  - D. 交换机

**【答案】C**

**【说明】**

18. 下面说法正确的是 ( )
- A. TCP 是一种可靠的无连接协议
  - B. UDP 是一种不可靠的无连接协议
  - C. Web 浏览器与 web 服务器通信采用的协议是 HTML
  - D. 数字数据只能通过数字信号传输

**【答案】B**

**【说明】A** 应该是连接协议，**C** 应该是 **HTTP**，**D** 应该还包括模拟信号

19. 对于如下 C 语言程序:

```
#include "csapp.h"
void *thread (void * arg)
{
 printf("Hello World") ;
 Pthread_detach(pthread_self()) ;
}
int main(void)
{
```

```

pthread_t tid;
int sta ;
sta = Pthread_create(&tid, NULL, thread, NULL);
if (sta==0)
 printf("Oops, I can not create thread\n");
exit(NULL);
}

```

在上述程序中，Pthread\_detach 函数的作用是 ( )

- A. 使主线程阻塞以等待线程 thread 结束
- B. 线程 thread 运行结束后会自动释放所有资源
- C. 线程 thread 运行后主动释放 CPU 给其他线程
- D. 线程 thread 运行后成为僵尸线程

**【答案】B**

**【说明】考查对 Posix 线程包 Pthreads 函数功能的理解。**

20. 两个线程中共享如下一段 C 代码：

```

for (j = 0; j < N; j++)
 count += 2;

```

假设其对应的汇编代码如下：

```

movq (%rdi), %rcx
testq %rcx,%rcx
jle .L2 Hi
movl $0, %eax
.L3:
movq count(%rip),%rdx Li
addq $2, %rdx Ui
movq %rdx, count(%rip) Si

addq $1, %rax
cmpq %rcx, %rax Ti
jne .L3
.L2:

```

请问在下列指令顺序对应的轨迹线中，哪一个是安全轨迹线？ ( )

- A. H<sub>1</sub>, H<sub>2</sub>, L<sub>2</sub>, L<sub>1</sub>, U<sub>2</sub>, U<sub>1</sub>, S<sub>1</sub>, S<sub>2</sub>, T<sub>1</sub>, T<sub>2</sub>
- B. H<sub>1</sub>, L<sub>1</sub>, U<sub>1</sub>, H<sub>2</sub>, L<sub>2</sub>, S<sub>1</sub>, T<sub>1</sub>, U<sub>2</sub>, S<sub>2</sub>, T<sub>2</sub>
- C. H<sub>2</sub>, L<sub>2</sub>, U<sub>2</sub>, H<sub>1</sub>, S<sub>2</sub>, L<sub>1</sub>, T<sub>2</sub>, U<sub>1</sub>, S<sub>1</sub>, T<sub>1</sub>
- D. H<sub>2</sub>, L<sub>2</sub>, H<sub>1</sub>, L<sub>1</sub>, U<sub>1</sub>, U<sub>2</sub>, S<sub>2</sub>, T<sub>2</sub>, S<sub>1</sub>, T<sub>1</sub>

**【答案】C**

**【说明】考查两个并发线程指令执行序列是否会导致不安全轨迹线。**

|    |
|----|
| 得分 |
|    |

## 第二题（10 分）汇编

阅读下面的 c 代码：

```
unsigned char d[256] = {
 0x00, 0x80, 0x40, 0xc0, 0x20, 0xa0, 0x60, 0xe0,
 0x10, 0x90, 0x50, 0xd0, 0x30, 0xb0, 0x70, 0xf0,
 0x08, 0x88, 0x48, 0xc8, 0x28, 0xa8, 0x68, 0xe8,
 0x18, 0x98, 0x58, 0xd8, 0x38, 0xb8, 0x78, 0xf8,
 0x04, 0x84, 0x44, 0xc4, 0x24, 0xa4, 0x64, 0xe4,
 0x14, 0x94, 0x54, 0xd4, 0x34, 0xb4, 0x74, 0xf4,
 0x0c, 0x8c, 0x4c, 0xcc, 0x2c, 0xac, 0x6c, 0xec,
 0x1c, 0x9c, 0x5c, 0xdc, 0x3c, 0xbc, 0x7c, 0xfc,
 0x02, 0x82, 0x42, 0xc2, 0x22, 0xa2, 0x62, 0xe2,
 0x12, 0x92, 0x52, 0xd2, 0x32, 0xb2, 0x72, 0xf2,
 0x0a, 0x8a, 0x4a, 0xca, 0x2a, 0xaa, 0x6a, 0xea,
 0x1a, 0x9a, 0x5a, 0xda, 0x3a, 0xba, 0x7a, 0xfa,
 0x06, 0x86, 0x46, 0xc6, 0x26, 0xa6, 0x66, 0xe6,
 0x16, 0x96, 0x56, 0xd6, 0x36, 0xb6, 0x76, 0xf6,
 0x0e, 0x8e, 0x4e, 0xce, 0x2e, 0xae, 0x6e, 0xee,
 0x1e, 0x9e, 0x5e, 0xde, 0x3e, 0xbe, 0x7e, 0xfe,
 0x01, 0x81, 0x41, 0xc1, 0x21, 0xa1, 0x61, 0xe1,
 0x11, 0x91, 0x51, 0xd1, 0x31, 0xb1, 0x71, 0xf1,
 0x09, 0x89, 0x49, 0xc9, 0x29, 0xa9, 0x69, 0xe9,
 0x19, 0x99, 0x59, 0xd9, 0x39, 0xb9, 0x79, 0xf9,
 0x05, 0x85, 0x45, 0xc5, 0x25, 0xa5, 0x65, 0xe5,
 0x15, 0x95, 0x55, 0xd5, 0x35, 0xb5, 0x75, 0xf5,
 0x0d, 0x8d, 0x4d, 0xcd, 0x2d, 0xad, 0x6d, 0xed,
 0x1d, 0x9d, 0x5d, 0xdd, 0x3d, 0xbd, 0x7d, 0xfd,
 0x03, 0x83, 0x43, 0xc3, 0x23, 0xa3, 0x63, 0xe3,
 0x13, 0x93, 0x53, 0xd3, 0x33, 0xb3, 0x73, 0xf3,
 0x0b, 0x8b, 0x4b, 0xcb, 0x2b, 0xab, 0x6b, 0xeb,
 0x1b, 0x9b, 0x5b, 0xdb, 0x3b, 0xbb, 0x7b, 0xfb,
 0x07, 0x87, 0x47, 0xc7, 0x27, 0xa7, 0x67, 0xe7,
 0x17, 0x97, 0x57, 0xd7, 0x37, 0xb7, 0x77, 0xf7,
```

```

 0x0f, 0x8f, 0x4f, 0xcf, 0x2f, 0xaf, 0x6f, 0xef,
 0x1f, 0x9f, 0x5f, 0xdf, 0x3f, 0xbf, 0x7f, 0xff,
};

```

```

static inline unsigned char a(unsigned char x)
{
 return d[x];
}

```

```

unsigned short b(unsigned short x)
{
 return (a(x & 0xff) << 8) | a(x >> 8);
}

```

```

unsigned int c(unsigned int x)
{
 return (b(x & 0xffff) << 16) | b(x >> 16);
}

```

1、根据程序逻辑，下面的结果是：

c(b(a(1))) = \_\_\_\_\_ (1)

a(b(c(1))) = \_\_\_\_\_ (2)

**答案（每项 1 分）：**

**(1) 0x800000**

**(2) 0**

**说明：考察信息表示和程序理解**

2、填写下面反汇编中的缺失的内容：（数组 d 的地址为 0x6009a0）

（提示：注意反汇编格式与汇编格式有所区别）

00000000004004d0 <b>:

```

4004d0: mov %edi,%eax
4004d2: movzbl %dil,%edx
4004d6: movzbl 0x6009a0(%rdx),%edx
4004dd: movzbl %ah,_____(1)_____
4004e0: movzbl 0x6009a0(%rax),%eax
4004e7: shl _____(2)_____,%edx
4004ea: or %edx,%eax
4004ec: retq

```

00000000004004f0 <c>:

```
4004f0: mov %edi,%eax
4004f2: push _____(3)_____
4004f3: mov %edi,%ebx
4004f5: shr _____(4)_____,%eax
4004f8: movzbl %bh,%ebx
4004fb: movzbl %al,_____(5)_____
4004fe: movzbl %ah,_____(6)_____
400501: movzbl 0x6009a0(%rdx),%edx
400508: movzbl 0x6009a0(%rax),%eax
40050f: shl _____(7)_____,%edx
400512: or %edx,%eax
400514: movzbl %dil,%edx
400518: movzbl 0x6009a0(%rdx),%ecx
40051f: movzbl 0x6009a0(%rbx),%edx
400526: movzwl %ax,%eax
400529: pop _____(3)_____
40052a: shl $0x8,%ecx
40052d: or %ecx,%edx
40052f: shl $0x10,%edx
400532: or %edx,_____(8)_____
400534: retq
```

**答案（每项 1 分）：**

**(1) %eax**

**(2) \$0x8**

**(3) %rbx**

**(4) \$0x10**

**(5) %edx**

**(6) %eax**

**(7) \$0x8**

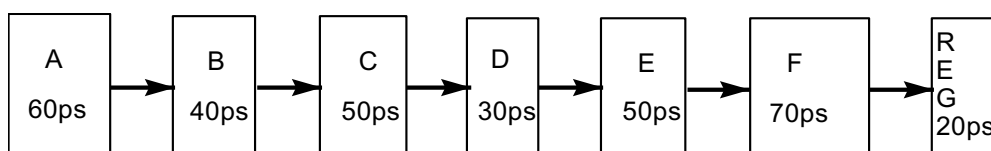
**(8) %eax**

**说明：考察汇编程序理解**

|    |
|----|
| 得分 |
|    |

### 第三题（10 分） 处理器

如图所示，每个模块表示一个单独的组合逻辑单元，每个单元的延迟已在图中标出。通过在两个单元间添加寄存器的方式，可以对该数据通路进行流水化改造。假设每个寄存器的延迟为 20ps。注意，由于电路互联特点 A 与 B 之间如果插入寄存器，B 本身的延迟将增加到 50ps。



1) 如果改造为一个二级流水线（只插入一个寄存器），为获得最大的吞吐率，该寄存器应在哪里插入？请计算该流水线的吞吐率，并说明计算过程。结果可以是分数形式也可以是小数形式。

**插入在 CD 间（1 分）**

**$1000 / (60 + 40 + 50 + 20) = 1000 / 170 = 5.88 \text{ GIPS}$ （过程和结果 1 分）**

2) 如果改造为一个三级流水线（插入两个寄存器），为获得最大的吞吐率，寄存器应在哪里插入？请计算该流水线的吞吐率，并说明计算过程。结果可以是分数形式也可以是小数形式。

**插入在 BC 间和 DE 间（1 分）**

**$1000 / (50 + 70 + 20) = 1000 / 140 = 7.143 \text{ GIPS}$ （过程和结果 1 分）**

3) 如果改造为一个四级流水线（插入三个寄存器），为获得最大的吞吐率，寄存器应在哪里插入？请计算该流水线的吞吐率，并说明计算过程。结果可以是分数形式也可以是小数形式。

**插入 AB 间、CD 间、EF 间（1 分）**

**$1000 / (50 + 50 + 20) = 1000 / 120 = 8.33 \text{ GIPS}$ （过程 1 分，结果 1 分）**

4) 不改变单元划分，为获得最大性能，该设计至少需要划分成几级？请计算对应的吞吐率，并说明计算过程。结果可以是分数形式也可以是小数形式。

**至少划分成 6 级（1 分）**

**$1000 / (70 + 20) = 1000 / 90 = 11.11 \text{ GIPS}$ （过程 1 分，结果 1 分）**

|    |
|----|
| 得分 |
|    |

第四题（10 分） 链接

考虑如下3个文件：main.c, fib.c和bignat.c:

```

/* main.c */
void fib (int n);
int main (int argc, char** argv) {
 int n = 0;
 sscanf(argv[1], "%d", &n);
 fib(n);
}

/* fib.c */
#define N 16

static unsigned int ring[3][N];

static void print_bignat(unsigned int* a) {
 int i;
 for (i = N-1; i >= 0; i--)
 printf("%u ", a[i]); /* print a[i] as unsigned int
 */
 printf("\n");
}

void fib (int n) {
 int i, carry;
 from_int(N, 0, ring[0]); /* fib(0) = 0 */
 from_int(N, 1, ring[1]); /* fib(1) = 1 */
 for (i = 0; i <= n-2; i++) {
 carry = plus(N, ring[i%3], ring[(i+1)%3],
 ring[(i+2)%3]);
 if (carry)
 { printf("Overflow at fib(%d)\n", i+2);
 exit(0); }
 }
 print_bignat(ring[n%3]);
}

```

另外，假设在文件 bignat.c 中定义了如下两个函数 plus 和 from\_int（具体定义略）：



```
int plus (int n, unsigned int* a, unsigned int* b, unsigned
int* c);
void from_int (int n, unsigned int k, unsigned int* a);
```

1. (5 分) 对于每个程序中的相应符号, 给出它的属性 (局部或全局, 强符号或弱符号) (提示: 如果某表项中的内容无法确定, 请画 x。)

main.c

|      | 局部或全局? | 强或弱? |
|------|--------|------|
| fib  |        |      |
| main |        |      |

fib.c

|      | 局部或全局? | 强或弱? |
|------|--------|------|
| ring |        |      |
| fib  |        |      |
| plus |        |      |

2. (3 分) 假设文件 bignat.c 被编译为一个静态库 bignat.a, 对于如下的 gcc 调用, 会得到什么样的结果 (请选择)?

- (A) 编译和链接都正确
- (B) 链接失败 (原因是包含未定义的引用)
- (C) 链接失败 (原因是包含重复定义)

| 命令                               | 结果 (A, B 或 C) |
|----------------------------------|---------------|
| gcc -o fib main.c fib.c bignat.a |               |
| gcc -o fib bignat.a main.c fib.c |               |
| gcc -o fib fib.c main.c bignat.a |               |

3. (2 分) 如果在文件 fib.c 中, 程序员在声明变量 ring 时, 不小心把它写成了:

```
static int ring[3][N];
```

会不会影响这些文件的编译、链接和运行结果? 为什么?

**答案:**

1. (5 分) 对于每个程序中的相应符号, 给出它的属性 (局部或全局, 强符号或弱符号) (提示: 如果某表项中的内容无法确定, 请画 x。)

main.c

|      | 局部或全局? | 强或弱? |
|------|--------|------|
| fib  | 全局     | 弱    |
| main | 全局     | 强    |

fib.c

|      | 局部或全局? | 强或弱? |
|------|--------|------|
| ring | 局部     | X    |
| fib  | 全局     | 强    |
| plus | 全局     | 弱    |

2. (3 分) 假设文件 bignat.c 被编译为一个静态库 bignat.a, 对于如下的 gcc 调用, 会得到什么样的结果 (请选择)?

- (A) 编译和链接都正确
- (B) 链接失败 (原因是包含未定义的引用)
- (C) 链接失败 (原因是包含重复定义)

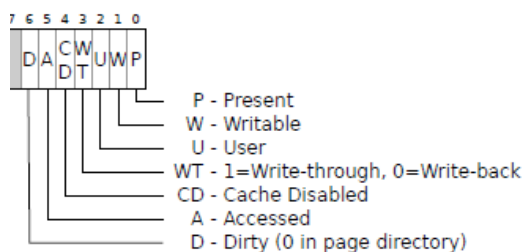
| 命令                               | 结果 (A, B 或 C) |
|----------------------------------|---------------|
| gcc -o fib main.c fib.c bignat.a | A             |
| gcc -o fib bignat.a main.c fib.c | B             |
| gcc -o fib fib.c main.c bignat.a | A             |

3. 对编译、链接和执行结果都没有影响。因为 signed 和 unsigned 之间的转换不会改变整数的表示形式, 因此函数调用会正常进行。

|    |
|----|
| 得分 |
|    |

第五题（10 分）虚拟存储

Intel 的 IA32 体系结构采用小端法和二级页表。其中两级页表的大小相同，页大小为 4KB。一级页表和二级页表的表项结构相同，其中页表项后六位的含义如下。



已知一级页表的地址为 0x0c23b000，物理内存中的部分内容如下图所示。

| 地址       | 内容 | 地址       | 内容 | 地址       | 内容 | 地址       | 内容 |
|----------|----|----------|----|----------|----|----------|----|
| 00023000 | E0 | 00023001 | BE | 00023002 | EF | 00023003 | BE |
| 00023120 | 83 | 00023121 | C8 | 00023122 | FD | 00023123 | 12 |
| 00023200 | 23 | 00023201 | FD | 00023202 | BC | 00023203 | DE |
| 00023320 | 33 | 00023321 | 29 | 00023322 | E5 | 00023323 | D2 |
| 00023FF8 | 29 | 00023FF9 | FF | 00023FFA | DE | 00023FFB | BC |
| 00055004 | 03 | 00055005 | D0 | 00055006 | 74 | 00055007 | 89 |
| 0005545C | 97 | 0005545D | C2 | 0005545E | 7B | 0005545F | 45 |
| 00055460 | 97 | 00055461 | D2 | 00055462 | 7B | 00055463 | 45 |
| 00055464 | 97 | 00055465 | E2 | 00055466 | 7B | 00055467 | 45 |
| 0C23B020 | 55 | 0C23B021 | EB | 0C23B022 | AE | 0C23B023 | 24 |
| 0C23B040 | 55 | 0C23B041 | AB | 0C23B042 | 2A | 0C23B043 | 01 |
| 0C23B080 | 05 | 0C23B081 | 5D | 0C23B082 | 05 | 0C23B083 | 00 |
| 0C23B09D | 05 | 0C23B09E | D3 | 0C23B09F | F2 | 0C23B0A0 | 0F |
| 0C23B274 | 05 | 0C23B275 | 3D | 0C23B276 | 02 | 0C23B277 | 00 |
| 0C23B9FC | 25 | 0C23B9FD | D2 | 0C23B9FE | 14 | 0C23B9FF | 23 |
| 2314D200 | 23 | 2314D201 | 12 | 2314D202 | DC | 2314D203 | 0F |
| 2314D220 | A9 | 2314D221 | 45 | 2314D222 | 13 | 2314D223 | D2 |
| 2314D4A0 | BD | 2314D4A1 | BC | 2314D4A2 | 88 | 2314D4A3 | D3 |
| 2314D890 | 00 | 2314D891 | 2D | 2314D892 | B3 | 2314D893 | 00 |

|          |    |          |    |          |    |          |    |
|----------|----|----------|----|----------|----|----------|----|
| 24AEE001 | 07 | 24AEE002 | A0 | 24AEE003 | 37 | 24AEE004 | C2 |
| 24AEE520 | D1 | 24AEE521 | DA | 24AEE522 | 8C | 24AEE523 | B5 |
| 29DE2504 | 02 | 29DE2505 | AD | 29DE2506 | FF | 29DE2507 | 56 |
| 29DE4400 | D0 | 29DE4401 | 5C | 29DE4402 | B4 | 29DE4403 | 2A |
| 29DE9402 | 00 | 29DE9403 | 20 | 29DE9404 | 73 | 29DE9405 | D4 |
| 29DEE500 | B0 | 29DEE501 | CD | 29DEE502 | 23 | 29DEE503 | 1A |

TLB 采用直接映射，TLB 的内容如下所示。

| 索引 | TLB 标记  | 内容       | 有效位 |
|----|---------|----------|-----|
| 0  | 0x08001 | 2314d220 | 1   |
| 1  | 0x01000 | 24aee520 | 0   |
| 2  | 0x005AE | 00055004 | 0   |
| 3  | 0x016BA | 0c23b09d | 1   |
| 4  | 0x0AA00 | 0005545c | 1   |
| 5  | 0x0000A | 29dee500 | 0   |
| 6  | 0x5AE82 | 00023320 | 1   |
| 7  | 0x28DFC | 00023000 | 1   |

1. (2 分) 某用户态进程试图写入虚拟地址：0x080016ba。该访问的最后结果是\_\_\_\_\_。

- (a) 该进程成功写入，未触发异常
- (b) 该进程触发了一个缺页异常
- (c) 该进程触发了一个非法访问异常

2. 下面描述了具体的访问过程，请填空。如果某个空在访问过程中已不可用，请填入“--”

2.1 TLB 的索引为\_\_\_\_\_，访问为\_\_\_\_\_ (a) 命中 (b) 不命中 (2 分)

2.2 一级页表表项地址为\_\_\_\_\_。(2 分)

2.3 二级页表表项地址为\_\_\_\_\_。(2 分)

2.4 最后物理地址为\_\_\_\_\_。(2 分)

**答案：**

**1. (c)；**

**2.1: 1, (b)；**

**2.2 0x0C23B080**

**2.3 0x00055004 (或 “--”)**

**2.4 8974D6BA (或 “--”)**

**说明：联合考察虚存、高速缓存（通过 TLB 考察）、大端法和小端法的知识。**

|    |
|----|
| 得分 |
|    |

第六题（10 分） ECF

1.（5 分）以下程序运行时系统调用全部正确执行，buffer.txt 文件的内容为 pekinguniv。请给出代码运行后打印输出的结果，并给出程序运行结束后 buffer.txt 文件的内容。

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
 char c;
 int file1 = open("buffer.txt", O_RDWR);
 int file2;

 read(file1, &c, 1);
 file2 = dup(file1);
 write(file2, &c, 1);
 printf("1 = %c\n", c);

 int pid = fork() ;
 if (pid == 0) {
 read(file1, &c, 1);
 write(file2, &c, 1);
 printf("2 = %c\n", c);
 read(file1, &c, 1);
 printf("3 = %c\n", c);
 close(file1);
 exit(0);
 } else {
 waitpid(pid, NULL, 0);
 close(file2);
 dup2(file1, file2);
 read(file2, &c, 1);
 write(file2, &c, 1);
 printf("4 = %c\n", c);
 }
 return 0;
}
```

答案:

1 = p

2 = k

3 = n

4 = g

buffer.txt 文件内容为 ppkknggniv

2.(5 分)某程序员实现了一个课程实验用的操作系统 ICSNIX,其系统函数 sleep 用以下代码实现。请分析该代码存在哪些问题。

```
1 #include <signal.h>
2 #include <unistd.h>
3 static void sig_alarm(int signo)
4 {
5 /* nothing to do, just return to wake up the pause */
6 }
7
8 unsigned int sleep(unsigned int seconds)
9 {
10 if (signal(SIGALRM, sig_alarm) == SIG_ERR)
11 return(seconds);
12
13 alarm(seconds); /* start the timer */
14 pause(); /* next caught signal wakes us up */
15 return(alarm(0)); /* turn off timer, return unslept time */
16 }
```

答案: (三个问题若只回答了 1 个或 2 个则每个 2 分, 全部回答了得 5 分)

问题 1) 由于操作系统调度的原因, alarm 信号触发时, pause 可能还未执行, 导致 sleep 调用永不会返回。

问题 2) 如果应用程序在调用 sleep 之前已经调用了 alarm, 则 sleep 中的 alarm 调用会取消之前设置的 alarm 闹钟。(若用户调用 alarm(5); sleep(10); 则第 5 秒 sleep 就应该唤醒; 若用户调用 alarm(20); sleep(10); 则 sleep 在 10 秒返回后, 再过 10 秒应继续产生一个 SIGALRM 信号。)

问题 3) sleep 的 signal 调用改变了整个程序的 SIGALRM 信号处理方式。因此 sleep 应该保留 signal 的返回值(旧的 SIGALRM 信号处理程序), 并在返回前恢复该值。

|    |
|----|
| 得分 |
|    |

第七题（10 分）系统 I/O

（用作试卷题目时，请把红字和红色箭头全部删去）

请阅读下面的代码：

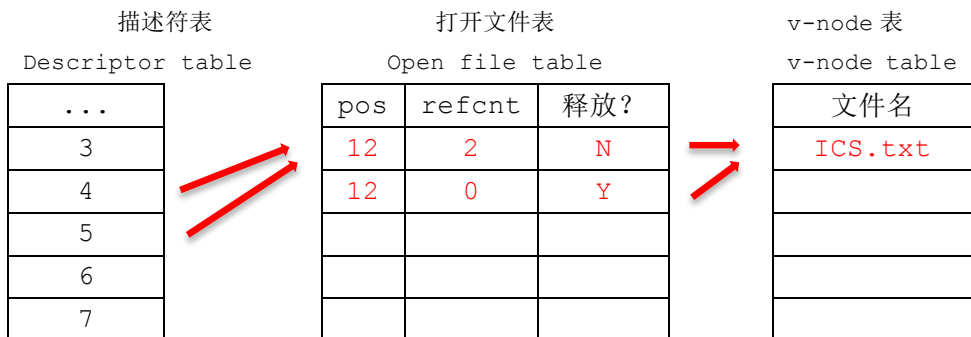
```

1: int main(int argc, char** argv) {
2: int fd1 = open("ICS.txt", O_CREAT|O_RDWR,
3: S_IRUSR|S_IWUSR);
4: write(fd1, "abc", 3);
5:
6: int fd2 = fd1;
7: int fd3 = dup(fd2);
8: int fd4 = open("ICS.txt", O_APPEND|O_RDWR);
9: write(fd2, "defghi", 6);
10: write(fd4, "xyz", 3);
11:
12: int fd5 = fd4;
13: dup2(fd3, fd5);
14: write(fd4, "pqr", 3);
15:
16: close(fd1);
17:
18: return 0;
19: }
```

1. 请填写在第 16 行代码刚刚执行完之后，下面的打开文件表和 v-node 表中表项的部分值，并画出表项之间的指向关系。（6 分）

初始时，ICS.txt 文件不存在。程序执行时，所有的系统调用均会成功，所有表项均会从上到下依次分配，描述符表一开始被占用掉前 3 个表项。对于已经释放的打开文件表表项，请填写释放前那一刻的值和指向的 v-node 表表项。

对于多余的表项，请直接忽略。



说明：考查三层表结构的基本概念，以及 Unix I/O 的基本用法。因为第 8 章出题会融合第 10 章内容，所以没涉及到第 8 章内容。

错一空或者多/少填一空，扣 1 分。多画/少画一个箭头扣 1 分。扣完为止。

2. 请填写在第 16 行代码刚刚执行完之后，下列变量的值（2 分）

| fd1 | fd2 | fd3 | fd4 | fd5 |
|-----|-----|-----|-----|-----|
| 3   | 3   | 4   | 5   | 5   |

说明：错 1 空扣 1 分，扣完为止。

3. 请写出程序执行完之后，ICS.txt 文件中的内容（2 分）

abcdefghijklpqr

说明：写成 abcxyzghipqr 给 1 分，出现此错误是因为不熟悉 O\_APPEND 用法。



|    |
|----|
| 得分 |
|    |

第八题（10 分）网络

1. (1 分) 以下问题默认为 IPv4 协议。一个服务器拥有四个独立的固定 IP 地址，那么它在 web 应用端口 80，理论上可以最多再监听\_\_\_\_\_个来自一个客户端独立的 socket 连接（客户端只有一个固定 IP 地址）。

答案：  $4 \times 2^{16} - 4$

| 服务器端       | 客户端                                 | 结果                                                                                                       |
|------------|-------------------------------------|----------------------------------------------------------------------------------------------------------|
| 4 个独立固定 IP | 一个 32 位固定 IP<br>任意 16 位 port number | $(4 \times 2^{16} - 4)$ ，因为 80 端口本身已用于监听端口<br>或者<br>$(4 \times 2^{16} - 4 \times \text{well-known 端口数})$ |

2. (2 分) 在 client-server 模型中，一个连接（connection）可以由 IP 地址，端口号的组合来表示。假设客户端 IP 地址为 162.105.192.178，内网 IP 为 192.168.100.121。HTTP 服务器端 IP 地址为 208.216.181.15。

服务器使用的是默认监听端口号。

指出下面这个网页浏览器应用的 Connection socket pair 有什么错误，并简要说明原因？

| 客户端 IP: 端口号           | 服务器端 IP: 端口号      |
|-----------------------|-------------------|
| 192.168.100.121:15321 | 208.216.181.15:25 |

答案：

192.168.100.121 不对，原因：不能用内网 IP；

25 应该是 80，原因：网页浏览器应用的默认监听端口是 80 端口。

3. (4 分) 在 Echo Server 程序中，客户端（Client）与服务器端（Server）通过 socket 进行一系列的命令和数据交互。

注意：客户端 Connect 命令包含在其 Open\_clientfd 命令中。

请在下图中用单向箭头标出这些交互步骤。例如，当 Client 给 Server 端发送某个命令或者数据时，则需要在 Client 端相应代码行，朝向 Server 端相应代码行画一条单向箭头。

| Echo client code                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | Echo Server code                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> int main(int argc, char **argv){     int clientfd, port;     char *host, buf[MAXLINE];     rio_t rio;     host = argv[1];     port = atoi(argv[2]);      clientfd = Open_clientfd(host, port);      Rio_readinitb(&amp;rio, clientfd);      printf("type:");     fflush(stdout);      while(Fgets(buf, MAXLINE, stdin) != NULL) {          Rio_writen(clientfd, buf, strlen(buf));          Rio_readlineb(&amp;rio, buf, MAXLINE);         printf("echo:");         Fputs(buf, stdout);         printf("type:"); fflush(stdout);     }      Close(clientfd);     exit(0); } </pre> | <pre> int main(int argc, char **argv) {     int listenfd, connfd, port, clientlen;     struct sockaddr_in clientaddr;     struct hostent *hp;     char *haddrp;     unsigned short client_port;     port = atoi(argv[1]);      listenfd = open_listenfd(port);     while (1) {         clientlen = sizeof(clientaddr);          connfd = Accept(listenfd, (SA*)&amp;clientaddr, &amp;clientlen);          hp = Gethostbyaddr((const char*)&amp;clientaddr.sin_addr.s_addr,             sizeof(clientaddr.sin_addr.s_addr), AF_INET);          haddrp = inet_ntoa(clientaddr.sin_addr);          client_port = ntohs(clientaddr.sin_port);         printf("server connected");          size_t n; char buf[MAXLINE]; rio_t rio;          Rio_readinitb(&amp;rio, connfd);          while((n = Rio_readlineb(&amp;rio, buf, MAXLINE)) != 0) {             upper_case(buf);              Rio_writen(connfd, buf, n);         }          Close(connfd);     } } </pre> |

答案:

```

clientfd = Open_clientfd(host, port); ----->
connfd=Accept(listenfd, (SA*)&clientaddr, &clientlen);

Rio_writen(clientfd, buf, strlen(buf)); ----->
while((n=Rio_readlineb(&rio,buf, MAXLINE)) != 0)

Rio_readlineb(&rio, buf, MAXLINE); <-----
Rio_writen(connfd, buf, n);

Close(clientfd); -----> Close(connfd);

```

4.关于 Tiny Server 程序，请回答下列问题。

- a. (1 分) 下面这段服务器代码用来生成内容的文件是哪个参数?
- b. (1 分) 所生成的内容是静态还是动态? 请简述原因。
- c. (1 分) 如果支持多个客户端请求，下面程序需要添加一个什么功能?

```
/* Return first part of HTTP response */
sprintf(buf, "HTTP/1.0 200 OK\r\n");
Rio_writen(fd, buf, strlen(buf));
sprintf(buf, "Server: Tiny Web Server\r\n");
Rio_writen(fd, buf, strlen(buf));

/* Real server would set all CGI vars here */
setenv("QUERY_STRING", cgiargs, 1);
Dup2(fd, STDOUT_FILENO); /*Redirect stdout to socket and
client */
Execve(filename, emptylist, environ);/* Run CGI prog */
```

**答案**

**a. filename**

**b. 动态，因为调用了 exec 函数启动新的程序 (execve 执行文件)，而非静态 file name**

**c. 可以通过多进程、多线程或 IO 多路复用进行更改**

|    |
|----|
| 得分 |
|    |

第九题（10 分）并发

桌子上有一个水果盘，能容纳一个水果。一家四口人：爸爸、妈妈、儿子、女儿。爸爸专门往盘子里放苹果，妈妈专门往盘子里放桔子；儿子专等盘子里的苹果吃，女儿专等盘子里的桔子吃。

```

dad() {
 while(1) {
 准备好一个苹果；
 ①
 往果盘中放苹果；
 ②
 }
}

mom() {
 while(1) {
 准备好一个桔子；
 ③
 往果盘中放桔子；
 ④
 }
}

boy() {
 while(1) {
 ⑤
 从果盘中拿走苹果；
 ⑥
 吃苹果；
 }
}

girl() {
 while(1) {
 ⑦
 从果盘中拿走桔子；
 ⑧
 吃桔子；
 }
}

```

1. (2 分) 请设计若干信号量，给出每一个信号量的作用和初值。

参考答案（答对两条给 2 分）：

plate: 互斥信号量，标识能否往果盘中放入水果，其初值为 1。

apple: 信号量，标识果盘中是否有苹果，其初值为 0。

orange: 信号量，标识果盘中是否有桔子，其初值为 0。

2. (8 分) 请将信号量上对应的 PV 操作填写在代码中适当位置。

参考答案：

①P(plate);

②V(apple);

③P(plate);

④V(orange);

⑤P(apple);

⑥V(plate);

⑦P(orange);

⑧V(Plate);

# 北京大学信息科学技术学院考试试卷

考试科目：计算机系统导论姓名：学号：

考试时间：2016年1月4日小班教师：

| 题号       | 一                                                                           | 二 | 三 | 四 | 五 | 六 | 七 | 八 | 总分 |
|----------|-----------------------------------------------------------------------------|---|---|---|---|---|---|---|----|
| 分数       |                                                                             |   |   |   |   |   |   |   |    |
| 北京大学考场纪律 |                                                                             |   |   |   |   |   |   |   |    |
| 阅卷人      | 1、考生进入考场后，按照监考老师安排隔位就座，将学生证放在桌面上。无学生证者不能参加考试；迟到超过15分钟不得入场。在考试开始30分钟后方可交卷出场。 |   |   |   |   |   |   |   |    |

- 2、除必要的文具和主考教师允许的工具书、参考书、计算器以外，其它所有物品（包括空白纸张、手机、或有存储、编程、查询功能的电子用品等）不得带入座位，已经带入考场的必须放在监考人员指定的位置。
- 3、考试使用的试题、答卷、草稿纸由监考人员统一发放，考试结束时收回，一律不准带出考场。若有试题印制问题请向监考教师提出，不得向其他考生询问。提前答完试卷，应举手示意请监考人员收卷后方可离开；交卷后不得在考场内逗留或在附近高声交谈。未交卷擅自离开考场，不得重新进入考场答卷。考试结束时间到，考生立即停止答卷，在座位上等待监考人员收卷清点后，方可离场。
- 4、考生要严格遵守考场规则，在规定时间内独立完成答卷。不准交头接耳，不准偷看、夹带、抄袭或者有意让他人抄袭答题内容，不准接传答案或者试卷等。凡有违纪作弊者，一经发现，当场取消其考试资格，并根据《北京大学本科考试工作与学术规范条例》及相关规定严肃处理。
- 5、考生须确认自己填写的个人信息真实、准确，并承担信息填写错误带来的一切责任与后果。
- 学校倡议所有考生以北京大学学生的荣誉与诚信答卷，共同维护北京大学的学术声誉。

以下为试题和答题纸，共页。

|    |
|----|
| 得分 |
|    |

第一题单项选择题（每小题 1 分，共 20 分）

注：请将选择题答案填写在下表中

|    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|
| 题号 | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
| 答案 |    |    |    |    |    |    |    |    |    |    |
| 题号 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| 答案 |    |    |    |    |    |    |    |    |    |    |

1. 下面哪条指令不会引起 esp 的变化？

- A. `movl %esp, %ebp`
- B. `pushl %ebp`
- C. `call printf`
- D. `subl $20, %esp`

答案：A，考察汇编指令和栈的理解。

2. 单精度浮点数 27.5 实际存储在内存中的十六进制数值为：

- A. `0x41ee0000`
- B. `0x425c0000`
- C. `0x41dc0000`
- D. `0x025c0000`

参考信息：单精度浮点数阶码 8 位，尾数 23 位

答案：C，考察浮点数的理解。

3. 下面哪条指令不是 x86 正确的寻址方式

- A. `movl $34, (%eax)`
- B. `movl (%eax), %eax`
- C. `movl $23, 10(%edx, %eax)`
- D. `movl (%eax), 8(%ebx)`

答案：D。考察寻址方式的理解。

4. 以下关于静态库链接的描述中，正确的是：
- A. 链接时，链接器会拷贝静态库中的所有目标模块。
  - B. 使用库的时候必须把它们放在命令行的结尾处。
  - C. 如果库不是相互独立的，那么它们必须排序。
  - D. 每个库在命令行只须出现一次即可。

【答案】C。

解释：A：链接时只需拷贝用到的目标模块；B：静态库也可以放在目标文件的前面；C：正确；D：如果相互调用的库，在命令行必须重复出现。

5. 在 `foo.c` 文件中包含如下代码：

```
int foo(void) {
 int error = printf("You ran into a problem!\n");
 return error;
}
```

经过编译和链接之后，字符串 `"You ran into a problem!\n"` 会出现在哪个段中？

- A. `.bss`
- B. `.data`
- C. `.rodata`
- D. `.text`

【答案】C。

6. 一段程序中阻塞了 `SIGCHLD` 和 `SIGUSR1` 信号。接下来，向它按顺序发送 `SIGCHLD`, `SIGUSR1`, `SIGCHLD` 信号，当程序取消阻塞继续执行时，将处理这三个信号中的哪几个？
- A. 都不处理
  - B. 处理一次 `SIGCHLD`
  - C. 处理一次 `SIGCHLD`，一次 `SIGUSR1`
  - D. 处理所有三个信号

答案：C。

7. 学完本课程后，几位同学聚在一起讨论有关异常的话题，请问你认为他们中谁学习的结果有错误？



- A. 发生异常和异常处理意味着控制流的突变。
- B. 与异常相关的处理是由硬件和操作系统共同完成的。
- C. 异常是由于计算机系统发生了不可恢复的错误导致的。
- D. 异常的发生可能是异步的，也可能是同步的。

答案：C。异常不一定是不可恢复的，也可能是可恢复的、甚至是有意产生的。

8. 下列说法正确的是：

- A. SIGTSTP 信号既不能被捕获，也不能被忽略
- B. 存在信号的默认处理行为是进程停止直到被 SIGCONT 信号重启
- C. 系统调用不能被中断，因为那是操作系统的工作
- D. 子进程能给父进程发送信号，但不能发送给兄弟进程

答案：B

A: SIGTSTP 可以被忽略；既不能被捕获又不能被忽略的是 SIGKILL 和 SIGSTOP

C: 系统调用可以被中断，如 read 这样的慢速系统调用

D: 可以通过 kill(0, ...) 给整个组的进程发送信号

9. 在系统调用成功的情况下，下面哪个输出是可能的？

```
int main() {
 int pid = fork();
 if (pid == 0) {
 printf("A");
 } else {
 pid = fork();
 if (pid == 0) {
 printf("A");
 } else {
 printf("B");
 }
 }
 exit(0);
}
```

- A. AAB
- B. AAA
- C. AABB
- D. AA

答案：A

总共产生 3 个子进程，共输出 2 个 A 和 1 个 B

10. 以下四句都是关于 Unix I/O 的说法。其中正确的是：

- A. 从网络套接字 (socket) 读取内容时，可以通过反复读的方式处理不足值问题，直到读完所需要的数量或遇到 EOF 为止。
- B. 以 O\_RDWR 方式打开文件后，文件会有两个指针，分别记录读文件的当前位置和写文件的当前位置。
- C. 用 read 函数直接读取控制台输入的文本行，会自动在行末追加 '\0' 字符。
- D. 使用 dup2(4, 1) 成功进行重定向后执行 close(4)，会导致 1 号文件描述符也不可用。

参考信息: O\_RDWR 表示文件可读可写; dup2(oldfd, newfd) 表示将 oldfd 重定向给 newfd。

答案：A。

说明：B. 并没有两个指针，读写操作是共用一个指针的；C. read 操作不会追加 '\0' 字符；D. close(4) 后 1 号描述符仍然可用。

11. 下面是一段 C 程序代码：

```
#include <stdio.h>
#include "csapp.h"

int main()
{
 printf("2");
 if (Fork())
 {
 printf("33");
 Write(STDOUT_FILENO, "lol", 3);
 }
 else
 {
 Sleep(1);
 printf("233");
 Write(STDOUT_FILENO, "hhhh", 4);
 }
}
```

```
 }
 fflush(stdout);
 return 0;
}
```

编译后运行程序，程序正常退出。那么程序的输出是：

- A. 233lol233hhhh
- B. lol233hhhh2233
- C. 233lol2233hhhh
- D. 2lol33hhhh233

答案：B。

说明：printf 所属的标准 IO 是有缓冲区的，直到关闭文件、遇到换行符或者 fflush 时才会输出；write 则没有缓冲区立刻输出。

12. 根据编译器安全优化的策略，如下手工程序代码的优化，哪个达不到优化效果？

- A. 循环展开，以减少循环的迭代次数
- B. 消除不必要的存储器引用，减少访存开销
- C. 将函数调用移到循环内，以提高程序的模块性
- D. 分离多个累计变量，以提高并行性

答案：C

13. 动态管理器分配策略中，最适合“最佳适配算法”的空白区组织方式是：

- A. 按大小递减顺序排列
- B. 按大小递增顺序排列
- C. 按地址由小到大排列
- D. 按地址由大到小排列

答案：B。

说明：最佳适应算法的空白区是按小大递增顺利链接在一起。

14. 虚拟内存管理方式可行性的基础是：

- A. 程序执行的离散性
- B. 程序执行的顺序性
- C. 程序执行的局部性
- D. 程序执行的并发性

答案： C。考察虚拟内存管理方式可行性是利用了程序执行的局部性。

15. Intel 的 IA32 体系结构采用二级页表，称第一级页表为页目录（Page Directory），第二级页表为页表（Page Table）。页面的大小为 4KB，页表项 4 字节。以下给出了页目录与若干页表中的部分内容，例如，页目录中的第 1 个项索引到的是页表 3，页表 1 中的第 3 个项索引到的是物理地址中的第 5 个页。则十六进制逻辑地址 8052CB 经过地址转换后形成的物理地址应为十进制的（）

| 页目录 |     | 页表 1 |    | 页表 2 |    | 页表 3 |    |
|-----|-----|------|----|------|----|------|----|
| VPN | 页表号 | VPN  | 页号 | VPN  | 页号 | VPN  | 页号 |
| 1   | 3   | 3    | 5  | 2    | 1  | 2    | 9  |
| 2   | 1   | 4    | 2  | 4    | 4  | 3    | 8  |
| 3   | 2   | 5    | 7  | 8    | 6  | 5    | 3  |

- A. 21195
- B. 29387
- C. 21126
- D. 47195

答案： 4KB=2<sup>12</sup>，所以页内地址有 12 位。4KB/4B=1K，所以页目录和每个页表中的页表项数为 1K 个。因此，在 32 位的虚拟地址中，最低的 12 位为页内地址（Offset），最高的 10 位为页目录的虚拟地址（Dir），中间 10 位为页表的虚拟地址（Table）。十六进制逻辑地址 8052CB 转换为二进制后为 10 000000101001011001011，Dir 为 10，即 10 进制的 2，在表中对应到页表 1。Table 为 101，即 10 进制的 5，在表中对应到物理页面 7。因此，物理地址应为 7 的二进制 111 和 Offset 的拼合，即 111001011001011，转换为十进制为 29387，答案为 B。

16. 已知某系统页面长 8KB，页表项 4 字节，采用多层分页策略映射 64 位虚拟地址空间。若限定最高层页表占 1 页，则它可以采用多少层的分页策略？

- A. 3 层
- B. 4 层
- C. 5 层
- D. 6 层

答案：C。由题意，64 位虚拟地址的虚拟空间大小为  $2^{64}$ 。页面长为 8KB，页表项 4 字节，所以一个页面可存放 2K 个表项。由于最高层页表占 1 页，也就是说其页表项个数最多为 2K 个，每一项对应一页，每页又可存放 2K 个页表项，依次类推可知，采用的分页层数为：5 层。

17. HTTP 协议中，哪个命令可以用来获取动态内容？

- A. HEAD
- B. GET
- C. POST
- D. PUT

答案：B 或 C 都正确

说明：POST 只能用于获得静态内容

18. 下列关于计算机网络概念的说法中，哪一项是正确的？

- A. HUB 会把它任意端口上接收到的帧只转发到它的目的地去
- B. 当在不同的 LAN 中的主机 A 和主机 B 通信的过程中，他们的数据包中的 LAN frame header 不会变化
- C. 162.105.0.0 是一个 B 类地址
- D. 同一台主机每次进入相同的网络，通过动态地址分配的到的 IP 地址总是相同的

答案：C

说明：HUB 会广播它任意端口上接收到的帧；LAN frame header 的目的地地址在经历的不同的 LAN 中会不断调整；动态分配地址和主机地址没有唯一映射关系

19. 有如下代码：

```
int counter = 0;

void * thread(void * vargp)
{
 intthread_var = ((int *) vargp);
 staticintthread_counter = 0;
 thread_internal(thread_var);
 thread_counter ++;
```

```

 return NULL;
 }

 int main (int argc, const char ** argv)
 {
 int tid1, tid2;

 int var = atoi(argv[1]);

 Pthread_create(&tid1, NULL, thread, (void *)var);
 Pthread_create(&tid2, NULL, thread, (void *)var);

 Pthread_join(tid1, NULL);
 Pthread_join(tid2, NULL);

 return 0;
 }

```

则，线程 tid1 与线程 tid2 可以共享的变量是

- A. counter, var
- B. counter, thread\_counter
- C. var, thread\_counter
- D. thread\_var, thread\_counter

答案: B

(本题考查对线程中共享变量的概念的理解。因为 counter 是全局变量；thread\_counter 是静态局部变量，所以两个线程可以共享它们)

20. 有四个信号量，初值分别为: a=1, b=1, c=1, d=1。

| 线程①   | 线程②   | 线程③   |
|-------|-------|-------|
| P(a); | P(d); | P(d); |
| P(d); | P(a); | P(c); |
| P(c); | P(c); | P(b); |
| P(b); | P(b); | P(a); |
| V(c); | V(d); | V(c); |
| V(b); | P(d); | V(b); |
| V(d); | V(a); | V(a); |

|       |                         |       |
|-------|-------------------------|-------|
| V(a); | V(b);<br>V(c);<br>V(d); | V(d); |
|-------|-------------------------|-------|

下列哪两个线程并发执行时，一定不会发生死锁？

- A. ①, ②
- B. ①, ③
- C. ②, ③
- D. 以上选项均不正确

答案：D

（本题考查对死锁概念的理解，本题的情况是任意两个线程并发执行，都会产生死锁）

|    |
|----|
| 得分 |
|    |

## 第二题（12 分）汇编

下面分别是一个程序的 C 语言代码、汇编语言代码，以及其执行结果，请根据其逻辑分别填写空出来的内容：

### 1. C 语言代码

```
#include <stdio.h>

long f1(long x, long y)
{
 return _____ (1) _____;
}

long f2(long x, long y)
{
 return _____ (2) _____;
}

long a[6] = {1, 0, 0, 0, 0, 0};

void foo(void)
{
 long (*f)(long, long);
 _____ (3) _____ long count = 0;
 long i;

 for (i=0; i<_____ (4) _____; i++) {
 if ((count % 2) == 0)
 f = f1;
 else
 f = f2;

 a[i+1] = f(a[i], _____ (5) _____);
 count ++;
 }
```



```

 for (i=0; i<6; i++) {
 printf("%ld\n", a[i]);
 }
}

```

```

int main()
{
 foo();
 foo();
}

```

## 2. 汇编语言代码

.LC0:

```

 .string (6)

```

f1:

```

 leaq (%rdi,%rsi), %rax
 ret

```

f2:

```

 movq %rdi, %rax
 imulq %rsi, %rax
 ret

```

foo:

```

 pushq %r12
 pushq %rbp
 (7)
 movl $a, %ebx
 movl $a + (8), %r12d
 movl $f1, %ebp

```

.L5:

```

 movq count, %rsi
 movq %rsi, %rax
 andl $1, %eax
 movl $f2, %eax
 cmovbe %rbp, %rax

```

```

movq (%rbx), %rdi
call *%rax
movq %_____(9)_____, 8(%rbx)
addq $1, count
addq $8, %rbx
cmpq %r12, %rbx
jne.L5
movl $0, %ebx
.L6:
movq a(,%rbx,8), %rdx
movl $.LC0, %esi
movl $1, %edi
movl $0, %eax
call printf
addq $1, %rbx
cmpq $6, %rbx
jne.L6
popq %rbx
popq %rbp
_____(10)_____

ret

```

### 3. 输出结果

```

1
1
1
3
_____(11)_____

13
1
5
11
77
85
_____(12)_____


```

答案：（每空一分）

1) (x + y)

2) (x \* y)

3) static

4) 5

5) count

6) "%ld\n"（必需有引号）

7) pushq %rbx

8) 0x40 或者 64（40 也可给分，因为汇编默认是 16 进制）

9) rax

10) popq %r12

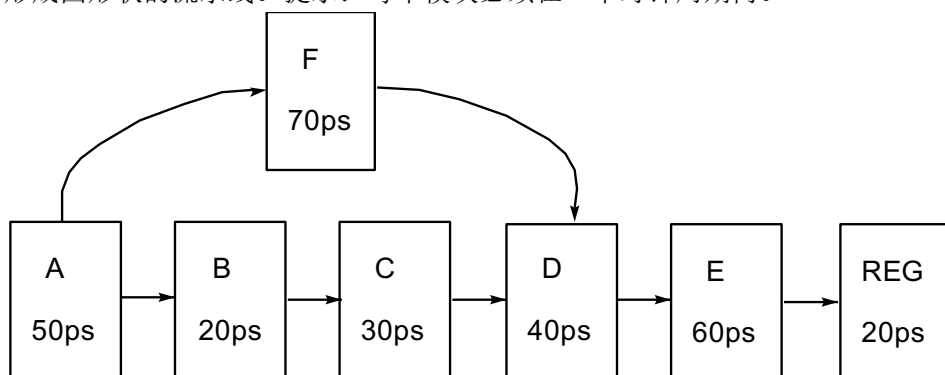
11) 9

12) 765

|    |
|----|
| 得分 |
|    |

### 第三题（13 分）处理器

如图所示，每个模块表示一个单独的组合逻辑单元，每个单元的延迟已在图中标出。通过在两个单元间添加寄存器的方式，可以对该数据通路进行流水化改造。假设每个寄存器的延迟为 20ps。设计人员考虑在额外增加一个模块 F 支持新的指令功能，形成图形状的流水线。提示：每个模块必须在一个时钟周期内。



1) 如果没有 F 模块，请计算该流水线改造前的吞吐率，并说明计算过程。结果保留小数点后两位。

$$1000 / (50 + 20 + 30 + 40 + 60 + 20) = 1000 / 220 = 4.55 \text{ GIPS (过程和结果 2 分)}$$

2) 如果有 F 模块，请计算该流水线改造前的吞吐率，并说明计算过程。结果保留小数点后两位。

$$1000 / (50 + 70 + 40 + 60 + 20) = 1000 / 240 = 4.17 \text{ GIPS (过程和结果 2 分)}$$

3) 如果有 F 模块，改造为一个二级流水线（可以插入多个寄存器），为获得最大的吞吐率，寄存器应在哪里插入？请计算该流水线的吞吐率，并说明计算过程。结果保留小数点后两位。

插入在 FD 间和 CD 间（2 分）

$$1000 / (50 + 70 + 20) = 1000 / 140 = 7.14 \text{ GIPS (过程和结果 2 分)}$$

4) 如果有 F 模块，改造为一个三级流水线（插入多个寄存器），为获得最大的吞吐率，寄存器应在哪里插入？请计算该流水线的吞吐率，并说明计算过程。结果保留小数点后两位。

插入在 AF 间、FD 间、CD 间（3 分，可以酌情部分给分）

$$1000 / (40 + 60 + 20) = 1000 / 120 = 8.33 \text{ GIPS (过程和结果 2 分)}$$

|    |
|----|
| 得分 |
|    |

第四题（10 分）链接

在x86\_64环境下，考虑如下2个文件：main.c和foo.c：

```

/* main.c */
#include <stdio.h>

longlong _____;
const char* foo(int);

int main(int argc, char **argv){
 int n = 0;
 sscanf(argv[1], "%d", &n);
 printf(foo(n));
 printf("%llx\n", a);
}

/* foo.c */
#include <stdio.h>

int a[2];

static void swapper(int num){
 int swapper;
 if (num % 2){
 swapper = a[0];
 a[0] = a[1];
 a[1] = swapper;
 }
}

const char* foo(int num){
 static char out_buf[50];
 swapper(num);

```

```

 sprintf(out_buf, "%x\n", _____);
 return out_buf;
 }

```

1. 对于每个程序中的相应符号，给出它的属性（局部或全局，强符号或弱符号）  
（提示：如果某表项中的内容无法确定，请画 x。）

main.c

|     | 局部或全局？ | 强或弱？ |
|-----|--------|------|
| a   |        |      |
| foo |        |      |

foo.c

|         | 局部或全局？ | 强或弱？ |
|---------|--------|------|
| a       |        |      |
| foo     |        |      |
| out_buf |        |      |

2. 根据如下的程序运行结果，补全程序【在程序空白处填空即可】。

```
$ gcc -o test main.cfoo.c
```

```
$./test 1
```

```
bffedead
```

```
cafebffedeadbeef
```

```
$./test 2
```

```
beefcafe
```

```
deadbeefcafebffe
```

3. 现在有一位程序员要为此程序编写头文件。假设新的头文件名称为 foo.h，内容如下：

```
extern long long a;
```

```
extern char *foo(int);
```

然后在 main.c 和 foo.c 中分别引用该头文件，请问编译链接能通过吗？请说明理由。

答案:

1. (5 分) 对于每个程序中的相应符号, 给出它的属性 (局部或全局, 强符号或弱符号) (提示: 如果某表项中的内容无法确定, 请画 x。)

(每格 0.5 分)

main.c

|     | 局部或全局? | 强或弱? |
|-----|--------|------|
| A   | 全局     | 强    |
| foo | 全局     | 弱    |

foo.c

|         | 局部或全局? | 强或弱? |
|---------|--------|------|
| A       | 全局     | 弱    |
| foo     | 全局     | 强    |
| out_buf | 局部     | x    |

2. (3 分) 根据程序运行结果, 补全程序

long long a = 0xdeadbeefcafebfef; (1 分)

\*(int\*)((unsigned long long)a + 2) (2 分)

3. 请问编译链接能通过吗? 请说明理由。

不能。无论如何声明 a 的类型都会造成在至少一个文件内引起声明和定义冲突。

结论 1 分, 理由 2 分。(结论错不得分)

|    |
|----|
| 得分 |
|    |

第五题（10 分）异常

以下程序运行时系统调用全部正确执行，且每个信号都被处理到。请给出代码运行后所有可能的输出结果。

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

int c = 1;

void handler1(int sig) {
 c++;
 printf("%d", c);
}

int main() {

 signal(SIGUSR1, handler1);
 sigset_t s;
 sigemptyset(&s);
 sigaddset(&s, SIGUSR1);
 sigprocmask(SIG_BLOCK, &s, 0);

 int pid = fork()?fork():fork();

 if (pid == 0) {
 kill(getppid(), SIGUSR1);
 printf("S");
 sigprocmask(SIG_UNBLOCK, &s, 0);
 exit(0);
 } else {
```



```
 while (waitpid(-1, NULL, 0) != -1);
 sigprocmask(SIG_UNBLOCK, &s, 0);
 printf("P");
 }
 return 0;
}
```

答:

答案:

共 5 种:

S2PS2P

SS2P2P

S2S2PP

S2SP2P

SS22PP

|    |
|----|
| 得分 |
|    |

#### 第六题（15 分）虚拟内存地址转换

为了提升虚拟内存地址的转换效率，降低遍历两级页表结构所带来的地址转换开销，英特尔处理器中引入了大页 TLB，即一个 TLB 项可以涵盖整个 4MB 对齐的地址空间（针对 32 位模式）。只要设置页目录页中页目录项（PDE）的大页标志位，即可让 MMU 识别这是一个大页 PDE，并加载到大页 TLB 项中。大页 PDE 中记录的物理内存页面号必须是 4MB 对齐的，并且整个连续的 4MB 内存均可统一通过该大页 PDE 进行地址转换。

在 32 位的 Linux 系统中，为了方便访问物理内存，内核将地址 0~768MB 间的物理内存映射到虚拟内存地址 3GB~3GB+768MB 上，并通过大页 PDE 进行进行该区间的地址转换。任何 0~768MB 的物理内存地址可以直接通过加 3G（0xC0000000）的方式得到其虚拟内存地址。在内核中，除了该区间的内存外，其他地址的内存通常都通过普通的两级页表结构来进行地址转换。

假设在我们使用的处理器中有 2 个大页 TLB 项，其当前状态如下：

| 索引号 | TLB 标记 | 页面号     | 有效位 |
|-----|--------|---------|-----|
| 0   | 0xC48  | 0x04800 | 1   |
| 1   | 0xC9C  | 0x09C00 | 1   |

有 4 个普通 TLB 项，当前的状态如下：

| 索引号 | TLB 标记  | 页面号     | 有效位 |
|-----|---------|---------|-----|
| 0   | 0xF8034 | 0x04812 | 1   |
| 1   | 0xF8033 | 0x09812 | 1   |
| 2   | 0xF4427 | 0x12137 | 1   |
| 3   | 0xF44AE | 0x17343 | 1   |

当前页活跃的目录页（PD）中的部分 PDE 的内容如下：

| PDE 索引 | 页面号     | 其他标志 | 大页位 | 存在位 |
|--------|---------|------|-----|-----|
| 786    | 0x04800 | ...  | 1   | 1   |
| 807    | 0x09C00 | ...  | 1   | 1   |
| 977    | 0x09C33 | ...  | 0   | 1   |
| 992    | 0x09078 | ...  | 0   | 1   |

注：普通页面大小为 4KB，并且 4KB 对齐。每个页面的页面号为其页面起始物理地址除以 4096 得到。大页由连续 1024 个 4KB 小页组成，且 4MB 对齐。

1. 分析下面的指令序列，

```
movl $0xC48012024, %ebx
movl $128, (%ebx)
movl $0xF8034000, %ecx
movl $36(%ecx), %eax
```

请问，执行完上述指令后，eax 寄存器中的内容是（）；在执行上述指令过程中，共发生了（）次 TLB miss？同时会发生（）次 page fault？

注：不能确定时填写“--”。

2. 请判断下列页面号对应的页面中，哪些一定是页表页？哪些不是？哪些不确定？

| 页面号     | 是否为页表页（是/不是/不确定） |
|---------|------------------|
| 0x04800 | 4                |
| 0x09C33 | 5                |
| 0x09812 | 6                |

3. 下列虚拟地址中哪一个对应着够将虚拟内存地址 0xF4427048 映射到物理内存地址 0x14321048 的页表项（）？

(A) 0x09C33027                      (B) 0xC9C3309C  
(C) 0xC9C33027                      (D) 0x09C3309C

通过上述虚拟地址，利用 movl 指令修改对应的页表项，完成上述映射，在此过程中，是否会产生 TLB miss？（）（回答：会/不会/不确定）

修改页表项后，是否可以立即直接使用下面的指令序列将物理内存地址 0x14321048 开始的一个 32 位整数清零？为什么？

```
movl $0xF4427048, %ebx
movl $0, (%ebx)
```

答：

答案：

第 1 小题（各 1 分）

(1) 128；(2) 0；(3) 0；

两个虚拟地址映射的是同一块物理内存；因此读出的就是写入的；此过程中全部 TLB 命中，因而既无 TLB miss，也不会有 page fault。

第 2 小题（各 2 分）

（1）不确定；因为是大页，一定不是当前页目录项对应的页表页，但不一定该页面不会用作其他页目录项对应的页表页。

（2）是；当前页目录项（977）对应的页表页。

（3）不确定；任何页面都可能用作页表页。

第 3 小题

B；（2 分）

虚拟地址对应的页表页的页面号（0x09C33）已知，通过其地址直接加 3G（即 0xC0000000），即可得到当前页表页的基地址（0xC9C33000），在加上对应的第 0x27 乘以 4 到页内偏移。

不会；（2 分）

因为地址 0xC9C33000 在大页映射范围内，已经被大页 TLB 项覆盖到了，会直接命中。

不能直接修改。（1 分）

因为 TLB 项中的内容和页表中的内容不一致，需要将对应的 TLB 项设置为失效，然后通过 TLB miss 重新加载页表结构中新的地址映射关系，之后才能访问对应的虚拟地址。（1 分）

|    |
|----|
| 得分 |
|    |

第七题（10 分）网络

1. 请根据 web 应用在计算机网络中的定义以及其在协议栈自上而下在软件中的实现，把以下关键字填入表格

注：同一个关键词可能被填入多次；不是每一个关键词都必须被填入

Streams (end to end), Datagrams, web content, IP, TCP, UDP, Kernel code, User code

| 协议   | 数据包类型 | 软件实现 |
|------|-------|------|
| HTTP |       |      |
|      |       |      |
|      |       |      |

答案：（每空 1 分，共 8 分）

| 协议   | 数据包类型       | 软件实现        |
|------|-------------|-------------|
| HTTP | Web content | User code   |
| TCP  | Streams     | Kernel code |
| IP   | Datagrams   | Kernel code |

2. 以下关于互联网的说法中哪些是正确的？并简要说明原因

- A. 在 client-server 模型中，server 通常使用监听套接字 listenfd 和多个 client 同时通信
- B. 在 client-server 模型中，套接字是一种文件标识符
- C. 准确地说，IP 地址是用于标识主机的 adapter (network interface card)，并非主机
- D. Web 是一种互联网协议
- E. 域名和 IP 地址是一一对应的
- F. Internet 是一种 internet

答：

答案：B, C, F（全对得 2 分，有错得 0 分，部分对得 1 分）

说明的原因供评分时参考

说明: `server` 会为每一个 `client` 单独创建一个套接字进行通信; `Web` 是一种基于 `HTTP` 协议的互联网应用; 一个域名可以对应多个 `IP` 地址, 一个 `IP` 地址也可以对应多个域名

|    |
|----|
| 得分 |
|    |

# 第八题（10 分）并发

有三个线程 PA、PB、PC 协作工作以解决文件打印问题：PA 将记录从磁盘读入内存缓冲区 Buff1，每执行一次读一个记录；PB 将缓冲区 Buff1 的内容复制到缓冲区 Buff2，每执行一次复制一个记录；PC 将缓冲区 Buff2 的内容打印出来，每执行一次打印一个记录。缓冲区 Buff1 可以放 4 个记录；缓冲区 Buff2 可以放 8 个记录。请用信号量及 P、V 操作实现上述三个线程以保证文件的正确打印。

```

PA() {
 while(1) {
 ①
 从磁盘读入一个记录
 ②
 将记录放入 Buff1
 ③
 }
}

PB() {
 while(1) {
 ④
 从 Buff1 中取出一个记录
 ⑤
 将记录放入 Buff2
 ⑥
 }
}

PC() {
 while(1) {
 ⑦
 从 Buff2 中取出一个记录
 ⑧
 打印
 }
}

```

1. 请设计若干信号量，给出每一个信号量的作用和初值。

2. 请将信号量上对应的 PV 操作填写在代码中适当位置。注意：每一标号处可以不填入语句（请标记成 x），或填入一条或多条语句。

| 标号 | 对应的操作 |
|----|-------|
| ①  |       |
| ②  |       |
| ③  |       |
| ④  |       |
| ⑤  |       |
| ⑥  |       |
| ⑦  |       |
| ⑧  |       |

答案：（本大题总得分按四舍五入取整数）



1. 3 分，答对 1 个 0.5 分

empty1, 初值 4;

full1, 初值 0;

empty2, 初值 8;

full2, 初值 0;

mutex1, 初值 1;

mutex2, 初值 2

2. 7 分

| 标号 | 对应的操作                                          |
|----|------------------------------------------------|
| ①  | X                                              |
| ②  | P(&empty1);<br>P(&mutex1);<br>注意：顺序不能错，各 0.5 分 |
| ③  | V(&full1);<br>V(&mutex1);<br>各 0.5 分           |
| ④  | P(&full1);<br>P(&mutex1);<br>注意：顺序不能错，各 0.5 分  |
| ⑤  | P(&empty2);<br>P(&mutex2);<br>注意：顺序不能错，各 0.5 分 |
| ⑥  | V(&mutex2)<br>V(&full2)<br>各 0.5 分             |
| ⑦  | P(&full2);<br>P(&mutex2);<br>注意：顺序不能错，各 0.5 分  |
| ⑧  | V(&mutex2)<br>V(&empty2)<br>各 0.5 分            |

# 北京大学信息科学技术学院考试试卷

考试科目：计算机系统导论 姓名：\_\_\_\_\_ 学号：\_\_\_\_\_

考试时间：2017 年 1 月 2 日 小班教师：\_\_\_\_\_

| 题号  | 一 | 二 | 三 | 四 | 五 | 六 | 七 | 八 | 总分 |
|-----|---|---|---|---|---|---|---|---|----|
| 分数  |   |   |   |   |   |   |   |   |    |
| 阅卷人 |   |   |   |   |   |   |   |   |    |

## 北京大学考场纪律

1、考生进入考场后，按照监考老师安排隔位就座，将学生证放在桌面上。无学生证者不能参加考试；迟到超过 15 分钟不得入场。在考试开始 30 分钟后方可交卷出场。

2、除必要的文具和主考教师允许的工具书、参考书、计算器以外，其它所有物品（包括空白纸张、手机、或有存储、编程、查询功能的电子用品等）不得带入座位，已经带入考场的必须放在监考人员指定的位置。

3、考试使用的试题、答卷、草稿纸由监考人员统一发放，考试结束时收回，一律不准带出考场。若有试题印制问题请向监考教师提出，不得向其他考生询问。提前答完试卷，应举手示意请监考人员收卷后方可离开；交卷后不得在考场内逗留或在附近高声交谈。未交卷擅自离开考场，不得重新进入考场答卷。考试结束时间到，考生立即停止答卷，在座位上等待监考人员收卷清点后，方可离场。

4、考生要严格遵守考场规则，在规定时间内独立完成答卷。不准交头接耳，不准偷看、夹带、抄袭或者有意让他人抄袭答题内容，不准接传答案或者试卷等。凡有违纪作弊者，一经发现，当场取消其考试资格，并根据《北京大学本科考试工作与学术规范条例》及相关规定严肃处理。

5、考生须确认自己填写的个人信息真实、准确，并承担信息填写错误带来的一切责任与后果。

学校倡议所有考生以北京大学学生的荣誉与诚信答卷，共同维护北京大学的学术声誉。

以下为试题和答题纸，共 页。

|    |
|----|
| 得分 |
|    |

第一题 单项选择题（每小题 1 分，共 20 分）

注：选择题的回答填写在下表中。

|    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|
| 题号 | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
| 回答 |    |    |    |    |    |    |    |    |    |    |
| 题号 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| 回答 |    |    |    |    |    |    |    |    |    |    |

1. C 语言中的 int 和 unsigned 类型的常数进行比较时，下列表达式及描述正确的是：(注：位宽为 32 位， $T_{\text{MIN}}=-2,147,483,648$ ， $T_{\text{MAX}}=2,147,483,647$ )
- A.  $0 == 0U$ ，按有符号数进行比较
  - B.  $2147483647U > -2147483647-1$ ，按无符号数进行比较
  - C.  $(\text{unsigned})-1 < -2$ ，按无符号数进行比较
  - D.  $2147483647 > (\text{int})2147483648U$ ，按有符号数进行比较

答案：D

A- 按无符号数进行比较

B- <

C- >

2. 现有一个二进制浮点的表示规则，其中 E 为指数部分，3 比特，且 bias 为 3；M 为小数部分，5 比特，采用二进制补码表示形式，且取值 ( $-\frac{1}{2} \leq |M| < 1$ )；s 是浮点的符号位；该形式包含一个值为 1 的隐藏位。

|   |   |   |
|---|---|---|
| s | E | M |
|---|---|---|

如果用该形式表示  $+5_{10}$ ，s、E 和 M 分别是：

- A. 0, 100, 01100
- B. 0, 101, 00100
- C. 0, 110, 11010
- D. 0, 111, 10101

答案：D

实际上 5 可以表示为  $4+1$ ， $8-3$ ， $16-11$ ，分别对应 B，C，D 但是前两种都不满足 ( $-\frac{1}{2} \leq |M| < 1$ )，因此只能选 D，A 是迷惑选项。

3. 缓冲区溢出会带来程序风险，下列避免方法中错误的是：

- A. 在栈中存放特殊字段用于检测是否发生缓冲区溢出
- B. 避免使用有风险的库函数，如 gets 等
- C. 随机设置栈的偏移地址
- D. 分配尽可能大的缓冲区数组

答案：D

分配大缓冲区无法从根本上解决溢出问题，输入足够长数据就仍然可以实现攻击。

4. 现有四级指令流水线，分别完成取指、取数、运算、传送结果 4 步操作。若完成上述操作的时间依次为 9ns、10ns、6ns、8ns，则流水线的操作周期应设计为 \_\_\_\_\_ ns。

- A. 6      B. 8      C. 9      D. 10

答案：D（考察对流水线运行原理的理解）

5. 对于下面这段程序，哪个描述是正确的： \_\_\_\_\_

```
void upper(char *s)
{
 for(int i=0; i<strlen(s); i++)
 if(s[i]>='a' && s[i]<='z')
 s[i] += ('A'-'a');
}
```

- A. 假设 s 的长度为 N，该程序的时间复杂度为  $O(N\log N)$
- B. 将“for(int i=0; i<strlen(s); i++){ }”修改为：“int len = strlen(s); for(int i=0; i<len; i++){ }”可以使程序运行时间与 s 的长度线性相关
- C. B 选项中的修改策略不影响程序的空间局部性与时间局部性
- D. B 选项中的修改策略仅影响程序的空间局部性，不影响时间局部性

答案：B（考察程序性能优化与程序局部性原理）

6. 某机器内存为 8 位地址，cache 设计参数为  $E=2, t=2, s=2, b=4$ ，cache 块大小为 16 字节。cache 为空的初始状态下，数据访问的地址序列为 0→4→34→162→128→192→2（以字节为单位），请问一共发生多少次 cache 命中？

答：（      ）

- A. 0                      B. 1                      C. 2                      D. 3

答案：B

00 00 0000 miss

```

00 00 0100 hit
00 10 0010 miss
10 10 0010 miss
10 00 0000 miss
11 00 0000 miss
01 00 0000 miss
00 00 0010 miss

```

7. C 源文件 f1.c 和 f2.c 的代码分别如下所示，编译链接生成可执行文件后执行，输出结果为 ( )
- A. 100                      B. 200  
C. 201                      D. 链接错误

|                                                                                                                                                                                   |                                                           |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| <pre>// f1.c #include &lt;stdio.h&gt; static int var = 100; int main(void) {     extern int var ;     extern void f() ;     f() ;     printf("%d\n", var) ;     return 0; }</pre> | <pre>//f2.c int var = 200;  void f() {     var++; }</pre> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|

答案: A(f1.c 中的 extern int var, 会搜索 f1.c 之前定义过的全局变量, 因为之前已经已经有了 var 的定义, 所以 printf 会打印该 var 的值 100)

8. C 源文件 m1.c 和 m2.c 的代码分别如下所示，编译链接生成可执行文件后执行，结果最可能为 ( )

```
$ gcc -o a.out m2.c m1.c ; ./a.out
```

```
0x1083020
```

- A. 0x1083018, 0x108301c                      B. 0x1083028, 0x1083024  
C. 0x1083024, 0x1083028                      D. 0x108301c, 0x1083018

|                                                                                                                                  |                                                                                                         |
|----------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| <pre>// m1.c #include &lt;stdio.h&gt;  int a1 ; int a2 = 2 ; extern int a4 ;  void hello() {     printf("%p ", &amp;a1); }</pre> | <pre>//m2.c int a4 = 10 ;  int main() {     extern void hello() ;      hello() ;     return 0 ; }</pre> |
|----------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|

|                                                               |  |
|---------------------------------------------------------------|--|
| <pre>printf("%p ", &amp;a2); printf("%p\n", &amp;a4); }</pre> |  |
|---------------------------------------------------------------|--|

答案: D (a2 和 a4 在 .data 中, a1 在 .bss 中, 按照布局, .data 地址比 .bss 要小, 又由于 m2.c 的编译先于 m1.c, 故选 D 不选 A )

9. 对于以下一段代码, 可能的输出为:

```
int count = 0;
int pid = fork();
if (pid == 0){
 printf("count = %d\n", --count);
}
else{
 printf("count = %d\n", ++count);
}
printf("count = %d\n", ++count);
```

A. 1 2 -1 0                      B. 0 0 -1 1  
C. 1 -1 0 0                      D. 0 -1 1 2

答案: A (printf 从后向前处理, 注意 fork 出的子进程与父进程执行顺序不确定)

10. 下列哪一事件不会导致信号被发送到进程?

- A. 新连接到达监听端口
- B. 进程访问非法地址
- C. 除零
- D. 上述情况都不对

答案: A (考察对各种情况的理解)

11. 在 C 语言中实现 Mark-and-Sweep 算法时, 可以基于以下哪个假设: (宿主主机为 32 位机器)

- A. 所有指针指向一个块的起始地址
- B. 所有指针数据都是 4 字节对齐
- C. 只需要扫描数据类型为指针的堆中的数据空间
- D. 只需要扫描所有长度为 4 字节的堆中的数据空间

答案: B

12. 在 Core i7 中, 以下哪个页表项属于 4 级页表项, 不属于 1 级页表项:

- A. G 位 (Global Bit)
- B. D 位 (Dirty Bit)
- C. XD 位 (Disable or enable instruction fetch)
- D. U/S 位 (User or supervisor mode access permission)

答案: B

13. 在 Core i7 中, 关于虚拟地址和物理地址的说法, 不正确的是:

- A.  $VPO = CI + CO$
- B.  $PPN = TLBT + TLBI$
- C.  $VPN1 = VPN2 = VPN3 = VPN4$
- D.  $TLBT + TLBI = VPN$

答案: B ( $VPN = TLBT + TLBI$ )

14. 考虑以下代码, 假设 result.txt 中的初始内容为 “666666”

```
char *str1 = "6666";
char *str2 = "2333";
char *str3 = "hhhh";
int fd1, fd2, fd3, i;

fd1 = open("result.txt", O_RDWR);
fd2 = open("result.txt", O_RDWR);
dup2(fd1, fd2);

for (i = 0; i < 5; ++i) {
 fd3 = open("result.txt", O_RDWR);
 write(fd1, str1, 4);
 write(fd2, str2, 4);
 write(fd3, str3, 4);
 close(fd3);
}

close(fd1); close(fd2);
```

假设所有系统调用均成功, 则这段代码执行结束后, result.txt 的内容中有 ( ) 个 “6”

- A. 6
- B. 16
- C. 20
- D. 22

答案: B (考察文件读写位置的理解)

15. 关于 IO 操作，以下说法中正确的是（）

- A. 由于 RIO 包的健壮性，所以 RIO 中的函数都可以交叉调用
- B. 成功调用 open 函数后，返回一个不小于 3 的文件描述符
- C. 调用 Unix I/O 开销较大，标准 I/O 库使用缓冲区来加快 I/O 的速度
- D. 和描述符表一样，每个进程拥有独立的打开文件表

答案：C （RIO 中带缓冲的函数不应与无缓冲的函数交叉调用，A 错误。若标准输入、标准输出或标准错误被关闭，则会返回一个小于 3 的文件描述符，B 错误。打开文件表是所有进程共享的，D 错误）

16. 关于 IP，以下说法正确的是：

- A. IP 协议提供了可信赖的主机与主机之间的数据包传输能力
- B. IP 地址的长度固定为 32 位
- C. 一个域名可以映射到多个 IP，但一个 IP 不能被多个域名映射
- D. 一个域名可以不映射到任何 IP

答案：D

A 错误，不可信赖；B 错误，如 IPv6 地址就是 128 位的；C 错误，存在多个域名映射到一个 IP 的情况

17. 关于以下说法，以下说法正确的是：

- A. HTTP 协议规定服务器端使用 80 端口提供服务
- B. 使用 TCP 来实现数据传输一定是可靠的
- C. Internet 上的两台的主机要通信必须先建立端到端连接
- D. 在 Linux 中只能通过 Socket 接口进行网络编程

答案：B

A 错误，80 是默认端口而非必须；C 错误，IP 协议本身不基于连接，传输层中的 UDP 协议便不基于连接；D 错误，可以通过网卡驱动接口直接收发数据。

18. 假设有一个 HTTPS（基于 HTTP 的一种安全的应用层协议）客户端程序想要通过一个 URL 连接一个电子商务网络服务器获取一个文件，并且这个服务器 URL 的 IP 地址是已知的，以下哪种协议是一定不需要的？

- A. HTTP
- B. TCP
- C. DNS
- D. SSL/TLS

答案：C

DNS 的作用是返回 URL 对应的 IP 地址，因为已知，所以不需要。其他协议都是必须的，因为 HTTPS 是基于 HTTP，SSL 以及 TCP 一种协议。



19. 下列关于“基于进程的并发服务器”的叙述中，哪一个是错误的？

- A. 父子进程共享文件表
- B. 父子进程共享文件描述符
- C. 父子进程不共享用户地址空间
- D. 进程控制和进程间通信开销大

答案：B（考核对基于进程的并发服务器优缺点的理解）

20. 请阅读如下代码：

```
sem_t s;

int main()
{
 int i;
 pthread_t tids[3];
 sem_init(&s, 0, 1);
 for (i=0; i <3; i++) {
 pthread_create(&tids[i], NULL, justdoit, NULL);
 }
 for (i=0; i <3; i++) {
 pthread_join(&tids[i], NULL);
 }
 return 0;
}

int j=0;

void *justdoit(void *arg)
{
 P(&s);
 j = j + 1;
 V(&s);
 printf("%d\n", j);
}
```

下列哪一个输出结果是不可能的？

- A. (1, 3, 2)
- B. (2, 3, 2)
- C. (3, 3, 2)
- D. (2, 1, 2)

答案：D（本题考查对多线程、信号量使用的掌握）

|    |
|----|
| 得分 |
|    |

## 第二题（12 分）

下面程序是一个完整 C 函数 myfunction 经编译器优化后生成的 32 位汇编语言代码：

```
myfunction:
 pushl %ecx
 movl 0x10(%esp), %ecx
 movl 8(%esp), %edx
 pushl %ebx
 movl 0x18(%esp), %ebx
 pushl %esi
 movl 0x14(%esp), %esi
 pushl %ebp
 xorl %eax, %eax
 pushl %edi
 jmp .L2
 leal (%esp), %esp
.L2:
 movl 8(%esi), %edi
 imull 4(%edx,%eax,8), %edi
 movl (%esi), %ebp
 imull (%edx,%eax,8), %ebp
 addl %ebp, %edi
 movl %edi, (%ecx)
 movl 0xc(%esi), %edi
 imull 4(%edx,%eax,8), %edi
 movl 4(%esi), %ebp
 imull (%edx,%eax,8), %ebp
 addl %ebp, %edi
 movl (%ecx), %ebp
 addl %edi, %ebp
 addl %ebp, %ebx
 movl %edi, 4(%ecx)
 incl %eax
 addl $8, %ecx
 cmpl $2, %eax
 jl .L2
 popl %edi
 popl %ebp
 popl %esi
 movl %ebx, %eax
 popl %ebx
 popl %ecx
```

```
ret
```

提示：32 位汇编使用栈来传递所有参数，参数压栈顺序为从右至左。

1. 请根据上述汇编代码，根据提示，补全 myfunction 的代码。注意，每行一条语句；不能定义新的变量。

```
int myfunction(int a[], int b[], int c[], int d) {
 int i;
 for (i = 0; i < _____; i++) {

 _____ = _____;

 _____ = _____;

 _____ = _____;
 }
 return _____;
}
```

2. 请给出以下程序的输出结果。

```
int main()
{
 int a[5] = {1, 2, 3, 4, 5};
 int b[5] = {9, 8, 5, 6, 4};
 int c[5] = {7, 9, 8, 10, 11};
 int d = 5;
 int ret = myfunction(a, b, c, d);
 printf("%d\n", ret);
 return 0;
}
```

**答案：**

1. for (i = 0; i < 2; i++) { // (1 分)

//下面第一二个等式左边都对得 1 分，第三个等式左边写对得 1 分

c[2\*i] = b[0] \* a[2\*i] + b[2] \* a[2\*i+1]; // (等式右边乘加乘序列得 1 分，等式右边写对 2\*i 和 2\*i+1 得 1 分)

c[2\*i+1] = b[1] \* a[2\*i] + b[3] \* a[2\*i+1]; // (等式右边乘加乘序列得 1 分，等式右边写对 2\*i 和 2\*i+1 得 1 分)

d = d + c[2\*i] + c[2\*i+1]; // (等式右边两个加号得 1 分，等式右边写对 2\*i 和 2\*i+1 得 1 分)

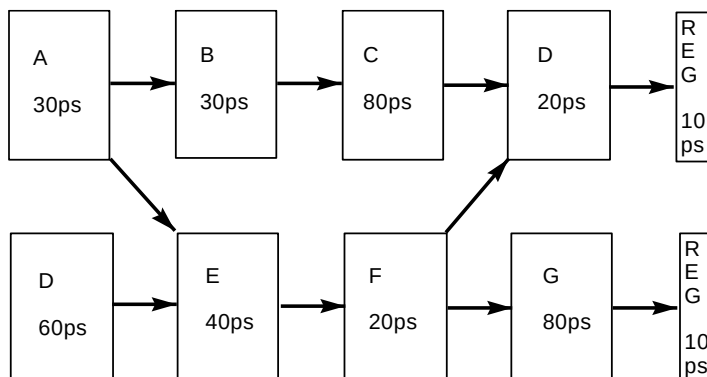
return d; // (1 分)

2. 139 (2 分)

|    |
|----|
| 得分 |
|    |

### 第三题（12 分）

如图所示，每个模块表示一个单独的组合逻辑单元，每个单元的延迟以及数据依赖关系已在图中标出。通过在两个单元间添加寄存器的方式，可以对该数据通路进行流水化改造。假设每个寄存器的延迟为 10ps。



1) 如果改造为一个二级流水线，为获得最大的吞吐率，该寄存器应在哪里插入？请计算该流水线的吞吐率，并说明计算过程。结果可以是分数形式也可以是小数形式。

**插入在 BC 间以及 EF（4 分，不完整酌情扣 1~2 分）**

**$1000 / (80 + 20 + 10) = 1000 / 110 = 9.09 \text{ GIPS}$ （正确结果 1 分，单位 1 分）。**

2) 如果改造为一个三级流水线，为获得最大的吞吐率，寄存器应在哪里插入？请计算该流水线的吞吐率，并说明计算过程。结果可以是分数形式也可以是小数形式。

**插入在 AE、FD、BC、CD、DE 和 FG 间（4 分，不完整酌情扣 1~2 分）**

**$1000 / (80 + 10) = 1000 / 90 = 11.11 \text{ GIPS}$ （正确结果 1 分，单位 1 分）。**

|    |
|----|
| 得分 |
|    |

#### 第四题 (10 分)

在 x86\_64 环境下, 考虑如下 4 个文件 (main.c, value.c, f1.c, f2.c):

```
/* main.c */
#include <stdio.h>

extern void f_void_void();
extern int f_int_void();
void *f;

int main()
{
 int a = 1, b, c;

 f = (void *)f_void_void;

 ((void (*)(int))f)(a);
 b = ((int (*)(void))f)();
 printf("b = %d\n", b);

 f = (void *)f_int_void;

 ((void (*)(void))f)(a);
 c = ((int (*)(int))f)(a);
 printf("c = %d\n", c);

 return 0;
}

/* value.c */
int BIG;

/* f1.c */
#include <stdio.h>

extern int BIG;
int small = 1;

void f_void_void() {
 small += 1;
 BIG += 1;
 printf("small = %d, BIG = %d\n", small, BIG);
}
```

```

/* f2.c */
#include <stdio.h>

extern int BIG;
static int small;

int f_int_void() {
 small += 1;
 BIG += 1;
 printf("small = %d, BIG = %d\n", small, BIG);
 return small + 1;
}

```

使用命令

```
gcc -o main main.c f1.c f2.c value.c
```

编译这四个文件。

使用

```
./main
```

运行编译好的程序。

1. 对于程序中的相应符号，请给出它的属性（局部或全局，强符号或弱符号），不确定的请画 X。

| 源文件     | 符号名   | 局部或全局? | 强符号或弱符号? |
|---------|-------|--------|----------|
| main.c  | f     |        |          |
| value.c | BIG   |        |          |
| f1.c    | small |        |          |
| f2.c    | small |        |          |

2. 请补全程序运行的输出，不确定的请画 X。

```

small = _____, BIG = _____
small = _____, BIG = _____
b = _____
small = _____, BIG = _____
small = _____, BIG = _____
c = _____

```

!!!! 答案

1. (4 分) 对于程序中的相应符号, 请给出它的属性 (局部或全局, 强符号或弱符号), 不确定的请填 x。

| 源文件     | 符号名   | 局部或全局? | 强符号或弱符号? |
|---------|-------|--------|----------|
| main.c  | f     | 全局     | 弱符号      |
| value.c | BIG   | 全局     | 弱符号      |
| f1.c    | small | 全局     | 强符号      |
| f2.c    | small | 局部     | x        |

评分标准: 每空 0.5 分

2. (6 分) 请补全程序运行的输出, 不确定的请填 x。

```
small = 2, BIG = 1
small = 3, BIG = 2
b = x
small = 1, BIG = 3
small = 2, BIG = 4
c = 3
```

评分标准: small 和 BIG 的值每空 0.5 分【取整, 错 1-2 个扣 1 分, 错 3-4 个扣 2 分, 以此类推】, b 和 c 的值每空 1 分  
b 值无法确定, 是因为传入参数无法确定。

!!!! 答案结束

|    |
|----|
| 得分 |
|    |

### 第五题（10 分）

#### Part I

请阅读以下程序，然后回答问题。假设程序中的函数调用都可以正确执行，并默认 printf 执行完会调用 fflush。

```
int main() {
 int cnt=1;
 int pid_1,pid_2;
 pid_1=fork();
 if(pid_1==0) {
 pid_2=fork();
 if(pid_2!=0) {
 wait(pid_2,NULL,0);
 printf("B");
 }
 printf("F");
 exit(0);
 }
 else {
 A
 B
 wait(pid_1,NULL,0);
 pid_2=fork();
 if(pid_2==0) {
 printf("D");
 cnt-=1;
 }
 if(cnt==0)
 printf("E");
 else
 printf("G");
 exit(0);
 }
}
```

(1) 如果程序中的 A、B 位置的代码为空，列出所有可能的输出结果：

**( 1 分 ) 3 个，全对才得分**

FBFGDE

FBFDGE

FBFDEG

(2) 如果程序中的 A、B 位置的代码为：



A:     printf("C");

B:     exit(0);

列出所有可能的输出结果:

(2分) 4个, 对1~3个给1分, 全对给2分

CFBF

FCBF

FBCF

FBFC

## Part II

请阅读以下程序, 然后回答问题 (假设程序中的函数调用都可以正确执行, 且每条语句都是原子动作):

```
pid_t pid;
int even = 0;
int counter1 = 0;
int counter2 = 1;
void handler1(int sig) {
 if (even % 2 == 0) {
 printf("%d\n", counter1);
 counter1 = ____ A ____ ;
 } else {
 printf("%d\n", counter2);
 counter2 = ____ B ____ ;
 }
 even = even + ____ C ____ ;
}
void handler2(int sig) {
 if (____ D ____) {
 counter1 = even*even;
 } else {
 counter2 = even*even;
 }
}
int main() {
 signal(SIGUSR1, handler1);
 signal(SIGUSR2, handler2);
 if ((pid = fork()) == 0) {
 while (1) {}
 }
 while (even < 30) {
 kill(pid, ____ E ____);
 sleep(1);
 }
}
```

```

 kill(pid, ____ F ____);
 sleep(1);
 even = even+____ G ____ ;
 }
 kill(pid, SIGKILL);
 exit(0);
}

```

(1) 完成程序，使得程序在输出的数字为以下 Q 队列的前 30 项，Q 队列定义如下：

$$Q_0 = 0, Q_1 = 1, Q_{n+2} = \begin{cases} Q_n + 1, & n \% 2 = 0 \\ Q_n \times 2, & n \% 2 \neq 0 \end{cases} \quad (n = 0, 1, 2, 3, \dots)$$

(若某个位置中的程序内容，对本次程序执行结果没有影响，请在相应位置填写“无关”)

(7 分) 7 个，对 1 个给 1 分

|          |                   |
|----------|-------------------|
| A: _____ | <u>counter1+1</u> |
| B: _____ | <u>counter2*2</u> |
| C: _____ | <u>1</u>          |
| D: _____ | <u>无关</u>         |
| E: _____ | <u>SIGUSR1</u>    |
| F: _____ | <u>SIGUSR1</u>    |
| G: _____ | <u>2</u>          |

|    |
|----|
| 得分 |
|    |

## 第六题（12 分）

程序 i16.c 如下：

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <ctype.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include "csapp.h"

int main(int argc, char **argv)
{
 int fd1, fd2;
 char *cs;
 struct stat stat;
 int sz, i;

 fd1 = Open(argv[1], O_RDWR, S_IRUSR | S_IWUSR);
 Fstat(fd1, &stat);
 sz = stat.st_size;
 fd2 = Open(argv[2], O_RDWR | O_TRUNC | O_CREAT, \
 S_IRUSR | S_IWUSR);

 cs = (char *)Mmap(NULL, sz, PROT_READ | PROT_WRITE, \
 _____, fd1, 0);
 if (Fork()) {
 Wait(&i);
 Write(fd2, cs, sz);
 } else {
 for (i = 0; i < sz; i++) {
 if (islower(cs[i]))
 cs[i] = (char) toupper(cs[i]);
 else if (isupper(cs[i]))
 cs[i] = (char) tolower(cs[i]);
 }
 _____;
 }
 Munmap(cs, sz);
 Close(fd1);
 Close(fd2);
}
```

}

1、假定 t1 内容为 ICSEXAM，根据 X/Y 处的内容，判断执行 ./i16 t1 t2 之后的结果：（均为单选题）

1.1、X 为 MAP\_PRIVATE，Y 为空时，结果为（ ）

- A. t1 为 ICSEXAM，t2 为 ICSEXAM
- B. t1 为 ICSEXAM，t2 为 icsEXAM
- C. t1 为 icsEXAM，t2 为 icsEXAM
- D. t1 为 icsEXAM，t2 为 ICSEXAM

1.2、X 为 MAP\_SHARED 时，Y 为空时，结果为（ ）

- A. t1 为 ICSEXAM，t2 为 ICSEXAM
- B. t1 为 ICSEXAM，t2 为 icsEXAM
- C. t1 为 icsEXAM，t2 为 icsEXAM
- D. t1 为 icsEXAM，t2 为 ICSEXAM

1.3、X 处的内容为 MAP\_PRIVATE，Y 为 Write(fd2, cs, sz) 时，t2 的结果为（ ）

- A. t2 为 ICSEXAM
- B. t2 为 icsEXAM
- C. t2 为 icsEXAMICSEXAM
- D. t2 为 icsEXAMicsEXAM

1.4、X 处的内容为 MAP\_SHARED，Y 为 Write(fd2, cs, sz) 时，t2 的结果为（ ）

- A. t2 为 ICSEXAM
- B. t2 为 icsEXAM
- C. t2 为 icsEXAMICSEXAM
- D. t2 为 icsEXAMicsEXAM

答案：

1.1: A

1.2: C

1.3: C

1.4: D

2、X 为 MAP\_SHARED，当子进程运行到 Y 处时，内存映射的内容如下：（只保留了 /proc/pid/maps 中的三部分内容：分别为 address/perms/pathname）

```
00400000-00405000 r-xp /home/tao/i16/i16
00604000-00605000 r--p /home/tao/i16/i16
00605000-00606000 A /home/tao/i16/i16
7f5df1e61000-7f5df2020000 r-xp /lib/x86_64-linux-gnu/libc-2.23.so
7f5df2020000-7f5df2220000 ---p /lib/x86_64-linux-gnu/libc-2.23.so
7f5df2220000-7f5df2224000 r--p /lib/x86_64-linux-gnu/libc-2.23.so
7f5df2224000-7f5df2226000 rw-p /lib/x86_64-linux-gnu/libc-2.23.so
7f5df2226000-7f5df222a000 rw-p
7f5df222a000-7f5df2250000 r-xp /lib/x86_64-linux-gnu/ B
7f5df243f000-7f5df2442000 rw-p
7f5df244c000-7f5df244d000 rw-s C
7f5df244d000-7f5df244f000 rw-p
7f5df244f000-7f5df2450000 r--p /lib/x86_64-linux-gnu/ B
7f5df2450000-7f5df2451000 rw-p /lib/x86_64-linux-gnu/ B
7f5df2451000-7f5df2452000 rw-p
7ffd4e35f000-7ffd4e380000 rw-p [D]
7ffd4e3b0000-7ffd4e3b2000 r--p [vvar]
7ffd4e3b2000-7ffd4e3b4000 r-xp [vdso]
fffffffffff600000-fffffffffff601000 r-xp [vsyscall]
```

根据程序执行情况，填充空白处：

A: \_\_\_\_\_  
B: \_\_\_\_\_  
C: \_\_\_\_\_  
D: \_\_\_\_\_

答案：

A: rw-p  
B: ld-2.23.so 或 ld.so  
C: /home/tao/i16/t1 或 t1  
D: stack

3、X 为 MAP\_SHARED，当子进程运行到 Y 处时：（均为单选题）

3.1、关于页表的描述，正确的是：（ ）

- A. 由于 i16 本身的代码和数据只占用了低 32 位空间，所以这部分空间只需使用 2 级页表
- B. 所有虚拟地址空间都是 48 位地址空间，都需要使用完整的 4 级页表
- C. 当页表项无效时（P 位为 0），MMU 不会使用到其它位的内容
- D. 除读写权限外，需要在页表项中为 COW 机制提供专门的支持

3.2、如果子进程在 Y 处访问 7f5df2224 这一页，可能发生以下异常：（ ）

- A. Page Fault 或 General Protection Fault
- B. General Protection Fault 或 Segmentation Fault
- C. Page Fault 或 Segmentation Fault
- D. Page Fault 或 General Protection Fault 或 Segmentation Fault

3.3、假设 TLB 有 64 个表项，4 路组相联，当访问 7f5df2224 这一页时，所对应的 TLBT 应为：（ ）

- A. 0111 1111 0101 1101 1111 0010 0010 0010
- B. 0111 1111 0101 1101 1111 0010 0010 0010 01
- C. 1111 0101 1101 1111 0010 0010 0010 0100
- D. 11 1111 0101 1101 1111 0010 0010 0010 0100

3.4、当发生 TLB 命中时，则意味着：（ ）

- A. 相应的页表项，在 L1 Cache 里
- B. 要访问的数据内容，在 L1 Cache 里
- C. 如果要访问的数据内容已在 L1 Cache 中，则该 Cache Line 的权限位中应具有相应的读写执行权限
- D. 以上都不对

答案：

3.1: C

3.2: B（执行代码，会引发 SEGV；COW 会引发 GPF）

3.3: A

3.4: D（Cache Line 没有读写权限位）

|    |
|----|
| 得分 |
|    |

第七题（12 分）

下面是一个基于进程的并发 echo server 的主要代码(省略了与本题无关的部分)，每段代码都注释了相应的功能，请补充空缺的代码（可能是 0 行或多行）。请确保任何由你打开的文件描述符都被显式地关闭。

```
int open_listenfd(char *port) {
 struct addrinfo hints, *listp, *p;
 int listenfd, optval = 1;

 /* Get a list of potential server addresses */
 memset(&hints, 0, sizeof(struct addrinfo));
 hints.ai_socktype = SOCK_STREAM;
 hints.ai_flags = AI_PASSIVE | AI_ADDRCONFIG;
 hints.ai_flags |= AI_NUMERICSERV;
 Getaddrinfo(NULL, port, &hints, &listp);

 /* Walk the list for one that we can bind to */
 for (p = listp; p; p = p->ai_next) {
 /* Create a socket descriptor. If failed, try the next */
 if ((listenfd = socket(p->ai_family,
 p->ai_socktype, p->ai_protocol)) < 0)
 (1)

 /* Eliminates error from bind */
 Setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR,
 (const void *)&optval, sizeof(int));
 /* Bind the descriptor to the address */
 if (bind(listenfd, p->ai_addr, p->ai_addrlen) == 0)
 (2) else /* Success, use the current one */
 (3) /* Bind failed, try the next */
 }
 (4)
 if (!p) return -1; /* No address worked */

 /* Make it a listening socket*/
 if (listen(listenfd, LISTENQ) < 0) {
 (5)
 return -1;
 }
 return listenfd;
}
```

```

void echo(int fd); /* service a client via fd */
void handler(int sig) {
 while (waitpid(-1, 0, WNOHANG) > 0);
 return;
}
int main(int argc, char **argv) {
 int listenfd, connfd;
 socklen_t clientlen;
 struct sockaddr_storage clientaddr;

 Signal((10), handler);
 listenfd = Open_listenfd(argv[1]);
 while (1) {
 clientlen = sizeof(struct sockaddr_storage)
 (6) = Accept((7), (SA *)&clientaddr, &clientlen);
 if (Fork() == 0) {
 /* Child services client and close fd */
 (8)
 exit(0); /* Child exits */
 }
 (9)
 }
}

```

(第(9)空 3 分，其余每空 1 分；共 12 分)

答案:

- (1) continue;
- (2) break;
- (3) Close(listenfd);
- (4) Freeaddrinfo(listp);
- (5) Close(listenfd);
- (6) connfd
- (7) listenfd
- (8) Close(listenfd); echo(connfd); Close(connfd);
- (9) Close(connfd);
- (10) SIGCHLD



|    |
|----|
| 得分 |
|    |

### 第八题（12 分）

1. 请阅读下列代码 badcnt.c:

```
/* Global shared variable */
volatile long cnt = 0; /* Counter */
int main(int argc, char **argv)
{
 long niters;
 pthread_t tid1, tid2;
 niters = atoi(argv[1]);
 Pthread_create(&tid1, NULL, thread, &niters);
 Pthread_create(&tid2, NULL, thread, &niters);
 Pthread_join(tid1, NULL);
 Pthread_join(tid2, NULL);
 /* Check result */
 if (cnt != (2 * niters))
 printf("BOOM! cnt=%ld\n", cnt);
 else
 printf("OK cnt=%ld\n", cnt);
 exit(0);
}

/* Thread routine */
void *thread(void *vargp)
{
 long i, niters = *((long *)vargp);

 for (i = 0; i < niters; i++)
 cnt++;

 return NULL;
}
```

运行后可能产生如下结果:

```
linux> ./badcnt 10000
OK cnt=20000
```

或者:

```
linux> ./badcnt 10000
BOOM! cnt=13051
linux>
```

为什么会产生不同的结果？请分析产生不同结果的原因。

2. 某辆公交车的司机和售票员为保证乘客的安全，需要密切配合、协调工作。司机和售票员的工作流程如下所示。请编写程序，用 P、V 操作来实现司机与售票员之间的同步。

司机进程：

售票员进程：

```
while(1) {
 ①
 启动车辆；
 ②
 正常行驶；
 ③
 到站停车；
 ④
}

while(1) {
 ⑤
 关门；
 ⑥
 报站名或维持秩序；
 ⑦
 到站开门；
 ⑧
}
```

- (1) 请设计若干信号量，给出每一个信号量的作用和初值。
- (2) 请将信号量对应的 PV 操作填写在代码中适当位置。
- 注：每一标号处可以不填入语句（请标记成 x），或填入一条或多条语句。

| 标号 | 对应的操作 |
|----|-------|
| ①  |       |
| ②  |       |
| ③  |       |
| ④  |       |
| ⑤  |       |
| ⑥  |       |
| ⑦  |       |
| ⑧  |       |

答案：

1. (4 分)

`cnt++` 在汇编里会被分成加载、更新、写回三个步骤 (2 分)。线程 1 把 `cnt` 读入寄存器，寄存器加 1，在未写回时，若线程 2 把 `cnt` 读入自己的寄存器，寄存器加 1，最终两个线程都是把 `cnt+1` 写入 `cnt`，这就造成了错误。(2 分)

2. (8 分)

(1) 设置两个信号量 `s1=0`、`s2=1` (2 分)。`s1` 用于司机进程是否能启动车辆 (1 分)，`s2` 用于售票员进程是否能开门 (1 分)。

(2) ① `P(s1)`

② `X`

③ `X`

④ `V(s2)`

⑤ `X`

⑥ `V(s1)`

⑦ `P(s2)`

⑧ `X`

评分：每空 0.5 分，共 4 分。

# 北京大学信息科学技术学院考试试卷

考试科目：计算机系统导论 姓名：\_\_\_\_\_ 学号：\_\_\_\_\_

考试时间：2019年1月7日 小班号：\_\_\_\_\_ 小班教师：\_\_\_\_\_

| 题号  | 一 | 二 | 三 | 四 | 五 | 六 | 七 | 八 | 总分 |
|-----|---|---|---|---|---|---|---|---|----|
| 分数  |   |   |   |   |   |   |   |   |    |
| 阅卷人 |   |   |   |   |   |   |   |   |    |

## 北京大学考场纪律

1、考生进入考场后，按照监考老师安排隔位就座，将学生证放在桌面上。无学生证者不能参加考试；迟到超过 15 分钟不得入场。在考试开始 30 分钟后方可交卷出场。

2、除必要的文具和主考教师允许的工具书、参考书、计算器以外，其它所有物品（包括空白纸张、手机、或有存储、编程、查询功能的电子用品等）不得带入座位，已经带入考场的必须放在监考人员指定的位置。

3、考试使用的试题、答卷、草稿纸由监考人员统一发放，考试结束时收回，一律不准带出考场。若有试题印制问题请向监考教师提出，不得向其他考生询问。提前答完试卷，应举手示意请监考人员收卷后方可离开；交卷后不得在考场内逗留或在附近高声交谈。未交卷擅自离开考场，不得重新进入考场答卷。考试结束时间到，考生立即停止答卷，在座位上等待监考人员收卷清点后，方可离场。

4、考生要严格遵守考场规则，在规定时间内独立完成答卷。不准交头接耳，不准偷看、夹带、抄袭或者有意让他人抄袭答题内容，不准接传答案或者试卷等。凡有违纪作弊者，一经发现，当场取消其考试资格，并根据《北京大学本科考试工作与学术规范条例》及相关规定严肃处理。

5、考生须确认自己填写的个人信息真实、准确，并承担信息填写错误带来的一切责任与后果。

学校倡议所有考生以北京大学学生的荣誉与诚信答卷，共同维护北京大学的学术声誉。

以下为试题和答题纸，共      页。

|    |
|----|
| 得分 |
|    |

第一题 单项选择题（每小题 2 分，共 30 分）

注：选择题的回答请填写在下表中。

|    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|
| 题号 | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
| 回答 |    |    |    |    |    |    |    |    |    |    |
| 题号 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| 回答 |    |    |    |    |    | /  | /  | /  | /  | /  |

1. 在如下的代码中，下列关于局部性的说法正确的是：

```
int add (int *a, int n){
 int sum = 0;
 for (int i = 0; i < n; ++i)
 sum += a[i];
 return sum;
}
```

- A. 变量 sum 具有良好的空间局部性.
- B. 数组 a 具有良好的空间局部性.
- C. 数组 a 具有良好的时间局部性
- D. 以上说法都正确

答案：B.

2. 有如下结构定义和程序片段

```
struct A
{
 char c;
 int i;
 double d;
 int array[10];
};

struct B
{
 int array[10];
```

```

 double d;
 char c;
 int i;
};

void foo(struct A *pa, struct B *pb, int index)
{
 pb->i = pa->array[index];
}

```

在 Linux 下使用 GCC 编译器，仅采用 -O2 选项，上述代码对应的汇编语言是：（将选项依次填入空格内）

```

movslq %edx, %rdx
movl __(%rdi,%rdx,__), %eax
movl %eax, __(%rsi)

```

- A. (16, 4, 52)    B. (24, 4, 52)    C. (16, 4, 49)    D. (24, 4, 49)

答案：A

（注意 char 和 int 共用一个 8 字节）

3. 下面关于缓存替换策略的说法哪个是正确的 (N 为 cache 的大小)

- A. FIFO 的性能总是优于随机替换
- B. LRU 适用于数组的顺序访问
- C. 假定 cache 初始状态为空，若某一输入片段使用 LRU 造成了  $N + 1$  次 miss，则对任意策略至少产生两次 miss
- D. 对于同一确定性替换策略，增大 N 的大小一定减少 miss 次数

答案：C

（至少有  $N+2$  个不同的输入，第一次一定 miss，后 N 次一定与第一次换入的不同，至少也有一次 miss）

4. 下列关于链接技术的描述，错误的是（ ）

- A. 在 Linux 系统中，对程序中全局符号的不恰当定义，会在链接时刻进行报告。
- B. 在使用 Linux 的默认链接器时，如果有多个弱符号同名，那么会从这些弱符号中任意选择一个占用空间最大的符号。

- C. 编译时打桩 (interpositioning) 需要能够访问程序的源代码, 链接时打桩需要能够访问程序的可重定位对象文件, 运行时打桩只需要能够访问可执行目标文件。
- D. 链接器的两个主要任务是符号解析和重定位。符号解析将目标文件中的全局符号都绑定到唯一的定义, 重定位确定每个符号的最终内存地址, 并修改对那些目标的引用。

**答案: 选 A。参考机械工业出版社第三版中文教材, 四个选项分别对应 P464 第 5 自然段、P471 页文末、P494 页文末、P496 文末的文字。其中选项 B 略有调整, 增加了“占用空间最大的”, 是 Linux binutils 中链接器实现的原则, 仍是正确选项。A 错误是因为全局符号的不恰当定义并不总会报警 (甚至不报告 warning)。**

5. 关于进程, 以下说法正确的是:

- A. 没有设置模式位时, 进程运行在用户模式中, 允许执行特权指令, 例如发起 I/O 操作。
- B. 调用 `waitpid(-1, NULL, WNOHANG & WUNTRACED)` 会立即返回: 如果调用进程的所有子进程都没有被停止或终止, 则返回 0; 如果有停止或终止的子进程, 则返回其中一个的 ID。
- C. `execve` 函数的第三个参数 `envp` 指向一个以 `null` 结尾的指针数组, 其中每一个指针指向一个形如 “`name=value`” 的环境变量字符串。
- D. 进程可以通过使用 `signal` 函数修改和信号相关联的默认行为, 唯一的例外是 `SIGKILL`, 它的默认行为是不能修改的。

**答案: C**

**说明: C 正确见 P521。A 不正确 见 P510。B 中 option 参数应使用 `|` 运算结合 (P517)。D 中 `SIGKILL` 不是唯一的例外, 例外共有两个 `SIGKILL`、`SIGSTOP` (P531)。**

6. 假设某进程有且仅有五个已打开的文件描述符: `0~4`, 分别引用了五个不同的文件, 尝试运行以下代码:

```
dup2(3, 2); dup2(0, 3); dup2(1, 10); dup2(10, 4); dup2(4, 0);
```

关于得到的结果, 说法正确的是:

- A. 运行正常完成, 现在有四个描述符引用同一个文件
- B. 运行正常完成, 现在进程共引用四个不同的文件
- C. 由于试图从一个未打开的描述符进行复制, 发生错误
- D. 由于试图向一个未打开的描述符进行复制, 发生错误

**答案: A**

**说明: 一开始打开文件描述符 (0, 1, 2, 3, 4) 对应文件 (A, B, C, D, E), 结束后打开描述符 (0, 1, 2, 3, 4, 10) 对应 (B, B, D, A, B, B), A 正确, B 应该为引用三个不同文件。教材**

637 页说明过可以向未打开的描述符进行复制，因此 D 错误；虽然教材未提及从未打开的描述符进行复制的后果，但执行过程中并没有发生这种情况，因此 C 错误。

7. 下列与虚拟内存有关的说法中哪些是不对的？

- A. 操作系统为每个进程提供一个独立的页表，用于将其虚拟地址空间映射到物理地址空间。
- B. MMU 使用页表进行地址翻译时，虚拟地址的虚拟页面偏移与物理地址的物理页面偏移是相同的。
- C. 若某个进程的工作集大小超出了物理内存的大小，则可能出现抖动现象。
- D. 动态内存分配管理，采用双向链表组织空闲块，使得首次适配的分配与释放均是空闲块数量的线性时间。

答案：D 首次释放不一定是空闲块数量的线性时间

8. 假定整型变量 A 的虚拟地址空间为 0x12345cf0，另一整型变量 B 的虚拟地址 0x12345d98，假定一个 page 的长度为 0x1000 byte，A 的物理地址数值和 B 的物理地址数值关系应该为：

- A. A 的物理地址数值始终大于 B 的物理地址数值
- B. A 的物理地址数值始终小于 B 的物理地址数值
- C. A 的物理地址数值和 B 的物理地址数值大小取决于动态内存分配策略，
- D. 无法判定两个物理地址值的大小

答案：B 考察物理地址和虚拟地址的映射关系 同一个 page

9. 虚拟内存中两层页表和单层页表相比，最主要的优势在于：

- A. 更快的地址翻译速度
- B. 能够提供页面更加精细的保护措施
- C. 能够充分利用代码的空间局部性
- D. 能够充分利用稀疏的内存使用模式

答案：D 主要是内存使用不连续，可以节省空间

10. 下面有关计算机网络概念的叙述中，正确的是

- A. 大写字母的 Internet 用来描述互联网的一般概念，而小写字母的 internet 用来描述一种具体的实现，也就是全球 IP 互联网。
- B. 在一个基于集线器 (hub) 的以太网 (Ethernet) 中，如果往一台主机发送一段数据帧 (frame)，那么其他主机无法看到这个帧。
- C. IP 协议提供基本的命名方法和递送机制，因此我们能够借助 IP 协议，从一台主机往另一台主机发送包，即使两台主机不在同一个 LAN 内。
- D. 当一段数据通过路由器，从 LAN1 被发送到 LAN2 时，附加的互联网络包头和局



域网帧头保持不变。

答案：C

A 选项出自书本 645 页，考察互联网的基本概念。internet 应该是描述一般概念，Internet 应该是特指一个具体实现

B 选项出自书本 643-644 页，考察 Ethernet 协议的概念和实现形式。早期的 Ethernet 协议使用 Hub 来连接网络主机，所有的帧传送都是广播形式。

C 选项出自书本 647 页，考察 IP 协议在网络协议栈中的角色。

D 选项出自书本 645-646 页，考察网络协议的基本概念。在通过路由器将数据从一个局域网传送到另一个局域网的过程中，原来的局域网帧头会被剥离，然后附加新的局域网帧头

11. 下面有关套接字接口(Socket API)的叙述中，**错误**的是

- A. 套接字接口常常被用来创建网络应用
- B. Windows 10 系统没有实现套接字接口
- C. getaddrinfo()和 getnameinfo()可以被用于编写独立于特定版本的 IP 协议的程序
- D. socket()函数返回的描述符，可以使用标准 Unix I/O 函数进行读写

答案：B

A 选项是套接字的基本概念

B 选项出自书本 652 页，考察对套接字接口的理解。几乎所有的现代操作系统都实现了套接字接口。

C 选项出自书本 656 页，考察套接字接口中协议无关的设计思想。

D 选项考察对 Socket API 和 Linux 文件描述符的基本理解

12. 使用浏览器打开网页 www.pku.edu.cn 的过程中，下列网络协议中，可能会被用到的网络协议有\_\_\_个

① DNS ② TCP ③ IP ④ HTTP

- A. 1      B. 2      C. 3      D. 4

答案：D

考察对于网络的整体理解。按照书上涉及的内容，要解析域名必须用到 DNS 协议，要传输网页必须用到 HTTP 协议，HTTP 协议构建于 TCP/IP 协议之上，因此，所有的协议都可能会被用到。因为存在诸如 HTTPS，QUIC 等其他传输网页的协议，故加了可能两字。

13. 下面关于线程安全和可重入的描述中，哪一个是正确的？
- A. 如果一个函数的所有参数都是值传递的且无返回值，该函数一定是可重入的
  - B. 函数的可重入版本一定比不可重入版本高效
  - C. 可重入函数一定是线程安全的
  - D. 以上说法都不正确

答案：C

14. 下列关于进程与线程的描述中，哪一个是不正确的？
- A. 一个进程可以包含多个线程
  - B. 进程中的各个线程共享进程的代码、数据、堆和栈
  - C. 进程中的各个线程拥有自己的线程上下文
  - D. 线程的上下文切换比进程的上下文切换快

答案：B

15. 给定下列代码片段：

```
char **ptr; /* global var */
int main(int main, char *argv[]) {
 long i; pthread_t tid;
 char *msgs[2] = {" Hello from foo", " Hello from bar" };
 ptr = msgs;
 for (i = 0; i < 2; i++)
 Pthread_create(&tid, NULL, thread, (void *)i);
 Pthread_exit(NULL);}

void *thread(void *vargp)
{
 long myid = (long)vargp;
 static int cnt = 0;
 printf("[%ld]: %s (cnt=%d)\n", myid, ptr[myid], ++cnt);
 return NULL;
}
```

下列哪一组变量集合是对等线程 1 引用的？

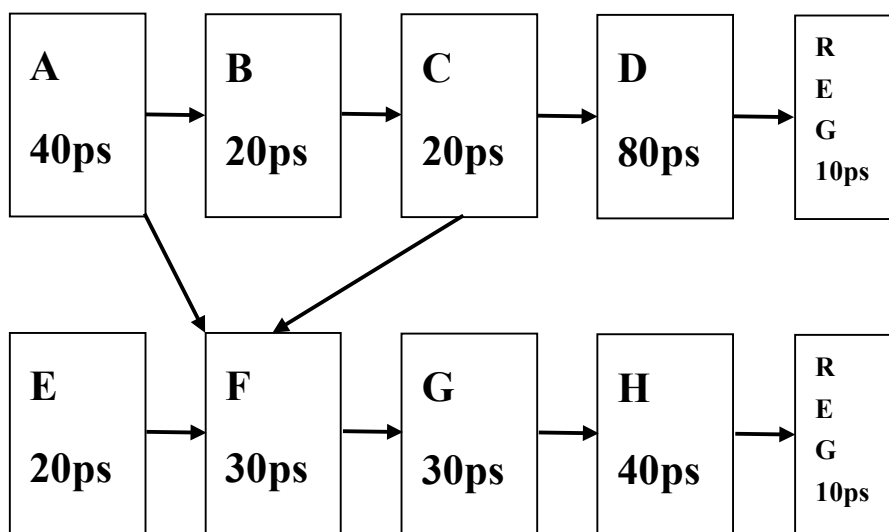
- A. ptr, cnt, i.m, msgs.m, myid.p0, myid.p1
- B. ptr, cnt, msgs.m, myid.p0
- C. ptr, cnt, msgs.m, myid.p1
- D. ptr, cnt, i.m, msgs.m, myid.p1

答案：C

|    |
|----|
| 得分 |
|    |

## 第二题（12 分）

如图所示，每个模块表示一个单独的组合逻辑单元，每个单元的延迟以及数据依赖关系已在图中标出。通过在两个单元间添加寄存器的方式，可以对该数据通路进行流水化改造。假设每个寄存器的延迟为 10ps。



1) 如果改造为一个二级流水线，为获得最大的吞吐率，该寄存器应在哪里插入？请计算该流水线的吞吐率，并说明计算过程。结果可以是分数形式也可以是小数形式。

（6 分）插入在 CD、CF、AF、EF（2 分）

$1000 / (30 + 30 + 40 + 10) = 1000 / 110 = 9.09 \text{ GIPS}$ （过程 1 分和结果正确包括单位 1 分）。

2) 如果改造为一个三级流水线，为获得最大的吞吐率，寄存器应在哪里插入？请计算该流水线的吞吐率，并说明计算过程。结果可以是分数形式也可以是小数形式。（请给出至少两种实现最大吞吐率的设计方案）

(6 分) 插入在 AB、AF、EF 间 (2 分)

$1000/(80+10) = 1000/90 = 11.11$  GIPS(过程 1 分和结果正确包括单位 1 分)。

其他解法如果正确也可以得分

|    |
|----|
| 得分 |
|    |

第三题（10 分）

本题基于下列 m.c 及 foo.c 文件所编译生成的 m.o 和 foo.o，编译过程未加优化选项。

|                                                                                           |                                                                                                                                                                                                                                    |
|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>//m.c void foo(); int buf[2] = {1, 2}; int main() {     foo();     return 0; }</pre> | <pre>//foo.c extern int buf[]; int *bufp0 = &amp;buf[0]; int *bufp1; void foo() {     static int count = 0;     int temp;     bufp1 = &amp;buf[1];     temp = *bufp0;     *bufp0 = *bufp1;     *bufp1 = temp;     count++; }</pre> |
|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

对于每个 foo.o 中定义和引用的符号，请用“是”或“否”指出它是否在模块 foo.o 的 .symtab 节中有符号表条目。如果存在条目，则请指出定义该符号的模块（foo.o 或 m.o）、符号类型（局部、全局或外部）以及它在模块中所处的节；如果不存在条目，则请将该行后继空白处标记为“/”。

第一问每行 1 分，该行全部答对才给分数，节名如果漏了“.”可以算对。

用英文回答的，如果正确也可以给分。

| 符号    | .symtab 条目? | 符号类型 | 定义符号的模块 | 节      |
|-------|-------------|------|---------|--------|
| bufp0 | 是           | 全局   | foo.o   | .data  |
| buf   | 是           | 外部   | m.o     | .data  |
| bufp1 | 是           | 全局   | foo.o   | COMMON |
| foo   | 是           | 全局   | foo.o   | .text  |
| temp  | 否           | /    | /       | /      |
| cnt   | 是           | 局部   | foo.o   | .bss   |

下图左边给出了 m.o 和 foo.o 的反汇编文件，右边给出了采用某个配置链接成可执行程序后再反汇编出来的文件。根据答题需要，其中的信息略有删减。

|                                                                    |                                                                            |
|--------------------------------------------------------------------|----------------------------------------------------------------------------|
| 0000000000.....<main>:<br>55                           push   %rbp | 00000000000000fe8 <main>:<br>fe8: 55                           push   %rbp |
|--------------------------------------------------------------------|----------------------------------------------------------------------------|

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> 48 89 e5      mov     %rsp,%rbp b8 00 00 00 00    mov     \$0x0,%eax e8 00 00 00 00    callq  e &lt;main+0xe&gt;      ① b8 00 00 00 00    mov     \$0x0,%eax 5d             pop     %rbp c3             req </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | <pre> fe9: 48 89 e5      mov     %rsp,%rbp fec: b8 00 00 00 00    mov     \$0x0,%eax ff1: e8      ①      callq  1000 &lt;foo&gt; ff6: b8 00 00 00 00    mov     \$0x0,%eax ffb: 5d             pop     %rbp ffc: c3             retq </pre> <p>.....略去部分和答题无关的信息.....</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <pre> 0000000000.....&lt;foo&gt;: 55             push    %rbp 48 89 e5      mov     %rsp,%rbp 48 c7 05 00 00 00 00 00 00 00 00                                 movq     \$0x0,0x0(%rip)                                 ③② 48 8b 05 00 00 00 00 00 mov     0x0(%rip),%rax      ④ 8b 00             mov     (%rax),%eax 89 45 fc      mov     %eax,-0x4(%rbp) 48 8b 05 00 00 00 00 00 mov     0x0(%rip),%rax      ⑤ 48 8b 15 00 00 00 00 00 mov     0x0(%rip),%rdx      ⑥ 8b 12             mov     (%rdx),%edx 89 10             mov     %edx,(%rax) 48 8b 05 00 00 00 00 00 mov     0x0(%rip),%rax      ⑦ 8b 55 fc      mov     -0x4(%rbp),%edx 89 10             mov     %edx,(%rax) 8b 05 00 00 00 00 00 00 mov     0x0(%rip),%eax ⑧ 83 c0 01      add     \$0x1,%eax 89 05 00 00 00 00 00 00 mov     %eax,0x0(%rip) ⑨ 90             nop 5d             pop     %rbp c3             retq </pre> | <pre> 00000000000001000 &lt;foo&gt;: 1000: 55             push    %rbp 1001: 48 89 e5      mov     %rsp,%rbp 1004: 48 c7 05 00 00 00 00 00 00 00 00                                 movq     ② , ③                                 (%rip) 100f: 48 8b 05 ?? ?? ?? ?? mov     0x????(%rip),%rax 1016: 8b 00             mov     (%rax),%eax 1018: 89 45 fc      mov     %eax,-0x4(%rbp) 101b: 48 8b 05 00 00 00 00 00 mov     ⑤ (%rip),%rax 1022: 48 8b 15 00 00 00 00 00 mov     0x????(%rip),%rdx 1029: 8b 12             mov     (%rdx),%edx 102b: 89 10             mov     %edx,(%rax) 102d: 48 8b 05 00 00 00 00 00 mov     0x????(%rip),%rax 1034: 8b 55 fc      mov     -0x4(%rbp),%edx 1037: 89 10             mov     %edx,(%rax) 1039: 8b 05 00 00 00 00 00 00 mov     0x????(%rip),%eax 103f: 83 c0 01      add     \$0x1,%eax 1042: 89 05 00 00 00 00 00 00 mov     %eax, ⑨ (%rip) 1048: 90             nop 1049: 5d             pop     %rbp 104a: c3             retq </pre> <p>.....略去部分和答题无关的信息.....</p> <pre> 00000000000002330 &lt;buf&gt;: .....略去部分和答题无关的信息..... 00000000000002338 &lt;bufp0&gt;: .....略去部分和答题无关的信息..... 00000000000003024 &lt;count.1837&gt;: .....略去部分和答题无关的信息..... 00000000000003028 &lt;bufp1&gt;: </pre> |

在上图对所涉及到的重定位条目进行用数字①至⑨进行了标记, 请根据下表中所提供的重定位条目信息, 计算相应的重定位引用值并填写下表。

| 编号 | 重定位条目信息                                                                       | 应填入的重定位引用值  |
|----|-------------------------------------------------------------------------------|-------------|
| ①  | r.offset = 0xa<br>r.type = R_X86_64_PC32<br>r.symbol = 本题不提供<br>r.addend = -4 | 0a 00 00 00 |
| ②  | r.offset = 0xb<br>r.type = R_X86_64_32<br>r.symbol = buf<br>r.addend = +4     | 0x2334      |
| ③  | r.offset = 0x7<br>r.symbol = bufp1                                            | 0x2019      |

|   |                                                                                                    |        |
|---|----------------------------------------------------------------------------------------------------|--------|
|   | r.type = R_X86_64_PC32      r.addend = -8                                                          |        |
| ⑤ | r.offset = 0x1e                      r.symbol = bufp0<br>r.type = R_X86_64_PC32      r.addend = -4 | 0x1316 |
| ⑨ | r.offset = 0x44                      r.symbol = 本题不提供<br>r.type = R_X86_64_PC32      r.addend = -4 | 0x1fdc |

|    |
|----|
| 得分 |
|    |

#### 第四题 (10 分)

Bob 是一名刚刚学完异常的同学，他希望通过配合 kill 和 signal 的使用，能让两个进程向同一个文件中交替地打印出字符。可惜他的 tshlab 做得不过关，导致他写的这个程序有各种 BUG。你能帮帮他吗？

```

1 #include "csapp.h"
2 #define MAXN 6
3 int parentPID = 0;
4 int childPID = 0;
5 int count = 1;
6 int fd1 = 1;
7 void handler1() {
8 if (count > MAXN)
9 return;
10 for (int i = 0; i < count; i++)
11 write(fd1, "+", 1);
12 X
13 kill(parentPID, SIGUSR2);
14 }
15 void handler2() {
16 if (count > MAXN)
17 return;
18 for (int i = 0; i < count; i++)
19 write(fd1, "-", 1);
20 Y
21 kill(childPID, SIGUSR1);
22 }
23
24 int main() {
25 signal(SIGUSR1, handler1);
26 signal(SIGUSR2, handler2);
27 parentPID = getpid();
28 childPID = fork();
29 fd1 = open("file.txt", O_RDWR);

```



```
30 if (childPID) {
31 Z
32 kill(childPID, SIGUSR1);
33 }
34 exit(0);
35 }
```

**注意：**假设程序能在任意时刻被系统打断、调度，并且调度的时间切片大小是不确定的，可以足够地长。在每次程序执行前，file.txt 是一个已经存在的空文件。

**Part A.** (1 分) 此时，X 处语句和 Y 处语句都是 count++;，Z 处语句是空语句。Alice 测试该代码，发现有时 file.txt 中没有任何输出！请解释原因。（提示：考虑 28 行语句 fork 以后，下一次被调度的进程，并从这个角度回答本题。不需要给出解决方案）

**Part B.** (6 分) Bob 根据 Alice 的反馈，在某两行之间加了若干代码，修复了 Part A 的问题。当 X 处代码和 Y 处代码都是 count++;、Z 处为空时，Bob 期望 file.txt 中的输出是：

+--+-----+--+-----+--+-----+--+-----

可 Alice 测评 Bob 的程序的时候，却发现有时 Bob 的程序在 file.txt 中的输出是：

+--+-----+--+-----

而与此同时，终端上出现了如下的输出：

+

Bob 找不到自己的代码的 BUG，只好向 Alice 求助。Alice 帮他做了如下分析：

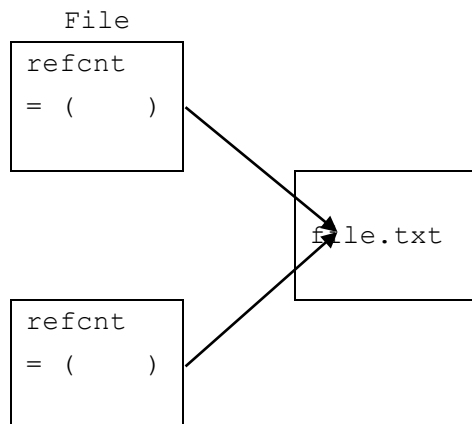
分析 1. 当程序第一次在终端上输出+的瞬间，请完成下表。要求：

- (1) 在“描述符表”一栏中，用“√”勾选该进程当前 fd1 的值。
- (2) 在“打开文件表”一栏中，填写该项的 refcnt（即，被引用多少次）。如果某一项不存在，请在括号中写“0”（并忽略其指向 v-node 表的箭头）。
- (3) 画出“描述符表”到“打开文件表”的表项指向关系。不需要画关于标准输入/标准输出/标准错误的箭头。评分时不对箭头评分，**请务必保证前两步的解答与箭头的连接情况匹配。**

|            |       |          |
|------------|-------|----------|
| 描述符表       | 打开文件表 | v-node 表 |
| Descriptor | Open  | V-node   |

|               |          |
|---------------|----------|
| 父进程<br>Parent | (    ) 0 |
|               | (    ) 1 |
|               | (    ) 2 |
|               | (    ) 3 |

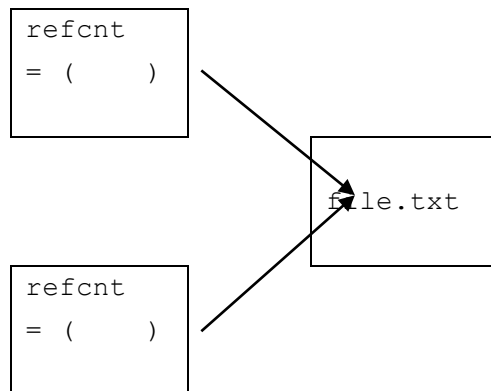
|              |          |
|--------------|----------|
| 子进程<br>Child | (    ) 0 |
|              | (    ) 1 |
|              | (    ) 2 |
|              | (    ) 3 |



分析 2. 当程序第一次在 **file.txt** 中输出+的瞬间，仿照上题要求完成下表：

|               |          |
|---------------|----------|
| 父进程<br>Parent | (    ) 0 |
|               | (    ) 1 |
|               | (    ) 2 |
|               | (    ) 3 |

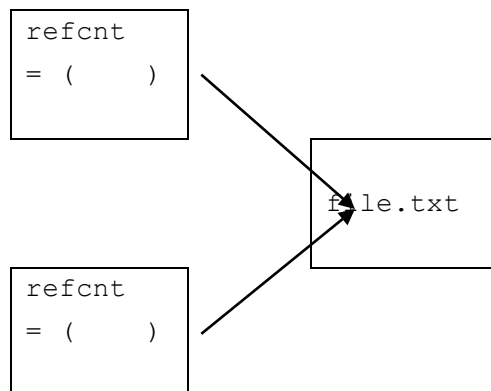
|              |          |
|--------------|----------|
| 子进程<br>Child | (    ) 0 |
|              | (    ) 1 |
|              | (    ) 2 |
|              | (    ) 3 |



分析 3. 如果要产生 Bob 预期的输出，三级表的关系应当是什么？仿照上题要求完成下表：

|               |          |
|---------------|----------|
| 父进程<br>Parent | (    ) 0 |
|               | (    ) 1 |
|               | (    ) 2 |
|               | (    ) 3 |

|              |          |
|--------------|----------|
| 子进程<br>Child | (    ) 0 |
|              | (    ) 1 |
|              | (    ) 2 |
|              | (    ) 3 |



**Part C.** (2 分) Bob 很高兴，他知道 Part B 的代码是怎么错的了！不过 Alice 仍然想考考 Bob。对于 Part B 的错误代码，如果终端上输出的是+++，那么

file.txt 中的内容是什么？请在下框中写出答案。

**Part D.** (1 分) Bob 修复了 Part B 的问题，使得代码能够产生预期的输出。现在，Bob 又希望自己的代码最终输出的是+---++-----+++++-----，为此，他对 X、Y、Z 处做了如下的修改。X、Y 处语句已做如下填写，请帮助 Bob 补上 Z 处语句。

X 处填写为: `count += 2;`

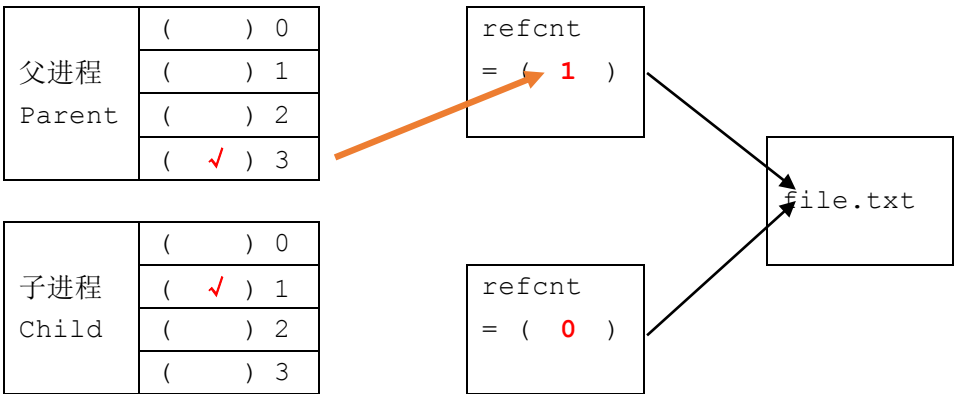
Y 处填写为: `count += 2;`

Z 处填写为:

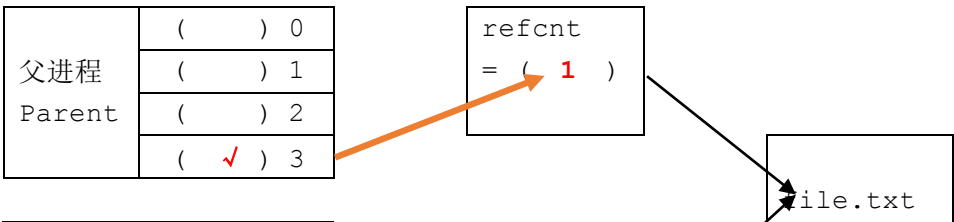
**【答案】Part A.** 如果 28 行 `fork` 执行过后，子进程先被调度了，并且执行完所有代码并退出，那么父进程的 `kill` 操作就无效了。

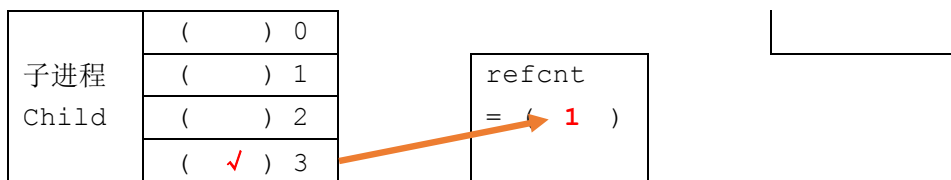
**Part B.**

分析 1

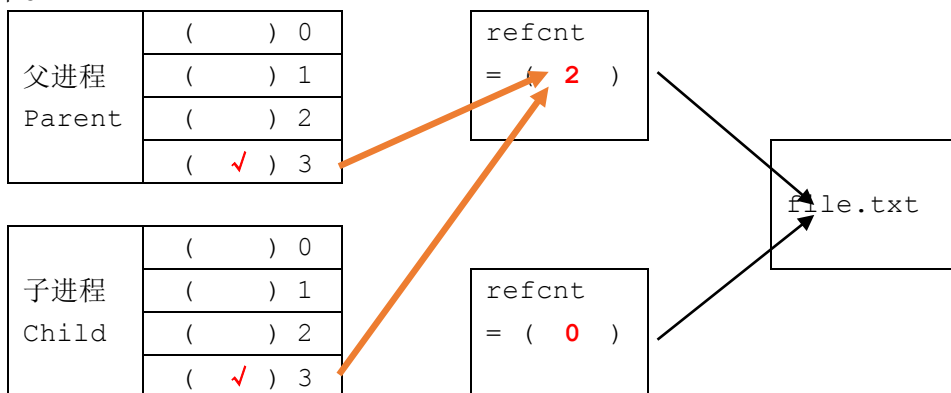


分析 2





分析 3



**Part C.** +++++--++-++----- (6 个+、1 个-、3 个+、2 个-、3 个+、6 个-)

**Part D.** count = 2; (或 count++; 等, 只要让 count 最终的值是 2 就可以了)

**【评分标准】**

Part A. 意思对即可, 1 分。

Part B. 打开文件表的两个条目可上下颠倒。每个分析 2 分: 勾选对父进程的 fd1, 0.5 分; 勾对子进程的 fd1, 0.5 分; 两个 refcnt 各 0.5 分。由于箭头可以被 refcnt 确定, 因此对箭头的连接不赋分。

Part C. 答案正确的 2 分。写对前 8 个字符的得 1 分, 作为“辛苦分”, 因为能写对前 8 个字符表明理解这道题是怎么回事了, 但是由于粗心而导致后面的模拟出错。

Part D. 1 分。漏分号的不扣分。

|    |
|----|
| 得分 |
|    |

第五题（12 分）

1. （2 分，每空一分，考察多级页表和单级页表的设计思想的理解）

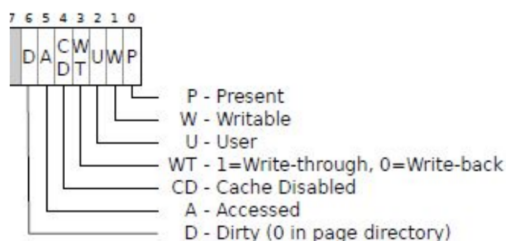
在进行地址翻译的过程中，操作系统需要借助页表(Page Table)的帮助。考虑一个 32 位的系统，页大小是 4KB，页表项(Page Table Entry)大小是 4 字节(Byte)，如果不使用多级页表，常驻内存的页表一共需要\_\_\_\_\_页。

考虑下图已经显示的物理内存分配情况，在二级页表的情况下，已经显示的区域 的页表需要占据\_\_\_\_\_页。

|                              |      |
|------------------------------|------|
| VP0                          | 已分配页 |
| ...                          |      |
| VP1023                       |      |
| VP1024                       |      |
| ...                          |      |
| VP2047                       |      |
| Gap                          | 未分配页 |
| 1023<br>unallocated<br>pages |      |
|                              |      |
| VP10239                      | 已分配页 |
| VP10240                      |      |
| ...                          |      |
| VP11263                      |      |

2. （6 分，每错一空扣一分，扣完为止。考察地址翻译过程）

IA32 体系采用小端法和二级页表。其中两级页表大小相同，页大小均为 4KB，结构也相同。TLB 采用直接映射。TLB 和页表每一项的后 7 位含义如下图所示。为简便起见，假设 TLB 和页表每一项的后 8~12 位都是 0 且不会被改变。注意后 7 位值为“27”则表示可读写。



当系统运行到某一时刻时，TLB 内容如下：

| 索引 | TLB 标记  | 内容         | 有效位 |
|----|---------|------------|-----|
| 0  | 0x04013 | 0x3312D027 | 1   |
| 1  | 0x01000 | 0x24833020 | 0   |
| 2  | 0x005AE | 0x00055004 | 1   |
| 3  | 0x00402 | 0x24AEE020 | 0   |
| 4  | 0x0AA00 | 0x0005505C | 0   |
| 5  | 0x0000A | 0x29DEE000 | 1   |
| 6  | 0x1AE82 | 0x00A23027 | 1   |
| 7  | 0x28DFC | 0x00023000 | 0   |

一级页表的基地址为 0x0C23B00，物理内存中的部分内容如下：

| 地址       | 内容 | 地址       | 内容 | 地址       | 内容 | 地址       | 内容 |
|----------|----|----------|----|----------|----|----------|----|
| 00023000 | E0 | 00023001 | BE | 00023002 | EF | 00023003 | BE |
| 00023120 | 83 | 00023121 | C8 | 00023122 | FD | 00023123 | 12 |
| 00023200 | 23 | 00023201 | FD | 00023202 | BC | 00023203 | DE |
| 00023320 | 33 | 00023321 | 29 | 00023322 | E5 | 00023323 | D2 |
| 0005545C | 97 | 0005545D | C2 | 0005545E | 7B | 0005545F | 45 |
| 00055464 | 97 | 00055465 | D2 | 00055466 | 7B | 00055467 | 45 |
| 0C23B020 | 27 | 0C23B021 | EB | 0C23B022 | AE | 0C23B023 | 24 |
| 0C23B040 | 27 | 0C23B041 | 40 | 0C23B042 | DE | 0C23B043 | 29 |
| 0C23B080 | 05 | 0C23B081 | 5D | 0C23B082 | 05 | 0C23B083 | 00 |
| 2314D200 | 23 | 2314D201 | 12 | 2314D202 | DC | 2314D203 | 0F |
| 2314D220 | A9 | 2314D221 | 45 | 2314D222 | 13 | 2314D223 | D2 |

|          |    |          |    |          |    |          |    |
|----------|----|----------|----|----------|----|----------|----|
| 29DE404C | 27 | 29DE404D | 42 | 29DE404E | BA | 29DE404F | 00 |
| 29DE4400 | D0 | 29DE4401 | 5C | 29DE4402 | B4 | 29DE4403 | 2A |

此刻，系统先后试图对两个已经缓存在 cache 中的内存地址进行写操作，请分析**完成写之后**系统的状态（写的地址和上面的内存地址无交集），完成下面的填空。  
若不需要某次访问或者缺少所需信息，请填“\”。

第一次向地址 0xD7416560 写入内容，TLB 索引为：\_\_\_\_\_，完成写之后该项 TLB 内容为：\_\_\_\_\_，

二级页表页表项地址为：\_\_\_\_\_，物理地址为：\_\_\_\_\_。

第二次向地址 0x0401369B 写入内容，

TLB 索引为：\_\_\_\_\_，完成写之后该项 TLB 内容为：\_\_\_\_\_

二级页表页表项地址为：\_\_\_\_\_，物理地址为：\_\_\_\_\_。

### 3. （2 分，考察对于虚拟内存独立地址空间的理解）

本学期的 fork bomb 作业中，大家曾用 fork 逼近系统的进程数量上限。下面有一个类似的程序，请仔细阅读程序并填空。

```
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>
#define N 4
int main() {
 volatile int pid, cnt = 1;
 for (int i = 0; i < N; i++) {
 if ((pid = fork()) > 0) {
 cnt++;
 }
 }
 while (wait(NULL) > 0);
 return 0;
}
```

整个过程中，变量 cnt 最大值是\_\_\_\_\_。假设所有的数据都已经存在于内存中，pid 和 cnt 在同一个物理页中。从第一个进程开始执行 for 语句开始，此过程对于 cnt 的操作至少会导致页表中\_\_\_\_\_次虚拟页对应的物理页被修改。

答案：

1: 1024, 5

2: 第一次试图向地址 0xD7416560 写入内容，TLB 索引为6，完成写之后该项 TLB 内容是0x00A23067，二级页表页表项地址为\，物理地址是0x00A23560。第二次试图向地址 0x0401369B 写入内容，TLB 索引为3，完成写之后该项 TLB 内容是0x00BA4067，二级页表页表项所在地址为0x29DE404C，物理地址是0x00BA469B。

3: 5, 15



|    |
|----|
| 得分 |
|    |

第六题（10 分）

1. 请在以下的表格中填入相应内容，每个空格仅需填一项。其中第 1 列可填选项包括：网络层、传输层、应用层；第 2 列可填选项 IP、UDP、TCP、HTTP；第 3 列可填选项包括：是、否。

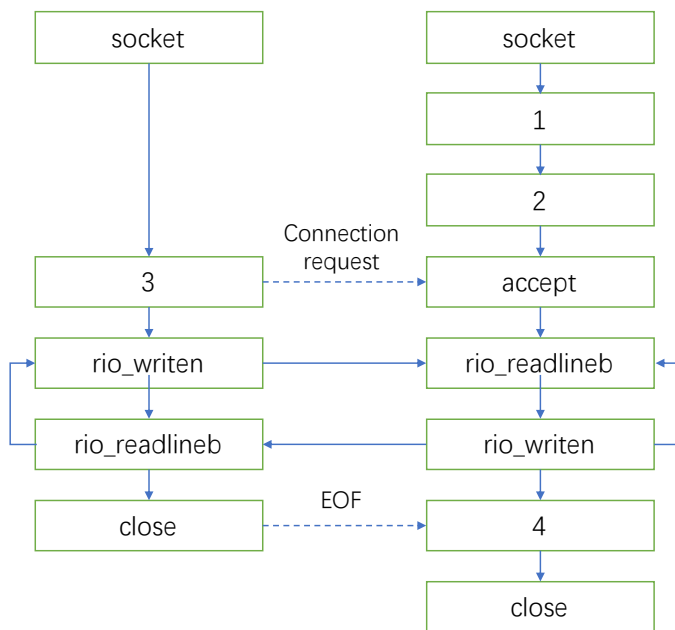
（说明：面向连接的协议保障数据按照发送时的顺序被接收）

| 协议层次 | 协议名称 | 是面向连接的吗？ |
|------|------|----------|
| 网络层  |      |          |
|      | HTTP |          |
|      | TCP  |          |

**答案：（每空 1 分，共 6 分）**

| 协议层次 | 协议名称 | 是面向连接的吗？ |
|------|------|----------|
| 网络层  | IP   | 否        |
| 应用层  | HTTP | 否        |
| 传输层  | TCP  | 是        |

2. 下图描述了客户端与服务器套接字连接和通信的过程，请在空格处补充相关步骤。



答案：（每空 1 分，共 4 分）

1: bind

2: listen

3: connect

4: rio\_readlineb

|    |
|----|
| 得分 |
|    |

第七题（10 分）

给定如下程序：

```
#include <stdio.h>
#include <pthread.h>
int i = 0;
int j = 0;
void *do_stuff1(void * arg __attribute__((unused))) {
 int a;
 for (a = 0; a < 1000; a++)
 {i++; j++;}
 return NULL;
}
void *do_stuff2(void * arg __attribute__((unused))) {
 int a;
 for (a = 0; a < 1000; a++)
 {j++; i++;}
 return NULL;
}
int main() {
 pthread_t tid1, tid2;
 pthread_create(&tid1, NULL, do_stuff1, NULL);
 pthread_create(&tid2, NULL, do_stuff2, NULL);
 pthread_join(tid1, NULL);
 pthread_join(tid2, NULL);
 printf("%d, %d\n", i, j);
 return 0;
}
```

- （2 分）用以下元素的编号写出 i++编译后的汇编代码。  
 (a) mov    (b) add    (c) 0x601040    (d) %eax    (e) \$0x1  
 比如，回答(a) (e) (d)表示 mov \$0x1, %eax

2. (4 分) 请回答该程序是否有可能输出如下结果, 并简述原因。

- (a) 2000, 2000
- (b) 1500, 1500
- (c) 1000, 1000
- (d) 2, 2

3. (2 分) 卜廷江同学在学习了信号量之后, 决定要让程序能稳定输出 2000, 2000, 于是对程序进行了如下改写。

```
1. #include <stdio.h>
2. #include <pthread.h>
3. int i = 0;
4. int j = 0;
5. sem_t si;
6. sem_t sj;
7. void *do_stuff1(void * arg __attribute__((unused))) {
8. int a;
9. for (a = 0; a < 1000; a++) {
10. P(&si);
11. P(&sj);
12. i++;
13. j++;
14. V(&si);
15. V(&sj);
16. }
17. return NULL;
18. }
19. void *do_stuff2(void * arg __attribute__((unused))) {
20. int a;
21. for (a = 0; a < 1000; a++) {
22. P(&sj);
23. P(&si);
24. j++;
25. i++;
26. V(&si);
27. V(&sj);
28. }
```

```

29. return NULL;
30. }
31. int main() {
32. pthread_t tid1, tid2;
33. Sem_init(&si, 0, 1);
34. Sem_init(&sj, 0, 1);
35. pthread_create(&tid1, NULL, do_stuff1, NULL);
36. pthread_create(&tid2, NULL, do_stuff2, NULL);
37. pthread_join(tid1, NULL);
38. pthread_join(tid2, NULL);
39. printf("%d\n", i);
40. return 0;
41. }

```

请问卜廷江同学的程序有什么潜在问题？为什么？

4. （2 分）如果对卜廷江同学的程序改动一处数字来消除上述问题，同时仍然保证输出结果稳定为 2000，2000，应该如何改动？

回答：将\_\_\_\_\_（填行号）行的\_\_\_\_\_（填数字）改为\_\_\_\_\_（填数字）。

答案

1. (a) (c) (d)

(b) (e) (d)

(a) (d) (c)

（写错任意一个不得分）

2. (a) 可能，do\_stuff 俩函数全串行执行即可

(b) 可能，do\_stuff1 先执行了读取 i 的操作，然后 do\_stuff2 执行 500 次循环，接下来 do\_stuff1 写回 i 完成该次循环，则 i 变成 1，j 变成 501。之后 do\_stuff2 执行了读取 j 的操作，然后 do\_stuff1 执行 500 次循环，接下来 do\_stuff2 完成该次循环，则 i 变成 501，j 变成 501。然后两个线程串行执行剩下的 499 次循环即可。

(c) 不可能。可以证明程序结束时一定有  $i+j \geq 2002$ 。只能当某线程先执行了读 i (j) 操作，之后另一个线程执行了若干次  $i++$  ( $j++$ )，然后 h 该线程执行了写 i (j) 操作可以将另一个线程的  $i++$  ( $j++$ ) 效果抵消掉。这个过程中，抵消掉的加法次数一定不会多于执行的加法次数。同时，由于两个线程交错执行  $i++$  和  $j++$  的顺序相反，所以头尾一定存在一次  $i++$  或  $j++$  无法被抵消，因此增加的总次数大于等于 2002。

(d) 不可能。理由同上。

（前一半对，后一半解释大致差不多得分，否则不得分。）

3. 死锁。两个线程分别执行到 11 行和 23 行时会发生。
4. 将 33 行或 34 行中的 1 改为任意大于 2 的数。（错任意一空不得分）

|    |
|----|
| 得分 |
|    |

#### 第八题（10 分，每空 1 分）

题目假设：软硬件系统为 64 位 Linux 操作系统，页大小为 4KB；栈的分配空间最大为 8MB；题目中的 echoserveri/echoserverp/echoservert 分别为原书中的迭代 echo 服务器、基于进程的并发 echo 服务器、基于线程的并发 echo 服务器；当客户端连入服务器时，假设客户端只进行了连接，没有进行后续的读写操作。

启动 echoserveri，查看内存占用情况如下：（输出结果中删除了部分没有用到的列）

```
linux$ pmap -X `pidof echoserveri`
18859: ./echoserveri 15213
Address Perm Offset Device Size Rss Pss Anonymous Mapping
00400000 r-xp 00000000 fc:02 20 20 20 0 echoserveri
00604000 r--p 00004000 fc:02 4 4 4 4 echoserveri
00605000 rw-p 00005000 fc:02 4 4 4 4 echoserveri
00a9a000 rw-p 00000000 00:00 132 8 8 8 (1)
7f56da7af000 r-xp 00000000 fc:00 1792 1220 11 0 libc-2.23.so
7f56da96f000 ---p 001c0000 fc:00 2048 0 0 0 0 libc-2.23.so
7f56dab6f000 r--p 001c0000 fc:00 16 16 16 16 libc-2.23.so
7f56dab73000 rw-p 001c4000 fc:00 8 8 8 8 libc-2.23.so
7f56dab75000 rw-p 00000000 00:00 16 16 16 16
7f56dab79000 r-xp 00000000 fc:00 96 92 1 0 libpthread-2.23.so
7f56dab91000 ---p 00018000 fc:00 2044 0 0 0 0 libpthread-2.23.so
7f56dad90000 r--p 00017000 fc:00 4 4 4 4 libpthread-2.23.so
7f56dad91000 rw-p 00018000 fc:00 4 4 4 4 libpthread-2.23.so
7f56dad92000 rw-p 00000000 00:00 16 4 4 4 4
7f56dad96000 r-xp 00000000 fc:00 152 152 1 0 (2)
7f56dafac000 rw-p 00000000 00:00 16 16 16 16
7f56dafbb000 r--p 00025000 fc:00 4 4 4 4 (2)
7f56dafbc000 rw-p 00026000 fc:00 4 4 4 4 (2)
7f56dafbd000 rw-p 00000000 00:00 4 4 4 4
7ffd382f8000 (3) 00000000 (4) 132 24 24 24 [stack]
7ffd38364000 r--p 00000000 00:00 12 0 0 0 [vvar]
```

```

7ffd38367000 r-xp 00000000 00:00 8 4 0 0 [vdso]
fffffffffff6000000 r-xp 00000000 00:00 4 0 0 0 [vsyscall]
=====
6540 1608 153 120 KB

```

填空：

- 1、 \_\_\_\_\_
- 2、 \_\_\_\_\_
- 3、 \_\_\_\_\_
- 4、 \_\_\_\_\_

在上面的表格中，Size 一列指的是 **VSS (Virtual Set Size)**，表示一个进程可访问的总的地址空间的大小 (the total accessible address space of a process)；**RSS (Resident Set Size)** 表示一个进程实际驻留在 RAM 中的空间大小 (the total memory actually held in RAM for a process)，其中包括了该进程所用到的所有共享空间(如共享库或私有 COW 空间)；**PSS (Proportional Set Size)** 与 RSS 的区别在于共享空间的尺寸，当某个共享空间为 30 页时，如果有三个进程共享该空间，则每个进程的该共享空间的 PSS 仅为 10 页，而每个进程的该共享空间的 RSS 仍为 30 页。根据上述定义，可以做出如下判断：VSS >= RSS >= PSS。

可以根据 echoserveri 的相关数值，对 echoserverp 和 echoservert 的运行情况进行估计：（单选题）

5、使用 echoserverp 作为服务器端，当十个客户端连入 echo 服务器时，总的 VSS 值可能为：

- A. 约 80MB
- B. 约 16MB
- C. 约 6MB
- D. 约 2MB

6、使用 echoserverp 作为服务器端，当十个客户端连入 echo 服务器时，总的 RSS 值可能为：

- A. 约 80MB
- B. 约 16MB



- C. 约 6MB
- D. 约 2MB

7、使用 echoserverp 作为服务器端，当十个客户端连入 echo 服务器时，总的 PSS 值可能为：

- A. 约 16MB
- B. 约 6MB
- C. 约 2MB
- D. 约 500KB

8、使用 echoserverv 作为服务器端，当十个客户端连入 echo 服务器时，总的 VSS 值可能为：

- A. 约 80MB
- B. 约 16MB
- C. 约 6MB
- D. 约 2MB

9、使用 echoserverv 作为服务器端，当十个客户端连入 echo 服务器时，总的 RSS 值可能为：

- A. 约 80MB
- B. 约 16MB
- C. 约 6MB
- D. 约 2MB

10、使用 echoserverv 作为服务器端，当十个客户端连入 echo 服务器时，总的 PSS 值可能为：

- A. 比 echoserverp 作为服务器时的总 PSS 值大
- B. 比 echoserverp 作为服务器时的总 PSS 值小

**答案：**

**1、[heap]或者 heap**

**2、ld-2.2.3.so 或者 ld.so**

**3、rw-p**

**4、00:00**

**5、A（实测：71940KB）**

- 6、C（实测：5696KB）
- 7、D（实测：415KB）
- 8、A（实测：88500KB）
- 9、D（实测：1548KB）
- 10、B（实测：263KB）

# ICS 2019、2020 期末考题分析

王畅

2021 年 12 月

注意：水平有限，不保证正确，祈请原谅。

最后更新于 2021 年 12 月 26 日 18:34

## 2020 年

### 选择题

1. C。A 是正确的，不能把 attack lab 和 bomb lab 混淆了。C 的错误原因在于可以支持可变栈帧，编译时不能确定大小，必须使用帧指针。
2. B。计算知该缓存有 4 组，每组 2 行 4 字节，所以容量为 32 字节。每行需要 4 字节存储块的内容，1 位有效位和 2 位 tag。鉴于课本没有讲解 cache physical size 的计算方法，而且实际上 cache line 可能还有一些额外的 flag bit，因此 1 项后半部分实属超纲。但额外的 3 位无论如何也占不到 2 字节，因此可判断 1 错误。3 说反了，B 的格式导致相邻的缓存块进入相同的组中，会影响空间局部性比较好的程序（顺序访问），这样一来 4 错误（参看书 P432）。最后来计算 2，注意到访问 1、7、8 之后，三个缓存块会进入 A 中的不同组，而 B 则会发生一次替换（8~11 和 0~3 在同一组），后来再访问就会发生 miss 而 A 会命中，所以 2 正确。综上 2 正确。
3. A。取  $x = \text{INT\_MIN}$  即可。B 正确，因为  $\sim x + \sim y + 1 = \sim x + \sim y + 2 - 1 = -(x + y) - 1 = \sim(x + y)$ 。讨论  $x, y$  的大小关系立刻知道 C 是正确的。由于  $\gg$  是向下舍入的，所以 D 正确。
4. B。送分题，float 不能精确表示绝对值  $2^{24} + 1$  及以上的整数。
5. D。送分题，RISC 中寄存器一般更多。
6. D。指令长度为 6 字节，内存地址需要用基址加上偏移计算。
7. D。只有编译时打桩需要访问源代码，A 错误。B，可执行目标文件中仍然会有 ABS 节（文件名）。程序的入口是 `_start`，后者跳到题干所说的位置，C 错误。D 是正确的（书 P489 原话），但是 `-fPIC`（大写）选项更好。
8. B。请注意编译器看不到两边  $x$  类型的不同，因此不会报错。链接器会将  $x$  初始化为 0，在 `fl.c` 中编译器生成的代码视其为浮点而在另一边视其为整数，所以自增结果是一个很小的浮点数，转为整数是 0。

9. B。注意 `printf` 是行缓冲，缓冲区会被复制，最后都一次性输出。另外 `fork() && fork()` 是父进程再 `fork()`，子进程不 `fork()`，而 `fork() || fork()` 则是子进程再 `fork()`，父进程不 `fork()`。故实质上一次产生 3 个进程，所以答案为  $3^2 \times 2 = 18$ 。（如果不考虑缓冲区则要去掉第二次产生的 6 个净拷贝，答案应为 12。）
10. C。送分题，外部 I/O 会触发中断，CPU 执行完当前指令之后可能会去处理该中断。
11. A。送分题，属于书上原话。
12. D。缺页是故障，这种故障需要重新执行引发故障的指令；系统调用是陷阱。二者都是同步的异常，而且处理完毕是从内核态回到用户态，会处理信号。
13. B。A 显然是正确的。B 我们实验过，一般这种读取对于用户来说是允许的，参看书 P511。C 一般同学不会实验过，需要推理一下，因为参数列表和环境变量实际上是程序自己处理的，而且 shell lab 中重定向是 shell 帮忙完成的，故推测 C 正确。D 是陆老师著名的“灵魂出窍”例子，即 `printf` 在信号处理程序中可能导致死锁。
14. B。根据 Core i7 的用法，每张页表恰好有一页，每个页表项 8 字节，所以每个 VPN 都是  $14 - 3 = 11$  位，于是四级 VPN 和 VPO 一共 58 位。
15. B。第二句话表明题目中的内存是按字节寻址的。首先计算 VPO 为 10 位，每个 VPN 都是 8 位，恰好是 34 位虚拟地址。最好情况是 1MB 虚拟地址映射到连续的页，即占有 1024 页，所以需要 1024 个三级页表条目（占有 4 页），1 个二级页表条目（占 1 页）和 1 个一级页表条目（占 1 页），即需要 6KB。最坏情况是每字节都恰好映射到不同的页面，也就是占有  $1024 \times 1024$  个不同的页面（可以实现）。这些页面可以用尽全部  $256 \times 256$  张三级页表（比如每 16 页用一个三级页表），所以需要  $256 \times 256 + 256 + 1 = 65793$  页，即 65973 KB。
16. B。和物理地址无关，VPO 为 13 位，VPN 均为 10 位，所以需要三级页表。
17. D。根据题目的描述就可以推断是 A 位和 D 位。
18. A。未解之谜。如果将共享区域理解为 `MAP_SHARED` 则 D 错误；否则共享库也是一种共享区域，会发生 COW。如果参照 Wikipedia 上的解说，COW 是一种技术，用户可以自行实现，那么对等线程之间也可能发生 COW（例如 `string y=x` 之后修改 `y`，C++ 中用 COW 实现）。A 可能是更好的选择。
19. A。其他段中都可能指针，其引用的内存需要管理。代码段中则没有这种问题。
20. C。服务器端的进程通过 `accept` 来建立已连接套接字。D 应阻塞在 `read`，参看课程投影片。
21. D。送分题，根据这个命令的字面意思也能看出获得的是主机名，参数 `-i` 表示返回点分十进制的 IP 地址。网络字节顺序和大端法是一回事。
22. C。a) 是共享的（在数据段），b) 也是共享的（栈是私有的），c) 是共享的，属于书上原话，或者从同一进程的概念中读解出，d) 是特别的上下文，是私有的。所以有 3 个。

23. C。此题意义不大。主要的困难在于最终的正确顺序并不是确定的一个全序，所以需要找一个“最优”的全序。大概估计一下， $L_2, U_2, S_2$  和  $L_4, U_4, S_4$  之间不能有重叠， $L_1, U_1, S_1$  和  $L_3, U_3, S_3$  之间亦然，所以可能是

$$L_2, L_1, U_1, S_1, L_3, U_2, S_2, L_4, U_4, U_3, S_4, S_3.$$

数出逆序数为 12。实际上，合法的全序只有十几种，可以枚举验证答案正确性。

24. B。注意主线程如果调用诸如 `pthread_exit` 的函数退出，是不会引发其他线程退出的，所以 A 为一错误描述。但是如果主线程调用了 `exit`（或者 `return`），则整个进程都会结束，自然其他线程也结束。
25. B。首先 1 和 2 分别获得 a 和 c 的锁，然后 2 再获得 b 的锁，1 和 3 分别等 d 的锁和 a 的锁。此时如果 2 释放 c 和 b 的锁，1 和 3 无论如何都将出现互相等待（a、d）的情况，因此发生了死锁。

### 解答题一

1. 10ns。送分题。
2. 60。题目中为一个 8 重循环，具体执行了什么不重要。每次循环出现数据冒险一次，故需要 7 个周期。最后分支预测错误有 2 个周期的惩罚，但是已经算在 4 个 `trailing cycle` 中了，所以需要 60 个周期。
3. 47；5；96。两个缓存都是 5 位偏移，10 位 tag（这里可能应该认为地址空间是 16 位）。因此指令缓存只会不命中一次，剩下 47 次都是命中的。数据缓存每块可以放下 4 个长整数；但注意起始位置不是 32 字节对齐的，所以会有 3 次不命中，剩下 5 次都是命中的。不命中一共额外造成了  $9 \times 4$  周期的惩罚，所以需要 96 周期。

### 解答题二

1. 答案如下：

| 符号    | .symtab 有条目? | 符号类型 | 定义符号的模块 | 节      |
|-------|--------------|------|---------|--------|
| buf   | 有            | 外部   | m.o     | .data  |
| bufp0 | 有            | 全局   | swap.o  | COMMON |
| count | 有            | 局部   | swap.o  | .bss   |
| func  | 有            | 全局   | swap.o  | .text  |
| temp  | 无            | /    | /       | /      |

送分题。注意外部符号要填定义的模块中的节。

2. 0x00002020, 0x000000be, 0x0000a000, 0x0000900d, 0x00decade。

1 处, ADDR=1000, offset=a, addend=-4, 结果为 302e-1000-a-4=0x00002020。3 处类似, 结果为 10e4-1000-22-4=0x000000be。4 处结合汇编代码知道目标符号是 buf 的地址移入, 所以是 0x0000a000 (绝对重定位)。9 处对应 bufp1, 下一条指令地址是知道的, 可以不用重定位算法, 所以是 a250-1243=0x0000900d。11 处对应 count, 下一条指令地址也是知道的, 所以是 dedd38-125a=0x00decade。具体填要注意小端法和立即数格式。

**解答题三** 4 行, 内容为

```
a b c 0
a b a 1
a a b 1
a a a 2
```

此题直接画进程树, 在结点旁边标注父进程和子进程, 并写出状态变化就可以。考点为每次父进程 open 时, 指针都重新回到文件开头; dup 则保持文件指针为同一个; count 四个进程相互独立。注意 (相对的) 父进程总是用 wait (NULL) 等待子进程, 所以输出的顺序一定是子子、子父、父子、父父。

**解答题四** 答案如下:

| $n$ | 缺页次数 | dTLB 失效次数 |           |           |           |           |        |
|-----|------|-----------|-----------|-----------|-----------|-----------|--------|
|     |      | 1         | 2         | 3         | 4         | 5         | 6      |
| 8   | 1    | 1         | 1         | 1         | 1         | 1         | 1      |
| 16  | 2    | 2         | 2         | 2         | 2         | 2         | 2      |
| 64  | 24   | 24        | 24        | 24        | 24        | 24        | 24     |
| 512 | 1536 | 266436157 | 268661500 | 252182591 | 252182591 | 235241284 | 521528 |

题目看起来非常复杂, 实际上分析一下就可以发现没有多少耦合, 数据也凑得比较好, 很容易拿到大部分分数 (但是此题意义不大, 而且没有考察虚拟内存的精华)。

不难看出, 缺页次数只和矩阵占的页数有关 (无论怎么访问), 而 TLB miss 等价于为一个缓存块 4KB, 64 行的全相联高速缓存的 miss 次数。这样一来, 数据占据页面的个数, 也就是缺页次数为  $\lceil 3n^2 \times 8/2^{12} \rceil$ , 分别为 1、2、24 和 1536。由于 TLB 是全相联的, 只有满了之后才会发生替换, 所以前三行的答案已经全部得出 (缺页次数 = TLB miss 次数 = 页面个数)。

$n = 512$  时的 TLB 行为模拟比较简单, 但仍然有一定的工作量, 这 1536 个页面每页恰好对应矩阵的一行, 所以 TLB miss 的次数就转变为对行访问的分析, 由于全相联缓存的行可以放在任何位置, 所以简单替换就可以, 通过计算可以得出答案。

**解答题五**

1. 死锁。可以看出，这实际上就是哲学家就餐问题，如果所有的同学同时拿起左边那个同学手机  $P(UP(i))$ ，就发生死锁。
2. 分别填 0 和 0。观察代码知道 Dave 的方法每次只有一位同学去找洞主，也就是说 26 个同学轮流去验证。由此知道只需要一个互斥锁即可。
3. ①②②①。根据 1 问的例子可以知道依赖关系成环是最大的问题。因此我们可以让除一位同学之外的同学正常工作，而那位同学则反过来工作，不难证明这个体系无死锁且无竞争（事实上，还有一种方法，即要求单号同学先拿左边的手机，而双号先拿右边的手机）。
4. 分别填 1、0、<、③、②、<=。根据代码容易看出 mutex 保护结构体，zero 是一个“等待信号量”，所以计数降到 0 以下时需要等待，由此发现其初值应该是 0。请注意 ③、② 不能反，否则会造成死锁。

## 2019 年

### 选择题

1. A。唯有这一项会先算  $d+a$ ，可能发生不结合的问题。
2. B。数据冒险一般是指当前指令没有写回，而下一条指令在流水线就已经要用到的情况。
3. A。除法向零取整，移位向下取整；2 相当于给负数添加一个修正，3 相当于转成正数后模拟除法行为。所以  $a=-2016$ ， $b=-2016$ ， $c=-2016$ 。
4. C。CISC 变长指令，有不少指令的长度是要比 RISC 短的。当代手机常常采用 ARM 架构，这是 RISC；实际上对能耗要求高或者结构简单的设备一般都用 RISC。
5. B。DRAM 常常组织成一个矩阵族，整行访问时效率比较高，B 错误。SSD 设备擦写的时间会比读取要高一个数量级。一般来说，SRAM 使用比较多的晶体管，而 DRAM 晶体管很少，主要基于电容，SSD 是闪存，晶体管也不会很多，因此后二者的存储密度相对高。
6. C。实际上对符号谈重定位是不正确的，应当对引用谈重定位，例如全局变量如果初始化为值且不使用，则不需要重定位。需要留意，如果是同一 C 语言源文件中定义的函数则一般不需要重定位。
7. C。除法错误是不可恢复故障，是同步的；I/O 中断是异步的，一般会执行完当前指令，再去处理，返回就不需要再处理了；缺页异常当然需要重新执行遇到问题的访存指令；时间片到中断属于时钟中断，属于异步异常。
8. 未解之谜，CD。并行一定并发。
9. C。fork 返回两次，setjmp 可返回多次，longjmp 和 execve 不返回。
10. A。高速缓存是物理寻址的，所以一定不用刷新。用户态和内核态的转换不会改变内存映射，而上下文切换会改变虚拟内存到物理地址的对应关系，所以 TLB 在上下文切换时要刷新。

11. C。VPO 为 11 位，页表均占一页，则 VPN 都为 8 位，所以映射满 48 位地址空间至少需要 5 级页表。
12. D。直接模拟就可以。
13. B。最好情况是高速缓存和 TLB 都命中，不需要访问内存；最坏情况是都不命中，需要访问四级页表项以及主存中的数据，一共 5 次。
14. B。参看课程投影片，迭代服务器的第二个客户端会在 read 阻塞。
15. C。网络字节序规定为大端序，在不同的主机中可能需要转换。
16. A。文件名和参数应该用 ? 分隔。
17. D。2 和 4 都是很明显的，6 仍然是陆老师著名的“灵魂出窍”例子，即 printf 在信号处理程序中可能导致死锁（加锁后打断，在信号处理程序再调用即死锁，因为需要信号处理程序返回后才能解锁）。
18. B。保持跨越多个调用的状态函数不是线程安全的，而 C 项是隐式可重入函数，如果合理使用就是线程安全的。
19. C。否则会在对等线程的赋值和主线程的 accept 之间引入竞争，如果赋值在 accept 之后完成，描述符值就错了。
20. D。D 是超纲选项，sem 型指令会阻塞整个进程，而 pthread\_mutex 型指令只会阻塞相应线程，后者的效率更高。A、B、C 都是容易看出正确的，所以可以用排除法做出。

### 解答题一

1. 8，送分题。
2. 48 周期，即  $8 \times 6$ 。
3. 50 周期。因为每个缓存块可存 8 个长整数，但起始地址不是 64 字节对齐，所以会发生 2 次 miss。
4. 42 周期。因为只有前两个 addq 可以一次执行完，这样能节省 8 周期的时间。
5. 26 周期。指令顺序可重新安排为

```
.L2:
 movq (%rdi), %rbx
 movq 8(%rdi), %rdx
 addq %rbx, %rax
 addq $16, %rdi
 addq $2, %rcx
 addq %rdx, %rax
```



```
cmpq $8, %rcx
jl .L2
```

此时每次循环只需要 6 周期，一共 4 次循环，即 24 周期，补上两次 miss 的惩罚得到 26 周期。

## 解答题二

1. 答案如下：

| 符号    | .symtab 有条目? | 符号类型 | 节      | 强弱符号 |
|-------|--------------|------|--------|------|
| bufp1 | 有            | 全局   | .data  | 强    |
| buf   | 有            | 外部   | UNDEF  | 弱    |
| bufp0 | 有            | 全局   | COMMON | 弱    |
| temp  | 无            | /    | /      | /    |
| count | 有            | 局部   | .bss   | 非强非弱 |

送分题。注意外部符号要填 foo.o 中的节，extern 的变量严格说不能区分强弱，但是如果问则回答“弱”。

2. 未知；bufp1.foo。弱弱选弱，强弱选强。

3. 0x000000be, 0x0000a000, 0x0000900d, 0x0000babe。

1 处, ADDR=f28, offset=16, addend=-4, 结果为 1000-f28-16-4=0x000000be。2 处结合汇编代码知道目标符号是 buf 的地址移入, 所以是 0x0000a000 (绝对重定位)。6 处对应 bufp1, 下一条指令地址是知道的, 可以不用重定位算法, 所以是 a050-1043=0x0000900d。9 处对应 count, 下一条指令地址也是知道的, 所以是 cb24-1066=0x0000babe。具体填要注意小端法和立即数格式。

## 解答题三

1. 3, 4。注意 0, 1, 2 是标准 I/O 流占用的。

2. 4, 5。因为 fd2 已经被关闭, 其描述符可复用; 另外 fd3, fd4 虽然指针相同, 但是描述符的号码是不同的。

3. 0, 1, 2, 3。父进程等待子进程结束, 所以子进程先输出, 另外父子进程的 count 当然是独立的。

4. 2019PKUCS。对 f1.txt 的写有三处, 其中 20、21 行是独立的, 得到 2019PKU, 最后只有子进程再次写文件, 此时 fd1 的指针在最后, 即 2019PKUCS。

5. PKUICS2019。鉴于 dup 产生 fd4, 而父子进程的文件描述符指向同样的打开文件表条目, 所以对 f2.txt 的写本质上只有一个指针, 因为父进程等待子进程结束, 所以先输出 PKUICS 再输出 2019。

**解答题四** 此题我们课上讲过了，原题有几个数据错误，条件也不够。摘取正确题目的分析如下：

1. 4096; 6; 10。送分题。

2. 0xEAB450D, 0x67F000, 0xAA3000, 0xC3F000。

考虑父进程执行 \*b 的写时的读写问题。在解析 \*b 时，第一步当然是找出一级页表项，因此从地址 0x67F0E8 读出 0x80AA32C4 就是从一级页表中，物理地址 0x67F0E8 的位置，读出一级页表项的内容 0x80AA32C4。根据题目对 PTE 结构的描述，我们知道 0xAA3 是二级页表页的页号，即 0xAA3000 是二级页表的起始物理地址。由于一级页表是 4 KB 对齐的，所以一级页表项的物理地址就是 0x67F000。

为什么会写两次？显然，我们知道 COW 机制在起作用，这段内存首先要被复制。0x80C3F110 这个数字当然是精心设计的，必然暗藏玄机。首先，一次写肯定是写 \*b，那么另一次写呢？注意这次写也是父进程，因此和子进程无关。由于 COW 后内存映射需要修改，结合后面也有类似于 0xC3F... 的物理地址，我们可以推定这个实际上就是 COW 后，b 对应的二级页表项。这样我们马上知道，其二级页表项的物理地址在 0xAA3AD0，而 b 指向的真实物理地址是 0xC3F50D（页号 0xC3F）。这时我们就可以计算出 b 的值（即虚拟地址）了。一级页表项的偏移是  $(0x67F0E8 - 0x67F000) / 4 = 0x3A$ ，二级页表项的偏移是  $(0xAA3AD0 - 0xAA3000) / 4 = 0x2B4$ ，VPO=0x50D，所以虚拟地址是它们的拼接。注意不是直接将 16 进制拼接，虚拟页号为 11 1010 10 1011 0100=0xEAB4，所以 b=0xEAB450D。

此题页表项中的最低 12 位是何含义，属于未解之谜。

### 解答题五

1. bind, connect, accept, writen, readlineb, readlineb, writen。容易读解，但注意 RIO 包的 API 名称不能记错。

2. bind, listp, listenfd。

### 解答题六

1. 7, 14, 7, 2, 0。这时两个读者都在正常读，因为是读者优先，所以写者在 14 行阻塞等待，写锁的值是 0。

2. 14, 7, 3, 0。完全同理。

3. 会出现竞争的错误。例如两位读者同时进入，由于 readcnt 的写不是原子的，可能导致进入后 readcnt=1，当一位读者离开后，写者就错误进入了。

# 北京大学信息科学技术学院考试试卷

考试科目： 计算机系统导论 姓名： \_\_\_\_\_ 学号： \_\_\_\_\_

考试时间： 2018 年 01 月 07 日 小班编号： \_\_\_\_\_ 小班教师： \_\_\_\_\_

| 题号  | 一 | 二 | 三 | 四 | 五 | 六 | 七 | 八 | 总分 |
|-----|---|---|---|---|---|---|---|---|----|
| 分数  |   |   |   |   |   |   |   |   |    |
| 阅卷人 |   |   |   |   |   |   |   |   |    |

## 北京大学考场纪律

1、考生要按规定的考试时间提前 5 分钟进入考场，隔位就坐或按照监考人员的安排就座，将学生证放在桌面。无学生证者不能参加考试；迟到超过 15 分钟不得入场；与考试无关人员不得进入考场。考生在考试开始 30 分钟后方可交卷出场；未交卷擅自离开考场，不得重新进入考场继续答卷；交卷后应离开考场，不得在考场内逗留或在考场附近高声交谈。

2、除非开卷考试中教师另有说明，除必要的文具和主考教师允许的工具书、参考书、计算器以外，其它所有物品（包括空白纸张、手机等）不得带入座位，已经带入考场的手机等电子设备必须关机，不得随身携带或放在座位旁边，应与其他物品一起放在监考人员指定的位置。

3、考试使用的试题、答卷、草稿纸由监考人员统一发放，考试结束时收回，一律不准带出考场。若有试题印制问题应向监考人员提出，不得向其他考生询问。考生提前答完试卷，应举手示意请监考人员收卷后方可离开；考试结束监考人员宣布收卷时，考生应立即停止答卷，在座位上等待监考人员收卷清点后，方可离场。

4、考生要严格遵守考场规则，在规定时间内独立完成答卷。不准旁窥、交头接耳、打暗号或做手势，不准携带与考试内容相关的材料参加考试，不准使用手机、非教师允许的计算器等具有信息发送、接受、存储功能的设备，不准抄袭或协助他人抄袭试题答案或者与考试内容相关的资料，不准传、接或者交换试卷、答卷、草稿纸，不准由他人代替考试或替他人参加考试等。凡违反考试纪律或作弊者，按《北京大学本科考试工作与学习纪律管理规定》给予相应处分。

5、考生须确认自己填写的个人信息真实、准确，并承担信息填写错误带来的一切责任与后果。

以下为试题和答题纸，共 21 页。

|    |
|----|
| 得分 |
|    |

第一题 单项选择题（每小题 1 分，共 20 分）

注：选择题的回答填写在下表中。

|    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|
| 题号 | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
| 回答 |    |    |    |    |    |    |    |    |    |    |
| 题号 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| 回答 |    |    |    |    |    |    |    |    |    |    |

1. 在 **gdb** 调试中，下一条指令是 **call func1**。下面哪条 **gdb** 指令能执行该指令并且停留在 **func1** 的第一条指令？

- A. **br func1**                  B. **si**                          C. **ni**                          D. **disas func1**

2. 浮点数的阶码没有设计成补码的形式的原因是：

- A. 为了加快四则运算                          B. 为了方便排序  
C. 为了保存更多数值                          D. 为了表示特殊值

3. 下面说法正确的是：

- A. 不同指令的机器码长度是相同的  
B. **test %rax, %rax** 恒等于 **cmp \$0, %rax**  
C. **switch** 编译后总是会产生跳转表  
D. 以上都不对

4. 分析下图的指令执行步骤，请问这是 Y86 指令系统的哪条指令？

|            |                                                          |
|------------|----------------------------------------------------------|
| Fetch      | $icode:ifun \leftarrow M_1[PC]$                          |
| Decode     | $valA \leftarrow R[\%esp]$<br>$valB \leftarrow R[\%esp]$ |
| Execute    | $valE \leftarrow valB + 4$                               |
| Memory     | $valM \leftarrow M_4[valA]$                              |
| Write back | $R[\%esp] \leftarrow valE$                               |
| PC update  | $PC \leftarrow valM$                                     |

- A. **call**  
B. **ret**  
C. **pushl**  
D. **popl**

---

5. 如果四路组相联的高速缓存大小是 32KB, 并且块(block)大小为 32 字节, 那么它每路(way)有多少行(line)?

- A. 32
- B. 64
- C. 128
- D. 256

6. 某个机械硬盘的部分指标数据如下, 则该硬盘标称的容量最有可能是多少?

- 1024 bytes/sector
  - 400 sectors/track (on average)
  - 20000 tracks/surface
  - 5 platters/disk
- A. 80 GB
  - B. 76 GB
  - C. 40 GB
  - D. 38 GB

7. C 源文件 f1.c 和 f2.c 的代码分别如下所示:

```
// f1.c
#include <stdio.h>
void f();
int x ;
int main(void)
{
 x = 1;
 f() ;
 printf("%x\n", x) ;
 return 0;
}
```

```
//f2.c
float x ;
void f()
{
 x = 2 ;
}
```

运行下面的命令后得到的结果是:

```
$ gcc f1.c f2.c
$./a.out
```

- A. 1
- B. 2
- C. 3f800000
- D. 40000000

8. 下面关于链接的说法, 正确的是:

- A. Linux 链接器在处理多重定义的同名弱符号时, 选择链接时遇到的第一个符号
- B. 链接发生在源代码编译之后、可执行目标程序运行之前
- C. C 程序静态局部变量和静态全局变量都在 ELF 可重定位目标文件的.data 段
- D. 链接器构造可执行目标文件时, 只复制静态库里被应用程序引用的目标模块

9. 下列程序输出的数字顺序可能是：

```
int count = 1;
if (fork() == 0) {
 if (fork() == 0) {
 printf("%d\n", ++count);
 }
 else {
 printf("%d\n", --count);
 }
}
printf("%d\n", ++count);
```

- A. 0 1 3 2 2                      B. 0 3 2 2 1  
C. 2 0 1 3 2                      D. 2 1 0 2 3

10. 下列关于信号的说法不正确的是：

- A. 在键盘上输入 Ctrl-C 会导致内核发送一个 SIGINT 信号到前台进程组中的每个进程  
B. 每种类型最多只能有一个未处理的信号  
C. SIGINT 的处理函数不能被另一个 SIGINT 信号中断  
D. 进程可以通过使用 signal 函数修改和 SIGSTOP 相关联的默认行为

11. 关于动态内存分配，下列说法中正确的是：

- A. 显式分配器可以重新排列请求顺序，从而最大化内存利用率  
B. 显式分配器可以修改已分配的块，把内容复制到别的位置，从而消除外部碎片  
C. 通常显式分配器会比隐式分配器更快  
D. C 语言中如果某个已分配块不再可达，那它就会被释放并返回给空闲链表

12. 在设计分配器时，下列说法中错误的是：

- A. 搜索空闲链表时，存储利用率为：best fit > next fit > first fit  
B. 带头部的隐式空闲链表，合并（内存中的）下一个空闲块可在常数时间内完成  
C. 如果采取立刻合并策略（immediate coalescing），会在某些请求模式中出现反复合并又分割的情况，于是会有较小的吞吐率  
D. 分配器使用二叉树结构，主要是为了能够更快地找到适配的空闲块

---

13. 下列说法错误的是:

- A. 虚拟内存存放在磁盘上, 这导致不命中时处罚非常大
- B. 使用 `fork` 函数时, 不会立刻拷贝内存空间
- C. 进程不会意外访问别的进程的内存空间
- D. 使用动态内存的主要原因是栈容易受到缓存溢出 (`buffer overflow`) 的腐蚀

14. 以下关于文件 I/O 的说法中, 正确的是:

- A. 文件重定向 (`dup` 和 `dup2`) 操作仅仅改变了文件描述符的指向, 不会改变打开文件表中的内容
- B. 进程调用 `fork()` 时, 可能对文件描述符表和打开文件表采用写时拷贝 (`Copy on Write`) 机制
- C. 对同一描述符, `rio_readlineb` 和 `rio_readnb` 可以任意交叉使用, `rio_readn` 和 `rio_writen` 也可以任意交叉使用
- D. RIO 中包括无缓冲的输入输出函数和带缓冲的输入输出函数, 使用带缓冲的输入输出函数时, 要先声明一个 `rio_t` 类型变量并调用 `rio_readinitb` 函数

15. 以下程序执行完成后, `ICS.txt` 文件中的内容是:

```
int main(int argc, char** argv) {
 int fd1 = open("ICS.txt", O_CREAT|O_RDWR,
 S_IRUSR|S_IWUSR);
 write(fd1, "ics ", 4);
 int fd2 = fd1;
 int fd3 = dup(fd2);
 int fd4 = open("ICS.txt", O_APPEND|O_RDWR);
 write(fd2, "segmentation fault ", 19);
 write(fd4, "tao", 3);
 int fd5 = fd4;
 dup2(fd3, fd5);
 write(fd4, "lab", 3);
 close(fd1);
 return 0;
}
```

- A. ics segmentation fault tao
- B. ics segmentation fault lab
- C. ics taomentation fault lab
- D. tao segmentation fault lab

---

16. 一台处于校园网内的笔记本电脑访问一台处于公网上的服务器上的网页服务。网页服务使用默认的 80 端口。以下答案有三项一定不正确，可能正确的那一项是：

- A. 笔记本在通信过程中，本地使用的端口号是 80
- B. 笔记本的 IP 地址是 192.168.1.101
- C. 服务器向笔记本传送网页内容时，使用的服务器端口号是 80
- D. 服务器的 IP 地址是 192.168.1.102

17. 下列关于计算机网络概念的说法中，正确的是：

- A. HUB 与交换机都会转发从任意端口上收到的帧，区别在于交换机转发速度更快
- B. UDP 是无连接的协议，而实时音视频一般需要先呼叫对方建立连接，因此这些应用不能基于 UDP 传输音视频数据
- C. 处于数据链路层的以太网协议和处于传输层的 TCP 协议都对数据进行了校验
- D. HTML 是互联网上应用最为广泛的超文本传输协议

18. 下列关于计算机网络的说法中，错误的是：

- A. 在 Linux 系统中，每台主机都有本地定义的域名 localhost，这个域名总是被映射为 IP 地址 127.0.0.1
- B. 在套接字编程中，既可以用网卡地址作为地址，也可以用 IP 地址作为地址
- C. Web 客户端和服务端间的交互采用的是基于文本的无连接协议
- D. IPv4 中的 IP 地址是一个 32 位无符号整数，IPv6 中的 IP 地址是一个 128 位无符号整数

19. 下列关于死锁的叙述中，不正确的是：

- A. 所谓死锁是指一组线程被阻塞，等待一个永远不会为真的条件
- B. 在用信号量及 PV 操作解决生产者消费者问题（多个缓冲区、多个生产者，多个消费者）时，如果先给缓冲区加锁 `P(&sp->mutex)`，再申请可用的缓冲区槽 `P(&sp->slots)`；则可能出现死锁
- C. `printf()` 是线程安全函数
- D. 在信号处理函数中使用 `printf()` 不会导致死锁



---

20. 某进程的主线程和对等线程的代码如下所示:

```
sem_t sem;

int main()
{
 int i;
 pthread_t tids[3];
 sem_init(&sem, 0, 1);

 for (i=0; i<3; i++) {
 pthread_create(&tids[i], NULL, thread, NULL);
 }

 for (i=0; i<3; i++) {
 pthread_join(&tids[i], NULL);
 }
 return 0;
}

int y = 15;
void *thread (void *arg)
{
 P(&sem);
 y = y - 3;
 V(&sem);
 printf("%d\n", y);
}
```

执行上述代码后, 会产生多少种不同的输出?

- A. 11                      B. 12                      C. 13                      D. 14

|    |
|----|
| 得分 |
|    |

第二题（12 分）

假设程序运行在 x86-64 的小端法机器上，编译器按照标准规则对齐，并且有 nan 参与的算术比较运算都返回 0。请回答下列问题：

1. 请根据左边 C 程序和右边对应的汇编代码，填写空白处的代码。

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> struct a {     char c[2][3];     int i; }; struct b {     float f;     struct a a;     char c; }; union c {     struct a a;     struct b b;     struct b* pb; }; void proc1(union c* u){     double a = u-&gt;b.f_____ ;     double b = u-&gt;b.f_____ ;     printf("%d\n", a==b); }  void proc2(union c* u){     char* a = u-&gt;a.c[1] + 1;     char* b = (u-&gt;a.c + 1)[1];     printf("%d\n", b - a); }  void proc3(union c* u){     char x = u-&gt;a.c[1][0]; </pre> | <pre> ..... proc1:     .....     movss    (%rcx), %xmm1     movss    .LC0(%rip), %xmm0     addss    %xmm1, %xmm0     cvtss2sd %xmm0, %xmm2     movsd    %xmm2, -8(%rbp)     movss    (%rcx), %xmm0     cvtss2sd %xmm0, %xmm0     movsd    .LC1(%rip), %xmm1     addsd    %xmm1, %xmm0     movq     %xmm0, -16(%rbp)     .....  proc2:     .....  proc3:     .....     _____     movb     %al, -1(%rbp)     movzbl   2(%rcx), %eax     movb     %al, -2(%rbp)     .....  proc4:     .....     movl     8(%rcx), %eax     movl     %eax, -4(%rbp)     movzbl   16(%rcx), %eax </pre> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> char y = _____; if (x == 0x7F &amp;&amp; (y &amp; 0x80) == 0x80)     printf("%d\n", (u-&gt;b.f &lt; u-&gt;a.i)); else     printf("0\n"); }  void proc4(union c* u){     int x = u-&gt;a.i;     int y = _____;     printf("%d\n", (x &gt; y) != (-x&lt;-y)); }  void main(){     unsigned int i;     printf("%d\n", sizeof(struct a));     printf("%d\n", sizeof(struct b));     printf("%d\n", sizeof(union c));      union c u;     ..... //随机初始化 u     proc1(&amp;u);     proc2(&amp;u);     proc3(&amp;u);     proc4(&amp;u); } </pre> | <pre> movsbl    %al, %eax movl      %eax, -8(%rbp) ..... main: ..... leaq 32(%rsp), %rbx movq      %rbx, %rcx call proc1 movq      %rbx, %rcx call proc2 movq      %rbx, %rcx call proc3 movq      %rbx, %rcx call proc4 ..... .align 4 .LC0: .long      1048576000            //十六进制 3E800000 .align 8 .LC1: .long      0 .long      1070596096            //十六进制 3FD00000 </pre> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

2. 程序执行会产生 7 行输出。请写出每一行所有可能的输出值，不同可能性之间逗号分隔

第一行: \_\_\_\_\_

第二行: \_\_\_\_\_

第三行: \_\_\_\_\_

第四行: \_\_\_\_\_

第五行: \_\_\_\_\_

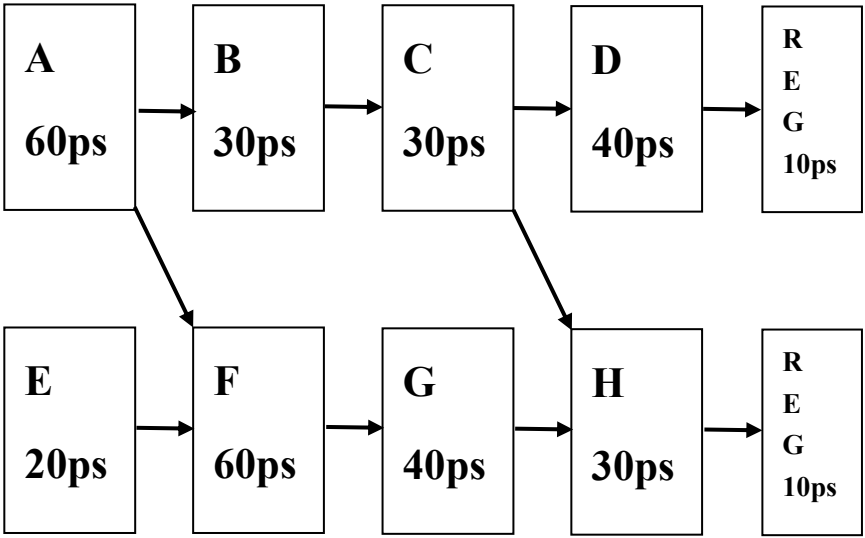
第六行: \_\_\_\_\_

第七行: \_\_\_\_\_

|    |
|----|
| 得分 |
|    |

第三题（12 分）

如图所示，每个模块表示一个单独的组合逻辑单元，每个单元的延迟以及数据依赖关系已在图中标出。通过在两个单元间添加寄存器的方式，可以对数据通路进行流水化改造。假设每个寄存器的延迟为 10ps。



1. 如果改造为一个二级流水线，为获得最大的吞吐率：
  - a) 需要在\_\_\_\_\_模块之间和\_\_\_\_\_模块之间插入寄存器；
  - b) 插入寄存器改造后二级流水线的最长延时路径为\_\_\_\_\_，最长延时为\_\_\_\_\_
  - c) 流水线最大吞吐率为：\_\_\_\_\_

（用分数表示，约分到最简形式，并写明单位）
  
2. 如果改造为一个三级流水线，为获得最大的吞吐率：
  - a) 需要在\_\_\_\_\_模块之间插入寄存器；
  - b) 插入寄存器改造后三级流水线的最长路径延时为：\_\_\_\_\_；
  - c) 流水线最大吞吐率为：\_\_\_\_\_

（用分数表示，约分到最简形式，并写明单位）

---

|    |
|----|
| 得分 |
|    |

#### 第四题（10 分）

考虑以下三个文件：

##### polygon.h

```
struct Node {
 float pos[2];
 int marked;
 struct Node* next;
 struct Node* prev;
};
typedef struct Node node;

node* alloc();
void init();
void gc();
```

##### main.c（函数中部分内容折叠）

```
#include "polygon.h"
node* root_ptr ;
int main(){
 node* p;
 init();
 p=alloc();
 root_ptr =p;
 ...
 gc();
 ...
 return 0;
}
```

##### gc.c（函数体被折叠）

```
#include "polygon.h"
#define N (1<<20)
static node polygon [N];
static node* free_ptr ;
static node* root_ptr ;
void mark(node* v){...}
void sweep() {...}
void gc() {...}
void init() {...}
node* alloc() {...}
```

使用命令 `gcc -o polygon main.c gc.c` 得到可执行文件 `polygon`。

- 
1. 对于每个程序中的相应符号，给出它的属性（局部或全局，强符号或弱符号）  
提示：如果某表项中的内容无法确定，请画 x。

**main.c**

|          | 局部或全局？ | 强或弱？ |
|----------|--------|------|
| root_ptr |        |      |
| init     |        |      |
| main     |        |      |

**gc.c**

|         | 局部或全局？ | 强或弱？ |
|---------|--------|------|
| N       |        |      |
| polygon |        |      |
| alloc   |        |      |

2. 解释为何其中一些符号被定义了多次，链接器仍然可以成功创建可执行文件。
3. gc.c 的功能是实现一个垃圾收集器。解释为何前面的命令能够编译、链接成功，但得到的执行文件中却存在潜在错误。并试提出如何修复这一 bug。

---

|    |
|----|
| 得分 |
|    |

第五题（10 分）

代码如下：（仅为示例代码，答题时不考虑任何出错的情况）

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/wait.h>

int counter1 = 1;
int counter2 = 1;

void handler(int sig) {
 if (sig == SIGUSR1) counter1++;
 if (sig == SIGUSR2) counter2++;
}

int main(int argc, char **argv)
{
 int i;
 pid_t ppid, cpid;
 int fd1, fd2;
 unsigned char buf1[64], buf2[64];

 signal(SIGUSR1, handler);
 signal(SIGUSR2, handler);

 fd1 = open("number.txt", O_RDWR);

 for (i = 0; i < N ; i++) {
 ppid = getpid();
 cpid = fork();
 if (cpid) {
```

---

```

 kill(cpid, A);
 while (sleep(4)) ;
 } else {
 kill(ppid, B);
 while (sleep(2)) ;
 }
 fd2 = open("letter.txt", O_RDWR);
 read(fdl, buf1, counter1);
 read(fd2, buf2, counter2);
 buf1[counter1] = '\0';
 buf2[counter2] = '\0';
 printf("%s %s\n", buf1, buf2);
 if (cpid) {
 waitpid(cpid, 0, 0);
 } else {
 E ;
 }
}
}

```

其中, number.txt 文件内容为:

123456789012345678901234567890

其中, letter.txt 文件内容为:

abcdefghijklmnopqrstuvwxyz

1. 当 N=2, A/B 为 SIGUSR1 或 SIGUSR2, E 为空时, 共有\_\_\_\_\_行输出
  - a) A 为 SIGUSR1, B 为 SIGUSR1 时, 最后一行输出为\_\_\_\_\_
  - b) A 为 SIGUSR1, B 为 SIGUSR2 时, 最后一行输出为\_\_\_\_\_
  - c) A 为 SIGUSR2, B 为 SIGUSR1 时, 最后一行输出为\_\_\_\_\_
  - d) A 为 SIGUSR2, B 为 SIGUSR2 时, 最后一行输出为\_\_\_\_\_
2. 当 N=3 时, A/B 为 SIGUSR1 或 SIGUSR2, E 为 exit(0) 时, 共有\_\_\_\_\_行输出
  - a) A 为 SIGUSR1, B 为 SIGUSR1 时, 最后一行输出为\_\_\_\_\_
  - b) A 为 SIGUSR1, B 为 SIGUSR2 时, 最后一行输出为\_\_\_\_\_
  - c) A 为 SIGUSR2, B 为 SIGUSR1 时, 最后一行输出为\_\_\_\_\_
  - d) A 为 SIGUSR2, B 为 SIGUSR2 时, 最后一行输出为\_\_\_\_\_



|    |
|----|
| 得分 |
|    |

第六题（12 分）

通常处理器中的 MMU 通过页表实现虚拟内存地址到物理内存地址的转换，而 TLB 被用于提升地址转换的效率。然而 TLB 必须和页表之间保持一致，才能保证 MMU 正确按页表转换虚拟地址。当修改页表内容时，需要通过 `invlpg` 指令将对应的 TLB 表项置为失效，以确保 TLB 与页表一致。

指令 `invlpg m` 把包含虚拟地址 `m` 的页面在 TLB 中的表项置为失效。

假设当前状态下，TLB 与页表是一致的。部分 TLB 表项如下（假设 TLB 是全相联的）：

| 有效位 | TLB 标记    | 页面号    |
|-----|-----------|--------|
| 1   | 0x8040201 | 0x4801 |
| 1   | 0x8040233 | 0x4812 |
| 0   | 0x8040382 | 0x9C33 |
| 1   | 0x8046740 | 0x4801 |
| 0   | 0x8046621 | 0x8845 |
| 1   | 0x80467CD | 0x6734 |

与上述 TLB 表项相关的两个页表页的地址及其中的部分页表项如下：

| 索引号   | 页面号           | 存在位    | 索引号   | 页面号           | 存在位    |
|-------|---------------|--------|-------|---------------|--------|
|       | Bits: 51 - 12 | Bit: 0 |       | Bits: 51 - 12 | Bit: 0 |
| 0x1   | 0x4801        | 1      | 0x10  | 0x2312        | 1      |
| 0x33  | 0x4812        | 1      | 0x21  | 0x8845        | 1      |
| 0x54  | 0x8745        | 1      | 0x73  | 0x4521        | 1      |
| 0x180 | 0x3212        | 1      | 0x140 | 0x4801        | 1      |
| 0x182 | 0x9C33        | 1      | 0x182 | 0x8ACD        | 1      |
| 0x1A1 | 0x9078        | 1      | 0x1CD | 0x6734        | 1      |

基地址为 0x4801000 的页表页      基地址为 0x4812000 的页表页

设处理器采用的是 64 位虚拟地址和 64 位物理地址，页面大小为 4KB，页表为 4 级。

1. 分析下面的指令序列：

```

1. movq $0x8040233000, %rbx
2. movq 0xA00(%rbx), %rcx
3. addq 0x11000, %rcx
4. movq %rcx, 0x108(rbx)
5. movq 0x8046621108, %rax

```

- 执行完第 2 行指令后，载入到%rcx 中的物理页面号为：\_\_\_\_\_；
- 执行完第 3 行指令后，%rcx 中的物理页面号为：\_\_\_\_\_；
- 执行完第 5 行指令后，%rcx 和%rax 中的内容之间有何关系？\_\_\_\_\_
- 在执行上述 5 行指令过程中，会发生\_\_\_\_\_次 TLB miss；
- 在执行上述 5 行指令过程中，会发生\_\_\_\_\_次 page fault；

2. 接着上面的的 5 行指令接着执行下面的指令：

```

6. movq 0x8040382C10, %r10
7. movq %rax, 0x8046740C10
8. movq 0x8040382C10, %r11
9. invlpg 0x8040382C10
10. movq 0x8040382C10, %r12

```

- 全部 10 条指令执行后，%r10 和%r11 中的内容之间有何关系？\_\_\_\_\_
- 全部 10 条指令执行后，%r11 和%r12 中的内容之间有何关系？\_\_\_\_\_
- 在执行上述 10 行指令过程中，共发生了\_\_\_\_\_次 TLB miss.

3. 请写出执行完上述 10 行指令后，TLB 和页表页中有变化的表项中的内容。

| 有效位 | TLB 标记    | 页面号 |
|-----|-----------|-----|
|     | 0x8040201 |     |
|     | 0x8040233 |     |
|     | 0x8040382 |     |
|     | 0x8046740 |     |
|     | 0x8046621 |     |
|     | 0x80467CD |     |

---

| 索引号   | 页面号           | 存在位    |
|-------|---------------|--------|
|       | Bits: 51 - 12 | Bit: 0 |
| 0x1   |               |        |
| 0x33  |               |        |
| 0x54  |               |        |
| 0x180 |               |        |
| 0x182 |               |        |
| 0x1A1 |               |        |

基地址为 0x4801000 的页表页

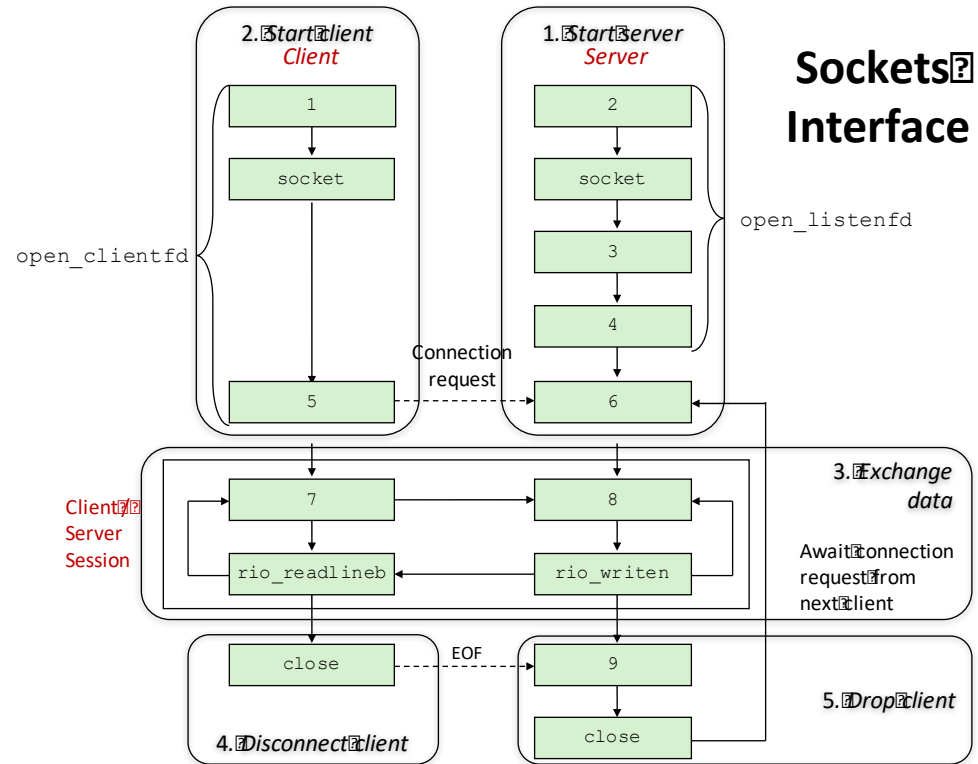
| 索引号   | 页面号           | 存在位    |
|-------|---------------|--------|
|       | Bits: 51 - 12 | Bit: 0 |
| 0x10  |               |        |
| 0x21  |               |        |
| 0x73  |               |        |
| 0x140 |               |        |
| 0x182 |               |        |
| 0x1CD |               |        |

基地址为 0x4812000 的页表页

|    |
|----|
| 得分 |
|    |

第七题（12 分）

1. 请根据 Echo server/client 应用的标准程序填空。



- |          |          |          |
|----------|----------|----------|
| 1. _____ | 2. _____ | 3. _____ |
| 4. _____ | 5. _____ | 6. _____ |
| 7. _____ | 8. _____ | 9. _____ |

2. 不定项选择题（全对得分，有错得 0 分）

- A. TCP 是一种实现在用户态的端到端可靠数据传输协议
  - B. 每一个连接在网络上的主机都会被分配一个固定的 IP 地址
  - C. IP 包在网际互联时通过其帧头（LAN frame header）和包头（Internet packet header）信息共同决定路由
  - D. HTTP1.0 支持在同一个连接里进行多个 transaction
  - E. Web 代理作为网络缓存功能的时候对用户是显式存在的
  - F. 10.0.0.0/12 和 172.16.0.0/16 和 192.168.0.0/20 属于私有地址字段
- 上述正确的有：\_\_\_\_\_

---

|    |
|----|
| 得分 |
|    |

第八题（12 分）

1. 基于进程的并发 Echo 服务器代码（简称 P 代码）和基于线程的并发 Echo 服务器代码（简称 T 代码）如下所示：

**P 代码：**

```
#include "csapp.h"
void echo(int connfd);

void sigchld_handler(int sig)
{
 if (waitpid(-1, 0, WNOHANG) > 0)
 ;
 return; }

int main(int argc, char **argv)
{ int listenfd, connfd;
 socklen_t clientlen;
 struct sockaddr_storage clientaddr;

 listenfd = Open_listenfd(argv[1]);
 while (1) {
 clientlen = sizeof(struct sockaddr_storage);
 connfd = Accept(listenfd, (SA*)&clientaddr, &clientlen);
 if (Fork() == 0) {
 echo(connfd);
 Close(connfd);
 exit(0);
 }
 Close(connfd);
 }
}
```

---

**T代码:**

```
#include "csapp.h"
void echo(int connfd);
void *thread(void *vargp);

int main(int argc, char **argv)
{ int listenfd, connfd1;
 socklen_t clientlen;
 struct sockaddr_storage clientaddr;
 pthread_t tid;

 listenfd = Open_listenfd(argv[1]);
 while (1) {
 clientlen = sizeof(struct sockaddr_storage);
 connfd1 = Accept(listenfd, (SA*)&clientaddr, &clientlen);
 Pthread_create(&tid, NULL, thread, connfd1);
 }
}

void *thread(void *vargp)
{
 int connfd2 = *((int *)vargp);
 Pthread_detach(pthread_self());
 echo(connfd2);
 Close(connfd2);
 return NULL;
}
```

问：以上 P 代码和 T 代码中存在几处错误，请指出出错的位置（请按照①、②等符号标注位置），解释出错的原因并给出修正错误的解决方案。

- 
2. 假设有 3 个线程，代码如下所示。已声明了三个信号量  $s_1$ 、 $s_2$  和  $s_3$ ，其初值分别为 1、0、0。请将信号量  $s_1$ 、 $s_2$ 、 $s_3$  对应的  $P()$  和  $V()$  操作填写在三个线程中的标号处（每空至多填写 1 个操作），要求三个线程并发执行的结果只能是  $x = 10$ 。

```
x = 1;

void thread1()
{
 ① _____
 x = x * 5;
 ② _____
}

void thread2()
{
 ③ _____
 x = x + 3;
 ④ _____
}

void thread3()
{
```

# 北京大学信息科学技术学院考试试卷

考试科目：计算机系统导论 姓名：\_\_\_\_\_ 学号：\_\_\_\_\_

考试时间：2019 年 12 月 30 日 小班号：\_\_ 小班教师：\_\_

| 题号  | 一 | 二 | 三 | 四 | 五 | 六 | 七 | 总分 |
|-----|---|---|---|---|---|---|---|----|
| 分数  |   |   |   |   |   |   |   |    |
| 阅卷人 |   |   |   |   |   |   |   |    |

## 北京大学考场纪律

1、考生进入考场后，按照监考老师安排隔位就座，将学生证放在桌面上。无学生证者不能参加考试；迟到超过 15 分钟不得入场。在考试开始 30 分钟后方可交卷出场。

2、除必要的文具和主考教师允许的工具书、参考书、计算器以外，其它所有物品（包括空白纸张、手机、或有存储、编程、查询功能的电子用品等）不得带入座位，已经带入考场的必须放在监考人员指定的位置。

3、考试使用的试题、答卷、草稿纸由监考人员统一发放，考试结束时收回，一律不准带出考场。若有试题印制问题请向监考教师提出，不得向其他考生询问。提前答完试卷，应举手示意请监考人员收卷后方可离开；交卷后不得在考场内逗留或在附近高声交谈。未交卷擅自离开考场，不得重新进入考场答卷。考试结束时间到，考生立即停止答卷，在座位上等待监考人员收卷清点后，方可离场。

4、考生要严格遵守考场规则，在规定时间内独立完成答卷。不准交头接耳，不准偷看、夹带、抄袭或者有意让他人抄袭答题内容，不准接传答案或者试卷等。凡有违纪作弊者，一经发现，当场取消其考试资格，并根据《北京大学本科考试工作与学术规范条例》及相关规定严肃处理。

5、考生须确认自己填写的个人信息真实、准确，并承担信息填写错误带来的一切责任与后果。

学校倡议所有考生以北京大学学生的荣誉与诚信答卷，共同维护北京大学的学术声誉。

以下为试题和答题纸，共 16 页。



|    |
|----|
| 得分 |
|    |

注：所有回答请填写在答题纸上,并注明大题和小题的题号。

第一题 单项选择题（每小题 2 分，共 40 分）

注：请在答题纸上按照下表样式画出表格并填写答案。

|    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|
| 题号 | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
| 回答 |    |    |    |    |    |    |    |    |    |    |
| 题号 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| 回答 |    |    |    |    |    |    |    |    |    |    |

- 已知浮点运算表达式： $t = a + b$ ； $x = t + c$ ； $y = d + t$ ；下列表达式中， $x$ ， $y$  不能都得到相同的计算结果的是：
  - $x = a + b + c$ ； $y = d + a + b$ ；
  - $x = a + b + c$ ； $y = d + (a + b)$ ；
  - $x = c + (a + b)$ ； $y = a + b + d$ ；
  - $x = (a + b) + c$ ； $y = d + (a + b)$ ；
- 在本课程的 PIPE 流水线中，下列情况会出现数据冒险的是：
  - 当前指令会改变下一条指令的目的操作数
  - 当前指令会改变下一条指令的源操作数
  - 下一条指令会改变当前指令的目的操作数
  - 下一条指令会改变当前指令的源操作数
- ```
int x = -2019;
int a = (x / 8) * 8;
int b = ((x + 7) >> 3) << 3;
int c = -(((~x)+1) >> 3) << 3;
```

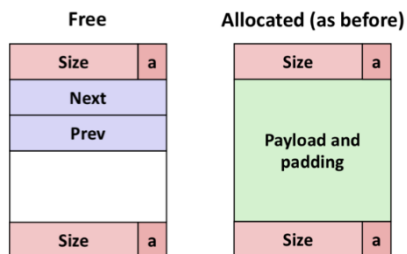
 a ， b ， c 大小关系是什么？
 - $a=b$ ， $b=c$
 - $a=b$ ， $b>c$
 - $a<b$ $b<c$
 - $a<b$ ， $b=c$
- 下述关于 RISC 和 CISC 的讨论，哪个是错误的
 - RISC 指令集包含的指令数量通常比 CISC 的少
 - RISC 的寻址方式通常比 CISC 的寻址方式少
 - RISC 的指令长度通常短于 CISC 的指令长度
 - 手机处理器通常采用 RISC，而 PC 采用 CISC
- 下述关于各类存储介质的讨论，哪个是错误的？

- A. 磁盘访问的延时和数据所在位置有关，连续数据访问的性能高于随机数据访问
- B. DRAM 是动态随机存储器，连续数据访问和随机数据访问延时是一样的
- C. 对 SSD 设备进行数据擦除操作的延时，远高于进行数据读取的延时
- D. SRAM 是静态随机存储器，它的存储密度比 DRAM、SSD 都要低
6. 在链接时，对于下列哪些符号需要进行重定位？
- (1) 不同 C 语言源文件中定义的函数
 - (2) 同一 C 语言源文件中定义的全局变量
 - (3) 同一函数中定义时不带 static 的变量
 - (4) 同一函数中定义时带有 static 的变量
- A. (1) (3) B. (2) (4) C. (1) (2) (4) D. (1) (2) (3) (4)
7. 关于 x86-64 系统中的异常，下面那个判断是正确的：
- A. 除法错误是异步异常，Unix 会终止程序；
- B. 键盘输入中断是异步异常，异常服务后会返回当前指令执行；
- C. 缺页是同步异常，异常服务后会返回当前指令执行；
- D. 时间片到时中断是同步异常，异常服务后会返回下一条指令执行；
8. 考虑 4 个具有如下开始和结束时间并运行在不同处理器上的进程，
- | 进程 | 开始时间 | 结束时间 | 运行处理器 |
|----|------|------|-------|
| A | 5 | 7 | P0 |
| B | 2 | 4 | P1 |
| C | 3 | 6 | P0 |
| D | 1 | 8 | P1 |
- 下面那个判断是正确的。
- A. 进程 A 和 B 不是并发运行的，但是并行运行的；
- B. 进程 A 和 C 是并发运行的，并且是并行运行的；
- C. 进程 B 和 D 是并发运行的，但不是并行运行的；
- D. 进程 A 和 D 是并发运行的，并且是并行运行的；
9. 进程管理相关函数的调用和返回行为，下列那些函数都是可能返回多于一次的？
- A. longjmp 和 fork
- B. execve 和 longjmp
- C. fork 和 setjmp
- D. setjmp 和 execve
10. 根据本课程介绍的 Intel x86-64 存储系统，填写表格中某一个进程从用户态切换至内核态时，和进程切换时对 TLB 和 cache 是否必须刷新。
- A. ①不必刷新 ②不必刷新 ③刷新 ④不必刷新
- B. ①不必刷新 ②不必刷新 ③不必刷新 ④不必刷新
- C. ①刷新 ②不必刷新 ③刷新 ④刷新
- D. ①刷新 ②不必刷新 ③不必刷新 ④刷新

11. 已知某系统页面长 2KB，页表项 8 字节，采用多层分页策略映射 48 位虚拟地址空间。若限定最高层页表占 1 页，则它可以采用多少层的分页策略？
- A. 3 层
B. 4 层
C. 5 层
D. 6 层

12. 现在有一个用户程序执行了如下调用序列

```
void *p1 = malloc(16);
void *p2 = malloc(32);
void *p3 = malloc(32);
void *p4 = malloc(48);
free(p2);
void *p5 = malloc(4);
free(p3);
void *p6 = malloc(56);
void *p7 = malloc(10);
```



内存分配器内部使用**显式空闲链表**实现，按照地址从低到高顺序来维护空闲链表节点顺序。分配块格式参照上图。每个分配块 16 字节对齐，头部和脚部的大小都是 4 字节。分配算法采用**首次适配**算法，将适配到的空闲块的第一部分作为分配块，剩余部分变成新的空闲块，并采用**立即合并**策略。假设初始空闲块的大小是 1KB。那么以上调用序列完成后，分配器管理的这 1KB 内存区域中，**内部碎片**的总大小是____，链表里**第一个**空闲块的大小是____。

- A. 58Byte, 848Byte
B. 26Byte, 32Byte
C. 74Byte, 48Byte
D. 74Byte, 16Byte
13. 在一个具有 TLB 和高速缓存的系统中，假设地址翻译使用四级页表来进行，且不发生缺页异常，那么在 CPU 访问某个虚拟内存地址的过程中，至少会访问（ ）次物理内存，至多会访问（ ）次物理内存。
- A. 0, 4
B. 0, 5
C. 1, 4
D. 1, 5
14. 采用迭代的 Echo 服务器服务多个客户端，假设 client1 先完成连接，并且没有结束，那么 client2 向服务器发出连接请求后，会被阻塞在下列哪个函数上？
- A. connect

- B. read
 - C. write
 - D. socket
15. 下列关于全球 IP 因特网的各种概念叙述中，哪一个是错误的？
- A. 一个套接字是连接的一个端点，对应了一个套接字地址
 - B. IPv4 的因特网中，一个 IP 地址是一个 32 位无符号整数
 - C. 因特网主机的字节顺序与 TCP/IP 定义的网络字节顺序是一致的
 - D. IP 协议提供了一种不可靠的从一台主机向其他主机的递送机制
16. 下列关于 Web 服务器向客户端提供动态内容（遵从 CGI 标准）的叙述中，哪一个是错误的？
- A. 客户端提出的 GET 请求的参数在 URI 中传递，并用“&”字符分隔文件名和多个参数
 - B. 接到客户端的请求，Web 服务器创建一个子进程执行相应的 CGI 程序
 - C. 通过设置环境变量，Web 服务器将 URI 中传递的参数传递给子进程
 - D. CGI 程序将动态内容重定向到与客户端关联的已连接描述符 `connfd`
17. 并发程序执行时会遇到下列各种情况：
- ① 程序执行结果的正确性依赖于一个线程要在另一个线程到达 y 点之前到达其控制流的 x 点
 - ② 程序员使用 P 和 V 操作顺序不当
 - ③ 外部事件和/或系统调度决策阻止了任务进度
 - ④ 不合理的资源分配策略导致程序难以向前运行
 - ⑤ 程序的执行结果取决于系统的调度决策
 - ⑥ 在信号处理函数中调用 `printf()`
- 请问下列哪些情况表示可能产生死锁现象？
- A. ①、②和⑤
 - B. ②、③和④
 - C. ③、④和⑥
 - D. ②、④和⑥
18. 称一个函数为线程安全的（thread-safe），当且仅当该函数被多个并发线程反复调用时会一直产生正确的结果。下列哪些函数不是线程安全的？
- A. 利用 P、V 操作保护了共享变量的函数
 - B. 保持跨越多个调用的状态的函数
 - C. 包括了调用者传递存放结果变量的地址的函数
 - D. 可重入函数
19. 在以下基于线程的并发 Echo 服务器代码中，哪一个代码片段会潜在引发不正确的程序执行结果？
- A.

```
int main(int argc, char **argv)
{
    /*假设有配对的*/
    int listenfd, connfd;
```

- ```

 socklen_t clientlen;
 struct sockaddr_storage clientaddr;
 pthread_t tid;
B. listenfd = Open_listenfd(argv[1]);
C. while (1) {
 connfd = Accept(listenfd, (SA *) &clientaddr,
 &clientlen);
 Pthread_create(&tid, NULL, thread, &connfd);
 }
D. void *thread(void *vargp)
 {
 int connfd = *((int *)vargp);
 Pthread_detach(pthread_self());
 echo(connfd);
 Close(connfd);
 return NULL;
 }

```
20. 下列关于信号量及 P、V 操作的叙述中，哪一个是不正确的？
- A. 信号量是一个非负整数，对信号量只能执行 P 操作和 V 操作
  - B. 当信号量的值为 0 时，调用 P 操作的线程被挂起
  - C. P(s) 的实现代码中语句：while (s == 0) wait(); s--; 应该由内核保证其执行的不可分割性
  - D. 在保护临界区的解决方案中使用 sem\_wait() 和 sem\_post() 比使用 pthread\_mutex\_lock() 和 pthread\_mutex\_unlock() 性能好

|    |
|----|
| 得分 |
|    |

## 第二题（10 分）

现有如下汇编代码段，相关寄存器的初始值也在右侧表格中给出：

.L2:

```

movq (%rdi), %rbx
addq %rbx, %rax
addq $8, %rdi
addq $1, %rcx
cmpq $8, %rcx
jl .L2

```

|      |        |
|------|--------|
| %rdi | 0x8008 |
| %rax | 0      |
| %rbx | 0      |
| %rcx | 0      |

- (1) 请在下面 C 代码中，补充缺失的部分。

```

for(i=0; i<_____ ; i++)
{
 sum=sum+a[i];
}

```

- (2) 现有一个类似课上 SEQ 的处理器来运行上述汇编代码，在理想情况下，其每个周期可以处理完一条汇编代码，请计算：理想情况下，执行完上述代码段（即结束循环）一共需要多少周期？
- (3) 现在考虑数据缓存（data cache）对该处理器的影响。  
该处理器配置了一个简单的直接映射（direct-mapped, E=1）数据缓存，总容量 32KB，每个缓存块的大小是 64Byte。对于 movq 指令，如果被访问的数据在该缓存中，则 1 个周期可以处理完成。否则，则需要 2 个周期才能完成。在运行上述汇编代码之前，数据缓存初始状态为空，其他设置同上一问。请再次计算：完成上述汇编代码段一共需要多少周期？
- (4) 为了提升处理器的性能，增加了一个加法器。因此，在**没有数据依赖**、并且**不改变指令顺序**的情况下，新的处理器可以同时处理两个 addq 指令、或者一个 addq 加一个 cmpq 指令。注意：movq 和跳转指令仍旧需要单独执行，其他设置同上一问。请再次计算：完成上述汇编代码段一共需要多少周期？
- (5) 为了进一步减少处理时间，程序员将代码进行循环展开，新的代码如下：

.L2:

```

movq (%rdi), %rbx
movq 8 (%rdi), %rdx
addq %rbx, %rax

```

```
addq %rdx, %rax
addq $16, %rdi
addq $2, %rcx
cmpq $8, %rcx
jl .L2
```

假设在不影响程序正确性的情况下，可以调整上述指令的顺序，请问最短需要多少个周期可以完成该代码段（注：其他设置和上一问相同）。

|    |
|----|
| 得分 |
|    |

### 第三题 (10 分)

本题基于下列 m.c 及 foo.c 文件所编译生成的 m.o 和 foo.o，编译和运行在 x86-64/Linux 下完成，编译过程未加优化选项。

|                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                               |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>//m.c void myswap(); char buf[2] = {1, 2}; void *bufp0; void *bufp1; char temp; int main(){     .....//略去题目无关代码     myswap(temp);     .....//略去题目无关代码     return 0; }</pre> | <pre>//foo.c extern int buf[]; char *bufp0; .....//略去题目无关代码 char *bufp1 = &amp;buf[1]; void myswap(char temp){     static int count = 0;     .....//略去题目无关代码     bufp0 = &amp;buf[0];     temp = *bufp0;     .....//略去题目无关代码     *bufp0 = *bufp1;     *bufp1 = temp;     .....//略去题目无关代码     count++; }</pre> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

(1) 对于每个 foo.o 中定义和引用的符号，请用“是”或“否”指出它是否在模块 foo.o 的 .symtab 节中有符号表条目。如果存在条目，则请指出在模块 foo.o 中的符号类型（局部、全局或外部）、它在 foo.o 模块中所处的节（.data、.text、.bss 或 COMMON）以及其强弱信息（强、弱、非强非弱）；如果不存在条目，则请将该行后继空白处标记为“/”（请将下表画到答题纸上）。

| 符号    | .symtab 条目? | 符号类型 | 节     | 强弱 |
|-------|-------------|------|-------|----|
| bufp1 | 是           | 全局   | .data | 强  |
| buf   |             |      |       |    |
| bufp0 |             |      |       |    |
| temp  |             |      |       |    |
| count |             |      |       |    |

(2) 我们使用 REF(myswap.m) -> DEF(myswap.foo) 表示链接器将把模块 m 中对符号 myswap 的任意引用与模块 foo 中对 swap 的定义关联起来。对于下面的示例，用这种方法来说明链接器如何解析每个模块对多重定义符号的引用。如果有链接时错误，请标记“错误”。如果链接器从定义中任意选择一个，请标记“未知”。

a) REF(bufp0.m) ->DEF(\_\_\_\_\_)

b) REF(bufp1.m) ->DEF(\_\_\_\_\_)

(3) 下图左边给出了 m.o 和 foo.o 的反汇编文件，右边给出了采用某个配置链接成可执行程序后再反汇编出来的文件。根据答题需要，其中的信息略有删减。



|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> 0000000000.....&lt;main&gt;: 55          push    %rbp 48 89 e5     mov     %rsp,%rbp 48 83 ec 10   sub     \$0x10,%rsp ..... b8 00 00 00 00    mov     \$0x0,%eax e8 00 00 00 00    callq   la &lt;main+0x1a&gt; ① ..... c9          leaveq c3          req </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | <pre> 0000000000000f28 &lt;main&gt;: f28: 55          push    %rbp f29: 48 89 e5     mov     %rsp,%rbp f2c: 48 83 ec 10   sub     \$0x10,%rsp .....略去部分和答题无关的信息..... f38: b8 00 00 00 00    mov     \$0x0,%eax f3d: e8          ①      callq   1000 &lt;myswap&gt; .....略去部分和答题无关的信息..... ffd: c9          leaveq ffe: c3          retq .....略去部分和答题无关的信息..... 00000000000001000 &lt;myswap&gt;: 1000: 55          push    %rbp 1001: 48 89 e5     mov     %rsp,%rbp .....略去部分和答题无关的信息..... 100d: 48 c7 05 ?? ?? ?? ?? ?? movq   ②,0x????(%rip) 1018:          48          8b 05 ?? ?? ?? ?? mov     0x????(%rip),%rax 101f: 0f b6 00     movzbl (%rax),%eax 1018: 88 45 fc     mov     %eax,-0x4(%rbp) .....略去部分和答题无关的信息..... 1035: 48 8b 05 ?? ?? ?? ?? mov     0x????(%rip),%rax 103c: 48 8b 15 ?? ?? ?? ?? mov     ⑥(%rip),%rdx 1043: 0f b6 12     movzbl (%rdx),%edx 1046: 88 10       mov     %dl,(%rax) 1048: 48 8b 05 ?? ?? ?? ?? mov     0x????(%rip),%rax 104f: 0f b6 55 fc     movzbl -0x4(%rbp),%edx 1053: 88 10       mov     %dl,(%rax) .....略去部分和答题无关的信息..... 1057: 8b 05 ?? ?? ?? ?? mov     0x????(%rip),%eax 105d: 83 c0 01     add     \$0x1,%eax 1060: 89 05 ?? ?? ?? ?? mov     %eax,⑨(%rip) 1066: 90          nop 1067: 5d          pop     %rbp 1068: c3          retq .....略去部分和答题无关的信息..... 000000000000a000 &lt;buf&gt;: .....略去部分和答题无关的信息..... 000000000000a050 &lt;bufp1&gt;: .....略去部分和答题无关的信息..... 000000000000cb24 &lt;count.1837&gt;: .....略去部分和答题无关的信息..... 000000000000cb28 &lt;bufp0&gt;: .....略去部分和答题无关的信息..... </pre> |
| <pre> 0000000000.....&lt;myswap&gt;: 55          push    %rbp 48 89 e5     mov     %rsp,%rbp ..... 48 c7 05 00 00 00 00 00 00 00 00 movq   \$0x0,0x0(%rip) ③② 48 8b 05 00 00 00 00 00 mov     0x0(%rip),%rax ④ 0f b6 00     movzbl (%rax),%eax 88 45 fc     mov     %al,-0x4(%rbp) ..... 48 8b 05 00 00 00 00 00 mov     0x0(%rip),%rax ⑤ 48 8b 15 00 00 00 00 00 mov     0x0(%rip),%rdx ⑥ 0f b6 12     movzbl (%rdx),%edx 88 10       mov     %dl,(%rax) 48 8b 05 00 00 00 00 00 mov     0x0(%rip),%rax ⑦ 0f b6 55 fc     movzbl -0x4(%rbp),%edx 88 10       mov     %dl,(%rax) ..... 8b 05 00 00 00 00 00 00 mov     0x0(%rip),%eax ⑧ 83 c0 01     add     \$0x1,%eax 89 05 00 00 00 00 00 00 mov     %eax,0x0(%rip) ⑨ 90          nop 5d          pop     %rbp c3          retq </pre> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

在上图对所涉及到的重定位条目进行用数字①至⑨进行了标记，请根据下表中所提供的重定位条目信息，计算相应的重定位引用值并填写下表。（请将下表的第1列和第3列画到答题纸上）

| 编号 | 重定位条目信息                                                                        | 应填入的重定位引用值 |
|----|--------------------------------------------------------------------------------|------------|
| ①  | r.offset = 0x16<br>r.type = R_X86_64_PC32<br>r.symbol = 本题不提供<br>r.addend = -4 |            |
| ②  | r.offset = 0x14<br>r.type = R_X86_64_32<br>r.symbol = buf<br>r.addend = +0     |            |
| ⑥  | r.offset = 0x3f<br>r.type = R_X86_64_PC32<br>r.symbol = bufp1<br>r.addend = -4 |            |
| ⑨  | r.offset = 0x62<br>r.type = R_X86_64_PC32<br>r.symbol = 本题不提供<br>r.addend = -4 |            |

|    |
|----|
| 得分 |
|    |

#### 第四题（10 分）

分析以下C程序，其中f1.txt和f2.txt为已有用户有读写的文件，初始文件内容为空。

```

1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <sys/types.h>
4. #include <sys/stat.h>
5. #include <fcntl.h>
6. #include <unistd.h>
7. #include <sys/types.h>
8. #include <sys/wait.h>
9.
10. int main()
11. {
12. int fd1,fd2,fd3,fd4;
13. int pid;
14. int c=1;
15. fd1=open("./f1.txt",O_WRONLY,0);
16. fd2=open("./f1.txt",O_WRONLY,0);
17.
18. printf("fd1=%d,fd2=%d;\n",fd1,fd2);
19.
20. write(fd1,"EECSPKU",7);
21. write(fd2,"2019",4);
22.
23. close(fd2);
24.
25. fd3=open("./f2.txt",O_WRONLY,0);
26. fd4=dup(fd3);
27.
28. printf("fd3=%d,fd4=%d;\n",fd3,fd4);
29.
30. pid=fork();
31. if((pid==0)) {
32. c--;
33. write(fd3,"PKU",3);
34. write(fd4,"ICS",3);
35. printf("c= %d\n",c);
36. }
37. else {
38. waitpid(-1,NULL,0);
39. c++;
40. write(fd3,"2019",4);

```

```

41. close(fd3);
42. close(fd4);
43. printf("c= %d\n",c);
44. }
45.
46. if(c)
47. write(fd1,"CS",2);
48. c++;
49. close(fd1);
50. printf("c=%d\n",c);
51. }

```

当程序正确运行后，填写输出结果：

- (1) 程序第 18 行：fd1= ①，fd2= ②；
- (2) 程序第 28 行：fd3= ①，fd4= ②；
- (3) 程序第 35、43、50 行输出 c 的值依次分别为：\_\_\_\_\_；
- (4) 文件 f1.txt 中的内容为：\_\_\_\_\_；
- (5) 文件 f2.txt 中的内容为：\_\_\_\_\_。

|    |
|----|
| 得分 |
|    |

### 第五题（10 分）

某 32 位机器有 24 位地址空间，采用二级页表，一级页表中有 64 个 PTE，二级页表中有 1024 个 PTE，一级页表 4KB 对齐，PTE 最高一位是有效位，没有 TLB 和 Cache。惠楚思程序员在该机器上执行了如下代码。

```
1. int* a=calloc(10000, sizeof(int));
2. int* b = a+5000;
3. fork();
4. *b=0x80C3F110;
```

（1） 假设编译后 b 的值保存在寄存器中，请问该机器页面大小为①\_\_字节，VPN1 长度为②\_\_\_\_，VPN2 长度为③\_\_\_\_。

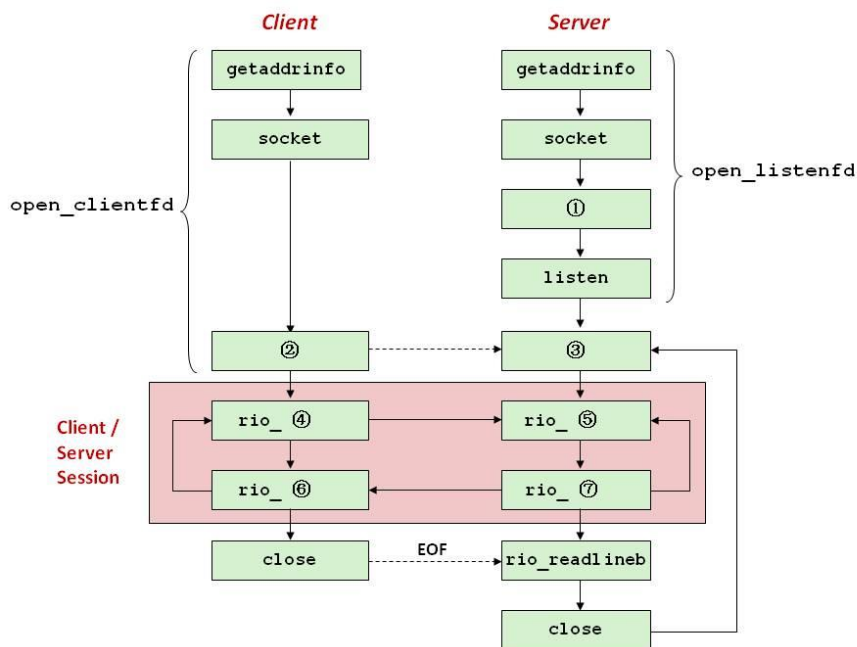
（2） 由于该执行结果不符合预期，惠楚思需要对程序执行过程进行还原。程序执行过程中硬件记录了部分内存访问记录，但访问记录并不完整。目前已知在父进程执行第四行的时候，进行了若干次物理内存的读写操作，其中第一次读内存操作从地址 0x67F0E8 读出 0x80AA32C4，另外有两次将 0x80C3F110 写入内存的操作，写入的地址分别是 0xC3F50D 和 0xAA3AD0，但写入顺序并不清楚。程序结束后变量 b 中保存的值是①\_\_\_\_\_。在解析 “\*b” 的过程中，一级页表的起始地址为②\_\_\_\_\_；二级页表的起始地址为③\_\_\_\_\_；物理页面的起始地址为④\_\_\_\_\_。（地址均用 16 进制表示）

|    |
|----|
| 得分 |
|    |

### 第六题（10 分）

下图是一个基于 echo 服务器的 client-server 框架

(1) 请给图中的编号填写相应的函数名。



(2) 请补全下面 server 端 open\_listenfd 函数中缺失的操作（Line 21, Line 26 和 Line 34）

```

Line 1: int open_listenfd(char *port)
Line 2: {
Line 3: struct addrinfo hints, *listp, *p;
Line 4: int listenfd, optval=1;
Line 5: /* Get a list of potential server addresses */
Line 6: memset(&hints, 0, sizeof(struct addrinfo));
Line 7: hints.ai_socktype = SOCK_STREAM;
Line 8: hints.ai_flags = AI_PASSIVE | AI_ADDRCONFIG;
Line 9: hints.ai_flags |= AI_NUMERICSERV;
Line 10: Getaddrinfo(NULL, port, &hints, &listp);

```

```

Line 11:
Line 12: for (p = listp; p; p = p->ai_next) {
Line 13: /* Create a socket descriptor */
Line 14: if ((listenfd = socket(p->ai_family, p->ai_socktype,
Line 15: p->ai_protocol)) < 0)
Line 16: continue; /* Socket failed, try the next */
Line 17: /* Eliminates "Address already in use" error from
bind */
Line 18: Setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR,
Line 19: (const void *)&optval , sizeof(int));
Line 20:
Line 21: if (___⑧___(listenfd, p->ai_addr, p->ai_addrlen) == 0)
Line 22: break; /* Success */
Line 23: Close(listenfd);
Line 24: }
Line 25: /* Clean up */
Line 26: Freeaddrinfo(___⑨___);
Line 27: if (!p) /* No address worked */
Line 28: return -1;
Line 29:
Line 30: if (listen(listenfd, LISTENQ) < 0) {
Line 31: Close(listenfd);
Line 32: return -1;
Line 33: }
Line 34: return ___⑩___;
Line 35: }

```

|    |
|----|
| 得分 |
|    |

### 第七题（10 分）

解决第一类读者-写者问题的代码如下：

```
int readcnt; /* Initially 0 */
sem_t mutex, w; /* Both initially 1 */
void reader(void)
{
(1) while (1) {
(2) P(&mutex);
(3) readcnt++;
(4) if (readcnt == 1) /* First in */
(5) P(&w);
(6) V(&mutex);
(7) /* Reading happens here, need 7 time unit */
(8) P(&mutex);
(9) readcnt--;
(10) if (readcnt == 0) /* Last out */
(11) V(&w);
(12) V(&mutex);
 }
}

void writer(void)
{
(13) while (1) {
(14) P(&w);
(15) /* Writing here, need 8 time unit */
(16) V(&w);
 }
}
```

假设有 5 个读者或写者到来，到达时刻和所需的时间如下：

| 线程 | 到达时刻 | 读写时间 |
|----|------|------|
| R1 | 0    | 7    |
| W1 | 1    | 8    |
| R2 | 2    | 7    |
| W2 | 4    | 8    |
| R3 | 5    | 7    |

注意，为简单起见，只考虑读写时间，忽略所有其他时间（包括代码中其他语句的执行时间、线程切换、调度等等），亦假设没有更多的读者或写者或其他线程。

(1) 在时刻 3 时，R1、W1 和 R2 所处的位置，请标出对应的代码行号。此刻，readcnt 和 w 的值分别是多少？R1: \_; W1: \_; R2: \_; readcnt: \_; w: \_。

(2) 在时刻 6 时，W2 和 R3 所处的位置，请标出对应的行号。此刻，readcnt 和 w 的值分别是多少？W2: \_; R3: \_; readcnt: \_; w: \_。

(3) 如果不使用 P(&mutex) 和 V(&mutex)，程序执行会不会出错？如果出错，会出什么错？

# 北京大学信息科学技术学院考试试卷

考试科目：计算机系统导论 姓名：\_\_\_\_\_ 学号：\_\_\_\_\_

考试时间：2021 年 1 月 11 日 小班号：\_\_\_\_\_

| 题号  | 一 | 二 | 三 | 四 | 五 | 六 | 总分 |
|-----|---|---|---|---|---|---|----|
| 分数  |   |   |   |   |   |   |    |
| 阅卷人 |   |   |   |   |   |   |    |

## 北京大学考场纪律

1、考生进入考场后，按照监考老师安排隔位就座，将学生证放在桌面上。无学生证者不能参加考试；迟到超过 15 分钟不得入场。在考试开始 30 分钟后方可交卷出场。

2、除必要的文具和主考教师允许的工具书、参考书、计算器以外，其它所有物品（包括空白纸张、手机、或有存储、编程、查询功能的电子用品等）不得带入座位，已经带入考场的必须放在监考人员指定的位置。

3、考试使用的试题、答卷、草稿纸由监考人员统一发放，考试结束时收回，一律不准带出考场。若有试题印制问题请向监考教师提出，不得向其他考生询问。提前答完试卷，应举手示意请监考人员收卷后方可离开；交卷后不得在考场内逗留或在附近高声交谈。未交卷擅自离开考场，不得重新进入考场答卷。考试结束时间到，考生立即停止答卷，在座位上等待监考人员收卷清点后，方可离场。

4、考生要严格遵守考场规则，在规定时间内独立完成答卷。不准交头接耳，不准偷看、夹带、抄袭或者有意让他人抄袭答题内容，不准接传答案或者试卷等。凡有违纪作弊者，一经发现，当场取消其考试资格，并根据《北京大学本科考试工作与学术规范条例》及相关规定严肃处理。

5、考生须确认自己填写的个人信息真实、准确，并承担信息填写错误带来的一切责任与后果。

学校倡议所有考生以北京大学学生的荣誉与诚信答卷，共同维护北京大学的学术声誉。

以下为试题，共 16 页。



第一题 单项选择题（每小题 2 分，共 50 分）

注：选择题答案请按照如下格式整理到答题纸上

|    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|
| 题号 | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
| 回答 |    |    |    |    |    |    |    |    |    |    |
| 题号 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| 回答 |    |    |    |    |    |    |    |    |    |    |
| 题号 | 21 | 22 | 23 | 24 | 25 |    |    |    |    |    |
| 回答 |    |    |    |    |    |    |    |    |    |    |

1. 下列关于 x86-64 汇编，说法**错误**的是：

- A. Bomblab 中 bomb 程序的代码段基址每次运行可能不同，因此其 switch 语句跳转表存储的不是目标代码的绝对地址，而是相对于某基址的偏移量。
- B. 本门课程主要讲述的是 ATT 格式的汇编，其 mov 等双操作数指令一般以第一个操作数作为源而第二个操作数作为目标，这种操作数顺序通常与 Intel 格式的汇编相反。
- C. 函数栈帧的长度在编译时被确定，因此 x86-64 中通用寄存器 %rbp 不会再作为帧指针使用，但 %rbp 依然是被调用者保存的寄存器。
- D. 多数同学 Archlab 书写 Y86-64 汇编时，在过程调用中修改了 %rbx 等部分寄存器而未事先保存，且未在过程调用结束时恢复，这种行为在 x86-64 汇编下是不可取的。

2. 假设内存空间为 64 字节，考虑两个高速缓存系统 A 与 B。它们均有  $s=E=b=2$ ，但高速缓存 A 要求组索引位在地址中间，而高速缓存 B 要求组索引位在地址的最高位，如图。设替换策略均采用 LRU，现用考虑如下访存序列(10 进制)：

1, 7, 8, 3, 4

|          |     |     |     |
|----------|-----|-----|-----|
| <b>A</b> | Tag | 组索引 | 块偏移 |
| <b>B</b> | 组索引 | Tag | 块偏移 |

则下列说法中**正确**的个数为：

- ①高速缓存 A 的容量为 32 字节，但 Tag 与 Valid 位的存在使得其实际占用的字节数是容量的 1.5 倍。
- ②若初始时缓存均为空，用该序列分别访存，其在高速缓存 A 中的 hit 次数多于 B。
- ③对于高速缓存 A，两个地址相邻的缓存块在相同的组中；而对于高速缓存 B，则在不同的组中。
- ④实际的缓存系统通常采用 A 的格式而不采用 B，是因为空间局部性较差时其能更好地利用缓存。

A. 0                      B. 1                      C. 2                      D. 3

3. 在int 类型值为32 位、用补码表示、采用算术右移，unsigned 类型值也为32 位的机器上，声明

```
int x, y;
```

```
unsigned ux = (unsigned)x;
```

```
unsigned uy = (unsigned)y;
```

则下列表达式中，当x 与y 取遍所有可能值时，可能为假的是

- A.  $(x < y) == (-x > -y)$
  - B.  $\sim x + \sim y + 1 == \sim(x + y)$
  - C.  $(ux - uy) == -(\text{unsigned})(y - x)$
  - D.  $((x >> 3) << 3) \leq x$
4. 下列哪种类型转换既可能导致溢出、又可能导致舍入？
- A. int 转 float    B. float 转 int    C. int 转 double    D. float 转 double
5. 下面对指令系统的描述中，错误的是：( )
- A. 通常 CISC 指令集中的指令数目较多，有些指令的执行周期很长；而 RISC 指令集中指令数目较少，指令的执行周期较短。
  - B. 通常 CISC 指令集中的指令长度不固定；RISC 指令集中的指令长度固定。
  - C. 通常 CISC 指令集支持多种寻址方式，RISC 指令集支持的寻址方式较少。
  - D. 通常 CISC 指令集处理器的寄存器数目较多，RISC 指令集处理器的寄存器数目较少。

6. Y86 指令 rmmovl 的 SEQ 实现如下图所示，其中①和②分别为：

| rmmovl rA, D(rB) |                                                                                                                                                                                      |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Fetch            | $\text{icode:ifun} \leftarrow M_1[\text{PC}]$<br>$\text{rA:rB} \leftarrow M_1[\text{PC}+1]$<br>$\text{valC} \leftarrow M_4[\text{PC}+2]$<br>$\text{valP} \leftarrow \textcircled{1}$ |
| Decode           | $\text{valA} \leftarrow R[\text{rA}]$<br>$\text{valB} \leftarrow R[\text{rB}]$                                                                                                       |
| Execute          | $\text{valE} \leftarrow \textcircled{2}$                                                                                                                                             |
| Memory           | $M_4[\text{valE}] \leftarrow \text{valA}$                                                                                                                                            |
| Write back       |                                                                                                                                                                                      |
| PC update        | $\text{PC} \leftarrow \text{valP}$                                                                                                                                                   |

- A .     $\text{PC} + 4, \quad \text{valB} + 4$                       B .             $\text{PC} + 4, \quad \text{valB} + \text{valC}$
  - C .     $\text{PC} + 6, \quad \text{valB} + 4$                       D .             $\text{PC} + 6, \quad \text{valB} + \text{valC}$
7. 下列关于链接的描述，正确的是：
- A . 打桩 (interpositioning) 机制需要能够访问程序的源代码。
  - B. 链接器生成的文件不再有 ABS、UNDEF 和 COMMON 伪节。
  - C. 对于 Glibc 和 GCC 生成的程序，其入口点位于 `__libc_start_main` 函数的第一条指令处。
  - D . 用户使用 GCC 进行共享库的编译必须使用 `-fpic` 选项指示生成位置无关代码。

8. 文件 f1.c 和 f2.c 的 C 源代码如下图所示：

|                                                                                                                                             |                                                    |
|---------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| <pre>//f1.c #include&lt;stdio.h&gt; extern float x; extern void f(); int main() {     f();     printf("%d\n",(int)x);     return 0; }</pre> | <pre>//f2.c int x = 0; void f() {     x++; }</pre> |
|---------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|

已知这两个文件在同一个目录下，在该目录下用“gcc -Og -o f f1.c f2.c”编译，然后用“./f”运行，这个过程中会出现的情况是：

- A. 编译错误
- B. 输出0
- C. 输出1
- D. 链接错误

9. C 语言中的代码如下：

```
fork() && fork();
printf("-");
fork() || fork();
printf("-");
```

这段代码一共输出（ ）个“-”字符。

- A. 12
- B. 18
- C. 20
- D. 32

10. 当一个网络数据包到达一台主机时，会触发以下哪种异常：

- A. 系统调用
- B. 信号
- C. 中断
- D. 缺页异常

11. 在键盘上输入 Ctrl+C 会导致内核发送一个（ ）信号到（ ）进程组中的每个进程

- A. SIGINT，前台
- B. SIGTSTP，前台
- C. SIGINT，后台
- D. SIGTSTP，后台

12. 以下关于缺页异常和系统调用的描述，不正确的是：
- A. 两个异常都是同步异常
  - B. 两个异常的触发都是由于执行某一条指令
  - C. 两个异常在正常退出时，都需要判断是否有非阻塞的待处理信号
  - D. 两个异常在正常退出后，都需要重新执行触发异常的指令
13. 对于 Linux 系统，下列说法错误的是：
- A. 在单核 CPU 上，所有看似并行的逻辑流实际上是并发的。
  - B. 用户模式不可访问/proc 文件系统中包含的内核数据结构的内容。
  - C. 在 `execve` 函数参数的 `argv` 数组加入 `> 1.txt`，不能自动实现 IO 重定向。
  - D. 即便在信号处理程序中调用 `printf` 前阻塞所有信号，也不一定安全。
14. 假设在 64 位系统下页大小被设置为 16KB，那么采用类似 Core i7 的地址翻译过程，四级页表最多可以索引多少位地址空间？
- A. 64
  - B. 58
  - C. 54
  - D. 52
15. 为了支持 16G 的虚拟地址空间，采用 3 级页表，页大小为 1KB，页表项大小依旧为 4 字节。现在映射总大小为 1MB 的虚拟地址，其分布未知但分布的最小单位限定为 1 字节，请问实现上述映射的页表结构占用的内存至少至多分别为多大？
- A. 6KB, 4113KB
  - B. 6KB, 65793KB
  - C. 6KB, 1052688KB
  - D. 3KB, 4113KB
16. 某 64 位系统，物理内存地址长度为 52 位，虚拟内存地址长度为 43 位，已知页的大小为 8KB，采用多级页表进行地址翻译，每级页表都占一页，则需要几级页表：
- A. 2 级    B. 3 级    C. 4 级    D. 5 级
17. 在页表条目中，以下哪个条目是由 MMU 在读和写时进行设置，而由软件负责清除：
- A. P 位，子页或子页表是否在物理内存中
  - B. G 位，是否为全局页（在任务切换时，不从 TLB 中驱逐出去）
  - C. CD 位，能/不能缓存子页或子页表
  - D. D 位，修改位，是否对子页进行了修改操作
18. 关于写时复制（copy-on-write）技术的说法，不正确的是：
- A. 写时复制既可以发生在父子进程之间，也可以发生在对等线程之间
  - B. 写时复制既需要硬件的异常机制，也需要操作系统软件的配合
  - C. 写时复制既可以用于普通文件，也可以用于匿名文件
  - D. 写时复制既可以用于共享区域，也可以用于私有区域

19. 垃圾收集器的根节点不包括以下哪个节里的数据:
- A. text
  - B. data
  - C. bss
  - D. stack
20. 按照课程中的描述, 套接字 socket 接口是一组函数, 它们与 UNIX I/O 函数一起使用可以构建网络应用。下列关于 socket 的描述中, 哪一个是错误的?
- A. 因特网的套接字地址存放在 `socketaddr_in` 结构中
  - B. 在使用套接字接口函数时, 要将对应协议的特定结构指针转换成通用结构 `sockaddr`
  - C. 当客户端通过 `connect()` 发起连接请求时, 由服务器端的服务进程通过 `listen()` 捕获该请求并建立监听套接字 `listenfd`
  - D. 对于迭代 echo 服务器, 当 `client1` 连接到该服务器且尚未断开连接时, `client2` 若想要连接该服务器会阻塞在从服务器读结果的操作上
21. 下列关于全球 IP 因特网的描述中, 哪一个是不正确的?
- A. IPV4 协议中, 一个 IP 地址是一个 32 位无符号整数
  - B. IP 地址结构中存放的地址总是以大端法存放的
  - C. 网络字节顺序是采用大端法的
  - D. 在 Linux 系统上, 用 `hostname -i` 命令可以确定自己主机的域名
22. 同一个进程内的不同线程共享以下几项?
- a) 函数体内部的静态变量
  - b) 堆
  - c) 文件描述符表
  - d) 条件码
- A. 1    B. 2    C. 3    D. 4
23. 令  $L_i$  表示加载共享变量到寄存器的指令,  $U_i$  表示更新寄存器的指令,  $S_i$  表示将更新值存回到共享变量的指令, 指令  $L_1$ 、 $S_1$ 、 $U_3$ 、 $S_3$  操作共享变量 `cnt`, 指令  $L_2$ 、 $S_2$ 、 $L_4$ 、 $S_4$  操作共享变量 `num`, 假定每次只允许交换相邻的两条指令, 则将下列指令序列变为正确的至少需要交换多少次:
- $L_2$ 、 $L_1$ 、 $U_1$ 、 $L_3$ 、 $L_4$ 、 $U_4$ 、 $U_3$ 、 $S_1$ 、 $U_2$ 、 $S_4$ 、 $S_3$ 、 $S_2$
- A.10    B.11    C.12    D.13

24. 以下程序在输出有限行之后就终止了，请问最有可能的原因是（假定所有函数都正常执行）

```
#include "csapp.h"
void *thread(void *dummy){
 while (1){
 printf("hello, world!\n");
 Sleep(1);
 }
}
int main(){
 pthread_t tid;
 Pthread_create(&tid, NULL, thread, NULL);
 Sleep(3);
}
```

- A. 主线程结束必然引发所有对等线程结束
- B. 主线程结束时调用了 `_exit` 导致进程结束
- C. 主线程结束后，内核发送 `SIGKILL` 杀死进程
- D. 主线程结束后，内核观察到对等线程运行时间过长，将其杀死

25. 有四个信号量，初值分别为：a=1, b=1, c=1, d=1

| 线程 1:   | 线程 2:   | 线程 3:   |
|---------|---------|---------|
| ① P(a); | ⑨ P(c); | ⑮ P(d); |
| ② P(b); | ⑩ P(b); | ⑯ P(a); |
| ③ P(d); | ⑪ V(c); | ⑰ V(d); |
| ④ P(c); | ⑫ V(b); | ⑱ P(b); |
| ⑤ V(d); | ⑬ P(a); | ⑲ V(a); |
| ⑥ V(a); | ⑭ V(a); | ⑳ V(b); |
| ⑦ V(b); |         |         |
| ⑧ V(c); |         |         |

上面的程序执行时，下列哪一个执行轨迹执行后已经出现死锁？

- A. ⑨①⑩②⑪⑮
- B. ①⑨⑩②⑮⑯
- C. ⑮①⑨⑩⑯②
- D. ①②③④⑨⑮

## 第二题 (10 分)

有一个五级流水线处理器，和课程中的 PIPE 处理器类似，包含：取指 (F)、译码 (D)、执行 (E)、访存 (M)、写回 (W) 五级，

- 1) 在理想设计情况下，流水线每一级的电路处理时间如下表所示，每两级流水线阶段之间插入的寄存器耗时为 1ns

|           | F | D | E | M | W |
|-----------|---|---|---|---|---|
| 处理时间 (ns) | 8 | 5 | 9 | 8 | 4 |

请问流水线运行的时钟周期，最快为 \_\_\_\_\_ ns

- 2) 需要执行如下 Y86-64 汇编代码，寄存器的初始状态如右下表格所示：

```
0x000 t: mrmovq (%rdi), %rbx
0x002 addq %rbx, %rax
0x004 addq $8, %rdi
0x006 addq $1, %rcx
0x008 xorq $8, %rcx
0x00a jne t
```

|      |        |
|------|--------|
| %rdi | 0x8008 |
| %rax | 0      |
| %rbx | 0      |
| %rcx | 0      |

流水线支持课上讲的数据前递 (data forwarding)，分支预测设计为“始终跳转” (predicted as taken)，请问执行上述代码，需要 \_\_\_\_\_ 个时钟周期？

- 3) 现需要考虑实际执行时，存储层次对于流水线执行时间的影响：

- 该处理器包含一个 L1 指令缓存 (I-cache) 和一个 L1 数据缓存 (D-cache)；
- I-cache 是一个直接映射 (direct-mapped, E=1) 缓存，总容量 32KB，每个缓存块的大小是 32Byte，代码执行前 I-cache 全部为空；
- D-cache 也是一个直接映射 (direct-mapped, E=1) 缓存，总容量 32KB，每个缓存块的大小是 32Byte，代码执行前 D-cache 全部为空；
- 取指 (F) 阶段，如果指令在 I-cache 中，则需要 1 个周期，否则需要 10 个周期；
- 访存 (M) 阶段，如果数据在 D-cache 中，则需要 1 个周期，否则需要 10 个周期；

请问执行上述代码 I-cache 缓存命中 \_\_\_\_\_ 次、D-cache 缓存命中 \_\_\_\_\_ 次？

在存储层次的影响下，执行完代码需要 \_\_\_\_\_ 个时钟周期？

第三题 (10 分)

本题基于下列 m.c 及 swap.c 文件所编译生成的 m.o 和 swap.o，编译和运行在 Linux/x86-64 下使用 GCC 完成，编译过程未加优化选项。

|                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                                   |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>//m.c void myswap(); void func(); char buf[2] = {1, 2}; void *bufp0; void *bufp1; char temp; int main(){     func();     .....//略去题目无关代码     myswap(temp);     .....//略去题目无关代码     return 0; }</pre> | <pre>//swap.c extern int buf[]; char *bufp0; .....//略去题目无关代码 char *bufp1 = &amp;buf[1]; void myswap(char temp){     static int count = 0;     .....//略去题目无关代码     bufp0 = &amp;buf[0];     temp = *bufp0;     .....//略去题目无关代码     *bufp0 = *bufp1;     *bufp1 = temp;     .....//略去题目无关代码     count++; } void func(){     .....//略去题目无关代码 }</pre> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

1) 对于每个 swap.o 中定义和引用的符号，请用“是”或“否”指出它是否在模块 swap.o 的 .symtab 节中存在符号表条目。如果存在条目，则请指出定义该符号的模块 (swap.o 或 m.o)、符号类型（局部、全局或外部）以及该符号在所属的模块（“即该符号在该模块中被定义”）中所处的节；如果不存在条目，则请将该行后继空白处标记为“/”。

| 符号    | .symtab 有条目? | 符号类型 | 定义符号的模块 | 节 |
|-------|--------------|------|---------|---|
| buf   |              |      |         |   |
| bufp0 |              |      |         |   |
| count |              |      |         |   |
| func  |              |      |         |   |
| temp  |              |      |         |   |

2) 下图左边给出了 m.o 和 swap.o 的反汇编文件，右边给出了采用某个配置链接成可执行程序后再反汇编出来的文件。根据答题需要，其中的信息略有删减。



|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> 0000000000.....&lt;main&gt;: 55                push   %rbp 48 89 e5          mov    %rsp,%rbp b8 00 00 00 00    mov    \$0x0,%eax e8 00 00 00 00    callq  e &lt;main+0xe&gt; ① ..... 0f b6 05 00 00 00 00 movzbl 0x0(%rip),%eax ② 0f be c0          movsbl %al,%eax 89 c7            mov    %eax,%edi b8 00 00 00 00    mov    \$0x0,%eax e8 00 00 00 00    callq  26 &lt;main+0x26&gt; ③ ..... b8 00 00 00 00    mov    \$0x0,%eax 5d              pop    %rbp c3              retq </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                      | <pre> 00000000000001000 &lt;main&gt;: 1000: 55                push   %rbp 1001: 48 89 e5          mov    %rsp,%rbp 1004: b8 00 00 00 00    mov    \$0x0,%eax 1009: e8 _____    callq  302e &lt;func&gt; ..... 1010: 0f b6 05 _____    movzbl 0x??????(%rip),%eax ② 1017: 0f be c0          movsbl %al,%eax 101a: 89 c7            mov    %eax,%edi 101c: b8 00 00 00 00    mov    \$0x0,%eax 1021: e8 _____    callq  10e4 &lt;myswap&gt; ③ ..... 10db: b8 00 00 00 00    mov    \$0x0,%eax 10e0: 5d              pop    %rbp 10e1: c3              retq </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <pre> 0000000000.....&lt;myswap&gt;: 55                push   %rbp 48 89 e5          mov    %rsp,%rbp 89 f8            mov    %edi,%eax 88 45 fc          mov    %al,-0x4(%rbp) ..... 48 c7 05 00 00 00 00 00 00 00 00 ⑤④                 movq    \$0x0,0x0(%rip) ..... 48 8b 05 00 00 00 00 00 mov    0x0(%rip),%rax ⑥ 0f b6 00          movzbl (%rax),%eax 88 45 fc          mov    %al,-0x4(%rbp) ..... 48 8b 05 00 00 00 00 00 mov    0x0(%rip),%rax ⑦ 48 8b 15 00 00 00 00 00 mov    0x0(%rip),%rdx ⑧ 0f b6 12          movzbl (%rdx),%edx 88 10            mov    %dl,(%rax) 48 8b 05 00 00 00 00 00 mov    0x0(%rip),%rax ⑨ 0f b6 55 fc          movzbl -0x4(%rbp),%edx 88 10            mov    %dl,(%rax) ..... 8b 05 00 00 00 00 00 00 mov    0x0(%rip),%eax ⑩ 83 c0 01          add    \$0x1,%eax 89 05 00 00 00 00 00 00 mov    %eax,0x0(%rip) ⑪ 90              nop 5d              pop    %rbp c3              retq </pre> | <pre> 000000000000010e4&lt;myswap&gt;: 10e4: 55                push   %rbp 10e5: 48 89 e5          mov    %rsp,%rbp 10e8: 89 f8            mov    %edi,%eax 10ea: 88 45 fc          mov    %al,-0x4(%rbp) ..... 10f4: 48 c7 05 _____    movq    _____, _____(%rip) ⑤ ④                 _____ ④, ⑤ ..... 1104: 48 8b 05 _____    mov    _____(%rip),%rax ⑥ 110b: 0f b6 00          movzbl (%rax),%eax 110e: 88 45 fc          mov    %al,-0x4(%rbp) ..... 1229: 48 8b 05 _____    mov    _____(%rip),%rax ⑦ 1230: 48 8b 15 _____    mov    _____(%rip),%rdx ⑧ 1237: 0f b6 12          movzbl (%rdx),%edx 123a: 88 10            mov    %dl,(%rax) 123c: 48 8b 05 _____    mov    _____(%rip),%rax ⑨ 1243: 0f b6 55 fc          movzbl -0x4(%rbp),%edx 1247: 88 10            mov    %dl,(%rax) ..... 124b: 8b 05 _____    mov    _____(%rip),%eax ⑩ 1251: 83 c0 01          add    \$0x1,%eax 1254: 89 05 _____    mov    %eax, _____(%rip) ⑪ 125a: 90              nop 125b: 5d              pop    %rbp 125c: c3              retq </pre> |
| <pre> 0000000000.....&lt;func&gt;: ..... </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | <pre> 0000000000000302e&lt;func&gt;: ..... .....略去部分和答题无关的信息..... 000000000000a000 &lt;buf&gt;: .....略去部分和答题无关的信息..... 000000000000a250 &lt;bufp1&gt;: .....略去部分和答题无关的信息..... 0000000000dedd38 &lt;count.1837&gt;: .....略去部分和答题无关的信息..... 0000000000dedd40 &lt;bufp0&gt;: .....略去部分和答题无关的信息..... </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |

在上图中对所涉及到的重定位条目进行用数字①至⑪进行了标记，请根据下表中所提供的重定位条目信息，计算相应的重定位引用值并填写下表。

| 编号 | 重定位条目信息                                                                             | 应填入的重定位引用值<br>一律填写 32 位 16 进制数 |
|----|-------------------------------------------------------------------------------------|--------------------------------|
| ①  | r.offset = 0x0a      r.symbol = 本题不提供<br>r.type = R_X86_64_PC32      r.addend = -4  |                                |
| ③  | r.offset = 0x22      r.symbol = 本题不提供<br>r.type = R_X86_64_PC32      r.addend = -4  |                                |
| ④  | r.offset = 0x17      r.symbol = 本题不提供<br>r.type = R_X86_64_32      r.addend = 0     |                                |
| ⑨  | r.offset = 0x15b      r.symbol = 本题不提供<br>r.type = R_X86_64_PC32      r.addend = -4 |                                |
| ⑪  | r.offset = 0x172      r.symbol = 本题不提供<br>r.type = R_X86_64_PC32      r.addend = -4 |                                |

第四题 (10 分)

C 语言中的代码如下:

```
int counter = 0;

void handler(int sig) {
 counter++;
}

int main(int argc, char *argv[])
{
 int fd1, fd2, fd3;
 char c1, c2, c3;
 char *fname = argv[1];
 int parent;

 signal(SIGUSR1, handler);

 fd1 = open(fname, O_RDONLY);
 read(fd1, &c1, 1);
 parent = getpid();
 if (fork()) {
 wait();
 fd2 = open(fname, O_RDONLY);
 } else {
 kill(parent, SIGUSR1);
 fd2 = dup(fd1);
 }
 read(fd2, &c2, 1);

 parent = getpid();
 if (fork()) {
 wait();
 fd3 = open(fname, O_RDONLY);
 } else {
 kill(parent, SIGUSR1);
 fd3 = dup(fd2);
 }

 read(fd3, &c3, 1);
 printf("%c %c %c %d\n", c1, c2, c3, counter);
 return 0;
}
```

当 fname 文件内容为 abcde 时, 这段代码的打印结果为\_\_\_\_行, 输出结果如下: (不考虑出错的情况)

### 第五题 (10 分)

考虑一对  $n \times n$  矩阵相乘问题:  $C=AB$ 。矩阵乘法函数通常是用 3 个嵌套的循环来实现的, 分别用索引  $i$ 、 $j$  和  $k$  来标识。如果改变循环的次序, 我们就能得到矩阵乘法的 6 个在功能上等价的版本, 如下图所示:

|                                                                                                                                                                                                |                                                                                                                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>for (i = 0; i &lt; n; i++)     for (j = 0; j &lt; n; j++) {         sum = 0.0;         for (k = 0; k &lt; n; k++)             sum += A[i][k]*B[k][j];         C[i][j] += sum;     }</pre> | <pre>for (j = 0; j &lt; n; j++)     for (i = 0; i &lt; n; i++) {         sum = 0.0;         for (k = 0; k &lt; n; k++)             sum += A[i][k]*B[k][j];         C[i][j] += sum;     }</pre> |
| <b>1) ijk 版本</b>                                                                                                                                                                               | <b>2) jik 版本</b>                                                                                                                                                                               |
| <pre>for (j = 0; j &lt; n; j++)     for (k = 0; k &lt; n; k++) {         r = B[k][j];         for (i = 0; i &lt; n; i++)             C[i][j] += A[i][k]*r;     }</pre>                         | <pre>for (k = 0; k &lt; n; k++)     for (j = 0; j &lt; n; j++) {         r = B[k][j];         for (i = 0; i &lt; n; i++)             C[i][j] += A[i][k]*r;     }</pre>                         |
| <b>3) jki 版本</b>                                                                                                                                                                               | <b>4) kji 版本</b>                                                                                                                                                                               |
| <pre>for (k = 0; k &lt; n; k++)     for (i = 0; i &lt; n; i++) {         r = A[i][k];         for (j = 0; j &lt; n; j++)             C[i][j] += r*B[k][j];     }</pre>                         | <pre>for (i = 0; i &lt; n; i++)     for (k = 0; k &lt; n; k++) {         r = A[i][k];         for (j = 0; j &lt; n; j++)             C[i][j] += r*B[k][j];     }</pre>                         |
| <b>5) kij 版本</b>                                                                                                                                                                               | <b>6) ikj 版本</b>                                                                                                                                                                               |

本题目具有以下假设:

- 每个数组都是一个 double 类型的  $n \times n$  的数组,  $\text{sizeof}(\text{double})=8$
- 虚拟存储分页管理, 页大小为 4K 字节, 且 A 的起始地址为页对齐, ABC 连续存放
- TLB 为全相联结构
- 初始时 TLB 为空, 数据全部在硬盘中
- 一级 dTLB 有 64 表项, 采用 LRU (Least-Recently-Used) 置换策略
- 分析过程只考虑矩阵操作本身的数据空间, 且  $i/j/k/n/r/\text{sum}$  等变量都存放在寄存器中
- 分析过程不考虑进程切换和异常处理程序的影响
- 不考虑对页表本身的访问

| n 的<br>取值 | 缺页<br>次数 | 一级 dTLB 失效次数 |       |       |       |       |       |
|-----------|----------|--------------|-------|-------|-------|-------|-------|
|           |          | 1-ijk        | 2-jik | 3-jki | 4-kji | 5-kij | 6-ikj |
| 8         |          |              |       |       |       |       |       |
| 16        |          |              |       |       |       |       |       |
| 64        |          |              |       |       |       |       |       |
| 512       |          |              |       |       |       |       |       |

## 第六题 (10 分)

1. 火锅洞洞主终于要请吃火锅啦! 由于防疫规定, 洞主决定分批邀请, Alice、Bob、……、Zach 这 26 位同学顺理成章地成为了第一批受到邀请的同学。他们围坐在一张圆桌旁, 按照首字母顺序排列 (即 Alice 的右边是 Bob, Bob 的右边是 Carol, …… , Zach 的右边是 Alice)。吃饭之前, 洞主想要验证他们的身份, 他提出了这样的要求:

- 1) 每位同学有一个编号, 即字母序号减 1, 也就是说, Alice 是 0 号, Bob 是 1 号, 以此类推。
- 2) 每位同学需要同时拿着旁边两位同学的手机去找洞主 (不必带上自己的手机), 按照洞主的要求在树洞里发信息以验证身份。
- 3) 可以同时有多位同学找洞主验证 (也就是说, Alice 需要拿着 Zach 和 Bob 的手机找洞主, 并且在同一时间, Bob 也可以拿着 Alice 和 Carol 的手机找洞主。但是, 如果 Alice 还没同时拿到 Zach 和 Bob 的手机, 就只能坐在座位上等待。)

所幸, 在座有多位字母君学过 ICS, 他们决定采用信号量解决问题。

1.1 Alice 给出了这样的代码, 其中 thread 函数的参数指明了字母君的编号:

```
1. #include "csapp.h"
2. #define N 26
3. #define UP(i) ((i + 1) % N)
4. #define DOWN(i) ((i - 1 + N) % N)
5. sem_t sem[N];
6.
7. void *thread(void *i){
8. int num = (int)i;
9. P(&sem[UP(num)]);
10. P(&sem[DOWN(num)]);
11.
12. /* verify identity */
13.
14. V(&sem[UP(num)]);
15. V(&sem[DOWN(num)]);
16. }
17. int main(){
18. for (int i = 0; i < N; i++)
19. Sem_init(&sem[i], 0, 1);
20. pthread_t tid[N];
21. for (int i = 0; i < N; i++)
22. Pthread_create(&tid[i], NULL, thread, (void *)i);
23. Pthread_exit(0);
24. }
```

Bob 一看, 皱起了眉头。请问这段代码可能会发生什么问题 (用一个专用名词表述即可)?

1.2 Carol 把 thread 函数换成了下面这样:

```
1. void *thread(void *i){
2. int num = (int)i;
3. for (int i = 0; i < N; i++)
4. P(&sem[i]);
5. /* verify identity */
6. for (int i = 0; i < N; i++)
7. V(&sem[i]);
8. }
```

Dave 指出, 这段代码固然正确, 但是效率太低了, 他认为只要把第 3、4 行的代码换成一个 P 操作, 把第 6、7 行的代码换成一个 V 操作, 就能实现同样的效果。请问 Dave 的方法是:

1.2.1 P 操作: P(&sem[\_\_\_\_\_]);

1.2.2 V 操作: V(&sem[\_\_\_\_\_]);

1.3 Eve 看了 Dave 的方案还是摇头, 这样执行效率太低了, 没能利用同时可以有多位同学找洞主验证的条件。借助“互斥锁加锁顺序规则”, 他给出了自己的代码。你能把代码补全吗?

```
1. void *thread(void *i)
2. {
3. int num = (int)i;
4. if (UP(num) < DOWN(num)){
5. P(&sem[___A___]);
6. P(&sem[___B___]);
7. }
8. else{
9. P(&sem[___C___]);
10. P(&sem[___D___]);
11. }
12. /* verify identity */
13. V(&sem[UP(num)]);
14. V(&sem[DOWN(num)]);
15. }
```

A:\_\_\_\_\_ B:\_\_\_\_\_ C:\_\_\_\_\_ D:\_\_\_\_\_

以上 4 空均填序号:

- ① UP(num)
- ② DOWN(num)
- ③ num

2. Hungary Alice 发现自己没吃上第一场的火锅, 一怒之下, 给上面的 26 位字母君出了一道难题: 用二元信号量实现信号量 (也就是值可以是任意非负整数的信号量)。精通 ICS 的你自然认为这是小儿科, 迅速补全了以下代码。

```

1. #include "csapp.h"
2. typedef struct{
3. int value;
4. sem_t mutex;
5. sem_t zero;
6. } my_sem_t;
7. void my_sem_init(my_sem_t *sem, unsigned int value){
8. sem->value = value;
9. Sem_init(&sem->mutex, 0, __A__);
10. Sem_init(&sem->zero, 0, __B__);
11. }
12. void my_P(my_sem_t *sem){
13. P(&sem->mutex);
14. sem->value--;
15. if (sem->value __C__ 0)
16. {
17. __D__;
18. __E__;
19. }
20. else
21. V(&sem->mutex);
22. }
23. void my_V(my_sem_t *sem){
24. P(&sem->mutex);
25. sem->value++;
26. if (sem->value __F__ 0)
27. {
28. V(&sem->mutex);
29. V(&sem->zero);
30. }
31. else
32. V(&sem->mutex);
33. }

```

A: \_\_\_\_\_ B: \_\_\_\_\_ C: \_\_\_\_\_  
 D: \_\_\_\_\_ E: \_\_\_\_\_ F: \_\_\_\_\_

D 和 E 请填序号:

- ① P(&sem->mutex)
- ② P(&sem->zero)
- ③ V(&sem->mutex)
- ④ V(&sem->zero)